

DISCIPLINA
ARQUITETURA DE SOFTWARE
E ENGENHARIA DE REQUISITOS



Sumário

Prefácio	13
O que este livro cobre	14
1. Introdução sobre arquitetura.....	15
1.1. O que exatamente é arquitetura de software?	15
1.2. Definição padrão ISO/IEC/IEEE 42010.....	15
1.3. O que constitui uma arquitetura de software?.....	16
1.4. Arquitetura de software é uma abstração.....	16
1.5. A arquitetura de software trata das coisas importantes	16
1.6. Por que a arquitetura de software é importante?.....	17
1.7. Definindo uma solução para atender aos requisitos	17
1.8. Habilitando e inibindo atributos de qualidade	18
1.9. Dando a você a capacidade de prever o sistema de software.....	18
1.10. Facilitando a comunicação entre as partes interessadas.....	18
1.11. Gerenciando a mudança	19
1.12. Fornecendo um modelo reutilizável	19
1.13. Melhorar as estimativas de custo e esforço	20
1.14. Um sistema de software com requisitos precisos que são refletidos no software	20
1.15. Serve como treinamento para membros da equipe	21
1.16. A arquitetura de software não é uma bala de prata.....	21
1.17. Quem são os consumidores de software arquiteturas?	21
1.18. Qual é a função do arquiteto de software?	22
1.19. Arquitetos de software são líderes técnicos.....	22
1.20. Os arquitetos de software desempenham uma série de funções	23
1.21. Arquitetos de software da torre de marfim.....	23
1.22. O que se espera que os arquitetos de software saibam?	24
1.23. Não fique sobrecarregado.....	25
1.24. A função de arquiteto de software é certa para você?.....	25
1.25. Resumo.....	26
2. Arquitetura de software em uma organização	27
2.1. Tipos de arquitetos de software	27
2.2. Arquiteto empresarial	28
2.3. Solução de arquitetura.....	28
2.4. Arquiteto de aplicativos	29
2.5. Arquiteto de dados/arquiteto de informação	29

2.6. Arquiteto de infraestrutura.....	30
2.7. Arquiteto de segurança da informação.....	30
2.8. Arquiteto de nuvem	31
2.9. Metodologias de desenvolvimento de software	31
2.10. O modelo em cascata.....	32
2.11. Metodologia das Fases da Cascata.....	32
2.12. Problemas com a metodologia em cascata	33
2.13. Metodologias ágeis de desenvolvimento de software	34
2.14. Valores e princípios ágeis	34
2.15. Uma metodologia iterativa	34
2.16. Adaptável em vez de preditivo.....	35
2.17. Reuniões diárias em pé	35
2.18. Gerenciamento de Projetos	36
2.18.1. A importância da estimativa de projetos de software.....	36
2.18.2. Colocar esforço nas estimativas.....	36
2.18.3. Ser realista (ou mesmo pessimista)	37
2.18.4. Equipe e fatores situacionais a serem considerados	37
2.18.5. Mudanças no cronograma do projeto	37
2.18.6. Colocar um projeto de volta no cronograma.....	37
2.18.13. Gestão de risco de software.....	41
2.18.14. Gerenciamento de configurações.....	43
2.19. Mudança de gestão	43
2.19. Linhas de produtos de software.....	45
2.19.1. Benefícios de uma linha de produtos de software	46
2.19.2. Ativos essenciais de uma organização	46
2.19.2. Riscos da engenharia da linha de produtos	46
2.20. Resumo.....	47
3. Compreendendo o domínio	48
3.1. Desenvolvendo visão de negócios	48
3.2. Familiaridade com tópicos gerais de negócios	49
3.3. Compreender os negócios da sua organização.....	49
3.4. Design orientado a domínio.....	50
3.5. Incentiva e melhora a comunicação.....	50
3.6. O que é uma linguagem onipresente?	50
3.7. Entidades, objetos de valor e agregados	52
3.7.1. Entidades.....	52

3.7.2. Objetos de valor	52
3.7.3. Agregados e entidades raiz	52
3.7.4. Separando o domínio em subdomínios	52
3.8. O que são contextos limitados?	53
3.9. Engenharia de Requisitos	54
3.9.1. Tipos de requisitos de software	55
3.9.2. A importância da engenharia de requisitos	57
3.9.3. Levantamento de requisitos	58
3.9.3. Técnicas para elicitar requisitos	59
3.9.4. Obtenha acesso às partes interessadas adequadas	63
3.10. Resumo	63
4. Atributos de qualidade de software	64
4.1. Externo ou interno	64
4.2. Atributos de qualidade e o SDLC	65
4.3. Atributos de qualidade de teste	65
4.4. Capacidade de Manutenção	66
4.5. Tipos de manutenção de software	66
4.5.1. Manutenção perfeita	67
4.5.2. Manutenção adaptativa	67
4.5.3. Manutenção preventiva	68
4.6. Modificabilidade	68
4.7. Extensibilidade e flexibilidade	68
4.7.1. Escopo de modificações	68
4.7.2. Projetando para sustentabilidade	68
4.7.3. Aumentando a coesão	69
4.7.4. Reduzindo o acoplamento	69
4.7.5. Medindo a capacidade de manutenção	69
4.7.5.1. Linhas de código (LOC)	70
4.7.5.2. Complexidade ciclomática	70
4.7.5.3. Árvore de profundidade de herança (DIT)	71
4.7.5.4. Usabilidade	72
4.7.6. Permitindo que os usuários concluam suas tarefas com eficiência	72
4.7.6.1. Aprendizagem	72
4.7.6.2. Fornecendo feedback útil	73
4.7.6.3. Acessibilidade	73
4.7.6.4. A usabilidade precisa ser considerada durante requisitos	73

4.7.6.5. Testando usabilidade	74
4.7.6.6. Design visual atraente	74
4.7.6.7. Fornecendo um bom sistema de ajuda.....	75
4.7.6.8. O software deve ser útil e não apenas utilizável	75
4.7.7. Disponibilidade.....	76
4.7.7.1. Calculando a disponibilidade com base no tempo	76
4.7.7.2. Calculando a disponibilidade com base no sucesso da solicitação avaliar	76
4.7.7.3. Falhas, erros e falhas.....	77
4.7.7.4. Detectando falhas	77
4.7.7.4.1. Resposta de ping/eco.....	77
4.7.7.4.2. Batimento cardíaco	77
4.7.7.4.2. Timestamp.....	78
4.7.7.4.3. Votação	78
4.7.7.4.4. Teste de sanidade/verificação de sanidade	78
4.7.7.4.5. Monitoramento de condição	78
4.7.7.4.6. Autotestes	78
4.7.7.5. Recuperando-se de falhas.....	78
4.7.7.5.1. Manipulação de exceção.....	78
4.7.7.5.2. Estratégia de nova tentativa	79
4.7.7.5.3. Níveis variáveis de redundância.....	79
4.7.7.5.4. Rollback	80
4.7.7.5.5. Degradação graciosa	80
4.7.7.5.6. Ignorando o comportamento defeituoso	80
4.7.7.6. Prevenção de falhas	80
4.7.7.6.1. Remoção do serviço	80
4.7.7.6.2. Transações.....	80
4.7.7.6.2. Aumento de conjuntos de competências	80
4.7.7.6.3. Prevenção de exceção.....	81
4.7.7.7. Portabilidade	81
4.7.7.8. Adaptabilidade	81
4.7.7.9. Instalabilidade	81
4.7.7.10. Substituibilidade.....	82
4.7.7.11. Internacionalização e localização.....	82
4.7.7.12. Manter a portabilidade	83
4.7.7.13. Interoperabilidade	83
4.7.7.13.1. Desafios com interoperabilidade	83

4.7.7.13.2. Localizando e trocando informações com outro sistema	84
4.7.7.13.3. Padrões de interoperabilidade.....	85
4.7.7.13.4. Teste de interoperabilidade	85
4.7.7.14. Testabilidade	85
4.7.7.14.1. Controlabilidade	86
4.7.7.14.2. Observabilidade	86
4.7.7.14.3. Isolabilidade	86
4.7.7.14.4. Automatabilidade.....	86
4.7.7.14.5. Complexidade do software	87
4.7.7.14.6. Importância da documentação de teste	88
4.7.7.15. O que é um bom testador?	88
4.7.7.16. Resumo.....	89
5. Design de Arquiteturas de Software	90
5.1. Tomando decisões de design	91
5.2. Termos de design de arquitetura de software.....	91
5.2.1. Estrutura.....	92
5.2.2. Elemento	92
5.2.3. Sistema	92
5.2.4. Subsistema	92
5.2.5. Módulo	92
5.2.6. Componente.....	93
5.3. A importância da arquitetura de software Projeto.....	93
5.4. Tomando decisões importantes.....	93
5.5. Evitar decisões de design pode incorrer em dívida de aspectos técnicos	93
5.6. Comunicar a arquitetura a outras pessoas	94
5.7. Fornecendo orientação para desenvolvedores.....	94
5.7.1. Influenciando partes não técnicas do projeto	95
5.8. Design de cima para baixo versus de baixo para cima aproximações	95
5.8.1. Abordagem de cima para baixo	95
5.8.2. Vantagens da abordagem de cima para baixo	95
5.8.3. Desvantagens da abordagem de cima para baixo	96
5.8.4. Abordagem de baixo para cima	96
5.8.5. Vantagens da abordagem ascendente.....	97
5.8.6. Desvantagens da abordagem ascendente	97
5.8.7. Qual abordagem devo usar?	98
5.9. Software greenfield versus brownfield	99

5.9.1. Sistemas Greenfield	99
5.9.2. Sistemas brownfield	100
5.10. Drivers arquitetônicos	100
5.11. Objetivos de design	100
5.12. Requisitos funcionais primários	101
5.13. Cenários de atributos de qualidade	102
5.14. Priorizando cenários de atributos de qualidade	102
5.15. Restrições	103
5.16. Preocupações arquitetônicas	103
5.17. Aproveitando os princípios de design e soluções	104
5.18. Selecionando um conceito de design	104
5.19. Padrões de arquitetura de software	104
5.20. Arquiteturas de referência	105
5.20.1. Benefícios das arquiteturas de referência	105
5.21. Refatorando uma arquitetura de referência para suas necessidades	105
5.22. Criando sua própria arquitetura de referência	106
5.22.1. Táticas	106
5.23. Software desenvolvido externamente	107
5.24. Comprar ou construir?	107
5.25. Vantagens/desvantagens da construção	107
5.26. Vantagens/desvantagens de comprar	108
5.27. Pesquisando software externo	108
5.28. Devo usar software de código aberto (OSS)?	108
5.29. Vantagens de usar software de código aberto	109
5.30. Desvantagens de usar software de código aberto	110
5.31. Documentando a arquitetura do software Projeto	110
5.31.1. Esboçando o projeto de arquitetura	110
5.31.2. Documentando a lógica do design	111
5.31.3. Justificativa de design para avaliação de design	112
5.31.4. Fundamentação do projeto para verificação do projeto	112
5.31.5. Justificativa de design para transferência de conhecimento de design	112
5.31.6. Justificativa de design para comunicação de design	112
5.31.7. Justificativa de design para documentação de design	113
5.31.8. Justificativa de design para reutilização de design	113
5.32. Usando uma abordagem sistemática para software projeto de arquitetura	113
6. Padrões de arquitetura de software	114

6.1. Padrões de arquitetura de software	114
6.2. Usando padrões de arquitetura de software	115
6.3. Uso excessivo de padrões de arquitetura	115
6.4. Compreendendo a diferença entre estilos de arquitetura e padrões de arquitetura ...	115
6.5. Arquitetura em camadas	116
6.5.1. Camadas abertas versus camadas fechadas	116
6.5.2. Camadas lógicas versus camadas físicas	117
6.5.3. Vantagens das arquiteturas em camadas	117
6.5.4. Desvantagens das arquiteturas em camadas.....	118
6.6. Arquitetura cliente-servidor (arquitetura de duas camadas)	119
6.6.1. Usando procedimentos armazenados para lógica de aplicativo	120
6.7. Arquitetura N-tier.....	121
6.7.1. Camada de apresentação.....	121
6.7.2. Camada de negócios	122
6.7.3. Camada de dados	122
6.8. Arquitetura orientada a eventos.....	122
6.8.1. Canais de eventos.....	123
6.8.2. Filas de mensagens.....	123
6.8.3. O padrão de canal ponto a ponto	123
6.8.4. Tópicos de mensagens	123
6.8.5. O padrão publicar-assinar	123
6.8.6. Topologias de arquitetura orientada a eventos.....	124
6.8.7. Implementações de mediador de eventos	125
6.8.8. A topologia do broker	125
6.8.9. Estilos de processamento de eventos.....	126
6.8.9. Processamento de evento simples (SEP)	126
6.8.10. Processamento de fluxo de eventos (ESP).....	127
6.8.11. Processamento de eventos complexos (CEP)	127
6.8.12. Tipos de funcionalidade orientada a eventos	127
6.8.13. Notificação de evento	127
6.8.14. Transferência de estado realizada por evento.....	128
6.8.15. Fonte de eventos.....	128
6.9. O padrão Model-View-Controller.....	129
6.9.1. Modelo	130
6.9.2. Visualização	130
6.9.3. Controlador	130

6.9.4. Vantagens do padrão MVC.....	130
6.9.5. Desvantagens do padrão MVC.....	131
6.10. O padrão Model-View-Presenter	131
6.10.1. Modelo	131
6.10.2. Visualização	132
6.10.3. Apresentador	132
6.11. O padrão Model-View-ViewModel	132
6.11.1. Modelo	133
6.11.2. Visualização	133
6.11.3. ViewModel	133
6.12. A Responsabilidade da Consulta de Comando.....	133
6.12.1. Padrão de segregação	133
6.13. O modelo de consulta e o modelo de comando	134
6.13.1. Usando fonte de eventos com CQRS	135
6.13.2. Vantagens do CQRS.....	136
6.13.3. Desvantagens do CQRS	136
6.14. Arquitetura Orientada a Serviços.....	137
6.14.1. O que torna SOA diferente de outras distribuídas soluções?	137
6.14.2. Benefícios de usar um SOA	137
6.14.3. Aumenta o alinhamento entre negócios e tecnologia.....	137
6.14.4. Promove a federação dentro de uma organização.....	138
6.14.5. Permite diversidade de fornecedores.....	138
6.14.6. Aumenta a interoperabilidade intrínseca	138
6.14.7. Funciona bem com metodologias de desenvolvimento ágil.....	138
6.14.8. Análise de custo-benefício de SOA.....	139
6.14.9. Desafios com SOA	139
6.14.10. Princípios-chave para orientação a serviços	140
6.14.11. Contrato de serviço padronizado.....	140
6.14.12. Serviço de acoplamento solto	140
6.14.13. Abstração de serviço	141
6.14.14. Reutilização de serviço.....	141
6.14.15. Autonomia de serviço	141
6.14.16. Apátrida de serviço	141
6.14.17. Descoberta de serviço	142
6.14.18. Composibilidade de serviço	142
6.14.19. Estratégias de entrega SOA.....	142

6.14.20. A estratégia de cima para baixo	142
6.14.21. A estratégia de baixo para cima	142
6.14.22. A estratégia ágil	143
6.14.23. Análise orientada a serviços	143
6.14.24. Identificação de sistemas de automação existentes	144
6.14.25. Serviços candidatos de modelagem	144
6.14.26. Camadas de serviço e modelos de serviço	144
6.14.27. Serviço de tarefas	145
6.14.28. Serviço de entidade	145
6.14.29. Serviço de utilidade	146
6.14.30. Design orientado a serviços	146
6.14.31. Design de interface de serviço	146
6.14.32. Granularidade da interface de serviço	147
6.14.33. Registros de serviço	147
6.14.33. Descrições de serviço	148
6.14.34. Estruturação de namespaces	149
6.14.35. Orquestração e coreografia	149
6.15. Resumo	150
7. Arquitetando aplicativos modernos	151
7.1. Arquitetura monolítica	151
7.1.1. Benefícios de uma arquitetura monolítica	152
7.1.2. Desvantagens de uma arquitetura monolítica	152
7.2. Arquitetura de micro serviço	153
7.2.1. SOA bem-feito	154
7.2.2. Características da arquitetura de micro serviço	154
7.2.3. Interfaces de serviço bem definidas	155
7.2.4. Serviços autônomos e implantáveis de forma independente	155
7.2.5. Armazenamento de dados independente	155
7.2.6. Melhor isolamento de falhas	155
7.2.7. Comunicação com protocolos de mensagem leves	156
7.2.8. Criação de micros serviços políglotas	156
7.2.9. Programação políglota	157
7.2.10. Persistência políglota	157
7.2.11. Usando muitas tecnologias	158
7.3. Micros serviços sem estado versus micro serviços com estado	160
7.3.1. Descoberta de serviço	160

7.3.2. Usando um registro de serviço.....	160
7.3.3. Padrão de auto registro.....	161
7.3.4. Padrão de registro de terceiros.....	161
7.3.5. Tipos de descoberta de serviço.....	162
7.3.6. Padrão de descoberta do lado do cliente	162
7.3.7. Padrão de descoberta do lado do servidor	163
7.4. Usar micro serviços não é para todos	164
7.5. Arquitetura sem servidor	165
7.4.5. Programas	165
7.5.1. Função como serviço (FaaS)	166
7.5.2. Back-end como serviço (BaaS)	167
7.5.3. Vantagens de arquiteturas sem servidor	167
7.6. Concentre-se na construção de seus produtos principais	168
7.6.1. Desenvolvimento poliglota	168
7.6.2. Desvantagens das arquiteturas sem servidor	168
7.6.3. Dificuldades com depuração e monitoramento	168
7.6.4. Problemas de multilocalização	168
7.6.5. Bloqueio do fornecedor	168
7.6.6. Complexidade de projetar muitas funções	169
7.6.7. Não há tantas otimizações de tempo de execução	169
7.6.8. Ainda imaturo.....	169
7.7. Adotando uma abordagem híbrida para sem servidor	169
7.7.1. Implantação de função.....	169
7.7.2. Invocação de função	170
7.8. Aplicativos nativos da nuvem.....	172
7.8.1. Razões para mudar para a nuvem.....	173
7.9. O que são aplicativos nativos da nuvem?	174
7.9.1. Containerized	174
7.9.2. Orquestrado dinamicamente	174
7.10. Orientado a micro serviços	175
7.10.1. Sem tempo de inatividade	175
7.11. Entrega contínua	176
7.12. Suporte para uma variedade de dispositivos.....	176
7.13. Aplicativos de doze fatores	176
7.13.1. Base de código	177
7.13.2. Dependências.....	177

7.13.3. Configuração	178
7.13.4. Serviços de apoio	178
7.13.5. Construir/Liberar/Executar	178
7.13.6. Processos.....	179
7.13.7. Ligação de porta	180
7.13.8. Simultaneidade	180
7.13.9. Descartabilidade.....	180
7.13.10. Paridade de desenvolvimento/produção.....	180
7.13.11. Histórico	181
7.13.12. Processos administrativos.....	181
7.14. Resumo.....	181

Prefácio

Os sistemas de software modernos são complexos e a função do arquiteto de software é desafiadora. Este livro foi escrito para ajudar a transição de desenvolvedores de software para a função de um software arquiteto e para ajudar os arquitetos de software existentes a serem bem-sucedidos em suas funções. Isto ajuda os leitores a entender como ser um arquiteto de software é diferente de ser um desenvolvedor e o que é necessário para ser um arquiteto de software eficaz.

Este guia abrangente para arquitetura de software começa explicando qual software arquitetura envolve, as responsabilidades da posição do arquiteto de software, e o que você irá saber. Os arquitetos de software devem ter habilidades técnicas e não técnicas, e eles devem ter amplitude e profundidade de conhecimento.

O livro avança para cobrir tópicos não técnicos, como a importância de compreender os negócios da sua organização, trabalhar no contexto de uma organização e coleta de requisitos para sistemas de software. Em seguida, analisa profundamente os tópicos técnicos como atributos de qualidade de software, design de arquitetura de software, desenvolvimento de software melhores práticas, padrões de arquitetura, como melhorar o desempenho e segurança considerações.

Depois de ler este livro, você deve estar familiarizado com os muitos tópicos relacionados a arquitetura de software e entender como ser arquiteto de software. Tecnologias e as práticas podem mudar com o tempo, mas o livro estabelece uma base sólida sobre a qual você pode construir uma carreira de sucesso como arquiteto de software.

O que este livro cobre

O Capítulo 1, O Significado da Arquitetura de Software, começa o livro fornecendo uma definição da arquitetura de software. O livro estabelece o que constitui uma arquitetura de software e as razões pelas quais é importante para um sistema de software. Também detalha o arquiteto de software função, incluindo as responsabilidades dos arquitetos de software e o que se espera deles saber.

Capítulo 2, Arquitetura de software em uma organização, enfoca a arquitetura de software no contexto de uma organização. Abrange os diferentes tipos de funções de arquiteto de software e metodologias de desenvolvimento de software que você possa encontrar. Tópicos não técnicos, como como gerenciamento de projeto, política de escritório e gerenciamento de risco são explicados. O desenvolvimento de linhas de produtos de software e a criação de ativos arquitetônicos centrais também são abordados.

Capítulo 3, Compreendendo o domínio, discute os aspectos comerciais de ser um software arquiteto. Abrange tópicos como familiarizar-se com os negócios da sua organização, design orientado a domínio (DDD) e como elicitar requisitos para o software de maneira eficaz sistema das partes interessadas.

Capítulo 4, Atributos de qualidade de software, cobre os atributos de qualidade de software e seus importância para uma arquitetura de software. Alguns atributos comuns de qualidade de software são apresentados, incluindo manutenção, usabilidade, disponibilidade, portabilidade, interoperabilidade e testabilidade.

O Capítulo 5, Projetando arquiteturas de software, concentra-se no importante tópico de software projeto de arquitetura. Ele detalha o que está envolvido com o design de arquitetura e sua importância para um sistema de software. O capítulo discute diferentes abordagens para o projeto de arquitetura, os motivadores para isso e os princípios de design que podem ser aproveitados durante o processo. O capítulo apresenta o uso de várias abordagens sistemáticas para a arquitetura de software design, incluindo design orientado a atributos (ADD), técnica da Microsoft para arquitetura e design, o método de design centrado na arquitetura (ACDM) e a arquitetura método de desenvolvimento (ADM).

Capítulo 6, Padrões de Arquitetura de Software, discute um dos softwares mais úteis conceitos de projeto de arquitetura. Aprender os padrões de arquitetura que estão disponíveis para você e quando aplicá-los adequadamente é uma habilidade chave para arquitetos de software. O capítulo detalha um número de padrões de arquitetura de software, incluindo arquitetura em camadas, orientada a eventos arquitetura (EDA), Model-View-Controller (MVC), Model-View-Presenter (MVP), Model-View-ViewModel (MVVM), Segregação de Responsabilidade de Consulta de Comando (CQRS) e Arquitetura Orientada a Serviços (SOA).

Capítulo 7, Arquitetando Aplicativos Modernos, explica os padrões de arquitetura de software e paradigmas que são usados com aplicativos modernos implantados na nuvem. Depois de descrever uma arquitetura monolítica, o capítulo detalha a arquitetura de micros serviços, sem servidor arquitetura e aplicativos nativos da nuvem.

1. Introdução sobre arquitetura

Uma visão abrangente da arquitetura de software deve primeiro começar com sua definição. Esse capítulo fornece razões pelas quais a arquitetura de software desempenha um papel importante em um projeto de software e os benefícios de ter um bom projeto arquitetônico.

Também é importante compreender as partes interessadas e os membros da equipe que são afetados por a arquitetura de software de um sistema. O capítulo entrará em detalhes sobre o software função do arquiteto, o que os arquitetos de software devem saber e se a função é certo para você.

Neste capítulo, cobriremos os seguintes tópicos:

- O que é arquitetura de software?
- Por que a arquitetura de software é importante?
- Quem são os consumidores de arquiteturas de software?
- Qual é a função do arquiteto de software?
- O que é arquitetura de software?

1.1. O que exatamente é arquitetura de software?

Você provavelmente tem suas próprias ideias sobre o que é, com base em seus conhecimentos e experiências. Certamente, existem muitas definições lá. Se você fizer uma pesquisa online ou perguntar a vários amigos e colegas, você obterá respostas. A definição é um tanto subjetiva e influenciada pelos pontos de vista e percepções do indivíduo que está fornecendo a definição. No entanto, existem alguns conceitos que são essenciais para a arquitetura de software, e antes de nos aprofundarmos em tópicos mais profundos, estabelecer um entendimento comum sobre o que a arquitetura de software envolve é imperativo.

Usando a palavra arquitetura para software originado de semelhanças com a construção indústria. Quando o termo foi usado pela primeira vez, a metodologia de desenvolvimento de software Waterfall era comum e ditava que projetos grandes e iniciais precisavam ser concluídos antes de qualquer o código foi escrito. Semelhante à arquitetura de um edifício, que exige muito planejamento antes da construção, o mesmo acontecia com o software.

No design de software moderno, a relação entre a construção e o software indústrias não está mais tão perto. Metodologias de software agora se concentram no desenvolvimento de software aplicativos que são altamente adaptáveis e podem ser alterados facilmente ao longo do tempo, resultando em menos da necessidade de um planejamento inicial e rígido. No entanto, a arquitetura de software ainda consiste em decisões de design que podem ser difíceis de alterar posteriormente.

1.2. Definição padrão ISO/IEC/IEEE 42010

Existe uma definição padrão para arquitetura de software, que resultou de um esforço conjunto entre a Organização Internacional de Normalização (ISO) e o Instituto de Engenheiros

Elétricos e Eletrônicos (IEEE). Sistemas e software ISO/IEC/IEEE 42010 descrição da arquitetura da engenharia é um padrão internacional que define software arquitetura como:

"Conceitos ou propriedades fundamentais de um sistema em seu ambiente incorporados em seus elementos, relacionamentos e nos princípios de seu design e evolução. "

O padrão apresenta os seguintes pontos principais:

- Uma arquitetura de software é uma parte fundamental de um sistema de software
- Um sistema de software está situado em um ambiente e sua arquitetura de software leva em consideração o ambiente em que deve operar
- Uma descrição da arquitetura documenta a arquitetura e se comunica com interessados como a arquitetura atende às necessidades do sistema
- Vistas de arquitetura são criadas a partir da descrição da arquitetura, e cada vista cobre uma ou mais questões de arquitetura das partes interessadas

1.3. O que constitui uma arquitetura de software?

No livro, *Software Architecture in Practice*, 2ª edição, uma definição de arquitetura de software é dada como:

"A arquitetura de software de um programa ou sistema de computação é a estrutura ou estruturas do sistema, que compreendem elementos de software, as externamente visíveis propriedades desses elementos e as relações entre eles. "

Um sistema de software contém estruturas, e esta definição observa que um sistema de software é composto por um ou mais deles. É a combinação destes que forma o conjunto arquitetura de software. Um grande projeto de software pode ter várias equipes trabalhando nele, cada um responsável por uma estrutura particular.

1.4. Arquitetura de software é uma abstração

A arquitetura de software é uma abstração de um sistema de software. As estruturas de um software sistema consiste em seus elementos. A arquitetura de software se preocupa em definir e detalhando as estruturas, seus elementos e as relações desses elementos com cada de outros.

A arquitetura de software enfoca os aspectos públicos dos elementos e como eles interagem um com o outro. Para os elementos, isso pode assumir a forma de suas interfaces públicas. Isso não lidar com os detalhes de implementação privada dos elementos. Embora o comportamento dos elementos não precisa ser exaustivamente documentado, deve-se tomar cuidado entender como os elementos devem ser projetados e escritos para que possam interagir um com o outro.

1.5. A arquitetura de software trata das coisas importantes

O cientista da computação Ralph Johnson, coautor de *Design Patterns: Elements of Reusable Software Object Oriented*, disse certa vez:

"Arquitetura é sobre as coisas importantes. Seja o que for."

Os projetos de software variam, e a quantidade de esforço de design, tempo, foco e documentação dedicado a aspectos específicos de uma arquitetura de software diferem. Em última análise, software a arquitetura consiste em importantes decisões de design que moldam o sistema. Isto é constituído por as estruturas e componentes que são significativos para a qualidade, longevidade e utilidade do sistema.

A arquitetura de software consiste em algumas das primeiras decisões feitas por um software sistema e alguns dos mais difíceis de mudar. No desenvolvimento de software moderno, o a arquitetura deve antecipar as mudanças e ser projetada de forma a maximizar o potencial de adaptação e evolução para esta mudança.

1.6. Por que a arquitetura de software é importante?

Por que devemos nos preocupar com a arquitetura de software? Às vezes, um desenvolvedor apenas quer começar a programar imediatamente. A arquitetura de software é a base de um sistema de software. Como outros tipos de engenharia, a fundação tem um efeito profundo na qualidade do que é construído em cima dela.

Como tal, é de grande importância em termos de desenvolvimento bem-sucedido, e eventual manutenção, do sistema. A arquitetura de software é uma série de decisões. Algumas das primeiras decisões vêm de projetar a arquitetura, e estes têm muita importância porque afetam as decisões que vêm depois disso.

Outra razão pela qual a arquitetura de software é importante é porque todos os sistemas de software têm uma arquitetura. Mesmo que compreendesse apenas uma estrutura com um elemento, há uma arquitetura. Existem sistemas de software que não possuem um design formal e outros que não documentam formalmente a arquitetura, mas mesmo esses sistemas ainda têm uma arquitetura.

Quanto maior o tamanho e a complexidade de um sistema de software, mais você precisará de um poço arquitetura pensada para ter sucesso. A arquitetura de software fornece uma série de benefícios quando feito corretamente, o que aumenta muito as chances de que o sistema de software terá sucesso. Uma base adequada estabelecida pela arquitetura de um sistema de software produz uma série de benefícios. Vamos examinar mais detalhadamente esses benefícios.

1.7. Definindo uma solução para atender aos requisitos

Software se esforça para atender a todos os aspectos funcionais, não funcionais, técnicos e operacionais requisitos. Trabalhar em estreita colaboração com as partes interessadas, como especialistas de domínio, negócios analistas, proprietários de produtos e usuários finais, permite que os requisitos sejam identificados e entendido. Uma arquitetura de software define uma solução que atenderá a esses requisitos.

A arquitetura de software é a base para o software, portanto, os sistemas de software que carecem de um sólido arquitetura torna mais difícil atender a todos os requisitos. Arquiteturas ruins irão levar a implementações que não atendem às metas mensuráveis de atributos de qualidade, e eles são normalmente difíceis de manter, implantar e gerenciar.

1.8. Habilitando e inibindo atributos de qualidade

A arquitetura de software habilita atributos de qualidade ou os inibe. Atributos de qualidade são propriedades mensuráveis e testáveis de um sistema. Alguns exemplos de atributos de qualidade incluem manutenção, interoperabilidade, segurança e desempenho.

Eles são requisitos não funcionais de um sistema de software em oposição aos seus recursos, que são requisitos funcionais. Atributos de qualidade e como eles satisfazem as partes interessadas do sistema são essenciais, e a arquitetura de software desempenha um grande papel em garantir que a qualidade atributos são satisfeitos. O design de uma arquitetura de software pode ser feito para se concentrar em certos atributos de qualidade em detrimento de outros. Os atributos de qualidade podem estar em conflito com uns aos outros. Uma arquitetura de software, quando projetada de forma adequada, visa atingir o acordo e requisitos validados relacionados aos atributos de qualidade.

1.9. Dando a você a capacidade de prever o sistema de software

Quando você olha para uma arquitetura de software e sua documentação, você pode prever as qualidades do sistema de software. Tomada de decisões de arquitetura com base em atributos de qualidade torna mais fácil cumprir esses requisitos. Você quer começar a pensar em qualidade atributos o mais cedo possível no processo de desenvolvimento de software, pois é muito mais difícil (e caro) fazer alterações para atendê-los mais tarde. Ao pensar sobre eles desde o início, e usando técnicas de modelagem e análise, podemos garantir que a arquitetura de software pode atender aos seus requisitos não funcionais.

Se você não for capaz de prever se um sistema de software atenderá aos atributos de qualidade até que seja implementado e testado, então um retrabalho caro e demorado pode ser necessário. UMA arquitetura de software permite prever as qualidades de um sistema de software e evitar custos retrabalho.

1.10. Facilitando a comunicação entre as partes interessadas

A arquitetura do software e sua documentação permitem que você comunique o software arquitetura e explicá-la a outros. Pode servir de base para discussões relacionadas a aspectos do projeto, como custos e duração. Discutiremos mais esse tópico quando formos em detalhes sobre a arquitetura de software em uma organização.

Uma arquitetura de software é abstrata o suficiente para que muitas partes interessadas, com pouco ou nenhuma orientação, deve ser capaz de raciocinar sobre o sistema de software. Embora diferentes as partes interessadas terão diferentes preocupações e prioridades em termos do que desejam saber sobre a arquitetura, fornecendo uma linguagem comum e artefatos de design de arquitetura permite que eles entendam o sistema de software. É particularmente útil para grandes e complexas sistemas que de outra forma seriam muito difíceis de entender completamente. Como requisitos e outras decisões iniciais são feitas para o sistema de software, uma arquitetura de software formal desempenha um papel importante e facilita negociações e discussões.

1.11. Gerenciando a mudança

Mudanças em um sistema de software são inevitáveis. O catalisador para a mudança pode vir do mercado, novos requisitos, mudanças nos processos de negócios, avanços tecnológicos e bug correções, entre outras coisas.

Alguns veem a arquitetura de software como inibidora da agilidade e preferem apenas deixá-la emergir sem design inicial. No entanto, uma boa arquitetura de software ajuda com ambos implementação e gerenciamento de mudanças. As alterações se enquadram em uma das seguintes categorias:

- Limitado a um único elemento
- Envolvem uma combinação de elementos, mas não requerem nenhuma mudança arquitetônica
- Exigir uma mudança arquitetônica

A arquitetura de software permite que você gerencie e entenda o que seria necessário para fazer uma mudança particular. Além disso, uma boa arquitetura reduz a complexidade para que a maioria das mudanças que precisam ser feitas podem ser limitadas a um único elemento ou apenas alguns elementos, sem ter que fazer alterações arquitetônicas.

1.12. Fornecendo um modelo reutilizável

Uma arquitetura estabelecida pode ser usada novamente dentro de uma organização para outros produtos em uma linha de produtos, especialmente se os produtos tiverem requisitos semelhantes. Vamos discutir linhas de produtos da organização, reutilização da arquitetura e os benefícios do próximo capítulo. Para agora, simplesmente reconheça que, uma vez que uma arquitetura de software é concluída, documentada, entendida e usada em uma implementação bem-sucedida, ela pode ser reutilizada.

Quando o código é reutilizado, recursos, como tempo e dinheiro, são economizados. Mais importante, a qualidade do software que aproveita a reutilização é aumentada porque o código já foi testado e comprovado. O aumento da qualidade por si só se traduz em economia de recursos.

Quando uma arquitetura de software é reutilizada, não é apenas o código que é reutilizado. Todos os primeiros as decisões que moldaram a arquitetura original também são aproveitadas. O pensamento e o esforço que foi para os requisitos necessários para a arquitetura, particularmente não funcional requisitos, podem ser aplicáveis a outros produtos. O esforço que foi feito para fazer aqueles as decisões não precisam necessariamente ser repetidas. A experiência adquirida com o original o projeto arquitetônico pode ser aproveitado para outros sistemas de software.

Quando uma arquitetura de software é reutilizada, é a própria arquitetura, e não apenas o software produto, que se torna um ativo para a organização. Impondo restrições de implementação. Uma arquitetura de software apresenta restrições na implementação e restringe o design escolhas. Isso reduz a complexidade de um sistema de software e impede que os desenvolvedores tomarem decisões incorretas.

Se a implementação de um elemento está em conformidade com a arquitetura projetada, então é obedecer às decisões de design feitas pela arquitetura. Arquitetura de software, quando feito adequadamente, permite que os desenvolvedores cumpram seus objetivos e os impede de implementar as coisas incorretamente.

1.13. Melhorar as estimativas de custo e esforço

Os gerentes de projeto fazem perguntas como: Quando isso será feito? Quanto tempo está indo pegar? Quanto vai custar? Eles precisam desse tipo de informação para planejar adequadamente recursos e monitorar o progresso. Uma das muitas funções de um arquiteto de software é ajudar gerenciamento de projetos, fornecendo este tipo de informação e auxiliando na determinação as tarefas e estimativas necessárias para essas tarefas.

O design da arquitetura de software em si afeta quais tipos de tarefas serão necessárias para implementação. Como resultado, a divisão do trabalho das tarefas depende do software arquitetura e o arquiteto de software podem auxiliar o gerenciamento de projetos com a criação de as tarefas.

Dois abordagens principais para a estimativa de gerenciamento de projetos são as seguintes:

- Abordagem de cima para baixo: começa com os resultados e metas e intervalos transformá-los em pacotes menores de trabalho
- Abordagem ascendente: começa com tarefas específicas primeiro e as agrupa juntos em pacotes de trabalho

Para alguns projetos, um gerente de projeto pode adotar uma abordagem mais de cima para baixo, enquanto desenvolvedores que trabalharão em tarefas específicas podem fazer uma análise de baixo para cima perspectiva. Com a experiência e o conhecimento que a maioria dos arquitetos de software possui, eles podem ajudar potencialmente com qualquer abordagem. Uma combinação dessas abordagens, onde as tarefas são observadas de ambos os pontos de vista, podem levar às melhores estimativas.

Pode ser útil quando os gerentes de projeto, o arquiteto de software e os desenvolvedores trabalham juntos para fornecer estimativas. As estimativas mais precisas podem ser obtidas por mútuas discussões entre os membros da equipe até que um consenso seja alcançado. Às vezes durante a construção de consenso, alguém da equipe fornecerá uma visão que outros não tinham considerado anteriormente, permitindo que todos repensem sua posição e, possivelmente, revisem suas estimativas.

1.14. Um sistema de software com requisitos precisos que são refletidos no software

Arquitetura pode evitar retrabalho caro que seria necessário se os principais requisitos fossem esquecidos. Além disso, uma arquitetura bem pensada reduz a complexidade, permitindo que seja facilmente raciocinado e compreendido. A complexidade reduzida pode resultar em custos mais precisos e estimativas de esforço.

1.15. Serve como treinamento para membros da equipe

A arquitetura do sistema e sua documentação servem como treinamento para os desenvolvedores na equipe. Aprendendo as várias estruturas e elementos do sistema, e como eles são supostamente interagir, eles aprendem a maneira adequada como a funcionalidade deve ser implementado.

Uma equipe de desenvolvimento de software pode passar por mudanças, como ter novos membros da equipe aderir ou deixar os existentes. A introdução e orientação de novos membros para uma equipe muitas vezes leva tempo. Uma arquitetura bem pensada pode tornar mais fácil para os desenvolvedores transição para a equipe.

A fase de manutenção de um sistema de software pode ser uma das fases mais longas e caras de um projeto de software. Como os novos membros da equipe introduzidos durante o desenvolvimento, é comum para diferentes desenvolvedores trabalharem no sistema ao longo do tempo, incluindo aqueles introduzido para mantê-lo. Ter uma arquitetura sólida disponível para ensinar e trazer a bordo novos desenvolvedores podem fornecer uma vantagem importante.

1.16. A arquitetura de software não é uma bala de prata

The Mythical Man-Month de Frederick P. Brooks é um dos textos seminais em software Gerenciamento de Projetos. Ele contém vários ensaios sobre engenharia de software. Embora isso livro foi escrito há algum tempo, e algumas das referências agora estão desatualizadas, ele fornece conselhos instigantes sobre o desenvolvimento de software que são atemporais e ainda aplicáveis hoje:

“Não existe um desenvolvimento único, seja em tecnologia ou técnica de gestão, que pôr em si promete até mesmo uma melhoria de ordem de magnitude dentro de uma década na produtividade, em confiabilidade, em simplicidade. ”

Ensaio de Fred Brooks 1986, No Silver Bullet - Essence and Accident in Software Engineering, que está incluído na edição do vigésimo aniversário do livro, começa com esta citação. Isto essencialmente transmite a ideia de que não existe uma solução mágica no desenvolvimento de software.

A arquitetura de software também não é uma bala de prata. Embora tenhamos coberto uma série de razões pelas quais a arquitetura de software é importante, não há uma arquitetura específica ou combinação de componentes que servirá como uma bala de prata. Não pode ser pensado como uma solução mágica que resolverá todos os problemas. Como aprenderemos com mais detalhes posteriormente, o software arquiteturas são sobre compromissos entre diferentes e às vezes conflitantes requisitos. Cada abordagem arquitetônica tem prós e contras que devem ser avaliados e avaliado. Nenhuma abordagem deve ser vista como uma bala de prata.

1.17. Quem são os consumidores de software arquiteturas?

Quando criamos uma arquitetura de software, para quem ela se destina? Há uma variedade de partes interessadas em um sistema de software, como os usuários finais do sistema, analistas de negócios, especialistas de domínio, pessoal de garantia de qualidade, gerentes, aqueles que podem se integrar com o sistema, e membros da equipe de operações. Cada uma dessas partes interessadas é afetada pelo software arquitetura em algum grau.

Embora certas partes interessadas tenham acesso e sejam interessados em examinar a arquitetura do software e sua documentação, outros não.

Algumas dessas partes interessadas são consumidores indiretos da arquitetura, na medida em que se importam sobre o software, e porque a arquitetura do software é a base do sistema, eles se tornam consumidores indiretos da arquitetura. Como arquiteto de software, você está servindo esses tipos de consumidores além dos consumidores diretos. Por exemplo, os usuários finais são talvez uma das partes interessadas mais importantes e deva ser o foco principal. O software a arquitetura deve permitir a implementação para satisfazer os requisitos dos usuários finais.

Quando discutimos os consumidores de uma arquitetura de software, não podemos omitir os desenvolvedores que trabalham nesse software. Como arquiteto de software, você precisa pensar sobre os seus desenvolvedores, cujo trabalho é diretamente afetado pela arquitetura do software. Eles são os únicos que trabalhará no software diariamente.

1.18. Qual é a função do arquiteto de software?

Agora que sabemos o que é arquitetura de software, a importância e os benefícios dela, e entender que há uma variedade de partes interessadas que são afetadas por isso, vamos examine a função do arquiteto de software. O que torna alguém um arquiteto de software? O que significa ser um arquiteto de software?

Certamente, os sistemas de software podem ser desenvolvidos sem um arquiteto de software. Você pode ter trabalhado em um projeto no qual ninguém desempenhava a função de arquiteto de software. Em alguns desses casos, o projeto pode ter sido bem-sucedido apesar disso, ou pode ter falhado por causa disso.

Quando ninguém recebe especificamente o título de arquiteto de software, alguém da equipe pode encerrar tomando decisões arquitetônicas. Tal indivíduo às vezes é chamado de acidental arquiteto. Eles não receberam o título de arquitetos de software, mas estão desempenhando alguns dos mesmos deveres e tomar os mesmos tipos de decisão. Ocasionalmente, quando há não é arquiteto de software, o projeto arquitetônico resulta de uma colaboração entre vários desenvolvedores.

Quanto menor e menos complexo for o sistema de software, mais você poderá ter sucesso sem um arquiteto de software. No entanto, se um projeto é grande em tamanho e/ou complexidade, você é mais propenso a precisar de alguém para desempenhar o papel formal de arquiteto de software.

1.19. Arquitetos de software são líderes técnicos

Os arquitetos de software são líderes técnicos de um projeto de software e devem estar comprometidos com o projeto, não importa quais desafios surjam. Eles fornecem orientação técnica para gerenciamento, clientes e desenvolvedores. Como tal, são frequentemente uma ligação entre recursos técnicos e não técnicos.

Embora os arquitetos de software tenham muitas responsabilidades, a principal delas é ser responsável pelos aspectos técnicos dos sistemas de software. Enquanto o arquiteto de software colabora com os outros, como líder técnico, o arquiteto de software é, em última

análise, responsável pela arquitetura de software, seu design e a documentação da arquitetura para um sistema de software.

1.20. Os arquitetos de software desempenham uma série de funções

Os arquitetos de software são obrigados a realizar diferentes tipos de funções, nem todos os quais são técnico. Os arquitetos de software combinam sua experiência, conhecimento e habilidades, ambos técnicos e não técnicos, para cumprir tais funções. Espera-se que os arquitetos de software ter um firme domínio de projetar arquiteturas de software, padrões de arquitetura e melhores práticas.

Os arquitetos de software devem ter a capacidade de prever possíveis problemas e design arquiteturas para superá-los. Eles devem ser capazes de mitigar riscos e avaliar soluções de forma que eles possam selecionar a mais adequada para resolver um problema específico. Enquanto algumas das habilidades e funções de um arquiteto de software são semelhantes às de um desenvolvedor sênior pode fazer, é uma função muito diferente. Os arquitetos de software suportam uma maior quantidade de responsabilidade, e há uma expectativa maior do que um arquiteto de software traz para um projeto.

Os desenvolvedores seniores têm um conhecimento profundo sobre as tecnologias que usam em um projeto. Eles são altamente proficientes em linguagens, ferramentas, estruturas e bancos de dados que são usados em seus sistemas de software. Embora se espere que os arquitetos de software tenham isso profundidade de conhecimento, eles também devem possuir uma ampla amplitude de conhecimento. Elas precisam estar familiarizado com tecnologias que não estão sendo usadas atualmente na organização para que possam tomar decisões informadas sobre o design da arquitetura.

Idealmente, os arquitetos de software têm a amplitude de conhecimento para estar cientes de várias soluções para um problema e compreender os trade-offs entre eles. Pode ser tão importante para um arquiteto de software entender por que uma solução específica não funcionará como é entender por que alguém o fará.

1.21. Arquitetos de software da torre de marfim

Se você se encontra no papel de um arquiteto de software, vai querer evitar ser um arquiteto de torre de marfim. Um arquiteto de software que está em uma torre de marfim refere-se a alguém que, seja pela forma como abordam sua posição ou por causa de como uma organização funciona, é isolado de outros.

Se um arquiteto de software está trabalhando em uma torre de marfim, ele pode estar criando uma arquitetura baseado em um ambiente de mundo perfeito que realmente não reflete cenários reais. Além disso, eles podem não estar trabalhando em estreita colaboração com os desenvolvedores que criarão as implementações com base na arquitetura.

Quanto mais um arquiteto de software trabalha por conta própria, isolado das partes interessadas e outros desenvolvedores, é mais provável que eles percam o contato com as necessidades daqueles indivíduos. Como resultado, eles podem estar projetando arquiteturas de software que não atendem às necessidades e requisitos variados de um grupo diversificado de partes interessadas.

Os arquitetos de software devem ter uma abordagem mais prática. Deveres de um arquiteto de software já deve incluir o envolvimento em uma série de fases do ciclo de vida de um software, mas ser prático ajuda a evitar perder o contato. Por exemplo, um arquiteto de software pode fazer parte da codificação com a equipe para ficar mais envolvido. Liderando pelo exemplo, como, como usar seu próprio código para servir de referência para outros, é uma maneira de praticar abordagem enquanto mantém suas habilidades afiadas.

Uma abordagem envolvida ajudará você a se manter informado sobre quais problemas e dificuldades os desenvolvedores pode estar enfrentando, e o que a arquitetura pode estar faltando. Liderar das trincheiras pode ser muito mais eficaz do que liderar de uma torre de marfim, e é mais provável que você ganhe o confiança e respeito de seus companheiros de equipe. Se um arquiteto de software estiver fora de contato ou mal-informado, mesmo que a percepção seja imprecisa, sua eficácia como líder será diminuída.

Um arquiteto de torre de marfim pode ser alguém visto como comandante de cima. UM arquiteto de software deve usar sua experiência e conhecimento para ensinar outras pessoas, e não pregar. Aproveite as oportunidades para tornar seus colegas de equipe melhores ensinando, mas também olhe ansiosos para aprender com os outros. Os colegas de equipe podem e irão fornecer informações valiosas e perspicazes feedback sobre seus projetos.

Uma organização não deve ter processos e/ou uma hierarquia organizacional em vigor que separar o arquiteto das partes interessadas. Eles não devem ser separados do técnico implementação porque isso afastará o arquiteto da tecnologia e habilidades que os tornaram um bom candidato para ser um arquiteto de software em primeiro lugar.

1.22. O que se espera que os arquitetos de software saibam?

Espera-se que os arquitetos de software tenham habilidades e conhecimento em uma variedade de tópicos. Esse livro concentra-se em muitos desses tópicos. Eles incluem funções não técnicas, como:

- Oferecendo liderança
- Auxiliar a gestão do projeto, incluindo estimativa de custo e esforço
- Membros da equipe de mentoria
- Ajudando a selecionar os membros da equipe
- Compreender o domínio do negócio
- Participar na coleta e análise de requisitos
- Comunicar-se com uma variedade de partes interessadas técnicas e não técnicas
- Ter uma visão para produtos futuros

Os tópicos técnicos com os quais os arquitetos de software devem estar familiarizados incluem:

- Compreender requisitos não funcionais e atributos de qualidade
- Ser capaz de projetar arquiteturas de software com eficácia
- Noções básicas sobre padrões e melhores práticas para desenvolvimento de software
- Ter um conhecimento profundo dos padrões de arquitetura de software, seus prós e contras, e saber quando escolher um sobre o outro
- Saber como lidar com questões transversais
- Garantir que os requisitos de desempenho e segurança sejam atendidos
- Ser capaz de documentar e revisar arquiteturas de software

- Ter uma compreensão do DevOps e do processo de implantação
- Saber como integrar e trabalhar com aplicativos legados
- Ser capaz de projetar arquiteturas de software que se adaptam às mudanças e evoluem ao longo do tempo

1.23. Não fique sobrecarregado

Se você se encontra na função de arquiteto de software pela primeira vez, ou se está ingressando em uma equipe que está trabalhando em um sistema de software existente há algum tempo, pode ser natural sentir-se oprimido por tudo o que você não conhece. Vai demorar para envolver sua cabeça em torno de tudo o que você eventualmente precisará saber.

Conforme sua experiência crescer, você se sentirá mais confortável ao iniciar um novo projeto. Como qualquer coisa, a experiência em diferentes situações o deixará mais confortável com enfrentando novos desafios. Você também entenderá que levará algum tempo para se tornar familiarizado com o domínio do negócio, pessoas, processos, tecnologia, detalhes e complexidades que vêm com cada sistema de software.

1.24. A função de arquiteto de software é certa para você?

Se você se preocupa com o software em que está trabalhando e todas as suas partes interessadas, incluindo usuários finais e desenvolvedores do software, então você se preocupa com o design importante decisões que entram na construção do software. Em última análise, isso significa que você se preocupa com sua arquitetura. Pode ser desafiador preocupar-se com as decisões mais importantes, mas pode ser agradável e gratificante por esse motivo.

Os arquitetos de software precisam se comunicar com uma variedade de partes interessadas e, às vezes, servir como uma ponte entre a gerência, a equipe técnica e a equipe não técnica. Se este não for algo com o qual você deseja se envolver, ser um arquiteto de software pode não ser a melhor opção para você.

Os arquitetos de software são apaixonados por tecnologia. Eles têm uma compreensão profunda de as tecnologias com as quais estão trabalhando e mantêm essas habilidades atualizadas, praticando seu ofício e estar envolvido com projetos. Eles devem ter uma grande amplitude de conhecimento e ter uma familiaridade com tecnologias que podem não estar usando atualmente em um projeto. Isto é necessário para acompanhar o ritmo acelerado de mudança em áreas como idiomas, ferramentas e frameworks. Estar ciente de uma variedade de tecnologias permitirá que você recomende a melhor solução para um problema específico.

Os arquitetos de software deveriam amar aprender e brincar com novas tecnologias, porque ser um arquiteto de software requer aprendizado contínuo. Como alguém com muita sabedoria para compartilhar, e quem será o líder de uma equipe, você deve gostar de orientar e ensinar os outros.

Fazer com que aqueles que trabalham ao seu redor sejam melhores no trabalho faz parte do seu trabalho. Todos os aplicativos de software têm um propósito. Bons arquitetos de software fazem todos os esforços para garantir que os aplicativos de software em que trabalham atendam ao

seu propósito da melhor maneira posso. Se você se preocupa com isso, a função de arquiteto de software pode ser a certa para você.

1.25. Resumo

Arquitetura de software é a estrutura ou estruturas de um sistema, seus elementos e o relações entre esses elementos. É uma abstração de um sistema de software. Programas arquitetura é importante porque todos os sistemas de software têm uma arquitetura, e que a arquitetura é a base do sistema de software.

A arquitetura de software oferece uma série de benefícios, como habilitar e inibir atributos de qualidade, permitindo prever as qualidades do sistema de software, facilitando a comunicação com as partes interessadas e permitindo que você faça alterações com mais facilidade. Ele também fornece um modelo reutilizável que pode ser usado em vários produtos de software, impõe implementação restrições que reduzem a complexidade e minimizam os erros do desenvolvedor, melhoram o custo/esforço estimativas e serve de treinamento para novos membros da equipe.

Os arquitetos de software são líderes técnicos que são responsáveis por questões técnicas decisões, a arquitetura e sua documentação. Eles desempenham uma série de funções e espera-se que tenham conhecimento de uma variedade de tópicos, tanto técnicos quanto não técnicos.

Embora a função possa ser desafiadora, se você se preocupa com o software que está trabalhando em e todos os seus stakeholders, então a função de arquiteto de software pode ser extremamente gratificante.

No próximo capítulo, exploraremos a arquitetura de software em uma organização. A maioria dos softwares arquitetos operam dentro do contexto de uma organização, por isso é importante entender o dinâmica de desenvolvimento de software dentro de um. O capítulo irá detalhar tópicos como as várias funções de arquiteto de software que você normalmente encontrará em uma organização, software metodologias de desenvolvimento que são utilizadas, trabalhando com projeto e configuração gerenciamento, navegando na política do escritório e criando linhas de produtos de software que alavancam reutilização arquitetônica.

2. Arquitetura de software em uma organização

No capítulo anterior, discutimos a arquitetura de software e a função do software arquiteto. Neste capítulo, exploraremos esses tópicos mais detalhadamente, mas no contexto de uma organização.

Os sistemas de software são desenvolvidos para satisfazer as metas de negócios de uma organização. Muitos arquitetos de software trabalham como parte de uma organização. Como resultado, o negócio da organização metas, objetivos, partes interessadas, gerenciamento de projetos e processos afetam muito o arquiteto de software e seu trabalho.

Este capítulo enfoca tópicos com os quais um arquiteto de software deve estar familiarizado ao trabalhar dentro de uma organização. Vamos dar uma olhada nos vários tipos de arquitetura de software funções que são comumente encontradas em organizações e o desenvolvimento de software metodologias que adotam. Você terá uma boa compreensão de como você pode ser esperado trabalhar com gerenciamento de projetos e a dinâmica da política de escritório.

Gerenciamento de risco e gerenciamento de configuração são dois outros aspectos do trabalho em projetos de software dentro de uma organização. Por fim, daremos uma olhada nas linhas de produtos de software, e como a reutilização arquitetônica pode criar ativos essenciais que podem tornar a construção de software produtos mais rápidos, eficientes e de maior qualidade.

Neste capítulo, cobriremos os seguintes tópicos:

- Tipos de arquitetos de software
- Metodologias de desenvolvimento de software
- Gerenciamento de Projetos
- A política do escritório
- Gestão de risco de software
- Gerenciamento de configurações
- Linhas de produtos de software

2.1. Tipos de arquitetos de software

A função de um arquiteto de software pode variar de organização para organização. Você pode ter também ouvido falar de uma variedade de cargos relacionados a arquitetos de software, como os seguintes:

- Arquiteto empresarial
- Solução de arquitetura
- Arquiteto de aplicativos
- Arquiteto de dados/arquiteto de informação
- Solução de arquitetura
- Arquiteto de segurança
- Arquiteto de nuvem

Algumas organizações têm um ou mais arquitetos que desempenham uma combinação dessas funções. Eles podem ser atendidos pelo título de arquiteto de software ou pelo título de uma dessas funções. Em outras organizações, diferentes indivíduos desempenham diferentes funções

arquitetônicas. Algumas empresas organizar seus arquitetos de software para que fiquem em uma equipe de arquitetura. Eles colaboram com a equipe em tarefas de arquitetura, mas também trabalhar em outras equipes para projetar e implementar produtos de software.

Este livro não se concentra em nenhum tipo de arquiteto de software. Trata principalmente de tópicos técnicos e, portanto, é voltado para uma série de funções de arquiteto técnico. Muitos das habilidades técnicas, não técnicas e sociais descritas neste livro são exigidas por mais de um tipo de arquiteto de software. Mesmo em organizações que têm diferentes tipos de arquitetos, há uma sobreposição em suas responsabilidades e deveres. Vamos dar uma olhada mais de perto nos diferentes tipos de funções de arquiteto de software e o que eles normalmente significam.

2.2. Arquiteto empresarial

Os arquitetos corporativos são responsáveis pelas soluções técnicas e direção estratégica de uma organização. Eles devem trabalhar com uma variedade de partes interessadas para compreender um mercado da organização, clientes, produtos, domínio de negócios, requisitos e tecnologia.

O arquiteto corporativo garante que os objetivos estratégicos e de negócios de uma organização estejam em sincronizar com as soluções técnicas. Eles precisam ter uma visão holística para garantir que seus projetos de arquitetura, e os projetos de outros arquitetos, estão em consonância com a organização.

Eles devem ter um conhecimento profundo e amplo da tecnologia para que possam fazer as recomendações e projetos de arquitetura adequados. Eles também devem olhar para o futuro para garantir que as soluções estejam em linha com as necessidades existentes, bem como uns.

Além de documentos de design de arquitetura de alto nível, os arquitetos corporativos trabalham com outros arquitetos, como arquitetos de aplicativos, para garantir que as soluções atendam a todos os requisitos definidos. Os arquitetos corporativos elaboram e mantêm as melhores práticas para coisas como designs, implementações e políticas. Para organizações que possuem vários produtos de software, eles irão analisá-los para identificar áreas de reutilização arquitetônica.

Os arquitetos corporativos fornecem orientação, mentoria, aconselhamento e liderança técnica para outros arquitetos e desenvolvedores.

2.3. Solução de arquitetura

Um arquiteto de soluções converte os requisitos em uma arquitetura para uma solução. Eles trabalham em estreita colaboração com analistas de negócios e proprietários de produtos para entender os requisitos para que eles podem projetar uma solução que satisfaça esses requisitos.

Os arquitetos de soluções selecionam as tecnologias mais adequadas para o problema que precisa ser resolvido. Eles podem trabalhar com arquitetos corporativos, ou se tal função não existir na organização, assumir as responsabilidades de um arquiteto corporativo, para considerar uns objetivos estratégicos gerais da organização e princípios de arquitetura empresarial ao projetar sua solução.

Os designs criados por arquitetos de soluções podem ser reutilizados em vários projetos. É comum em uma organização para reutilizar componentes arquitetônicos e reutilizar padrões em

arquiteturas em diferentes áreas de solução. Em grandes organizações que têm arquitetos atuando funções diferentes, os arquitetos de soluções preenchem uma lacuna entre os arquitetos corporativos e arquitetos de aplicativos.

2.4. Arquiteto de aplicativos

Os arquitetos de aplicativos se concentram em um ou mais aplicativos e em sua arquitetura. Elas garantir que os requisitos para a sua aplicação sejam satisfeitos pelo design desse aplicativo. Eles podem servir como um elo entre a equipe técnica e não técnica trabalhando em um aplicativo.

Na maioria das vezes, os arquitetos de aplicativos estão envolvidos em todas as etapas do software processo de desenvolvimento. Eles podem recomendar soluções ou tecnologias para um aplicativo, e avaliar abordagens alternativas para problemas. Os indivíduos nesta função precisam se manter atualizados com tendências e novas tecnologias. Eles sabem quando usá-los para resolver um problema ou aproveite uma oportunidade. Quando apropriado, eles estão envolvidos com como os aplicativos em uma organização funcionarão e se integrarão uns aos outros.

Os arquitetos de aplicativos garantem que a equipe de desenvolvimento esteja seguindo as melhores práticas e padrões durante a implementação. Eles fornecem orientação e liderança para a equipe membros. Eles podem estar envolvidos na revisão de projetos e código. Arquitetos de aplicativos trabalhar com arquitetos corporativos para garantir que as soluções projetadas para um indivíduo aplicação alinhada com a estratégia geral da organização.

2.5. Arquiteto de dados/arquiteto de informação

Os arquitetos de dados são responsáveis por projetar, implantar e gerenciar a arquitetura de dados. Eles se concentram em sistemas de gerenciamento de dados, e seu objetivo é garantir que os consumidores apropriados dos dados de uma organização têm acesso aos dados da maneira certa lugar na hora certa.

Os arquitetos de dados são responsáveis por todas as fontes de dados de uma organização, tanto internas quanto externo. Eles garantem que os requisitos de dados estratégicos de uma organização sejam atendidos. Elas criar projetos e modelos e decidir como os dados serão armazenados, consumidos e integrados nos vários sistemas de software da organização. Os arquitetos de dados também garantem a segurança de os dados da organização e definir processos para backup de dados, arquivamento de dados e banco de dados recuperação.

Eles mantêm o desempenho do banco de dados monitorando ambientes e podem ser encarregados de identificar e resolver vários problemas, incluindo problemas em ambientes de produção. Os arquitetos de dados podem apoiar os desenvolvedores auxiliando no design e codificação de seus bancos de dados trabalhar.

Algumas organizações têm a função de arquitetos de informações. Embora o arquiteto de dados e as funções de arquiteto da informação estão relacionadas e podem até ser desempenhadas pela mesma pessoa, há uma diferença entre as duas funções.

Enquanto os arquitetos de dados focam sua atenção em bancos de dados e estruturas de dados, as informações arquitetos colocam seu foco nos usuários. Eles estão preocupados com a intenção do usuário relacionada aos dados e como os dados afetam a experiência do usuário. Eles estão principalmente interessados em como os dados estão vai ser usado e o que vai ser feito com ele.

Os arquitetos de informação desejam fornecer uma experiência positiva ao usuário e garantir que os usuários possam interagir facilmente com os dados. Eles querem projetar soluções para que os usuários possam encontrar intuitivamente as informações de que precisam. Eles podem realizar testes de usabilidade para coletar feedback para que possam determinar quais mudanças, se houver, devem ser feitas a um sistema. Eles trabalham com designers de UX e outros para desenvolver estratégias que irão melhorar a experiência do usuário.

2.6. Arquiteto de infraestrutura

Os arquitetos de infraestrutura se concentram no design e implementação de uma organização infraestrutura empresarial. Este tipo de arquiteto é responsável pela infraestrutura ambiente atendendo às metas de negócios da organização e fornecer hardware, soluções de rede, sistema operacional e software para satisfazê-los.

A infraestrutura deve suportar os processos de negócios e aplicativos de software da organização. Esses arquitetos estão envolvidos com componentes de infraestrutura, como o Segue:

- Servidores: servidores físicos ou virtuais para ambientes em nuvem ou locais
- Elementos de rede: elementos como roteadores, switches, firewalls, cabeamento e balanceadores de carga
- Sistemas de armazenamento: sistemas de armazenamento de dados, como redes de área de armazenamento (SAN) e armazenamento conectado à rede (NAS)
- Instalações: A localização física do equipamento de infraestrutura, e garantindo as necessidades de energia, refrigeração e segurança sejam atendidas

Os arquitetos de infraestrutura oferecem suporte ao fornecimento de aplicativos de software corporativos. Esse inclui a concepção e implementação de soluções de infraestrutura e integração de novos sistemas de software com uma infraestrutura existente ou nova. Uma vez em produção, eles também garantir que os sistemas de software existentes continuem atendendo aos requisitos afetados por infraestrutura e funcionar em níveis ideais. Os arquitetos de infraestrutura podem fazer recomendações, como o uso de novas tecnologias ou hardware, o que irá melhorar uma infraestrutura da organização.

Para atender às demandas da empresa, eles monitoram e analisam características como carga de trabalho, rendimento, latência, capacidade e redundância para que um equilíbrio adequado seja os níveis de desempenho alcançados e desejados sejam atendidos. Eles usam gerenciamento de infraestrutura ferramentas e serviços para auxiliá-los na gestão da infraestrutura.

2.7. Arquiteto de segurança da informação

Um arquiteto de segurança é responsável pela segurança do computador e da rede de uma organização. Eles criam, supervisionam e mantêm as implementações de segurança de uma organização. Segurança os arquitetos devem ter um entendimento completo dos sistemas e infraestrutura de uma organização para que eles podem projetar sistemas seguros.

Os arquitetos de segurança realizam avaliações de segurança e testes de vulnerabilidade para identificar e avaliar ameaças potenciais. Os arquitetos de segurança devem estar familiarizados com os padrões de segurança, práticas recomendadas e técnicas que podem ser usadas para combater quaisquer ameaças identificadas. Elas reconhecer lacunas de segurança nas arquiteturas de software existentes e propostas e recomendar soluções para fechar essas lacunas.

Uma vez que os componentes de segurança são colocados no lugar, os arquitetos de segurança estão envolvidos no teste para garantir que funcionem conforme o esperado. Quando um incidente de segurança ocorre, a segurança os arquitetos estão envolvidos em sua resolução e conduzem uma análise pós-incidente. Os resultados da análise são usados para tomar as medidas adequadas para que um incidente semelhante não ocorra novamente.

Um arquiteto de segurança pode supervisionar o programa de conscientização de segurança de uma organização e ajudar a implementar as políticas e procedimentos de segurança corporativa de uma organização.

2.8. Arquiteto de nuvem

Agora que as implantações na nuvem são a norma, ter alguém em uma organização dedicado para a adoção da nuvem, com o conhecimento relevante, tornou-se cada vez mais comum e necessário. Um arquiteto de nuvem é alguém responsável pela nuvem de uma organização estratégia e iniciativas de computação. Eles são responsáveis pela arquitetura de nuvem usada para a implantação de sistemas de software. Uma organização que tem alguém que se concentra em a arquitetura da nuvem leva a maiores níveis de sucesso com a adoção da nuvem.

As responsabilidades dos arquitetos de nuvem incluem a seleção de um provedor de nuvem e a seleção do modelo (por exemplo, SaaS, PaaS ou IaaS) que é mais apropriado para a organização precisa. Eles criam planos de migração para a nuvem para aplicativos existentes que ainda não estão na nuvem, incluindo a coordenação do processo de adoção. Eles também podem estar envolvidos na concepção novos aplicativos nativos da nuvem que são desenvolvidos do zero para a nuvem.

Os arquitetos de nuvem supervisionam o gerenciamento da nuvem e criam políticas e procedimentos para governança. Eles usam ferramentas e serviços para monitorar e gerenciar implantações em nuvem. A experiência que os arquitetos de nuvem possuem normalmente significa que eles estão envolvidos na negociação contratos com provedores de serviços em nuvem e garantindo que os acordos de nível de serviço (SLAs) estão satisfeitos.

Os arquitetos de nuvem devem ter um firme entendimento das questões de segurança, como proteção dados implantados em diferentes tipos de nuvem e sistemas em nuvem/híbridos. Eles trabalham com arquitetos de segurança, ou se tal função não existir na organização, assumir as responsabilidades de um arquiteto de segurança, para garantir que os sistemas implantados na nuvem sejam seguros.

Para organizações que não migraram totalmente para a nuvem, uma das tarefas para uma nuvem arquiteto deve liderar uma mudança cultural dentro da organização para a adoção da nuvem. Nuvem as estratégias podem falhar se a cultura da organização não as abraçar totalmente. Parte de O trabalho do arquiteto da nuvem é evangelizar a adoção da nuvem, comunicando os muitos benefícios, e influenciar mudanças de comportamento em direção à adoção da nuvem que, em última instância, levarão a alterar.

2.9. Metodologias de desenvolvimento de software

Como arquiteto de software que trabalha em uma organização, você normalmente precisará usar a metodologia de desenvolvimento de software que vem sendo adotada pela organização. Contudo, em alguns casos, o arquiteto de software pode desempenhar um papel na decisão de qual software metodologia de desenvolvimento é usada. De qualquer forma, os arquitetos de software podem fornecer entrada para os processos de uma organização, dando-lhes a capacidade de fazer sugestões que podem melhorar esses processos.

Por essas razões, é bom ter uma compreensão dos softwares mais comuns metodologias de desenvolvimento. Há uma variedade de tipos diferentes que podem ser empregados para um projeto de software, cada um com seus próprios pontos fortes e fracos. Hoje, Agile metodologias são muito mais amplamente utilizadas do que as tradicionais, mas mesmo entre as metodologias consideradas Agile, existem inúmeras variações.

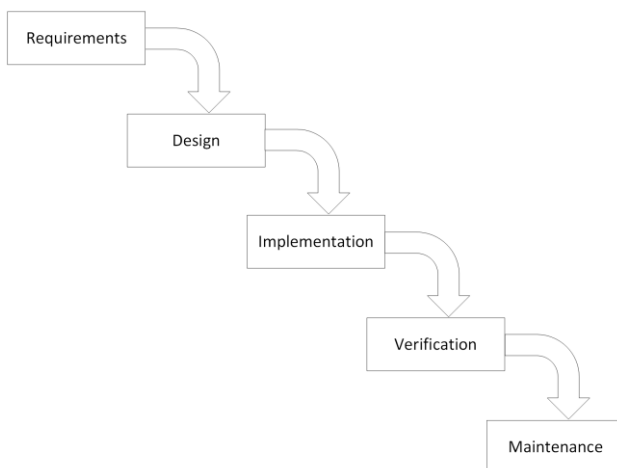
Infelizmente, às vezes, um projeto de software avança com um desenvolvimento de software metodologia que não é apropriada para o projeto. Antes de escolher um, você deve considere qual seria o mais apropriado para usar. Nas seções a seguir, vamos dê uma olhada em dois tipos de metodologias de desenvolvimento de software: o modelo em cascata e ágil.

2.10. O modelo em cascata

A metodologia de desenvolvimento do software Waterfall é sequencial, em que cada etapa do ciclo de vida deve ser concluída em sua totalidade antes de passar para o próximo estágio. Alguma das vantagens deste modelo incluem o seguinte:

- O modelo é simples e fácil de entender.
- As partes interessadas saberão o que esperar em termos de cronograma, funcionalidade e custo.
- Devido aos artefatos que devem ser produzidos fora de cada fase, diferentes tipos de documentação sobre o sistema estarão disponíveis. A documentação é benéfica para aqueles que farão a manutenção do software daqui para frente. Pode também facilita trazer novos funcionários a bordo e minimizar o impacto de qualquer rotatividade de funcionários.

No diagrama a seguir do modelo em Cachoeira, você pode ver como o fluxo das etapas lembra uma cachoeira:



2.11. Metodologia das Fases da Cascata

A seguir estão as etapas mais comuns no modelo em cascata:

- Requisitos: durante esta fase inicial, os requisitos são extraídos de partes interessadas e analisados. Os resultados são colocados em um documento de requisitos.
- Design: Na fase de design, as especificações técnicas do design são criadas. As especificações detalham como os requisitos serão atendidos pelo técnico solução.
- Implementação: esta fase representa o trabalho real de codificação. O design as especificações são implementadas no código.

- Verificação: durante esta fase, o teste é realizado para garantir que a implementação funciona corretamente e que os requisitos são cumpridos.
- Manutenção: Uma vez que o software foi implantado, o ciclo de vida entra nesta fase para correção de bugs e outras melhorias.

Também há uma variação no modelo em cascata, em que os nomes das etapas individuais diferem. Por exemplo, a codificação pode ser usada em vez da implementação, ou o teste pode ser usado em vez de verificação. Algumas organizações adicionam mais etapas ao processo. Pode haver uma etapa separada para análise que ocorre antes da fase de design, ou uma para implantação que ocorre após a fase de verificação.

O modelo em cascata saiu de moda, mas se o seguinte for verdadeiro (o que muitas vezes não é caso), o projeto pode ser candidato a este modelo:

- A equipe já tem muito conhecimento sobre o domínio de negócios do projeto, regras de negócios e funcionalidade
- Os requisitos são bem compreendidos e não devem ser alterados
- O escopo do projeto está definido e estável
- A equipe entende bem a tecnologia e a arquitetura, e essas são improváveis que mude
- O projeto não é grande em tamanho e não possui muita complexidade
- É aceitável que uma versão funcional do software não esteja disponível para partes interessadas internas ou externas até as fases posteriores
- O projeto é limitado por um período/prazo que já foi decidido

2.12. Problemas com a metodologia em cascata

Existem vários problemas com a abordagem em cascata. O teste não começa até depois toda a implementação está concluída, portanto, não há feedback de teste até o final do ciclo de vida. Os usuários não recebem uma versão funcional do software até o final, tão valioso o feedback não pode ser fornecido até esse momento. Os usuários podem ter dificuldade para visualizar o final produto apenas da documentação de requisitos e pode perder requisitos importantes.

Embora maquetes e wireframes possam minimizar esse problema, não é o mesmo que ter uma versão de trabalho do software por meio de uma abordagem iterativa. Se os requisitos forem perdidos ou incorretos, fazer alterações no final do processo de desenvolvimento pode ser um grande esforço, resultando em estouros de custo e/ou tempo. Se o tempo está se esgotando em um projeto de desenvolvimento de software, e o teste ocorre no final, o teste pode ser interrompido, diminuindo a qualidade do software.

A rigidez do modelo em cascata pode ser um grande problema se os requisitos estiverem sujeitos para mudar ou são mal compreendidos. Mesmo com projetos considerados previsíveis, a natureza do desenvolvimento de software é que a mudança é quase inevitável de uma forma ou outro. Assim, um processo muito rígido pode colocar o projeto em desvantagem.

Para projetos que não são adequados para um modelo tradicional em cascata, um ciclo de vida mais flexível pode ser necessário modelo. Os processos de desenvolvimento ágil podem fornecer esse tipo de flexibilidade. Devido às limitações dos modelos tradicionais e aos benefícios que as metodologias Agile fornecer, eles se tornaram mais populares no desenvolvimento de software moderno.

2.13. Metodologias ágeis de desenvolvimento de software

Metodologias ágeis de desenvolvimento de software foram criadas por profissionais de software líderes com base em suas experiências do mundo real no desenvolvimento de software. Eles abordam muitos das limitações das metodologias de desenvolvimento tradicionais e, como resultado, tornaram-se muito mais populares.

Existem vários tipos diferentes de processos Ágile disponíveis, como Scrum, Kanban, Extreme Programming (XP) e Crystal. Cada um deles tem suas diferenças, mas, em essência, todos eles se concentram em serem adaptáveis às mudanças. Eles tentam encontrar um equilíbrio entre não ter processos suficientes e muitos deles.

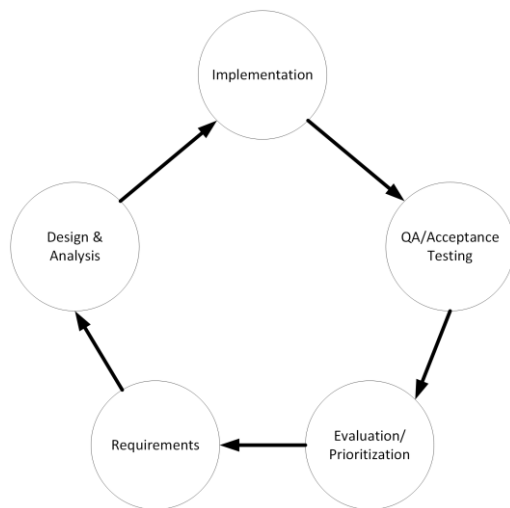
2.14. Valores e princípios ágeis

As metodologias ágeis têm certos valores e seguem certos princípios de software desenvolvimento. Estes foram documentados no Manifesto Ágil e os 12 princípios de Software Ágil, escrito pelos líderes de pensamento que criaram o software Ágil abordagem de desenvolvimento. A seguir estão os quatro valores fundamentais do Manifesto Ágil:

- Indivíduos e interações são mais valorizados do que processos e ferramentas. Valorizando as pessoas são mais importantes do que processos e ferramentas. As pessoas são aquelas que compreender o negócio e impulsionar o desenvolvimento do software. As equipes vão ser menos responsivo às necessidades de negócios se os processos e ferramentas forem mais importantes do que as pessoas.
- O software funcional é mais importante do que a documentação. Métodos tradicionais podem ter se concentrado em uma grande quantidade de documentação, mas o software que funciona e atender aos requisitos é mais importante. Só precisa haver a suficiente documentação necessária e não deve ter precedência sobre o trabalho Programas.
- A interação com o cliente e o envolvimento contínuo são valorizados em relação ao contrato negociação. Ágil enfatiza a importância da interação com o cliente e engajamento em todo o processo de desenvolvimento de software. Isso aumenta a probabilidade de que o software atenda às necessidades dos clientes.
- Responder às mudanças é mais importante do que seguir um plano. Tradicional as metodologias de desenvolvimento de software dão grande importância ao início planejamento e um desejo de evitar desvios desse plano. Ágil abraça a mudança, incluindo mudanças de requisitos em todo o processo de desenvolvimento. Baixo iterações fornecem uma oportunidade de adicionar e alterar requisitos a qualquer momento, como contanto que agregue valor e satisfaça os clientes.

2.15. Uma metodologia iterativa

Ao contrário da abordagem em cascata, que é sequencial, as metodologias Ágil são iterativas:



O software é construído de forma incremental, com cada iteração satisfazendo uma parte do total requisitos. No final de cada iteração está uma versão funcional do software, ao invés de tentando entregar todo o software de uma vez. O teste ocorre na mesma iteração que a codificação. Há um feedback contínuo, e se algo precisa ser mudado, torna-se aparente mais cedo.

2.16. Adaptável em vez de preditivo

As metodologias ágeis são adaptativas em vez de preditivas. Embora metodologias como Foco em cascata em uma grande quantidade de planejamento inicial para detalhar e prever resultados no futuro, as metodologias Agile se concentram na adaptação às mudanças conforme elas ocorrem.

Embora seja difícil saber qual será o estado do software no futuro, e cada vez mais quanto mais longe no futuro você deseja examinar, as metodologias Agile permitem que você faça alterações após o planejamento inicial. Feedback dos usuários e testadores podem ser levados em consideração imediatamente e incorporados ao trabalho de iterações subsequentes.

2.17. Reuniões diárias em pé

Se você vai adotar uma metodologia ágil de desenvolvimento de software, uma prática comum que você deve considerar implementar em sua equipe é a reunião diária em pé. Se você está seguindo a estrutura do Scrum, esta reunião é conhecida como scrum diário. É uma breve reunião, normalmente realizada no início do dia, na qual todos na equipe fornecem uma atualização de status.

Normalmente, um facilitador dirige a reunião e essa função deve ser alternada regularmente. Não deve haver dependência de um único facilitador. Isso também mantém a equipe conversando com entre si, que é o objetivo da reunião, e não apenas para um único facilitador.

Durante o stand-up, cada pessoa fornece seu status diário, respondendo a três perguntas:

- No que eu trabalhei ontem?
- No que vou trabalhar hoje?
- Há algum impedimento que estou enfrentando?

A reunião deve ser breve, para que se o status de alguém comece a se expandir para uma discussão, o facilitador deve pedir que tais discussões sejam colocadas offline, para ser discutido em outro momento.

Existem vários benefícios em realizar esta reunião:

- Todos ficam informados sobre as tarefas atuais que estão sendo trabalhadas por outros na equipe.
- Quando um membro da equipe declara um impedimento, torna o impedimento conhecido para o time. Oferece uma oportunidade para os membros da equipe se ajudarem.
- Alguém da equipe pode ter ideias de como resolver o problema. Além disso, outros membros da equipe podem ter o mesmo problema. Mesmo se não o fizerem, eles se tornarão cientes do problema e da possível solução, e podem ajudar a si próprios ou outra pessoa no futuro, caso enfrente o mesmo problema.
- Ele mantém os membros da equipe identificados como uma equipe. Em vez de trabalhar isoladamente, ter todos interagindo uns com os outros regularmente ajudará todos a identificar com a equipe.

2.18. Gerenciamento de Projetos

Os arquitetos de software geralmente auxiliam o gerenciamento de projetos ao longo do ciclo de vida de um sistema de software. Uma das maneiras pelas quais você será solicitado a fazer isso é estimando o nível de esforço para tarefas técnicas. Como as estimativas são derivadas e quem participa estimativa varia de organização para organização, mas normalmente, arquitetos de software, com seus conhecimentos e experiência, deverão fornecer algum nível de contribuição durante o processo de estimativa de tarefas.

2.18.1. A importância da estimativa de projetos de software

A importância da estimativa não pode ser subestimada. As estimativas e o planejamento do projeto são fatores importantes para o sucesso de um projeto.

A estimativa pode ser difícil e deve levar em consideração vários fatores. No entanto, o a equipe deve trabalhar em conjunto para chegar a estimativas precisas de gerenciamento de projetos depende deles. As estimativas são usadas para organizar o trabalho e planejar as liberações. Estimativas imprecisas e o planejamento deficiente do projeto estão entre os principais motivos pelos quais os projetos de software falham.

2.18.2. Colocar esforço nas estimativas

Às vezes, a estimativa é feita informalmente. Você pode ser solicitado a fornecer uma estimativa ou uma opinião in loco, sem a oportunidade de analisar o trabalho mais a fundo. Você quer estabelecer uma cultura dentro da organização que valorize o esforço nas estimativas, em vez de estimativas improvisadas que representam pouco mais do que suposições.

Se alguém lhe pedir para fornecer uma estimativa para uma tarefa de improviso, e você não se sentirem confortáveis fazendo isso, pergunte se é aceitável voltar para eles. Isso vai desacelerar o processo para baixo, e fornece a você a oportunidade de conduzir uma análise adequada antes de fornecer uma estimativa. Às vezes, se você estiver no local para fornecer uma estimativa, é fácil ser impreciso e depois se arrepender.

Ao estimar tarefas, algumas envolverão um trabalho que é compreendido, tornando mais fácil fornecer estimativas. É o trabalho que envolve incógnitas que pode ser desafiador. Se você é colocado nessa posição, veja se você pode reservar algum tempo para realizar alguma análise, ou, se apropriado, até mesmo criar uma prova de conceito (POC). Um POC permitirá que você

crie um protótipo solução para obter uma melhor compreensão do esforço envolvido, ou se a solução é até mesmo viável.

2.18.3. Ser realista (ou mesmo pessimista)

As pessoas que fornecem estimativas são otimistas, realistas ou pessimistas. Se você é geralmente um otimista, ou os outros indivíduos que estão trabalhando com você para fazer as estimativas são geralmente otimistas, considere fazer um esforço consciente para ser mais realista ou pessimista. Pense no que pode se tornar um desafio no trabalho e certifique-se de não estão deixando nada de fora ao considerar sua estimativa. Seu conhecimento e experiência como arquiteto de software, junto com a abordagem adequada, é o que é necessário para produzir uma estimativa útil.

2.18.4. Equipe e fatores situacionais a serem considerados

Se você estiver envolvido com o planejamento do projeto, considere também fatores situacionais, como a infraestrutura existente da organização (instalações, equipamentos), a cultura da organização, a maturidade dos processos da organização e as ferramentas disponibilizadas para o projeto a ser usado.

Certifique-se de levar em consideração o nível de habilidade e experiência de sua equipe. Esse inclui não apenas os desenvolvedores, mas todos na equipe, como os analistas de negócios e testadores de controle de qualidade. Pode haver um pouco de variação dependendo desse fator. Talvez você não saiba a quem será atribuída uma tarefa específica, mas considere os níveis gerais de habilidade da equipe, juntamente com seus pontos fortes e fracos, durante o planejamento do projeto.

Qualquer método que sua organização escolha adotar, use-o de forma consistente e, em seguida, analise os resultados. Dependendo de como funciona, você pode refinar seus métodos com base no tempo nos resultados que você coleta, o que deve produzir resultados ainda melhores no futuro.

2.18.5. Mudanças no cronograma do projeto

Seria ótimo se os projetos sempre estivessem dentro do cronograma, mas como todos sabemos, uma variedade de fatores pode fazer com que um projeto fique atrasado. Para esses tipos de situações, você vai ter que trabalhar com o gerenciamento do projeto para discutir o que poderia ser feito para trazer o projeto de volta ao cronograma, ou se os cronogramas do projeto podem ser ajustados.

Você também pode se encontrar em uma situação em que os prazos foram antecipados. Isso pode ser iniciada pela gestão interna, mas as forças externas também podem ditar uma mudança no cronograma do projeto. Uma nova oportunidade de negócio pode se apresentar, como a possibilidade de um novo cliente, ou sua organização pode ser convidada para um importante feira de negócios e deseja demonstrar o aplicativo de software. Quando surgem situações como essas, dependendo da importância e a direção dada pela gestão, a equipe pode precisar fazer cada tentativa de se conformar ao novo período.

Mesmo se você tiver adotado uma metodologia ágil de desenvolvimento de software, que antecipa mudar e ajudar a equipe a se adaptar a ela, você precisará cumprir determinados prazos do projeto.

2.18.6. Colocar um projeto de volta no cronograma

A fim de colocar um projeto de volta no cronograma, ou para se ajustar a um novo prazo, há um

número de abordagens. Vamos dar uma olhada neles.

2.18.6.1. Hora extra

Uma abordagem é fazer com que a equipe trabalhe mais horas, permitindo-lhes realizar mais em a mesma quantidade de tempo. No entanto, você corre o risco de diminuir o moral e a produtividade à medida que as horas aumentam.

Uma maneira de motivar sua equipe com essa abordagem é pagar a mais por horas extras, ou emitir tempo compensatório que pode ser usado posteriormente. Claro, isso requer gerenciamento aprovação. Se o pagamento extra estiver envolvido, o orçamento do projeto deve ser levado em consideração. O pagamento de horas extras, especialmente se houver empreiteiros no projeto, pode somar rapidamente.

2.18.6.2. Reduzindo o escopo

A equipe pode escolher revisitar o escopo do projeto. Se você puder identificar qualquer coisa supérfluo no cronograma do projeto, você pode notificar a gerência do projeto para tê-lo removido. Além disso, qualquer revestimento de ouro, que se refere à adição de recursos extras que não são necessários, deve ser eliminado, empurrado para um momento posterior ou reduzido em escopo.

Se nada óbvio puder ser encontrado, você pode trabalhar com as partes interessadas apropriadas, como especialistas de domínio, analistas de negócios, usuários finais e proprietários de produtos, para considerar a mudança requisitos. A funcionalidade de prioridade mais baixa pode ser descartada ou atrasada até que um lançamento futuro.

2.18.6.3. Adicionando recursos

Outra maneira comum de as organizações tentarem colocar um projeto de volta no cronograma é adicionar mais recursos. No entanto, esta abordagem não permite necessariamente que um projeto seja concluído mais rápido e pode fazer com que o projeto demore ainda mais. Lei de Brooks, de *The Mythical Man-Month*, afirma que adicionar recursos humanos a um projeto de software atrasado o torna mais tarde. Conforme você adiciona mais pessoas a um projeto, os caminhos de comunicação aumentam em um multiplicativo caminho. Além de levar mais tempo para se comunicar, muito mais oportunidades para falhas de comunicação são criadas. Produtividade e o número de defeitos (ambos em requisitos, bem como design e implementação) são conhecidos por aumentar à medida que os projetos começam maior.

Se a equipe for adicionar mais recursos, certifique-se de que ela está preparada para apoiá-los. Os novos membros da equipe exigirão orientação e mentoria nas fases iniciais, mas isso esforço é necessário para que eles possam se tornar o mais produtivos possível no menor tempo de tempo.

2.18.6.4. Realocando recursos

Às vezes, não se trata tanto de adicionar recursos, mas de garantir que os você recebeu as tarefas certas. Alguns recursos podem ser mais bem informados ou eficiente em certas tarefas. Um arquiteto de software que está trabalhando junto com a equipe pode ser capaz de fornecer informações valiosas para o gerenciamento de projetos para realocar recursos.

Se você vir recursos que não estão trabalhando em itens críticos, pode sugerir que eles sejam reatribuídos a tarefas mais importantes. Se um projeto está atrasado, todos precisam estar trabalhando no caminho crítico para concluir as tarefas mais importantes primeiro.

Se houver tarefas sendo trabalhadas em série, o que pode causar um atraso, pense sobre como as tarefas podem ser potencialmente reatribuídas para que possam ser trabalhadas em mais forma paralela.

2.18.6.5 Identificando áreas problemáticas

Tente entender por que o projeto está atrasado. O primeiro passo para encontrar uma solução é descobrir o problema. Se você vai fazer sugestões para o gerenciamento de projetos sobre como voltar ao cronograma, faça todos os esforços para identificar os principais problemas.

Considere obter ideias da equipe sobre quais melhorias podem ser feitas. Vocês não querem transformá-lo em um exercício de apontar o dedo, mas a equipe pode fornecer um bom insight. Pode haver algum processo que esteja causando um atraso, que pode ser ajustado ou eliminado.

Por exemplo, talvez haja uma reunião regular que não agrega muito valor, mas ocupa tempo valioso, e a reunião poderia ser eliminada ou feita para acontecer com menos frequência. Se houver atrasos causados por forças externas, examine se algo pode ser feito para aliviar esses tipos de problemas.

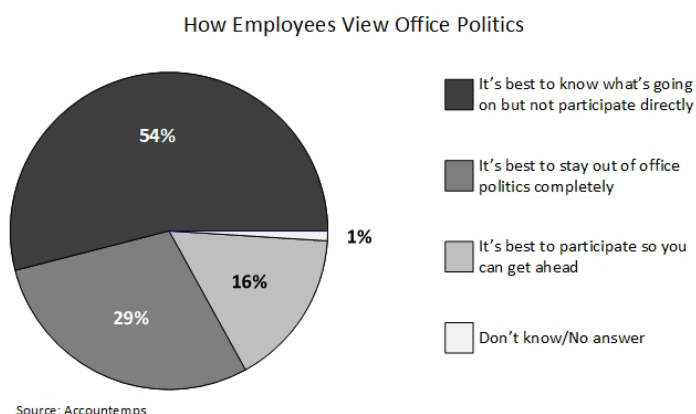
2.18.6.6. Agindo o mais cedo possível

Qualquer que seja a abordagem que você adote, uma coisa que você não quer fazer é encurtar o tempo de teste. Não realizar a quantidade adequada de testes pode ter sérias implicações para o sucesso do projeto. O software que não atende aos requisitos e/ou contém defeitos pode colocar todo o projeto em risco. O custo de testes inadequados pode ser muito maior do que qualquer economia de tempo ou custo que ela ofereça.

A melhor coisa que você pode fazer é agir o mais cedo possível. Comunique-se com o projeto gestão para que tenham uma compreensão detalhada do estado atual do projeto. Use sua experiência para reconhecer sinais de alerta e armadilhas em potencial para a equipe. Não adie a atuação em problemas potenciais, porque quanto mais cedo no projeto você percebe que há um problema, mais opções você tem para resolvê-lo.

2.18.7. A política do escritório

Quase todo mundo em uma organização tem que lidar com a política do escritório em algum grau. Como você podemos ver no gráfico de pizza a seguir, a maioria das pessoas acham que é melhor ter uma consciência da política do escritório em sua própria organização:



Como arquiteto de software, você pode se envolver na política do escritório mais do que você no passado, como quando você era um desenvolvedor sênior. Isso é porque você será interagido

e se comunicando com uma grande variedade de partes interessadas, e você pode estar fazendo tão mais frequentemente.

Em uma organização, indivíduos de diversas origens, com seus próprios objetivos e pontos de vista, reúnem-se para trabalhar em prol dos objetivos de uma organização. Uma variedade de fatores pode moldar as motivações das pessoas, o que pode levar a políticas de escritório.

Quando você deseja alcançar algo, pode precisar da ajuda de outras pessoas na organização. Diferentes ações que você realiza podem fazer com que você gaste, receba ou peça empréstimos políticos capital. Pode ser útil ter habilidades políticas ao trabalhar dentro de uma organização.

Prefiro me concentrar na construção de um ótimo software e não lidar com a política do escritório, mas às vezes é inevitável. A seguir estão algumas coisas que você pode fazer como um software arquiteto para ter sucesso na política de escritório.

2.18.8. Compreender os objetivos da sua organização

Esteja ciente dos objetivos estratégicos e da direção de sua organização. Tanto quanto possível, alinhe seus objetivos com os objetivos da organização. Haverá menos atrito com os outros e aqueles que estão em conflito com tais objetivos estarão em conflito com a organização.

Ter uma compreensão de como sua organização ganha dinheiro e como esse dinheiro é normalmente investido, pode ser útil. Se você está procurando recursos para o seu projeto, como membros adicionais da equipe, equipamentos e licenças para ferramentas, requer um investimento da organização. A organização vai querer um retorno sobre esse investimento, então seja preparado para explicar como se alinha com as metas e objetivos da organização.

2.18.9. Lidando com as preocupações dos outros

Em uma organização, muitas partes interessadas podem vir ao arquiteto de software com preocupações sobre um produto ou processo de software. Embora a prioridade e legitimidade dessas preocupações irá variar, procure resolver essas questões o mais rapidamente possível.

Se um problema não for resolvido, ele pode ser escalado para outras pessoas na organização. Pode parecer ser um problema maior do que realmente é. Quando alguém traz uma preocupação para você, você quer ter certeza de que essa pessoa sabe que você ouviu a preocupação dela, entendeu, e tomará as medidas adequadas para isso. Se nenhuma ação for realizada, explique os motivos por trás dessa decisão e certifique-se de que a pessoa sabe que pelo menos sua preocupação tem sido observada.

2.18.10. Ajudar as pessoas com seus objetivos

Quando você tiver a oportunidade, ajude outras pessoas em sua organização a atingir suas metas ou resolver seus problemas. Todos vocês estão trabalhando para a mesma organização. Contanto que sejam tentar realizar não entra em conflito com seus próprios valores ou os objetivos da empresa, ajude as pessoas quando puder.

Faça isso sem qualquer expectativa de receber nada em troca. Você não quer uma reputação como alguém que só faz favores esperando algum benefício pessoal. Se você ajudar seu companheiro funcionários, vocês serão vistos com gentileza. Em algum momento, você pode precisar de outra pessoa ajuda, e mesmo que você não esteja devendo nada, será mais provável que você receba ajuda.

2.18.10. Saber quando comprometer

Como arquiteto de software, você se verá em discussões com outras pessoas em uma variedade de tópicos. Você pode estar negociando algo, ou em conflito com alguém, ou pode estar fazer um pedido de algo. Seja qual for o motivo, quando se torna evidente que você pode não conseguir o que deseja, esteja disposto a se comprometer e saiba quando fazer isso.

Uma razão para se comprometer é quando você está simplesmente sem influência e se você não comprometer, você não pode cumprir nenhuma parte de seu pedido ou objetivo. Um compromisso pode fazer sentido nessa situação porque você ainda pode obter algo adequado com o intercâmbio. Um acordo também faz sentido se você acha que será benéfico a longo prazo.

2.18.11. Desistir de algo agora pode valer a pena em algum momento no futuro.

Esteja preparado para comprometer desde o início, pensando proativamente sobre o quão importante a questão é para você, bem como o que seria ou não um compromisso aceitável.

Ouçá a pessoa ou grupo com quem você está lidando em relação ao problema e tente compreender seus pontos de vista e raciocínio. Compreender o outro lado pode fornecer a você com pontos adicionais que você pode fazer, ou pelo menos, dar uma ideia do que pode ser um meio termo aceitável.

2.18.12. Estar ciente das diferenças culturais

Muitas organizações têm escritórios e clientes em todo o mundo e podem terceirizar trabalho para recursos em outros países. Lembre-se das diferenças culturais quando comunicar-se com pessoas de outros países. Diferentes culturas podem preferir frases ou abordagens diferentes. Reserve um tempo para aprender sobre as culturas das pessoas que você lidará com para maximizar a clareza de suas comunicações e evitar mal-entendidos.

2.18.13. Gestão de risco de software

Os riscos são problemas potenciais, e há muitos riscos envolvidos na concepção e desenvolvimento de software. As organizações e as pessoas que trabalham para elas têm níveis diferentes de tolerância ao risco. Qualquer que seja esse nível, uma organização deve ter um plano de gestão de risco.

Como arquiteto de softwares, você precisará auxiliar os gerentes de projeto no gerenciamento desses riscos. Se não forem gerenciados, os riscos podem levar a estouro de custo/tempo, retrabalho, falhas operacionais e possivelmente falha total do projeto.

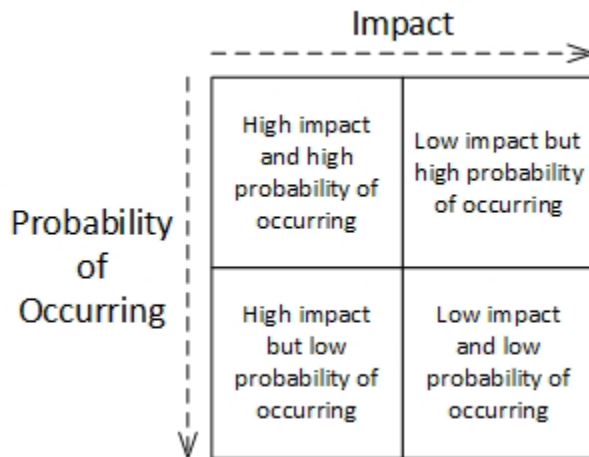
A primeira etapa para auxiliar no gerenciamento de riscos é ser capaz de identificar os riscos. O time deve apresentar e documentar os riscos potenciais. Usando seu conhecimento e experiência como arquiteto de software, você pode ajudar a identificar os riscos que as partes interessadas, projetam gestão e outros membros da equipe não.

Alguns exemplos de possíveis tipos de riscos incluem o seguinte:

- Riscos funcionais: Requisitos incorretos, falta de participação dos usuários finais e analistas de negócios, objetivos de negócios conflitantes
- Riscos técnicos: Grau de complexidade, tamanho do projeto, novo linguagens/ferramentas/estruturas com as quais a equipe não está familiarizada, dependências dos fornecedores e subcontratados fora da organização
- Riscos de pessoal: Membros da equipe sem a experiência adequada ou conjunto de habilidades, incapacidade de contratar o projeto, problemas de produtividade com os membros da equipe

- Riscos financeiros: falta de financiamento suficiente para o projeto, retorno do investimento (ROI) restrições que serão difíceis de cumprir
- Riscos legais: necessidade de obedecer às regulamentações governamentais, mudanças legais requisitos, mudanças contratuais
- Riscos de gestão: Falta de experiência e conjunto de habilidades adequados, incorreto planejamento, falta de comunicação, questões organizacionais

Uma vez que um risco é identificado, ele deve ser avaliado por seu impacto potencial e a probabilidade que isso vai ocorrer. Riscos que têm um alto impacto, bem como uma alta probabilidade de ocorrendo são os mais críticos para um projeto:



Depois que os riscos são avaliados, os arquitetos de software podem ser solicitados pela organização para ajudar com um plano sobre como gerenciar os riscos. Existem quatro técnicas principais que podem ser usadas para lidar com riscos: prevenção de risco, transferência do risco para outra parte, mitigação de risco e risco aceitação.

2.18.13.1. Prevenção de riscos

Uma técnica para lidar com o risco é a prevenção de risco, que envolve a mudança do projeto em alguma forma de eliminar o risco por completo. Um exemplo de um risco que poderia ser tratado com, desta forma, seria uma equipe de projeto que está pensando em usar um programa de linguagem e/ou tecnologia com a qual a equipe não está familiarizada. Avançando com tal a escolha traz consigo alguns riscos potencialmente significativos.

O projeto pode demorar mais do que o previsto devido à curva de aprendizado, ou pode ser descobriu que os recursos que a equipe deseja implementar não são adequados para o escolhido tecnologia. Uma maneira de lidar com esse risco seria selecionar uma tecnologia que a equipe esteja já familiarizada, desde que possa satisfazer as necessidades do projeto. Isso permitiria a equipe para efetivamente evitar o risco.

Lembre-se de que nem todos os riscos podem ser evitados, e evitar um risco pode levar a outros riscos. Além disso, assumir riscos às vezes é necessário para tirar proveito de uma oportunidade. Estas devem ser algumas das suas ideias ao considerar se deve ou não evitar um risco.

2.18.13.2. Transferindo o risco para outra parte

Outra técnica de gerenciamento de risco é transferir o risco para outra parte. Um exemplo disso pode ser uma equipe de projeto que contrata um subcontratado para implementar partes de um projeto, como o design e o desenvolvimento de uma interface de usuário para um aplicativo

da web. Existe um risco que o subcontratado pode não concluir suas entregas a tempo ou que suas entregas podem não atender a certos padrões de qualidade. Uma maneira de lidar com esse risco seria transferi-lo para o subcontratado, incorporando penalidades no contrato que serão incorridas se o as entregas estão atrasadas ou não atingem o nível de qualidade desejado.

2.18.13.3. Mitigação de risco

Pode-se também empregar mitigação de risco para lidar com um risco, o que está reduzindo a probabilidade de que um risco ocorrerá. Digamos que um projeto de software contratou recentemente um novo membro da equipe. Essa pessoa não é tão experiente quanto alguns dos outros membros da equipe, então há o risco de que o trabalho do novo membro da equipe pode ser atrasado devido a uma curva de aprendizado ou ao trabalho produzido pode não estar correto devido à falta de conhecimento sobre as normas técnicas ou o domínio do negócio. Uma forma de mitigar ou diminuir esse risco seria atribuir o novo funcionário um mentor. Este mentor pode dedicar algum tempo ao ensino, respondendo a perguntas, e revisar o trabalho do novo membro da equipe.

2.18.13.4. Aceitação de risco

A aceitação do risco é outra maneira de lidar com o risco. Essa abordagem é simplesmente aceitar o risco e quaisquer consequências possíveis. Por exemplo, talvez sua empresa e outra empresa estejam desenvolvendo um novo produto de software para atender a um novo mercado. Idealmente, seu produto seria o primeiro ao mercado, mas como você começou mais tarde, há o risco de não ser. Não ser o primeiro a chegar ao mercado pode resultar na perda de quotas de mercado. Depois de analisar o risco, você pode decidir aceitar quaisquer consequências de não ser o primeiro no mercado em troca para desenvolver um produto melhor, não apressando.

Um ponto a ter em mente é que as ações tomadas para resolver um risco, mitigando ou transferi-lo pode ter seu próprio conjunto de riscos. Isso deve ser levado em consideração como parte da análise desse risco. Se, no último exemplo, a empresa decidiu mitigar o risco ao tentar concluir o produto mais cedo, riscos adicionais podem ter sido criados, como uma maior probabilidade de que os requisitos possam ser perdidos ou que a qualidade possa sofrer.

2.18.14. Gerenciamento de configurações

Como arquiteto de software em uma organização, espera-se que você esteja envolvido com gerenciamento de configurações. Em muitas organizações, existe uma configuração de software equipe de gerenciamento (SCM). Além disso, pode haver uma equipe SCM no nível do projeto. Para pequenos projetos, pode ser uma única pessoa, ou pode até ser assumido por alguém que não é até mesmo dedicado à função, como um membro da equipe que desempenha outras funções.

Algumas das responsabilidades de uma equipe de gerenciamento de configuração de software incluem identificando itens de configuração (software, documentos, modelos, planos), implementando um processo de controle de mudanças e gerenciamento do processo e das ferramentas usadas para compilações. Nós vamos examinar compilações automatizadas e integração contínua (CI) no Capítulo 14, Arquitetura Aplicativos legados.

2.19. Mudança de gestão

As organizações irão implementar um processo formal de controle de mudanças para lidar com as mudanças em um produto de software. Isso inclui todos os aspectos de um sistema de software, como requisitos, código-fonte e documentação. Existem muitos motivos pelos quais

uma mudança pode ser proposta, incluindo a correção de um problema (um bug no software), a implementação de uma mudança funcional (uma mudança em uma regra de negócios) ou adição de novas funcionalidades ao software.

O objetivo do processo é garantir que, se forem feitas alterações, seja apropriado fazê-lo e para minimizar o esforço, dificuldade e interrupção ao implementar a mudança. Alguns processos de controle de mudanças envolvem um conselho de controle de mudanças (CCB). Este é um grupo de partes interessadas do projeto que são designadas para analisar as mudanças propostas e decidir se eles devem ser implementados. Como arquiteto de software, você pode fazer parte de um escritório formal ou CCB informal. Seu conhecimento e experiência podem ser aproveitados pelo conselho para ajudar com:

- Avaliar uma mudança proposta para decidir se a mudança é algo que deve ser implementado no projeto
- Priorizando a mudança proposta com base em sua importância e gravidade (se for um defeito)
- Fornecer uma estimativa quanto ao esforço envolvido para implementar a mudança

Existem várias abordagens para o gerenciamento de mudanças que uma organização pode usar. UMA o arquiteto de software deve estar familiarizado com o processo de sua organização. Como um software arquiteto, você pode oferecer informações para alterar um processo existente ou pode precisar de ajudar formular o processo pela primeira vez.

Uma abordagem para a gestão da mudança, que é a mais formal, requer que um CCB reveja todas as mudanças, independentemente de quão grande ou complexa a mudança possa ser. Tem a vantagem de permitir que várias pessoas considerem a mudança. Ter mais pessoas conscientes da mudança proposta e dar a eles a oportunidade de discuti-la aumentará as chances de tomar a decisão certa. Especialmente com sistemas grandes e complexos, pode ser muito difícil ou mesmo impossível para uma única pessoa estar ciente de todas as técnicas e ramificações funcionais de uma mudança particular. Ter vários pares de olhos olhando para ele certamente pode ajudar. Infelizmente, esse processo vai demorar mais tempo. O CCB poderia facilmente se tornar um gargalo. Não só mais tempo será gasto tomando decisões, às vezes, o tempo passará apenas marcando as reuniões para reunir todos. Para situações em que muitas mudanças ou decisões precisam ser feitas em tempo hábil, tendo um CCB revisando todas as mudanças podem não ser práticas.

A segunda abordagem para o gerenciamento de mudanças é o oposto do primeiro método. Em vez do que fazer com que um CCB analise todas as alterações, nesta abordagem de controle de alterações, não há CCB.

Os desenvolvedores individuais tomam suas próprias decisões para todas as mudanças. A vantagem disso abordagem é que as mudanças podem passar pelo processo muito mais rapidamente do que qualquer um das outras duas abordagens. Além disso, capacitar os desenvolvedores pode dar-lhes um certo nível de confiança e satisfação. No entanto, apenas uma pessoa está pensando sobre cada mudança, e conforme mencionado anteriormente, isso pode ter um efeito prejudicial. A qualidade alcançada por um determinado a mudança dependerá unicamente do desenvolvedor que fez a mudança. Alguns desenvolvedores podem ser novos no software ou inexperientes em geral, ou podem simplesmente não ser muito bons.

Seja qual for o motivo, a mudança que eles decidem fazer pode acabar causando ainda mais problemas do que aquilo que está tentando resolver. Como foi mencionado anteriormente, mesmo com um desenvolvedor experiente e de qualidade, pode ser difícil com grandes sistemas

estar ciente de todas as ramificações de uma mudança particular. Para pequenos defeitos, isso não é um grande problema, mas pode ser para grandes mudanças.

A terceira abordagem para o gerenciamento de mudanças busca um equilíbrio entre a primeira e a segunda. Nessa abordagem, um CCB apenas analisa algumas das mudanças. Normalmente, isso significa revisar as mudanças mais importantes ou mais complexas. Embora as alterações não sejam concluídas tão rápido quanto a segunda abordagem, e as mudanças não estarão sob tanto escrutínio quanto a primeira abordagem, pelo menos as mudanças que poderiam causar a maioria dos problemas são revisadas, e o CCB não é um gargalo para as demais mudanças. Simplesmente pode não ser prático ter um CCB analisa tudo, mas você também não quer sacrificar todas as revisões. Por exemplo, o CCB pode revisar a maioria das melhorias e defeitos de alta prioridade/gravidade, mas não os defeitos de menor prioridade/gravidade (embora um analista, desenvolvedor e testador ainda tenha uma oportunidade de pensar sobre todas as correções enquanto estão trabalhando em um defeito). É realmente depende mais de quão grande e complexa é a mudança, e quão importante a funcionalidade é que será afetada, e não tanto se é um defeito ou um Aprimoramento. Às vezes, uma correção de defeito é mais complicada do que um aprimoramento, então aqueles fatores são sempre levados em consideração.

Talvez a melhor abordagem para a gestão da mudança, e a mais pragmática, seja sempre use um CCB, mas crie vários níveis dele. A organização pode designar diferentes níveis de autoridade, e as mudanças com impacto e escopo limitados podem ser aprovadas em níveis mais baixos de autoridade, enquanto as mudanças que têm um impacto maior são escaladas para níveis mais altos de autoridade dentro de um CCB. Quando há vários níveis de um CCB, o estágio do projeto (ou seja, o quão perto a equipe está de lançar) às vezes é considerado ao determinar o que CCB nível é apropriado. Uma mudança normalmente ocorre quando um projeto se aproxima do lançamento, com o foco indo de querer flexibilidade para querer mais estabilidade e controle sobre as mudanças.

2.19. Linhas de produtos de software

Crescentes pressões competitivas combinadas com a complexidade do desenvolvimento de software sistemas torna uma necessidade para as organizações aumentar a eficiência sempre que possível.

Uma maneira de fazer isso é criar linhas de produtos de software. Uma variedade de indústrias, incluindo a indústria de software, usaram a engenharia de linha de produtos (PLE) com sucesso.

As linhas de produtos são vários produtos de uma única empresa que atendem a uma necessidade específica ou mercado. Esses produtos poderiam ser vendidos sob a mesma marca, com a ideia de que os clientes que usam um produto da marca têm maior probabilidade de comprar outro produto da mesma marca.

As organizações que desenvolvem software podem ter vários produtos de software existentes e/ou outros produtos de software em desenvolvimento. Esses produtos de software podem funcionar de forma semelhante forma, têm requisitos funcionais e/ou não funcionais semelhantes, e têm um semelhante aparência/sensação.

Sem qualquer tipo de reutilização, a mesma funcionalidade pode ser gravada várias vezes. Quando trabalhando como arquiteto de software em uma organização, você deve procurar reutilizar a arquitetura construção de sistemas de software a partir de ativos essenciais. Uma arquitetura de software que fornece uma solução para um produto de software também pode

ser útil para outro. Componentes arquitetônicos podem ser reutilizados, permitindo uma solução genérica para resolver o mesmo problema em vários produtos.

Além de construir software, as organizações podem adquirir software ou outras organizações que possuem produtos de software existentes, e este software pode se tornar uma nova linha de software para a organização adquirente. O software adquirido também pode ser mesclado com uma linha de produtos existente.

2.19.1. Benefícios de uma linha de produtos de software

Alguns dos benefícios da reutilização arquitetônica planejada e estratégica em uma linha de produtos de software inclui o seguinte:

- Esforço de desenvolvimento reduzido
- Custos mais baixos
- Produtividade aumentada
- Maior qualidade
- Tempo reduzido para o mercado

2.19.2. Ativos essenciais de uma organização

O objetivo de uma linha de produtos de software é ter componentes reutilizáveis, conhecidos como ativos essenciais, que compartilham uma arquitetura comum. Alguns dos componentes que podem ser reutilizados e designados como ativos principais incluem o seguinte:

- Análise de requisitos
- Modelos de domínio e análise
- Projeto de arquitetura de software
- Planos de teste e casos de teste
- Planos de trabalho, cronogramas, orçamentos
- Processos, métodos, ferramentas
- O conhecimento, habilidades e experiência dos funcionários
- Guias do usuário e documentação técnica

Os ativos principais precisam ser construídos, tendo em mente que eles serão usados em vários produtos em uma linha de produtos. Cada produto terá variação, então, para dar conta disso, ativos essenciais devem ser construídos com pontos de variação, lugares que fornecem uma oportunidade de aproveitar levar em consideração esta variação e permitir que a equipe adapte o ativo para uso em uma produtos.

Uma vez que esses ativos tenham sido compilados, exigirá menos esforço, tempo e custo para reutilizar os ativos do que criá-los sempre que forem necessários. Construir um sistema de software então requer que a equipe reutilize os ativos apropriados, usando os pontos de variação conforme apropriado, e construir o sistema a partir daí.

2.19.2. Riscos da engenharia da linha de produtos

Embora a engenharia da linha de produtos para software possa resultar em benefícios, esteja ciente de que há são riscos relacionados à sua implementação bem-sucedida. Mudar para este tipo de abordagem requer adotando uma nova estratégia técnica para a organização como um todo. Requer coordenação e suporte gerencial.

Deve ficar claro para as equipes de desenvolvimento quais ativos essenciais estão disponíveis para eles e quais os pontos de variação são para que possam utilizá-los adequadamente. O escopo da linha de produtos precisa ser definido de forma adequada, de modo que não seja nem muito amplo nem muito estreito.

Se a organização está criando todos os produtos na mesma linha de produtos, ela exige execução bem-sucedida pelos arquitetos de software e equipes de desenvolvimento, bem como a gestão organizacional adequada. Se a organização está adquirindo produtos de software para ser incluídos em uma linha de produtos de software, então os recursos adequados, tanto técnicos quanto gerencial, deve estar disponível e preparado para realizar o esforço necessário para identificar e utilizar os componentes comuns desses produtos.

As organizações que não estão preparadas para adotar totalmente essa abordagem podem experimentar o fracasso.

2.20. Resumo

Ao trabalhar como arquiteto em uma organização, você deve estar ciente dos vários tipos de funções de arquiteto de software. As várias funções têm semelhanças e diferenças em termos de responsabilidades e deveres. Se você for obrigado a usar um desenvolvimento de software metodologia que já foi selecionada para sua organização, é importante familiarize-se com ele e suas práticas. Se você estiver em posição de selecionar o software metodologia de desenvolvimento, certifique-se de ter conhecimento sobre as escolhas para que um apropriado pode ser selecionado para as necessidades do projeto.

Os arquitetos de software usam seu conhecimento e experiência para trabalhar com gerenciamento de projetos em estimativas de tarefas, planejamento do projeto e controle e monitoramento do projeto durante sua vida útil.

Devido à comunicação com muitas partes interessadas e gestão, escritório política é uma realidade com a qual um arquiteto de software terá que lidar mais do que aqueles em outros funções de desenvolvimento.

As organizações vão querer o envolvimento de arquitetos de software para gerenciamento de risco. Os arquitetos de software podem identificar riscos e propor abordagens para lidar com eles. Os arquitetos de software trabalham com a equipe SCM para coordenar compilações, desenvolvimento ambientes e fornece informações para o gerenciamento de mudanças.

Os arquitetos de software devem considerar a introdução de linhas de produtos de software para muitos benefícios que eles fornecem. Reutilizar componentes arquitetônicos para criar produtos de software permite que sejam criados com maior rapidez e qualidade.

No próximo capítulo, examinaremos os requisitos de software e como eles afetam a arquitetura de software. Os arquitetos de software devem ter uma compreensão do negócio domínio e requisitos a fim de projetar uma solução apropriada. Vamos aprender sobre engenharia de requisitos, como extrair e resumir os requisitos das partes interessadas, e atributos de qualidade arquitetônica.

3. Compreendendo o domínio

Mudar de outras funções, como uma função de desenvolvedor de software, para ser um arquiteto de software requer que se expanda e amplie sua esfera de conhecimento para incluir um profundo conhecimento de seu domínio. Projetar com eficácia uma solução para um espaço problemático requer conhecimento sobre o domínio e os requisitos do software.

Este capítulo começa descrevendo a base para a compreensão de um domínio, que inclui conhecimento geral de negócios e uma compreensão aprofundada do negócio. Abordaremos os conceitos importantes de design orientado a domínio (DDD), e como isso pode ajudar sua equipe de projeto de software a lidar com a complexidade e modelar uma solução em torno dos conceitos do mundo real. O capítulo também detalha os diferentes tipos de requisitos de software e as técnicas para obtê-los dos principais interessados.

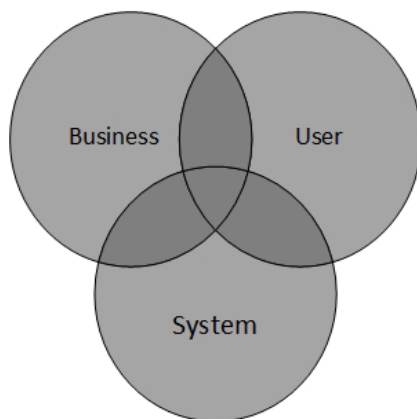
Neste capítulo, cobriremos os seguintes tópicos:

- Desenvolvendo visão de negócios
- Design orientado a domínio
- Engenharia de Requisitos
- Levantamento de requisitos

3.1. Desenvolvendo visão de negócios

Embora ser um arquiteto de software exija um conhecimento técnico profundo, para ter sucesso em a função também requer um entendimento completo dos negócios da sua organização. Em ordem de projetar uma arquitetura apropriada, você precisa ter conhecimento dos problemas de negócios que você está tentando resolver e as oportunidades de negócios que sua organização está buscando explorar. Um sistema de software tecnicamente avançado não tem utilidade se não atingir seus objetivos.

Ao projetar uma arquitetura de software, a fim de garantir que a solução seja apropriada, você deve considerar os objetivos da empresa, os usuários e o software sistema:



Cada uma dessas áreas de foco tem seus próprios objetivos, que podem se sobrepor e impactar significativamente cada um dos outros. Por exemplo, uma meta de negócios de um tempo específico e agressivo para o mercado pode significar que não há tempo suficiente para coisas como análise de requisitos adequada ou qualidade de teste de garantia, que pode impactar significativamente os objetivos do usuário. Quando as metas estão em conflito umas com as outras, requer que você encontre um equilíbrio aceitável entre eles ao projetar sua arquitetura de software.

3.2. Familiaridade com tópicos gerais de negócios

Ter uma compreensão prática e competente do negócio e sua terminologia será útil para você se tornar um arquiteto completo. Embora sua paixão possa ser por tecnologia, como arquiteto de software, você se beneficiará do conhecimento de negócios mais do que outras funções, como um desenvolvedor de software. Os arquitetos de software precisam interagir com uma variedade de partes interessadas, e compreender a linguagem do negócio irá garantir que você tenha um entendimento comum com eles.

Em última análise, você está projetando uma arquitetura de software que atenderá às metas de negócios e sua compreensão dos negócios o guiará nessa tarefa. Ter negócios em geral conhecimento de tópicos como finanças, operações, gestão e marketing irá ajudá-lo para entender o valor que seu software deve agregar a uma organização. O negócio as decisões serão tomadas com base em coisas como cálculos de retorno sobre o investimento (ROI) para o projeto de software e análise de custo-efetividade de diferentes abordagens. Uma boa compreensão de esses conceitos o ajudará a agregar valor a essas discussões.

Uma forma de adquirir esse conhecimento é por meio da educação formal. Se isso não for viável, há são outras maneiras de obter pelo menos uma compreensão rudimentar dos tópicos de negócios. Existem aulas online disponíveis, algumas das quais você pode assistir gratuitamente. Outra opção é obter e leia um ou mais livros sobre os tópicos relevantes.

3.3. Compreender os negócios da sua organização

Depois de ter algum conhecimento geral de negócios, você desejará obter uma boa compreensão dos negócios da sua organização. É um aspecto crucial para ser bem-sucedido arquiteto de software e separa um bom arquiteto, que só tem conhecimento técnico, de um ótimo.

Um bom ponto de partida é obter uma compreensão dos produtos da sua organização e serviços e o valor que eles fornecem aos clientes. Como sua organização faz dinheiro? Se você é o arquiteto de software de um produto específico, preste atenção especial a esse produto. Invista tempo para entender os vários processos de negócios de sua organização.

Você deve aprender sobre o mercado em que sua organização atua e suas tendências. Isto é prudente se familiarizar com os concorrentes de sua organização. Você deve procurar respostas a perguntas como:

- O que seus concorrentes fazem de diferente?
- O que eles fazem de semelhante?
- Quais são os pontos fortes e fracos de seus concorrentes?

Mais importante ainda, dedique tempo para entender os clientes da sua organização. Os produtos de software que você ajuda a projetar são para seus clientes, e talvez sejam os mais aspectos importantes dos negócios da sua organização. O que o negócio deles faz? Como eles usam seus produtos e serviços? Por que eles escolheram seus produtos e serviços as de um concorrente?

Depois de se familiarizar com os negócios de sua organização, o mercado em que ela opera, seus produtos/serviços e seus clientes, você está no caminho para compreender o domínio da sua organização.

3.4. Design orientado a domínio

Compreender o domínio de seu aplicativo de software é parte do que é necessário para descobrir a arquitetura apropriada para quaisquer soluções que você precise desenvolver. O domínio é o assunto e o corpo de conhecimento em que o software será aplicado.

O termo design orientado a domínio (DDD) foi cunhado por Eric Evans em seu livro *Domain-Design Orientado: Enfrentando a Complexidade no Coração do Software*. DDD é uma abordagem para desenvolver software que visa torná-lo melhor focando no domínio. Isto tem um conjunto de conceitos e padrões que foram usados com sucesso para resolver problemas e construir um ótimo software.

DDD é particularmente útil para grandes aplicativos de software que possuem modelos. O DDD ajuda você a resolver problemas complexos. Mesmo se você decidir não seguir DDD totalmente durante a arquitetura de seus aplicativos, alguns conceitos DDD podem ser úteis para você. Isto é benéfico para se familiarizar com esses conceitos, pois alguns deles são referenciados em outros áreas deste livro e caso você as encontre em seu trabalho.

3.5. Incentiva e melhora a comunicação

Um dos benefícios do DDD é o fato de estimular e melhorar a comunicação. A comunicação entre todos os membros da equipe é incentivada. Em particular, DDD enfatiza a importância de interagir com especialistas no domínio.

Um especialista de domínio, ou especialista no assunto (SME), é alguém que possui experiência sobre, e é uma autoridade em, uma área particular. Compreender o domínio do seu software a aplicação é altamente benéfica e os especialistas em domínio ajudarão toda a equipe a obter isso entendimento.

Além de encorajar a comunicação com especialistas de domínio, o DDD melhora comunicação entre todos os membros da equipe e partes interessadas, introduzindo o conceito de uma linguagem onipresente.

3.6. O que é uma linguagem onipresente?

A equipe de desenvolvimento pode não ter um bom conhecimento do domínio e não pode familiarizar-se com os termos e conceitos usados pelas partes interessadas, incluindo os especialistas do domínio.

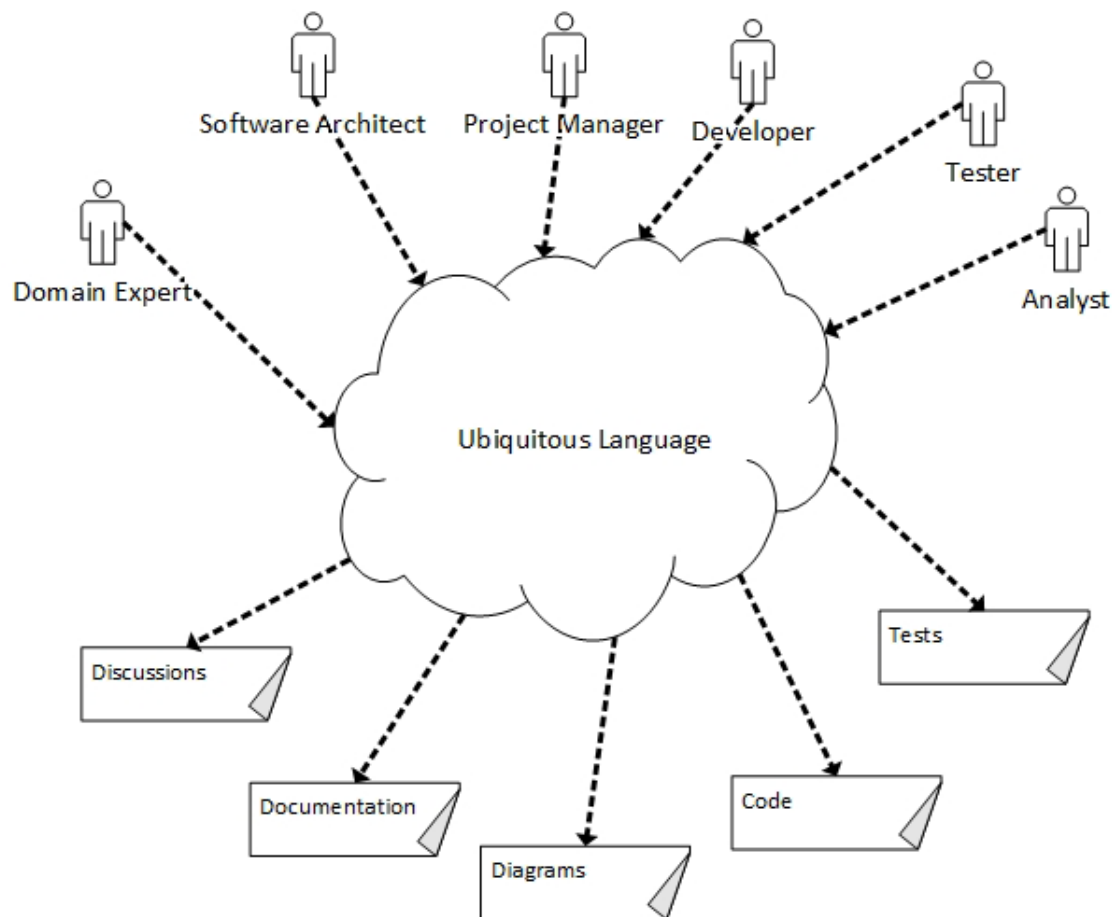
Eles podem usar sua própria linguagem ao discutir a funcionalidade e discutir o domínio em termos de seu projeto técnico. As partes interessadas, incluindo os especialistas do domínio, usará seu próprio jargão ao discutir seu domínio e pode não ter uma boa compreensão dos termos técnicos. Porque diferentes pessoas podem usar diferentes linguagens para descrever os mesmos conceitos em um domínio específico, pode demorar mais para comunicar ideias, e pode levar a mal-entendidos.

Em *Domain-Driven Design: Lidando com a Complexidade no Coração do Software*, Eric Evans descreveu este problema:

"A terminologia das discussões do dia a dia está desconectada da terminologia embutido no código (em última análise, o produto mais importante de um projeto de software). E até mesmo a mesma pessoa usa uma linguagem diferente na fala e na escrita, de modo que a maioria expressões incisivas do domínio muitas vezes surgem de uma forma transitória que nunca é capturada no código ou mesmo na escrita. A tradução embota a

comunicação e torna anêmico o esmagamento do conhecimento. No entanto, nenhum desses dialetos pode ser uma linguagem comum porque nenhum atende a todas as necessidades. "

Alguns membros da equipe podem se familiarizar com a terminologia do domínio e agir como tradutores para o resto da equipe, mas podem se tornar gargalos. Para mitigar esses tipos de riscos, Eric Evans criou o conceito de uma onipresente língua. É uma linguagem comum entre todos os membros da equipe e partes interessadas com base no modelo de domínio:



O desenvolvimento de uma linguagem onipresente pode levar tempo e pode evoluir e crescer à medida que a equipe compreensão das mudanças de domínio. Especialistas em domínio devem usar seu conhecimento de o domínio para apontar termos que não expressam corretamente uma ideia, e todos podem olhar para inconsistências e ambiguidades em um esforço para melhorar a linguagem onipresente.

Embora exija esforço, uma vez que você tem uma linguagem onipresente, isso simplifica a comunicação e leva a um maior entendimento entre todos os envolvidos com o projeto. Não tradução será necessária porque todos concordaram e entendem os vários termos. O importante é usá-lo de forma consistente e durante todo o projeto. A linguagem ubíqua deve ser usada durante as discussões e em todos os artefatos do projeto como documentação, diagramas, código e testes.

3.7. Entidades, objetos de valor e agregados

Alguns dos blocos de construção básicos do DDD são entidades, objetos de valor e agregados. Quando modelando-os, a linguagem onipresente deve ser usada.

3.7.1. Entidades

Entidades são objetos definidos por sua identidade e não por seus atributos. Eles são mutáveis porque os valores de seus atributos podem mudar sem alterar sua identidade. Se dois objetos têm os mesmos valores para seus atributos, além de seu identificador único, eles não são considerados iguais. Por exemplo, se você tivesse dois objetos *Person* com os mesmos valores de nome e sobrenome para esses atributos correspondentes, eles ainda são dois objetos diferentes porque têm identidades diferentes. Isso também significa que um valor para um atributo, como o sobrenome, pode ser alterado em um objeto *Person* e ainda representa a mesma pessoa.

3.7.2. Objetos de valor

Ao contrário das entidades, os objetos de valor são objetos que descrevem alguma característica ou atributo, mas não tem conceito de identidade. Eles são definidos pelos valores de seus atributos e são imutáveis. Se dois objetos têm os mesmos valores atribuídos às suas propriedades, eles podem ser considerados iguais.

Por exemplo, se dois objetos que representam pontos em um gráfico usando coordenadas cartesianas têm os mesmos valores de *x* e *y*, eles podem ser considerados iguais e seriam modelados como um objeto de valor.

3.7.3. Agregados e entidades raiz

Agregados são agrupamentos de entidades e objetos de valor que são tratados como uma única unidade. Um limite é definido que os agrupa. Sem agregados, domínio complicado modelos podem se tornar difíceis de gerenciar, à medida que muitas entidades e suas dependências aumentam grande em número. Recuperar e salvar uma entidade e todos os seus objetos dependentes pode tornar-se difíceis e sujeitos a erros.

Um exemplo de um agregado é um objeto de pedido que contém um objeto de endereço e uma coleção de objetos de item de linha. O objeto de endereço e a coleção de objetos de item de linha são todos os objetos separados, mas são tratados como uma única unidade para recuperação de dados e alterações.

3.7.4. Separando o domínio em subdomínios

Uma prática do DDD é separar o modelo de domínio em vários subdomínios. Enquanto um domínio é todo o espaço do problema para o qual a solução de software está sendo desenvolvida, um subdomínio é uma parte particionada do domínio geral. Isso é particularmente útil para grandes domínios, onde não é viável ter um modelo de domínio grande e pesado.

Ao se concentrar em um subdomínio de cada vez, ele reduz a complexidade e torna o trabalho geral mais digerível. Em vez de tentar resolver muitos problemas de uma vez, dividir seu domínio em subdomínios fornece mais uma abordagem de dividir e conquistar. Por exemplo, em um sistema de informação do aluno, você pode ter subdomínios para contato gestão, admissões, auxílio financeiro, contas de alunos e acadêmicos, entre outros.

Um ou mais dos subdomínios podem ser designados como um domínio principal, que normalmente é a parte do domínio que é fundamental para a organização. Se houver uma parte

do domínio que diferencia a organização dos concorrentes, é provavelmente um dos principais domínios. Domínios centrais são a razão pela qual vale a pena escrever o software, ao invés de comprar software existente da prateleira ou terceirizar o trabalho. Os especialistas de domínio no projeto podem ajudar a identificar os domínios principais, bem como a divisão de domínios em subdomínios.

3.8. O que são contextos limitados?

Um modelo de domínio é um modelo conceitual baseado no domínio e inclui ambos os comportamentos e dados. Ele representa uma parte da solução geral que atende aos objetivos do negócio.

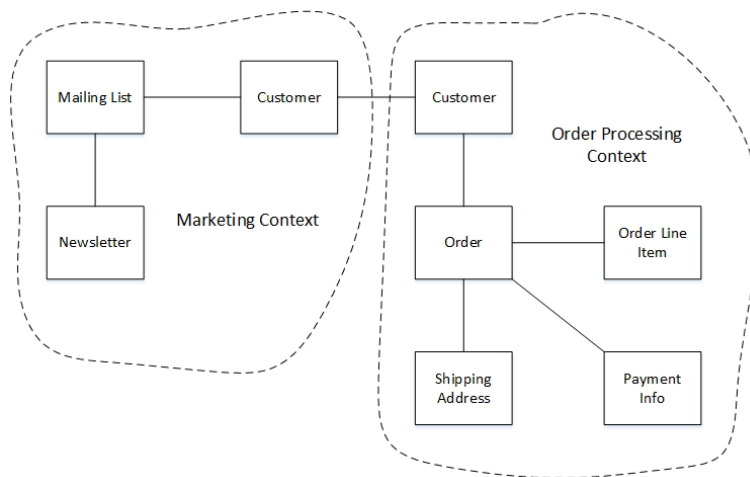
Contextos limitados são um padrão em DDD que representam partições no modelo de domínio. Semelhante a subdomínios, que são partições no domínio, contextos limitados são partições no modelo de domínio. Como é o caso dos subdomínios, a criação de partições e limites reduz a complexidade geral.

Um contexto limitado pode ser mapeado para um único subdomínio, mas lembre-se de que nem sempre é o caso. O modelo de domínio para um subdomínio pode exigir vários contextos limitados para a solução geral desse subdomínio.

Por exemplo, se estivéssemos criando um sistema de software para uma empresa que vende roupas online, podemos permitir que os clientes se inscrevam em um boletim informativo que contenha ofertas e descontos.

Outra parte do aplicativo permitiria aos clientes fazer pedidos e fornece Informação de pagamento. Com essas duas peças de funcionalidade, alguns conceitos são compartilhados, enquanto outros não. Se diferentes equipes de desenvolvimento, ou diferentes desenvolvedores em uma única equipe, estão trabalhando esses dois conjuntos de funcionalidade, não está claro qual sobreposição, se houver, existe. Se houver sobreposição, o que deve ou não deve ser compartilhado entre essas duas partes da funcionalidade? Isto é onde o conceito de contextos limitados é aplicável. Um modelo de domínio se aplica a um contexto particular, para que possamos definir os vários contextos para esclarecer algumas das ambiguidades que existem.

Neste exemplo, poderíamos criar um contexto limitado para marketing (contexto de marketing), e um para processamento de pedidos (Contexto de Processamento de Pedidos). Cada contexto limitado pode ter entidades que são exclusivas para ele. Por exemplo, o Contexto de Processamento de Pedidos tem o conceito de um item de linha de pedido, enquanto o Contexto de gerenciamento de contatos não. No entanto, ambos contextos limitados têm o conceito de um cliente. O cliente está se referindo ao mesmo conceito em ambos os contextos limitados? Ao separá-los, podemos começar a responder a esta pergunta:



No contexto de marketing, tudo o que pode ser necessário para uma entidade Cliente é uma identidade (identificador único), nome, sobrenome e endereço de e-mail. No entanto, no contexto de fazer um pedido, a entidade Cliente exigiria informações adicionais, como um endereço de entrega e informações de pagamento.

Você pode criar uma entidade Cliente, mas usá-la para contextos diferentes adiciona complexidade e pode levar a inconsistências. A validação que requer informações de pagamento só se aplica no Contexto de Processamento de Pedidos, e não no Contexto de Marketing. O comportamento requerido para um cliente no contexto de processamento do pedido não deve impedir um cliente de sendo criado no Contexto de Marketing, onde apenas o nome, sobrenome e e-mail endereço são obrigatórios.

Discutiremos o princípio de responsabilidade única (SRP) posteriormente neste livro, mas o básico. A ideia é que cada classe seja responsável por um único aspecto da funcionalidade. A entidade do cliente ainda é pequena agora, mas você pode começar a ver como ela pode crescer rapidamente. Se isso fossem usados em vários contextos, seria uma tentativa de cumprir muitas responsabilidades e quebrar o SRP.

O contexto de cada modelo deve ser claramente definido e deve haver uma fronteira entre contextos limitados. Eles são criados para que todos na equipe, ou em várias equipes, pode ter o mesmo entendimento do que pertence a cada contexto.

Embora o exemplo usado seja simplista, um grande modelo de domínio terá muitas entidades e contextos, e geralmente não é imediatamente claro o que é único ou comum entre contextos diferentes e como cada contexto deve interagir uns com os outros.

DDD e o conceito de contextos limitados funcionam bem com micro serviços, que seremos discutidos em mais detalhes posteriormente neste livro. Agora que temos uma melhor compreensão do Conceitos DDD, vamos entrar em detalhes sobre os requisitos. Trabalhar com especialistas de domínio e outras partes interessadas, precisamos ter uma compreensão dos requisitos antes do design.

3.9. Engenharia de Requisitos

Para modelar seu domínio e projetar uma arquitetura apropriada, você precisará conheça os requisitos do software que você está construindo. Engenharia de Requisitos envolve o estabelecimento da funcionalidade exigida pelas partes interessadas, juntamente com as

restrições sob as quais o software deve ser desenvolvido e operar. Abrange todos os as tarefas envolvidas na obtenção, análise, documentação, validação e manutenção dos requisitos do sistema de software. Como arquiteto de software, você participará de essas tarefas, portanto, é útil estar familiarizado com elas.

3.9.1. Tipos de requisitos de software

Existem diferentes tipos de requisitos de software, e os arquitetos de software devem ser bem-informado sobre eles. Os principais tipos de requisitos de software incluem:

- Requisitos de negócio
- Requisitos funcionais
- requisitos não funcionais
- Restrições

Vamos examinar mais de perto cada um desses tipos.

3.9.1.1. *Requisitos de negócio*

Os requisitos de negócios representam os objetivos de negócios de alto nível da construção da organização o software. Este tipo de requisito define os problemas de negócios que o software irá resolver ou as oportunidades de negócios que serão abordadas pelo software.

Os requisitos de negócios podem incluir requisitos que vêm do mercado. Uma organização pode querer garantir que eles não estão excluindo alguma funcionalidade que um concorrente está fornecendo. Eles também podem querer se diferenciar de um concorrente fornecendo funcionalidade que um concorrente não oferece, ou fornece a mesma funcionalidade, mas em alguma forma aprimorada (por exemplo, tempos de resposta mais rápidos). Como resultado, os negócios os requisitos frequentemente influenciam os atributos de qualidade de um sistema de software.

3.9.1.2. *Requisitos funcionais*

Os requisitos funcionais descrevem a funcionalidade do software, em outras palavras, o que o sistema de software deve fazer. Eles detalham as capacidades do sistema de software em termos de comportamento. A funcionalidade e recursos descritos pelos requisitos funcionais permitir que as partes interessadas realizem suas tarefas.

Os requisitos funcionais incluem a interação do software com seu ambiente. Elas consistem nas entradas, saídas, serviços e interfaces externas que devem ser incluídas com o software. Lembre-se de que os requisitos podem vir de várias fontes, como as seguintes:

- Requisitos organizacionais: requisitos baseados em requisitos organizacionais políticas e procedimentos
- Requisitos legislativos: requisitos não funcionais que detalham quaisquer leis e regulamentos que o software deve seguir
- Requisitos éticos: Quaisquer requisitos para a operação ética do software, como preocupações relacionadas à privacidade ou segurança
- Requisitos de entrega: Requisitos que estão relacionados à entrega e implantação do software
- Requisitos de padrões: requisitos para quaisquer padrões que devem ser seguidos para o desenvolvimento do software ou como o software deve operar
- Requisitos externos: requisitos que se originam externamente, como requisitos de sistemas externos que devem se integrar com o sistema de software sendo projetado

3.9.1.3. Requisitos não funcionais

Requisitos não funcionais são condições que devem ser atendidas para que a solução ser eficazes, ou restrições que devem ser levadas em consideração. Analistas de negócios e outras partes interessadas tendem a fazer um bom trabalho na captura de requisitos funcionais, mas eles nem sempre se concentra tanto em requisitos não funcionais. No entanto, não funcional os requisitos são uma parte importante da engenharia de requisitos. O sucesso do projeto depende dos requisitos não funcionais e se eles são ou não atendidos.

Quando uma arquitetura de software é projetada, o arquiteto de software deve garantir que o não funcional requisitos podem ser satisfeitos. Requisitos não funcionais podem ter um impacto significativo no design da arquitetura. Por esse motivo, eles são de grande importância para arquitetos de software. Os arquitetos de software precisam desempenhar um papel ativo na elicitacão requisitos não funcionais das partes interessadas e garantindo que eles sejam capturados.

Os atributos de qualidade são um subconjunto de requisitos não funcionais e incluem as *ilidades, como quanto à manutenção, usabilidade, testabilidade e interoperabilidade. Entraremos em mais detalhes sobre alguns dos diferentes atributos de qualidade no próximo capítulo.

3.9.1.4. Restrições

Restrições são algum tipo de restrição na solução e podem ser técnicas ou não técnicas na natureza. Algumas restrições em um projeto podem ser capturadas e classificadas como requisito funcional ou não funcional, ou podem ser categorizados explicitamente como uma limitação. De qualquer forma, o importante é que sejam decisões que já foram feitos e deve ser homenageado. Normalmente, uma restrição não pode ser alterada e o software o arquiteto não tem nenhum controle sobre ele. No entanto, se você tiver razões pelas quais acredita que uma restrição deve ser alterada ou removida, pode haver situações em que você poderia forneça sua opinião.

As restrições podem referir-se a vários aspectos de um projeto de software. Os seguintes são alguns exemplos de restrições:

- Uma organização pode ter um acordo existente com um fornecedor específico ou já comprou uma determinada tecnologia ou ferramenta que você deverá usar
- Pode haver uma lei ou regulamento que o software deve seguir
- Pode haver um prazo específico para um marco ou a entrega final do software que não pode ser alterado
- A gestão pode ditar que um certo número de recursos seja atribuído ao projeto, ou que o projeto deve utilizar recursos terceirizados
- Se a equipe de desenvolvimento já existe e eles são qualificados em um determinado linguagem de programação, a organização pode exigir que a programação linguagem ser usada

As restrições devem ser consideradas ao projetar a solução, assim como outros tipos de requisitos.

3.9.2. A importância da engenharia de requisitos

A importância da análise de requisitos não pode ser exagerada. Requisitos adequados a análise é crucial para um projeto bem-sucedido, pois afeta todas as fases subsequentes. Se isso não for feito corretamente, será necessário trabalho adicional, resultando em estouros de tempo e custos.

Quanto mais tarde no ciclo de vida esses problemas forem encontrados, mais custará e o levará mais tempo para corrigir esses erros. Quando um problema com os requisitos é descoberto mais tarde no ciclo de vida, as entregas que já foram produzidas em fases subsequentes, como design e desenvolvimento, podem exigir refatoração. Em código Completo (segunda edição), Steve McConnell explica que o princípio é encontrar um erro o mais próximo quanto possível ao momento em que foi introduzido. Quanto mais tempo o defeito permanece na comida do software corrente, mais danos ela causa mais abaixo na corrente.

Alguns dos muitos benefícios da análise de requisitos adequada incluem:

- Retrabalho reduzido
- Menos recursos desnecessários
- Custos de melhoria mais baixos
- Desenvolvimento mais rápido
- Custos de desenvolvimento reduzidos
- Melhor comunicação
- Estimativas de teste de sistema mais precisas
- Níveis mais altos de satisfação do cliente

É imperativo que a gestão compreenda a importância e os benefícios de uma adequada engenharia de requisitos. Se não o fizerem, uma tentativa deve ser feita para comunicar isso a para que o tempo adequado possa ser agendado para a engenharia de requisitos.

3.9.2.1. Os requisitos de software devem ser mensuráveis e testáveis

Ao definir os requisitos de software, eles devem ser completos de forma que todos eles sejam definidos e consistentes no sentido de que são claros e não se contradizem. Cada O requisito de software deve ser inequívoco, mensurável e testável. O teste deve ser considerado quando os requisitos são escritos. Os requisitos precisam ser específicos o suficiente para eles podem ser verificados.

Os analistas de negócios e outras partes interessadas que estão definindo os requisitos devem escrever de uma forma que sejam mensuráveis e testáveis. Como arquiteto de software, se você vir requisitos que não atendem a essas condições, você precisa apontá-los para que pode ser modificado.

Se um requisito deve ser considerado mensurável, ele deve fornecer valores ou limites específicos. Para que um requisito seja testável, deve haver uma maneira prática e econômica de determinar se o requisito foi satisfeito. Deve ser possível escrever um teste caso que pode verificar se o requisito foi cumprido ou não. Por exemplo, considere um requisito que afirma que a página da web deve ser carregada em tempo maneiras. O que exatamente isso significa? As partes interessadas e a equipe de desenvolvimento podem ter uma compreensão diferente do que irá satisfazer tal requisito. Deve ser escrito com um limite específico em mente, como a página da web deve ser carregada em dois segundos.

Um entendimento comum e expectativas mutuamente acordadas precisam ser definidas com partes interessadas para que não haja surpresas na entrega do produto.

3.9.2.2. Requisitos de software que afetam a arquitetura

Como arquiteto de software que está projetando uma arquitetura que atenderá aos requisitos, requisitos completos e validados são cruciais para o seu trabalho. Requisitos, especialmente os atributos de qualidade, podem afetar significativamente o projeto arquitetônico.

No entanto, o grau em que um determinado requisito afeta a arquitetura varia. Alguns não têm nenhum efeito, enquanto outros têm um efeito profundo. Você deve ser capaz de reconhecer os requisitos que podem afetar as decisões que você está tomando arquitetonicamente.

Muitas vezes, os requisitos que afetam o design de uma arquitetura de software são de qualidade atributos. Portanto, você deve prestar atenção especial a eles. Esteja ciente de que é comum a falta de definição de atributos de qualidade na definição de requisitos.

As partes interessadas podem se concentrar nos requisitos funcionais e não podem definir atributos de qualidade, ou se o fizerem, podem fazê-lo de uma forma que não seja mensurável e testável.

Os arquitetos de software podem precisar fazer um esforço extra para entender os atributos de qualidade que são importantes para as partes interessadas e os valores esperados para torná-los testáveis para obter os atributos de qualidade definidos e documentados. Na próxima seção, vamos examinar as maneiras de que você pode extrair requisitos, incluindo atributos de qualidade, das partes interessadas.

3.9.3. Levantamento de requisitos

Talvez você já tenha ouvido falar dos termos conhecidos, conhecidos desconhecidos e desconhecidos. Eles são usados para descrever aquilo que conhecemos, aquilo de que estamos cientes, mas não sabemos, e aquilo que nem mesmo estamos considerando porque não sabemos sobre eles.

Idealmente, os requisitos e o domínio de negócios de um projeto de software são bem compreendidos. No entanto, a equipe de desenvolvimento pode não ter esse entendimento desde o início do projeto. Mesmo para aqueles que têm o conhecimento, como algumas das partes interessadas, eles podem não saber exatamente o que querem do software.

Como resultado, você lidará com coisas conhecidas e desconhecidas. Parte dos requisitos engenharia envolve obter o máximo de conhecimento possível sobre os requisitos de o sistema que queremos construir. Procuramos eliminar as incógnitas desconhecidas e consideramos como muitos dos requisitos possíveis ao projetar o software.

O início desse processo é eliciar requisitos das partes interessadas, o que é conhecido como coleta de requisitos ou elicitação de requisitos. A coleta de requisitos parece implica simplesmente coletar requisitos que são fáceis de descobrir, embora normalmente envolva muito mais do que isso. Muitas vezes, é necessário extrair os requisitos de partes interessadas, porque nem todos estão na vanguarda dos pensamentos das partes interessadas. Isto é mais um processo proativo, e não reativo.

Como Andrew Hunt e David Thomas apontam em *The Pragmatic Programmer*:

"Os requisitos raramente ficam na superfície. Normalmente, eles estão profundamente enterrados sob camadas de suposições, equívocos e política. "

3.9.3. Técnicas para elicitar requisitos

Obter informações das partes interessadas exige esforço, mas existem técnicas comprovadas que pode ajudá-lo a desenhá-los. Cada técnica tem suas próprias vantagens e desvantagens, então selecione aquelas que têm maior probabilidade de funcionar de acordo com sua situação. Não esqueça que você pode usar mais de uma dessas técnicas. Usando várias técnicas para a conjunção entre si pode produzir os melhores resultados.

3.9.3.1. Entrevistas

Uma forma de elicitar requisitos é conduzir entrevistas com as partes interessadas. Entrevistas para esse propósito pode ser conduzido formal ou informalmente. Cada sessão de entrevista deve seja com uma única pessoa ou um pequeno grupo. Se for com mais de uma pessoa, você não deseja ter muitas pessoas na sessão ou corre o risco de não obter a quantidade máxima de informações de cada parte interessada.

Uma ou mais pessoas podem fazer perguntas, e pelo menos uma pessoa deve ser designada para responder notas. Faça perguntas abertas para estimular a discussão e obter informações, e fechadas perguntas podem ser usadas para confirmar fatos.

Como acontece com todas as técnicas, o sucesso das entrevistas depende do conhecimento do entrevistado, bem como a sua disponibilidade em participar. É bom entrevistar diferentes tipos de partes interessadas, a fim de obter perspectivas diferentes. Você precisa levar em consideração seu conhecimento e experiência ao revisar os resultados do entrevistas. As entrevistas nem sempre são uma boa maneira de chegar a um consenso, porque nem todos os as partes interessadas podem estar presentes, mas podem ser eficazes para obter informações.

3.9.3.2. Workshops de requisitos

Workshops de requisitos são um dos métodos de elicitação mais comuns e eficazes. Eles são usados para coletar e priorizar requisitos. Um grupo de partes interessadas relevantes são convidados a participar de uma sessão na qual fornecerão seus comentários. Um resultado inevitável de ter tais discussões será um nível mais alto de clareza sobre como o software deve funcionar, e o que é necessário fazer.

Uma agenda clara deve ser definida para cada workshop de requisitos. O escopo varia, mas você pode querer manter cada sessão restrita a uma determinada parte do processo de negócios ou aplicação de software. Alguém deve ser designado como facilitador que pode executar a reunião, e uma pessoa diferente deve tomar notas.

A duração de um workshop de requisitos pode variar e depende do escopo. Pode durar de uma hora a vários dias. A duração do workshop deve ser apropriada para seu escopo. Você pode adquirir muitas informações em um workshop de requisitos. Apenas certifique-se de ter um número ideal de pessoas participando. Se houver muitos, o processo pode ser diminuiu a velocidade e algumas pessoas podem não ter a oportunidade de compartilhar seus pensamentos. Sobre por outro lado, se não houver participantes suficientes, você pode não reunir o suficiente em formação.

Às vezes, pode ser difícil reunir todas as partes interessadas em um lugar ao mesmo tempo. Se você não pode organizar isso, você pode considerar a possibilidade de realizar vários workshops sobre o mesmo tópico.

3.9.3.3. Debate

Sessões de brainstorming envolvem obter pensamentos de um grupo espontaneamente e documentando esses pensamentos. Pode ser uma maneira divertida e produtiva de obter requisitos para um sistema. Se você vai conduzir tal sessão, certifique-se de convidar o responsável partes interessadas. Se houver muitos interessados, você pode considerar a realização de várias sessões e mantendo o atendimento de cada um entre cinco e dez pessoas.

Ao convidar as partes interessadas, certifique-se de ter uma variedade. Diferentes tipos de as partes interessadas terão perspectivas diferentes e podem fornecer ideias que outros não fariam considerou.

A sessão de brainstorming deve ter objetivos claros, e cada sessão não deve ser muito Largo. Você pode conduzir uma sessão de brainstorming para obter os requisitos para uma parte específica de funcionalidade dentro do sistema de software.

Tente realizar a sessão de brainstorming em um ambiente relaxado e confortável para que o os participantes se sentem à vontade para compartilhar suas ideias. Alguém deve ser designado como o facilitador. O facilitador pode precisar encorajar a participação, especialmente no início da sessão, já que alguns participantes podem conter seus pensamentos. Como um software arquiteto que deseja que o grupo gere ideias, você pode assumir a responsabilidade de criar com a primeira ideia para encorajar outros.

A crítica de ideias não deve ser tolerada, pois você não quer desencorajar ninguém de participando ainda mais. Embora possa não haver ideias ruins, às vezes haverá pensamentos que não são relevantes para os objetivos da reunião. Se uma discussão sair do assunto, o facilitador deve limitar a discussão e conduzi-la em outra direção.

O facilitador ou outra pessoa deve fazer anotações, de preferência em um quadro branco para que todos possam ver as ideias que foram dadas anteriormente. Para reuniões remotas onde todos não estão em uma sala juntos, alguém deve compartilhar sua tela para que o as ideias são visíveis.

Deve haver um limite de tempo para que todos saibam quando a sessão terminará. Se houver um final claro onde não há mais ideias novas sendo geradas, a sessão pode ser cancelada cedo.

3.9.3.4. Observação

Observação é uma técnica em que alguém estuda uma parte interessada em seu trabalho ambiente, executando tarefas relacionadas ao projeto de software. É particularmente útil quando você está tentando compreender um processo atual. É eficaz porque o observador pode observe coisas que não são mencionadas por meio de outras técnicas de elicitação. As partes interessadas podem esquecer certos requisitos ou pode até não estar ciente de que o que estão fazendo é um requisito que precisa ser documentado. Ao observar o trabalho real realizado, você às vezes pode coletar informações importantes.

Esta técnica pode ser realizada de forma passiva ou ativa, dependendo do o que foi acordado e o que seria mais eficaz. Se o observador é passivo, então ele ou ela faz todos os esforços para não perturbar. O observador não pergunta a muitas perguntas, nem interrompem as tarefas que a parte interessada está realizando. Se o observador está ativo, então eles podem ter um diálogo contínuo com as partes interessadas e fazer perguntas enquanto estão realizando suas tarefas.

Existem desvantagens nesta técnica. Pode ser demorado observar alguém realizando seu trabalho diário. A pessoa que está sendo observada pode achar isso perturbador ao trabalho deles, então você terá tempo limitado para realizar a observação.

Mesmo que gaste muito tempo observando, você pode não testemunhar todos os cenários possíveis, e estar ciente de cenários que não acontecem com tanta frequência ainda é importante para os requisitos do software.

Embora você não deva usar essa técnica como a única, ela pode ser útil como um suplemento a outras técnicas porque pode extrair requisitos que não seriam revelados usando outras técnicas de elicitación.

3.9.3.5. Grupos de foco

Os grupos focais podem ser organizados para elicitar requisitos. Esta técnica é mais formal do que brainstorming e envolve convidar um grupo de participantes para fornecer feedback. Essa técnica é comumente usada para aplicativos públicos que terão usuários externos. Naquilo caso, os participantes convidados são usuários ou especialistas externos que são externos à organização.

Um moderador dirige a sessão. O moderador selecionado geralmente é hábil em dirigir o foco grupos e é contratado especificamente para desempenhar essa função. O moderador faz as perguntas e incentiva a participação de todos os participantes. Os moderadores devem permanecer neutros durante a sessão.

As perguntas feitas durante um grupo de foco são normalmente abertas e promovem discussão. As respostas em um grupo de foco são geralmente faladas, ao invés de escritas. Naquilo tipo de ambiente, coisas como comunicação não verbal e interação em grupo podem ser observados. Os participantes do grupo focal podem promover novas ideias uns com os outros. Esta técnica pode ser mais rápida do que conduzir entrevistas individualmente.

Apesar das vantagens, existem algumas desvantagens nessa técnica. Grupos focais correm o risco de indivíduos seguirem a multidão ao ouvirem o feedback de outras pessoas no grupo. Algumas pessoas hesitam em compartilhar suas ideias em um ambiente de grupo, e o moderador pode precisar ser pago para conduzir o grupo de foco.

3.9.3.6. Pesquisas

Pesquisas podem ser criadas e fornecidas às partes interessadas para obter informações. As pesquisas devem ter um propósito claro. Em vez de criar uma grande pesquisa que cobre muitos tópicos, pode ser mais eficaz para criar várias pesquisas, cada uma cobrindo uma parte dos processos de negócios ou aplicação de software. Algumas pessoas serão avessas a preencher pesquisas extremamente longas.

As perguntas da pesquisa devem ser bem pensadas, claras e concisas. Embora as pesquisas podem ter perguntas abertas, normalmente as perguntas em uma pesquisa são fechadas uns. Isso torna mais fácil para os participantes fornecerem respostas e, mais o mais importante é que as respostas serão mais fáceis de analisar. Se você quiser fazer perguntas abertas perguntas em uma pesquisa, lembre-se de que será necessário mais esforço para analisar as respostas.

3.9.3.7. Análise de documentos

A análise de documentos utiliza a documentação existente para obter informações e requisitos. A documentação pode abranger os processos de negócios relevantes ou existentes sistemas de software. Se houver um sistema existente em uso, ele pode servir como um ponto de partida para os requisitos do novo sistema. A documentação pode vir na forma de documentação, manuais do usuário, contratos, declarações de trabalho, e-mails, materiais de treinamento e qualquer outra coisa que possa ser útil.

A documentação pode até ser um pacote de software comercial de prateleira (COTS) manuais. Pode haver software existente que fornece parte ou toda a funcionalidade que você está buscando implementar e, ao analisar essa documentação, você pode obter os requisitos para o seu sistema de software.

A análise de documentos é particularmente útil se as partes interessadas não estiverem disponíveis para outras técnicas de elicitação de requisitos.

3.9.3.8. Prototipagem

A prototipagem é uma técnica de elicitação de requisitos que envolve a construção de um protótipo que as partes interessadas podem usar até certo ponto, ou pelo menos ver. Algumas pessoas são mais visualmente orientadas do que outras, e ter um protótipo pode disparar ideias sobre os requisitos.

A desvantagem da prototipagem é que pode levar algum tempo para construir um protótipo. Contudo, com a maioria das tecnologias modernas, um protótipo pode ser construído rapidamente. Também existe a opção de simplesmente criando diagramas visuais do software, em vez de um protótipo. Para web aplicativos, isso envolve a criação de wireframes, que são representações visuais da web páginas que permitem que uma pessoa veja o layout e a estrutura das páginas da web.

O escopo de um protótipo pode ser tão amplo ou restrito quanto você desejar. Embora possa demonstrar um aplicativo inteiro, ele pode ser focado em uma parte específica da funcionalidade.

A prototipagem pode ser útil em conjunto com outras técnicas para que você possa validar requisitos e descobrir coisas que ainda não foram discutidas. A prototipagem também pode ser levada a um nível diferente, no qual uma versão funcional do software é produzida. Em uma situação em que a direção e o propósito do software foram ainda não totalmente avaliados, talvez porque as partes interessadas não saibam por onde começar, ou eles têm muitas ideias, mas não conseguem concordar entre si, um protótipo inicial pode ser desenvolvido. Se você estiver usando uma metodologia ágil, algumas iterações iniciais podem ocorrer, cada um terminando com uma versão funcional do software que pode ser compartilhada com as partes interessadas.

Uma vez que eles tenham algo concreto para olhar e usar, isso pode inspirá-los com novas ideias e abordagens. Todos serão capazes de ver o que funciona e, tão importante, o que funciona não funciona. Se feito corretamente, conforme ocorre a refatoração e outras iterações acontecem, os requisitos se tornarão mais evidentes à medida que o software tomar forma.

3.9.3.9. Engenharia reversa

A engenharia reversa é um método em que o código existente é analisado para determinar os requisitos. É semelhante à técnica de análise de documentos, pois assume que há são artefatos

existentes para analisar. Nem sempre é o caso ao projetar um novo software sistema. Também requer acesso ao código-fonte e alguém com habilidade técnica para analisar o código e extraia os requisitos dele.

É uma técnica demorada, mas pode ser usada como último recurso se outras técnicas forem não é possível. Por exemplo, se as partes interessadas não estão disponíveis para você, ou aqueles que estão disponíveis para você não tem conhecimento, muitas das outras técnicas podem não ser viáveis. Se também há falta de documentação, então a análise de documentos também pode não ser possível.

Este método não é apenas um curso de ação final quando outros não são possíveis. Quando usado apropriadamente, pode ser uma técnica poderosa. As partes interessadas podem ter limitado perspectivas e pode não pensar em tudo o que o software deve fazer. Se houver um sistema existente, olhar para o código é uma maneira de determinar exatamente o que precisa acontecer.

3.9.4. Obtenha acesso às partes interessadas adequadas

Mesmo armado com técnicas para elicitare requisitos, pode ser difícil obtê-los das partes interessadas adequadas. Você pode se encontrar em situações em que certas partes interessadas não estão disponibilizadas para você. Você também pode se encontrar em uma situação em que certas partes interessadas, por várias razões, não estão sendo úteis ou não querem participar do projeto.

Devido à importância da análise de requisitos, você deve fazer um esforço para obter acesso a as partes interessadas relevantes. Isso pode envolver falar com a gerência para obter o acesso. Embora isso possa ser mais fácil se você trabalhar para a mesma organização, muitas das partes interessadas serão externas à organização. O sucesso do projeto pode depender de isso, então você pode precisar escalar essa necessidade para sua própria gestão ou para a gestão de organização da parte interessada.

3.10. Resumo

Ser um arquiteto de software eficaz significa compreender o domínio do software que você está construindo. Adquirir conhecimento de tópicos gerais de negócios e uma compreensão profunda de o negócio da organização é a base para se tornar um especialista no espaço do problema para o qual você projetará uma solução. DDD é uma abordagem comprovada para modelar um domínio. Criar uma linguagem onipresente que irá simplificar e facilitar a comunicação entre todos os envolvidos no projeto de software, e trabalhar com especialistas no domínio, facilitará o aprendizado de um domínio específico.

Outras práticas, como separar seu domínio em subdomínios e criar contextos em seu modelo de domínio, irão minimizar a complexidade e permitir que você e sua equipe agarrar firmemente até mesmo domínios complexos. Uma das chaves para construir software com sucesso é a engenharia de requisitos adequada, incluindo saber como elicitare requisitos de forma eficaz das partes interessadas. Conhecendo os requisitos para o software é crucial para projetar uma solução apropriada.

No próximo capítulo, exploraremos mais um dos tipos mais importantes de software requisitos: atributos de qualidade. Construir software de qualidade exige que o arquiteto de software conhecer e compreender os detalhes dos atributos de qualidade que são importantes para as partes interessadas.

4. Atributos de qualidade de software

Os atributos de qualidade são de extrema importância para os arquitetos de software porque afetam decisões arquitetônicas. Neste capítulo, começaremos explicando quais atributos de qualidade são, e porque é importante considerá-los ao longo da vida de desenvolvimento de software ciclo. Alguns atributos de qualidade podem ser difíceis de testar, então vamos explorar o que é necessário para testar eles.

Entraremos em detalhes sobre alguns atributos de qualidade de software mais comuns, como manutenibilidade, usabilidade, disponibilidade, portabilidade, interoperabilidade e testabilidade. Depois de lendo este capítulo, você conhecerá as considerações que precisam ser levadas em consideração para um sistema de software atender a esses atributos de qualidade.

Neste capítulo, cobriremos os seguintes tópicos:

- Atributos de qualidade
- Capacidade de Manutenção
- Usabilidade
- Disponibilidade
- Portabilidade
- Interoperabilidade
- Testabilidade
- Atributos de qualidade

Atributos de qualidade são propriedades de um sistema de software e um subconjunto de seus requisitos. Como outros requisitos, eles devem ser mensuráveis e testáveis. Programas atributos de qualidade são referências que descrevem a qualidade do sistema de software e medem a adequação do sistema. Um sistema de software é composto de uma combinação de qualidade atributos, e o grau em que eles são cumpridos descreve a qualidade geral do Programas.

Os atributos de qualidade podem ter um impacto significativo no design da arquitetura, portanto, são de grande interesse para arquitetos de software. Eles afetam várias facetas de um sistema de software, como seu design, grau de manutenção, comportamento de tempo de execução e o usuário geral experiência.

Ao projetar sua arquitetura, é imperativo entender que a qualidade do software atributos podem afetar uns aos outros, e o grau em que um é atendido pode afetar o grau de que outros podem ser encontrados. É importante identificar potenciais conflitos entre qualidade atributos. Por exemplo, a necessidade de desempenho ultrarrápido pode entrar em conflito com a capacidade de alcançar escalabilidade extrema, e ter um alto nível de segurança pode diminuir o nível de usabilidade.

Esses trade-offs precisam ser analisados para fornecer um equilíbrio para que uma solução aceitável possa Ser criado. A prioridade de cada atributo de qualidade será um fator em seu design geral.

4.1. Externo ou interno

Os atributos de qualidade podem ser internos ou externos. Atributos de qualidade internos podem ser medidos pôr o próprio sistema de software e são visíveis para a equipe de desenvolvimento. Consequentemente, eles são mensuráveis durante e após o desenvolvimento. Exemplos de atributos de qualidade internos são aspectos do sistema de

software, como linhas de código (LOC), nível de coesão, legibilidade de o código e o grau de acoplamento entre os módulos.

Esses atributos refletem a complexidade do sistema de software. Apesar de qualidade interna os atributos não são visíveis para os usuários diretamente, eles afetam os atributos de qualidade externos. Um maior nível de qualidade interna muitas vezes leva a um maior nível de qualidade externa.

Atributos de qualidade externos são propriedades visíveis externamente; portanto, eles são perceptíveis para os usuários finais. Esses atributos de qualidade são medidos em relação a como o sistema de software se relaciona com seu ambiente. Ao contrário dos atributos de qualidade internos, um trabalho versão do software deve ser implantada para que possa ser testada. Exemplos de externos atributos de qualidade são o desempenho, confiabilidade, disponibilidade e usabilidade do sistema.

4.2. Atributos de qualidade e o SDLC

Os atributos de qualidade devem ser considerados ao longo do ciclo de vida de desenvolvimento de software (SDLC). O processo começa durante a engenharia de requisitos, garantindo que eles sejam capturados completamente e corretamente. No capítulo anterior, discutimos a importância de requisitos mensuráveis e testáveis. Isso deve ser enfatizado ainda mais em relação à qualidade atributos, já que alguns atributos de qualidade podem ser difíceis de medir. Os atributos de qualidade devem ser mensuráveis e testáveis, a fim de determinar se o sistema de software satisfaz as necessidades de partes interessadas.

O projeto da arquitetura de software deve garantir que ele atenda aos objetivos de qualidade. No decorrer teste, os atributos de qualidade devem ser verificados para garantir que o sistema de software satisfaça os requisitos.

4.3. Atributos de qualidade de teste

Que tipos de técnicas de teste devem ser usados para verificar os atributos de qualidade? Além de mais simples dos aplicativos de software, pode ser desafiador testar atributos de qualidade de software e todos os cenários possíveis que são afetados por eles. Uma variedade de técnicas de teste deve ser empregue para testar propriedades diferentes. Por exemplo, a seguir estão alguns testes que você pode precisar realizar:

- Testar manualmente o software quanto à usabilidade
- Criação de benchmarks e uso de ferramentas para testes de desempenho
- Execução de revisões de código e cálculo de métricas de código para testar a capacidade de manutenção

Executar testes de unidade automatizados para garantir que o sistema se comporte conforme o esperado cada técnica de teste vem com seu próprio conjunto de pontos fortes e fracos. Alguns software atributos de qualidade são difíceis de avaliar, então você pode precisar combinar vários testes técnicas para testar efetivamente um atributo de qualidade. Com recursos ilimitados, nós podemos realizar uma quantidade exaustiva de testes, mas os projetos são normalmente limitados por alguma combinação de custo e tempo. Como resultado, às vezes é necessário alcançar um equilíbrio entre a quantidade de testes e o tempo disponível.

Automatizar o máximo possível dos testes é a chave para ser capaz de executar testes em uma curta quantidade de tempo enquanto maximiza a cobertura do teste. Testes automatizados podem ser executados sob demanda ou como parte de um processo de entrega contínua (por exemplo, como parte de um processo automatizado construir). As equipes de desenvolvimento podem ter certeza de que ainda atendem aos objetivos de atributos de qualidade como eles continuam a fazer alterações no código.

4.4. Capacidade de Manutenção

A capacidade de manutenção se concentra na facilidade com que um sistema de software pode ser mantido. A manutenção de um sistema de software ocorre à medida que as alterações são feitas nele depois que o software é em operação. A manutenção é necessária para preservar o valor do software ao longo do tempo.

A mudança é constante no mundo real. Às vezes é esperado e pode ser planejado, enquanto outras vezes não. De qualquer forma, é inevitável que os sistemas de software experimentem mudança. Com o conhecimento de que a mudança é inevitável, é importante construir sistemas sustentáveis.

Décadas atrás, a maior parte dos custos de um projeto de software era destinada ao desenvolvimento de software. No entanto, ao longo dos anos, houve uma mudança na relação de custo de desenvolvimento para manutenção. Hoje, a maioria dos custos de vida de um sistema pode ser tipicamente atribuída a manutenção. Fazer o que for possível para manter esses custos baixos pode resultar em uma significativa diferença no dinheiro total gasto com o software durante sua vida útil.

O código de fácil manutenção permite que o trabalho de manutenção seja concluído mais rapidamente e por sua vez, ajudará a manter os custos de manutenção baixos. Quando um desenvolvedor está escrevendo código, ele ou ela deve levar em consideração não apenas o usuário final do software, mas também aqueles que vão mantê-lo.

Mesmo que o desenvolvedor original acabe sendo também responsável pela manutenção de uma parte específica do código, considere o fato de que o desenvolvedor pode deixar a organização.

Além disso, um desenvolvedor pode ter necessidade de revisitar seu próprio código depois de algum tempo, apenas ter esquecido suas complexidades. Em alguns casos, um desenvolvedor pode nem se lembrar de primeiro que eles foram os desenvolvedores originais do código! Código sustentável beneficia quem precisa mantê-lo, mesmo que seja o desenvolvedor original.

A capacidade de manutenção também afeta a facilidade com que um sistema de software pode sofrer engenharia reversa. Lá pode ser a necessidade de um sistema de software sofrer engenharia reversa, possivelmente para que possa ser migrado para uma tecnologia mais recente. Uma arquitetura que exibe um alto nível de modificabilidade será mais fácil de entender e raciocinar, tornando mais fácil a engenharia reversa.

4.5. Tipos de manutenção de software

A manutenção do software é realizada por vários motivos, como correção de defeitos, melhorando a qualidade de alguma forma ou atendendo a novos requisitos. Como resultado, o

software o trabalho de manutenção pode ser categorizado nos seguintes tipos diferentes de software manutenção:

- Corretivo
- Perfeito
- Adaptativo
- Manutenção preventiva
- Manutenção corretiva

A manutenção corretiva é o trabalho envolvido na análise e correção de defeitos no Programas. Embora não seja o único tipo de manutenção, é o tipo que as pessoas associam a maioria com trabalho de manutenção. Os defeitos podem ser encontrados internamente ou por usuários na produção.

A gravidade e a prioridade dos defeitos variam dependendo da natureza do bug. A gravidade representa o nível de impacto que o bug tem na operação do software. As organizações têm vários sistemas de classificação de gravidade, mas categorias como crítico, alto, médio e baixo são exemplos comuns.

A prioridade de um defeito é a ordem em que será corrigido. Normalmente, quanto maior a prioridade, mais rápido será corrigido. Assim como a gravidade, as organizações podem ter diferentes sistemas de classificação por prioridade, mas categorias como alta, média e baixa são comuns.

Outro sistema de classificação comum é P0, P1, P2, P3 e P4, com P0 sendo o mais alta prioridade. Os defeitos de P0 são críticos e considerados bloqueadores. Uma liberação será colocada em espera até que todos os defeitos P0 sejam corrigidos.

A capacidade de manutenção pode ser medida pelo tempo que leva para analisar e corrigir um defeito específico. Níveis mais altos de capacidade de manutenção permitem que essas tarefas sejam concluídas em uma quantidade menor de Tempo.

4.5.1. Manutenção perfeita

A manutenção perfeita é necessária quando o software precisa implementar novos ou requisitos atualizados. Esses tipos de mudanças são principalmente focados na funcionalidade de o software. Um exemplo de manutenção perfeita é um novo aprimoramento do software sistema.

O software que tem um nível mais alto de manutenção permitirá que mudanças perfeitas sejam feitas com menos esforço e, portanto, a um custo total inferior.

4.5.2. Manutenção adaptativa

A manutenção adaptativa é definida como o trabalho necessário para adaptar um sistema de software para mudanças no ambiente de software. Exemplos disso podem ser adaptar o sistema de software para um novo sistema operacional (SO), uma nova versão dele SO ou para usar um novo banco de dados sistema de gestão (DBMS).

O tempo que leva para adaptar um sistema de software às mudanças no ambiente é uma medida da capacidade de manutenção do software.

4.5.3. Manutenção preventiva

O objetivo das tarefas de manutenção preventiva é prevenir problemas no futuro, por aumentando a qualidade. Isso pode incluir a melhoria dos atributos de qualidade do sistema de software, como aumentar a capacidade de manutenção e confiabilidade.

A manutenção preventiva pode ser realizada para facilitar a manutenção no futuro. Um exemplo disso é a refatoração de um componente de software para torná-lo menos complexo.

4.6. Modificabilidade

A capacidade de modificação é um aspecto da manutenção. Modificabilidade é a facilidade com que as mudanças podem ser feitas no software sem introduzir defeitos ou reduzir a qualidade. É um atributo de qualidade importante porque há uma série de razões pelas quais o software precisa ser mudado.

Alguns softwares podem permanecer úteis na produção por anos ou mesmo décadas. Inevitavelmente, isso o código precisará ser modificado para os diferentes tipos de manutenção que foram descritos anteriormente. O tempo necessário a partir do momento em que uma mudança é especificada até que seja implementada é uma indicação da modificabilidade do sistema.

No mundo de hoje, as metodologias ágeis de desenvolvimento de software são as mais comuns. Esses projetos de software adotam mudanças. Além da nova funcionalidade, cada iteração do projeto pode envolver mudanças no código existente. Melhorar a capacidade de modificação não é apenas benéfico para manutenção, mas também para todo o desenvolvimento do software.

4.7. Extensibilidade e flexibilidade

Extensibilidade e flexibilidade são características adicionais relacionadas à manutenção. O nível de extensibilidade reflete o quão fácil é estender ou aprimorar o sistema de software. Os sistemas de software que são projetados para serem extensíveis levam em consideração o crescimento futuro antecipando a necessidade de adicionar novas funcionalidades.

A flexibilidade é semelhante, mas se concentra principalmente em como é fácil alterar uma capacidade para que ela possa ser usada de uma maneira que não foi projetada originalmente. Tanto a extensibilidade quanto a flexibilidade são características que ditam o nível de facilidade com que alguém pode realizar a perfeição manutenção.

4.7.1. Escopo de modificações

Nem todas as mudanças no software são iguais, e o escopo de uma modificação específica é um fator em como a mudança será difícil de implementar. Quanto maior e mais complexo a modificação, maior será o esforço para concluí-la.

Além do tamanho, se as mudanças exigirem mudanças no nível de arquitetura, isso aumentará o nível e escala do esforço envolvido. Alguns componentes e suas interações podem precisar ser refatorado extensivamente para grandes mudanças.

4.7.2. Projetando para sustentabilidade

A fim de projetar uma arquitetura de software que exiba capacidade de manutenção, você deve reduzir a dificuldade em implementar mudanças. Facilitando a implementação de mudanças, em grande parte, significa reduzir a complexidade da arquitetura e seus componentes.

Níveis mais altos de complexidade tornam os sistemas de software mais difíceis de entender, testar e manter. Por esse motivo, o nível de complexidade é uma medida preditiva de alguma qualidade atributos, incluindo manutenibilidade.

As evidências mostram que os módulos com maior complexidade também são mais propensos a conter erros. O que torna esta situação pior é que esses módulos também são mais difíceis de testar, o que significa que é mais provável que um erro não seja detectado.

Embora fazer medições e projetar arquiteturas para diminuir a complexidade pode requerer mais tempo (e, portanto, custam mais dinheiro) durante o desenvolvimento, a economia de custos ser realizada a longo prazo devido à maior qualidade e um maior nível de manutenção.

Algumas técnicas usadas para reduzir a complexidade e aumentar a capacidade de manutenção são:

- Reduzir tamanho
- Aumente a coesão
- Reduzir o acoplamento
- Reduzindo o tamanho

Módulos grandes tendem a ser mais complexos e são responsáveis por uma parcela maior de a lógica. Portanto, quanto maior for um módulo, mais difícil pode ser alterá-lo. Sua o design deve procurar reduzir o tamanho dos módulos individuais. Uma maneira de fazer isso seria dividir um módulo em vários.

4.7.3. Aumentando a coesão

A coesão representa o quão inter-relacionados os elementos em um módulo específico são. Desenhos deve procurar aumentar a coesão, não permitindo que um módulo em particular tenha muitos elementos díspares. Aumentar a coesão reduz a complexidade e permite uma série de atributos de qualidade diferentes, incluindo facilidade de manutenção. Alta coesão geralmente se correlaciona com acoplamento solto.

4.7.4. Reduzindo o acoplamento

O acoplamento refere-se à dependência dos diferentes módulos uns dos outros. Projetos deveriam procure um acoplamento fraco, de modo que os diferentes módulos sejam independentes uns dos outros ou quase independente. Se for necessário fazer alterações em um módulo altamente acoplado, é mais provável que outros módulos sejam afetados e, portanto, também exijam alterações. Solto o acoplamento reduz a complexidade e permite uma série de atributos de qualidade diferentes, incluindo manutenibilidade. O acoplamento fraco geralmente está relacionado à alta coesão.

4.7.5. Medindo a capacidade de manutenção

Uma série de métricas de software pode ajudar a medir a complexidade do software, e, portanto, também a manutenção do software. Apesar de medir a complexidade e a manutenção pode ser difícil, existem métricas de software que podem fornecer percepções sobre o nível de complexidade e facilidade de manutenção do seu software.

4.7.5.1. Linhas de código (LOC)

Uma dessas métricas de software são as linhas de código (LOC), também conhecidas como linhas de código-fonte (SLOC). Esta medição simplesmente representa o tamanho de um sistema de software, determinando o número de linhas em seu código-fonte.

Normalmente, os sistemas de software com mais linhas de código são maiores em complexidade e mais difícil de manter do que aqueles com menos linhas de código. No entanto, as comparações de LOC entre diferentes sistemas de software são realmente úteis apenas se envolverem uma ordem de diferença de magnitude nas linhas de código. Por exemplo, se um sistema de software tem 50.000 linhas de código, e outro tem 48.000 linhas de código, você não será capaz de fazer uma determinação sobre qual software é mais fácil de manter. No entanto, se você comparasse um sistema de software com 10.000 linhas de código com uma que tem 100.000 linhas de código, então você está mais propenso a fazer uma determinação útil.

Ferramentas de desenvolvimento e ambientes de desenvolvimento integrado (IDEs) podem contar com o número de linhas de código para você. No entanto, existem diferentes maneiras de calculá-lo, e é discutível qual é o mais útil. Também é um desafio se um sistema de software usa vários idiomas.

As duas formas principais de calculá-lo são o LOC físico e o LOC lógico. O LOC físico normalmente é apenas uma contagem de todas as linhas do código-fonte, excluindo comentários e linhas. O LOC lógico leva em consideração o número real da linguagem do programa declarações em uma tentativa de contar apenas linhas eficazes de código. Enquanto o LOC físico é mais fácil de calcular, é mais sensível a ser afetado por coisas como espaçamento entre linhas e outra formatação.

4.7.5.2. Complexidade ciclomática

A complexidade ciclomática é uma métrica de software quantitativa que reflete a complexidade de um módulo de software. Foi desenvolvido por Thomas J. McCabe e às vezes é referido como A complexidade ciclomática de McCabe. Ele mede o número de caminhos linearmente independentes por meio de um módulo ou elemento de design detalhado. Complexidades ciclomáticas mais altas indicam que o software é mais complexo.

A complexidade ciclomática pode ser calculada de algumas maneiras diferentes, mas uma das mais comum é usar a seguinte fórmula:

$$CC = E - N + 2P$$

Uma explicação para a fórmula anterior é a seguinte:

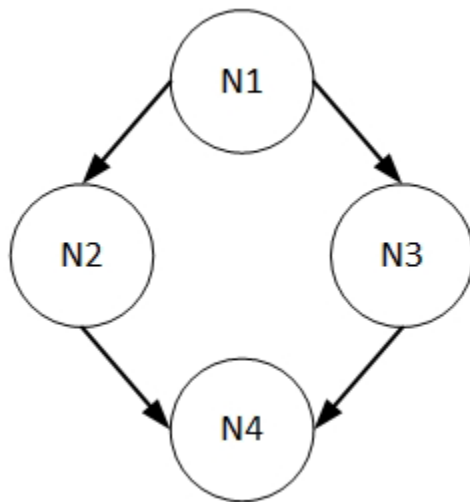
- CC = complexidade ciclomática
- E = o número de arestas do gráfico
- N = o número de nós do gráfico
- P = o número de partes conectadas no gráfico

Dado um gráfico de fluxo de controle criado examinando o código ou simplesmente examinando o código sem criar um gráfico de fluxo de controle, você deve primeiro contar o número de nós e arestas. Os nós são um único bloco de código com instruções executadas em uma sequência sem saltos. As bordas representam o fluxo de controle entre os nós.

Por exemplo, suponha que temos o seguinte pseudocódigo representando um simples estrutura if/then/else:

```
if (N1)
  then N2
  else N3
end if
N4
```

O gráfico de fluxo de controle para ele seria semelhante ao seguinte:

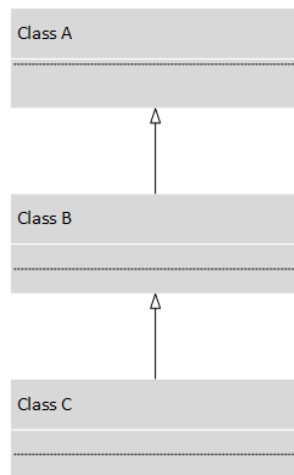


Há apenas um ponto de entrada (N1) e um ponto de saída (N2) para este exemplo, então há apenas um caminho ($P = 1$). Existem quatro arestas e quatro nós, então a complexidade ciclomática é $4 - 4 + (2 \times 1) = 2$.

Os valores de complexidade ciclomática maiores que 10 normalmente indicam módulos que são complexos, tornando-os mais sujeitos a erros e mais difíceis de testar. As equipes devem buscar refatorar módulos complicados para diminuir sua complexidade ciclomática.

4.7.5.3. Árvore de profundidade de herança (DIT)

A profundidade da árvore de herança (DIT) é uma métrica de código que é específica para orientação a objetos programação. Ele mede o comprimento máximo entre um nó e o nó raiz em uma hierarquia de classes. Dê uma olhada no seguinte diagrama de uma hierarquia de classes simples:



Neste exemplo, a Classe C herda da Classe B, que herda da Classe A. O DIT de A classe A é 0, o DIT da classe B é 1 e o DIT da classe C é 2. Um número maior para o DIT indica um maior nível de complexidade. Isso também significa que há um número maior de atributos e métodos sendo herdados, o que significa que há reutilização de código por meio herança, mas pode tornar mais difícil prever o comportamento. Um número menor para o DIT indica menos complexidade, mas também pode significar que há menos reutilização de código por meio herança.

Herança é um conceito poderoso em programação orientada a objetos e projetos devem esforçar-se para reutilizar o código. Portanto, não é que o DIT deva ser sempre reduzido ao mínimo. Um equilíbrio deve ser alcançado entre a reutilização de código e a complexidade. Como regra geral, se o DIT é maior que 5, você pode querer analisar as razões para isso e possivelmente reduzir complexidade por refatoração para que a árvore não seja tão profunda.

4.7.5.4. Usabilidade

Usabilidade descreve como é fácil para os usuários realizarem as tarefas necessárias usando o software sistema. A satisfação do usuário está diretamente relacionada ao seu nível de usabilidade. Os usuários são muito mais provavelmente ficarão satisfeitos com um sistema de software se for fácil de usar e fornecer um bom usuário experiência. A percepção dos usuários sobre a qualidade geral do sistema de software também será superior. Aumentar a usabilidade pode ser uma das maneiras mais fáceis e baratas de melhorar a qualidade de um sistema de software.

Usabilidade é um atributo de qualidade importante porque o software não é suficientemente utilizável diminuirá a produtividade dos usuários. Pior ainda, os usuários podem não querer usar o software em absoluto. Se o software for um site da Web e for difícil de usar, difícil de navegar, lento para carregar ou difícil de ler, os usuários simplesmente começarão a usar uma alternativa.

4.7.6. Permitindo que os usuários concluam suas tarefas com eficiência

Os sistemas de software que apresentam usabilidade permitem que os usuários concluam suas tarefas com eficiência. Uma vez os usuários aprenderam a usar o sistema, a velocidade com que executam ações reflete a usabilidade do sistema de software.

4.7.6.1. Aprendizagem

Aprendizagem é o grau em que novos usuários podem aprender como usar o software de maneira eficaz sistema. É definido na ISO/IEC 25010, que é uma série de padrões que se concentram em um número das características de qualidade do produto. Isso reflete como é

fácil para os usuários atingirem seus objetivos de aprender a usar o sistema de software. Além de novos usuários, a capacidade de aprendizagem também inclui como é fácil para usuários experientes aprenderem novas funcionalidades que são adicionadas ao sistema.

Se um sistema de software for utilizável, seu nível de aprendizagem será mais alto. O sistema deve ser intuitivo, de modo que não leve muito tempo ou esforço para aprender uma parte da funcionalidade isso faz parte do sistema de software.

4.7.6.2. Fornecendo feedback útil

Os sistemas de software utilizáveis ajudam os usuários evitando que cometam erros, minimizando o impacto dos erros e o fornecimento de feedback. Exemplos de feedback útil incluem:

- Validação apropriada e mensagens úteis quando a validação falha
- Mensagens amigáveis e informativas para o usuário
- Dicas de ferramentas
- Feedback para processos de longa duração, como uma barra de progresso ou notificações quando as operações assíncronas são concluídas com sucesso ou falham por algum motivo

4.7.6.3. Acessibilidade

Acessibilidade é o aspecto da usabilidade que fornece recursos que tornam mais fácil para pessoas com deficiências ou deficiências para usar o software de forma eficaz. Isso poderia potencialmente significar pensar sobre a visão (cegueira parcial ou total, daltonismo), audição (surdo ou com deficiência auditiva) e deficiências físicas (não ser capaz de digitar ou usar o mouse).

Projetar para acessibilidade pode incluir coisas como:

- Tornar o software utilizável ao usar apenas um teclado (sem mouse)
- Fornece suporte para tecnologias assistivas, como ampliadores de tela, tela leitores, software de conversão de texto em voz, teclados alternativos e dispositivos apontadores
- Garantir conteúdo não textual (por exemplo, controles, mídia baseada em tempo e CAPTCHAs) tem uma alternativa em texto
- Projetar o software para que haja maneiras de ajudar os usuários a navegar no software
- Fornecer aos usuários tempo suficiente para ler e usar a página da web ou tela (para por exemplo, permitindo que os usuários ajustem ou desativem os limites de tempo e forneça aos usuários a capacidade de pausar ou interromper o conteúdo, como atualização automática ou rolagem de conteúdo)
- Usar cores que levam em consideração o daltonismo
- Garantir que haja uma ordem de tabulação lógica para controles e campos de entrada

Uma chave para o sucesso nesta área é incluir pessoas com deficiência no requisito, design e processos de teste de usabilidade. O feedback deles pode ser extremamente útil em avaliar o quão acessível seu sistema de software é para eles.

4.7.6.4. A usabilidade precisa ser considerada durante requisitos

Durante a engenharia de requisitos, use a técnica de observação de elicitação de requisitos que foi mencionado anteriormente para observar como os usuários realizam seu trabalho. Isso inclui o processo que eles passam, bem como o software atual que eles usam (se esse software for relevante para aquele que está sendo construído). Por meio dessa técnica, você pode colher o que funciona para os usuários, e tão importante quanto, o que não funciona.

Outras técnicas de elicitação de requisitos, como entrevistas em que você pode fazer perguntas, pode fornecer informações sobre usabilidade também. Os resultados da elicitação de requisitos podem ser usados para criar requisitos relacionados à usabilidade.

4.7.6.5. Testando usabilidade

Assim que uma versão funcional do software estiver disponível para teste, o teste de usabilidade deve tomar lugar. Algumas das mesmas técnicas usadas para elicitação de requisitos também podem ser usadas para obter feedback de usabilidade. Providencie para que os usuários operem o software e realizem tarefas com enquanto observa o que funciona bem e quais os pontos problemáticos que eles podem ter. Usar entrevistas e grupos de foco para obter feedback dos usuários sobre o que gostaram e o que não gostaram sobre o software.

4.7.6.6. Design visual atraente

A estética, ou aparência visual, do sistema de software, é um componente da usabilidade. A aparência do aplicativo pode ter um grande efeito sobre os usuários, por isso vale a pena gastando tempo garantindo que o aplicativo seja projetado para ser visualmente atraente para os usuários.

Para aplicativos da web, há uma série de opções de design que podem ser feitas para aumentar usabilidade. Ao projetar aplicativos da web, considere o seguinte:

- Dê ênfase à legibilidade, incluindo o uso de cabeçalhos, espaçamento adequado, legibilidade fontes, cores atraentes e texto formatado apropriadamente
- Certifique-se de que os layouts de todas as páginas estão bem pensados
- Mantenha o conteúdo conciso evitando texto excessivo
- Certifique-se de que a navegação e os menus não sejam excessivamente complicados para evitar torná-los difícil para os usuários encontrarem o que desejam
- Certifique-se de que nenhum link está quebrado
- Fornece dicas de ferramentas, como usar o atributo title em hyperlinks para que o texto pop-up permitirá que os usuários saibam mais sobre um link
- Seja consistente em todo o aplicativo com coisas como cores, ícones, fontes, e legendas/termos
- Exibir indicadores de progresso quando o usuário tiver que esperar por uma resposta, como um aguarde animação ou um indicador de porcentagem concluída

A seguir estão alguns itens a serem considerados ao projetar interfaces de usuário para Windows aplicativos de desktop:

- Projete com espaçamento adequado de controles para que eles não fiquem muito próximos ou muito distantes de um para o outro
- Dimensionar janelas e controles, como botões, menus suspensos, grades e visualizações de lista apropriadamente com base no conteúdo esperado

- Se uma janela for redimensionada e o conteúdo estiver truncado, certifique-se de que mais conteúdo é mostrado conforme a janela fica maior
- Considere definir um tamanho mínimo de janela se o conteúdo não puder mais ser usado em um tamanho particular
- Rotule cada controle e cada grupo de controles
- Certifique-se de que a ordem das guias dos controles em uma janela está correta
- Designe as teclas de atalho (uma única tecla ou combinação de teclas) que permitirão aos usuários executar rapidamente comandos executados com frequência
- Atribuir teclas de acesso (teclas alfanuméricas usadas com a tecla Alt) para que os usuários possam ativar rapidamente um comando
- Garanta a capitalização adequada nos cabeçalhos, controles, rótulos e conteúdo da janela
- Forneça dicas úteis
- Seja consistente em todo o aplicativo com coisas como cores, ícones, fontes, e legendas/termos
- Fornece feedback ao usuário sobre o status do sistema quando ele está ocupado com um longo período processo, como mostrar uma barra de progresso
- O aplicativo deve exibir mensagens informativas para o usuário, como mensagens de confirmação, aviso e erro, quando apropriado

4.7.6.7. Fornecendo um bom sistema de ajuda

Projetar um sistema que seja altamente utilizável também significa que ele deve ser projetado para permitir usuários para aprender facilmente como usá-lo. Um sistema de ajuda completo e atualizado torna um aplicativo mais fácil de aprender. Esteja você desenvolvendo um aplicativo desktop ou baseado na web um, certifique-se de ter um sistema de ajuda que seja extenso o suficiente para o dado aplicativo para que seja útil para as pessoas. Dependendo do software, quem o está usando, e como está sendo usado, um sistema de ajuda geral pode incluir dicas de ferramentas, ajuda online, produto manuais, tutoriais, perguntas frequentes (FAQs), uma base de conhecimento/discussão fórum, eventos de treinamento e maneiras de enviar casos de suporte. Isso não quer dizer que um sistema de ajuda seja a principal forma de tornar um aplicativo utilizável.

O aplicativo deve ser intuitivo e fácil de usar com base em outros fatores, mas um sistema de ajuda apropriado complementa o esforço geral para aumentar a capacidade de aprendizagem e usabilidade.

4.7.6.8. O software deve ser útil e não apenas utilizável

Lembre-se de que, independentemente de quão utilizável o software seja, ele deve ter utilidade. Para sistema para ter utilidade, deve fornecer a funcionalidade de que seus usuários precisam. Ser utilizável é ótimo, mas para ser útil, também deve ter utilidade. Os usuários devem ser capazes de realizar seus objetivos e o software deve servir ao seu propósito.

O sistema de software está atendendo às necessidades dos usuários? É útil para eles? Mesmo se o sistema de software fornece uma experiência visual agradável, sem utilidade não será uma boa experiência de usuário.

4.7.7. Disponibilidade

A disponibilidade descreve o grau em que o sistema de software funcionará conforme necessário quando é necessário. É a probabilidade de que o sistema de software esteja operando corretamente quando necessário para um usuário e não está passando por tempo de inatividade não planejado devido a falha e reparo. É normalmente medido em termos de noves, como em quantos noves representam a disponibilidade nível (99,9%, 99,99% ou 99,999%).

4.7.7.1. Calculando a disponibilidade com base no tempo

A disponibilidade pode ser calculada com a seguinte fórmula:

$$\text{Disponibilidade} = \text{MTBF} / (\text{MTBF} - \text{MTTR})$$

MTBF é o tempo médio entre falhas, que é o tempo médio entre duas falhas de o sistema. MTTR é o tempo médio para reparo, que é o tempo médio para solucionar problemas e repare o sistema de volta ao seu estado operacional. Apenas interrupções não planejadas são consideradas ao calcular o tempo de inatividade do sistema.

Para alcançar alta disponibilidade, que é considerada cinco noves, ou 99,999% de disponibilidade, seu sistema de software pode ficar inativo por apenas 5 minutos e 15 segundos ou menos durante o curso de um ano inteiro! A tabela a seguir mostra alguns exemplos de cálculos de disponibilidade:

Availability	Downtime per year	Downtime per month	Downtime per week
99.00%	3.65 days	7.2 hours	1.68 hours
99.90%	8.76 hours	43.2 minutes	10.1 minutes
99.99%	52.6 minutes	4.32 minutes	60.5 seconds
100.00%	5.26 minutes	25.9 seconds	6.05 seconds

Níveis extremos de disponibilidade não são necessariamente ideais porque, em certo ponto, você vai experimentar retornos decrescentes. Considere que cada nove adicional requer uma toda melhoria da magnitude da disponibilidade. Isso tem um custo em termos de tempo e dinheiro.

Isso pode ter um efeito negativo no software porque limitará quanto tempo e o esforço pode ser gasto em outras coisas, como o lançamento de novos recursos. Além disso, considere o fato de que um usuário não será capaz de dizer a diferença entre 99,99% e Disponibilidade de 99,999%. Por exemplo, no caso de um aplicativo da web ou móvel, existem outros fatores como o dispositivo do usuário e a rede em que o usuário está, que podem não ser tão confiável quanto o sistema de software.

O arquiteto de software e as partes interessadas precisam pesar os benefícios de uma maior disponibilidade níveis contra o aumento dos custos de fornecê-los para determinar qual nível é adequado para eles melhor.

4.7.7.2. Calculando a disponibilidade com base no sucesso da solicitação avaliar

Se o seu software for distribuído globalmente, talvez aproveitando os serviços em nuvem fornecidos por empresas como Amazon, Microsoft ou Google, você pode estar experimentando disponibilidade muito alta, de forma que pelo menos parte do tráfego esteja sendo servido a qualquer momento. Como um resultado, calcular a disponibilidade com base no tempo, como fizemos anteriormente, pode não ser muito significativo.

Um cálculo alternativo seria basear a disponibilidade na taxa de sucesso da solicitação. Isso pode ser calculado da seguinte forma:

Disponibilidade = Requisições com Sucesso/Total de Requisições

Seu sistema de software terá uma variedade de solicitações, algumas das quais são mais importantes do que outras, mas olhar para todas as solicitações deve fornecer uma boa estimativa de tempo de inatividade.

Os cálculos de disponibilidade com base no tempo levam em consideração a duração total da falha, incluindo o tempo para reparar o sistema de software. Não faz distinção entre, por exemplo, duas falhas de 30 minutos e uma falha de uma hora, mas um cálculo baseado em a taxa de sucesso da solicitação sim. A consideração de ambas as métricas fornecem insights sobre facetas diferentes da disponibilidade do seu software, o que lhe dá uma imagem mais completa.

4.7.7.3. Falhas, erros e falhas

Uma parte da disponibilidade envolve como um sistema de software pode lidar e superar as falhas para que a duração de uma interrupção não planejada não exceda o valor especificado ao longo de um determinado período.

Uma falha de sistema é uma característica de um sistema de software que pode levar, por conta própria ou em conjunto com outras falhas, para uma falha do sistema. Existe uma falha de sistema em algum lugar do código. Um erro é um estado incorreto do sistema de software causado por uma ou mais falhas. Uma falha do sistema é um evento vivido por um usuário no qual o sistema de software não se comporte conforme o esperado. As falhas do sistema são causadas por uma ou mais falhas. A fim de lidar e superar as falhas e prevenir falhas, podemos tentar detectar falhas, recuperar de falhas ou prevenir falhas.

4.7.7.4. Detectando falhas

Antes que um sistema de software possa se recuperar de uma falha, ele deve primeiro detectar uma falha. A seguir são algumas técnicas que podem ser usadas para detectar falhas.

4.7.7.4.1. Resposta de ping/eco

Neste método de detecção de falhas, um componente, atuando como monitor do sistema, envia uma Solicitação de eco do protocolo de mensagens de controle da Internet (ICMP) para outro componente (faz ping o componente) e aguarda uma resposta de eco ICMP. Se o alvo não responder ao ping em um período predefinido, o componente atuando como o monitor do sistema reporta que o outro componente falhou.

4.7.7.4.2. Batimento cardíaco

Esta técnica requer um componente para enviar periodicamente uma mensagem (pulsção) para indicar que está operando normalmente. Se o componente de escuta não receber a mensagem de pulsção dentro do período predefinido, ela determina que uma falha do sistema ocorreu e toma as medidas adequadas.

4.7.7.4.2. Timestamp

Esta estratégia de detecção de falhas concentra-se em capturar uma sequência incorreta de eventos. Por usando um carimbo de data/hora, ou mesmo apenas uma sequência de números, as falhas podem ser relatadas se o a sequência está na ordem incorreta.

4.7.7.4.3. Votação

Um sistema de votação pode ser usado para relatar falhas. Uma dessas abordagens é modular tripla redundância (TMR), às vezes chamada de redundância de modo triplo. Ele utiliza três componentes para executar o mesmo processo.

A lógica de votação irá comparar os resultados para produzir uma única saída. Se todos os três componentes produzirem a mesma saída, então tudo funcionou conforme o esperado. Se dois de três componentes concordarem, eles podem corrigir a falha vencendo o terceiro componente. Uma falha é em seguida, relatado no terceiro componente.

4.7.7.4.4. Teste de sanidade/verificação de sanidade

Esta técnica de detecção de falhas usa um teste para avaliar se o resultado de um processo é razoável e possível. Se o teste determinar que o resultado não é, ele pode relatar uma falha. Esse teste é um simples, e não se destina a testar uma ampla variedade de problemas. É usado para detectar óbvio falhas de forma rápida.

4.7.7.4.5. Monitoramento de condição

O monitoramento de condição é um método pelo qual as condições são verificadas em um sistema de software em a fim de detectar uma falha ou uma situação na qual uma falha pode se desenvolver. Quando um problema é detectado, a falha pode ser tratada.

Um benefício do monitoramento de condição é que ele pode detectar um problema antes que uma falha ocorra, ou pelo menos antes de se tornar um problema ainda maior.

4.7.7.4.6. Autotestes

Os sistemas de software podem incorporar autotestes, permitindo que seus componentes detectem falhas, panes. Esse teste às vezes é referido como um autoteste integrado (BIST) ou um teste integrado (PEDAÇO). Se esta técnica for usada, componentes de software ou combinações de componentes são desenvolvidos com a lógica que lhes dá a capacidade de se testar para uma operação correta.

Os autotestes podem ser iniciados pelo componente que estão testando ou por um componente separado atuando como um monitor do sistema. Se um teste falhar, a falha será relatada.

4.7.7.5. Recuperando-se de falhas

Uma vez que as falhas são detectadas, uma estratégia pode ser empreendida para recuperá-las. A seguir são algumas maneiras de se recuperar de falhas.

4.7.7.5.1. Manipulação de exceção

Uma vez que uma exceção é detectada, o sistema de software pode utilizar um tratamento de exceção mecanismo. Como uma exceção é tratada depende da linguagem, estrutura e tipo de exceção, mas pode envolver qualquer coisa, desde simplesmente retornar um código de erro para retornando uma instância de alguma classe de erro que contém informações úteis sobre a exceção, como um código de erro, uma mensagem e um rastreamento de pilha.

O software pode usar informações de exceção para se recuperar potencialmente da falha, como corrigindo a causa da exceção e tentando novamente a operação, e/ou mostrando ao usuário uma mensagem amigável sobre o problema.

4.7.7.5.2. Estratégia de nova tentativa

Falhas transitórias são erros que ocorrem devido a algumas condições temporárias, como rede, problemas de conectividade, indisponibilidade temporária de um serviço, tempo limite de um serviço ou falhas no nível da infraestrutura.

Estratégias de repetição podem ser usadas para tentar repetir uma operação quando ela encontra um culpa. Uma política de nova tentativa pode ser adaptada com base na natureza do componente que causou o erro, que pode ditar coisas como quantas tentativas de repetição são feitas e a duração de tempo de espera entre as tentativas.

Alguns tipos de intervalo de repetição comuns incluem:

- Intervalos regulares: O sistema de software espera a mesma quantidade de tempo em entre cada tentativa de intervalo.
- Intervalos incrementais: O sistema de software espera um curto período antes a primeira tentativa e, em seguida, aumenta gradativamente a quantidade de tempo antes de cada nova tentativa subsequente. Por exemplo, as tentativas de repetição podem ocorrer em 2 segundos, 6 segundos, 12 segundos e assim por diante.
- Retirada exponencial: O sistema de software espera um curto período antes a primeira tentativa e, em seguida, aumenta exponencialmente a quantidade de tempo antes de cada nova tentativa subsequente.
- Nova tentativa imediata: uma nova tentativa pode ocorrer imediatamente. No entanto, não deve ser várias tentativas imediatas de repetição. Se uma única tentativa de repetição imediata falhar, então, todas as tentativas subsequentes devem usar um dos outros tipos de intervalo.
- Randomização: Qualquer um dos tipos de intervalo acima mencionados pode ser usado em conjunto com a randomização para evitar que novas tentativas sejam enviadas no ao mesmo tempo a partir de várias instâncias de um cliente.

4.7.7.5.3. Níveis variáveis de redundância

Uma maneira de se recuperar de falhas e obter disponibilidade é ter um mecanismo de failover. Existem vários níveis de redundância que podem ser fornecidos.

Em ambientes ativos/hot spare, cada componente tem outro que executa o mesmo processo com as mesmas entradas, de modo que, se um falhar, o outro componente pode assumir em a qualquer momento. O failover é geralmente transparente porque o tempo de recuperação é quase instantâneo.

Um ambiente de reposição passivo/quente é aquele em que apenas os componentes ativos desempenham processos de entradas, mas os componentes ativos fornecem os componentes de backup com atualizações periódicas de estado. Uma abordagem de redundância passiva não está tão disponível quanto uma active/hot spare, mas é menos caro para operar. Dependendo de quão frequentes as atualizações de estado são fornecidas para o backup, o tempo de recuperação pode ser em termos de segundos ou minutos.

Com uma abordagem cold spare, os componentes redundantes são mantidos fora de serviço até que sejam precisava. O componente que falhou ficará fora de operação até que possa ser reparado ou substituído. Usar um cold spare leva mais tempo para trazer o componente redundante para operação em comparação com peças sobressalentes quentes ou mornas. O tempo de recuperação pode levar algumas horas.

4.7.7.5.4. Rollback

A técnica de reversão retorna o sistema a um ponto de verificação no qual o sistema é conhecido por estiveram em bom estado. Isso exige que os pontos de verificação sejam persistidos de alguma maneira.

Depois que o sistema for revertido, as operações regulares podem continuar mais uma vez. Esta abordagem pode ser usada com redundância ativa ou passiva para que um componente com falha possa ser feito ativo novamente após uma reversão.

4.7.7.5.5. Degradação graciosa

A degradação normal é uma abordagem de recuperação de falhas em que alguma funcionalidade é feita indisponível para evitar que todo o sistema se torne inutilizável. Se as falhas forem impedindo que todo o sistema de software esteja operacional, então algumas funções podem ser caiu em favor de outros. Por exemplo, se um sistema está com poucos recursos, a maioria funções críticas podem ser mantidas funcionando enquanto outras são desligadas.

4.7.7.5.6. Ignorando o comportamento defeituoso

Outra abordagem para o tratamento de falhas é simplesmente ignorar a falha. Se um tipo particular de falha de uma determinada fonte é conhecido como aquele que pode ser ignorado, então o sistema pode simplesmente ignore a falha e o processamento pode continuar.

4.7.7.6. Prevenção de falhas

Uma alternativa para detectar e se recuperar de falhas é evitar que ocorram em o primeiro lugar. Existem várias estratégias que podem ser empregadas para prevenir falhas.

4.7.7.6.1. Remoção do serviço

Uma maneira de evitar falhas é remover o componente ofensivo de serviço. Um software o componente pode ser removido da operação em antecipação a falhas. Ele pode então ser restaurado quando estiver operacional novamente.

4.7.7.6.2. Transações

As transações podem ser usadas para evitar falhas. Várias etapas em um processo podem ser agrupadas juntos em uma transação, de modo que, se uma etapa falhar, todo o pacote pode ser desfeito. Essa abordagem pode evitar que os dados sejam salvos em um estado incorreto, ou impedir a corrida condições quando mais de um processo tenta acessar ou alterar os mesmos dados.

4.7.7.6.2. Aumento de conjuntos de competências

O conjunto de competências para um determinado componente em um sistema de software determina quais estados e condições que ele pode lidar. O nível de tolerância a falhas de um componente e os casos em que é projetado para lidar com dependem do conjunto de lógicas

que compõem seu conjunto de competências. UMA componente pode ser modificada para que ele trate mais casos, reduzindo as exceções que são jogados.

Por exemplo, se um método não manipular um valor nulo para um parâmetro específico, uma exceção pode ser lançada, levando a uma falha do sistema. O código pode ser modificado para lidar com valores nulos, aumentando o conjunto de competências do software e evitando uma falha.

4.7.7.6.3. Prevenção de exceção

Essa estratégia envolve escrever o código de forma que as exceções possam ser evitadas. Para exemplo, os métodos podem realizar a verificação de limites em argumentos e tratá-los elegantemente, o que evitaria que exceções fossem lançadas.

4.7.7.7. Portabilidade

A portabilidade descreve a eficiência e eficácia com que um sistema de software pode ser transferido de um ambiente para outro. Alguns fatores que influenciam a portabilidade são adaptabilidade, instalabilidade e substituíbilidade.

4.7.7.8. Adaptabilidade

Adaptabilidade é o grau em que um sistema de software pode ser adaptado para diferentes ambientes, como diferentes hardwares, sistemas operacionais ou outras características. A fim de testar a adaptabilidade, o teste funcional deve ser realizado para garantir que o sistema de software pode executar todas as suas tarefas em todos os ambientes de destino.

4.7.7.9. Instalabilidade

A instalabilidade é a facilidade com a qual um sistema de software pode ser instalado ou desinstalado em um ambiente especificado. O processo de instalação deve ser fácil de entender e pode fornecer opções de configuração como parte da instalação. Por exemplo, o processo pode solicitar ao usuário para configurar coisas como o local para a instalação, banco de dados informações de conexão e outras opções de configuração do software.

Outro aspecto da instalabilidade é como o software lida com um processo de atualização/atualização. O sistema de software deve fornecer um processo de atualização amigável e utilizável para que um usuário possa atualize o software para uma versão mais recente. O processo de atualização deve limpar a antiga versão, possivelmente desinstalando-o automaticamente primeiro.

A instalabilidade também abrange qualquer funcionalidade para desinstalar o aplicativo. Quando um sistema de software é desinstalado, deve remover o sistema de software e quaisquer componentes inteiramente da máquina. Ele também pode excluir quaisquer pastas que não sejam mais precisava.

Um recurso que você pode considerar incluir que aumenta a capacidade de instalação está fornecendo aos usuários a capacidade de cancelar um processo de instalação, atualização ou desinstalação depois de iniciado. Idealmente, o processo será limpo após si mesmo de forma adequada se o usuário o abortar ou se ele falhar.

O teste de instalabilidade deve garantir que todas as funcionalidades relacionadas à instalação, atualizar e desinstalar o software funciona corretamente e sem erros. Deve testar se todas as opções de configuração funcionam corretamente e verifique como o processo lida com

situações como quando há uma quantidade insuficiente de espaço em disco disponível para instalar o software ou uma atualização.

4.7.7.10. Substituibilidade

Substituibilidade é a capacidade de um sistema de software de substituir outro sistema de software para a mesma finalidade, no mesmo ambiente. Isso pode implicar apenas a substituição de um ou mais componentes de software dentro de um sistema de software.

Um bom exemplo de capacidade de substituição é quando um sistema de software é atualizado. Desejo dos clientes um sistema que é altamente substituível para que uma atualização para uma versão mais recente do software de uma versão mais antiga corra bem.

O teste de substituibilidade deve confirmar que, após qualquer substituição, o sistema de software ainda funciona corretamente. Todas as funcionalidades relevantes devem ser testadas para verificar se funciona como esperado.

4.7.7.11. Internacionalização e localização

Internacionalização e localização fazem parte da portabilidade e consistem em adaptar um sistema de software para uso com diferentes idiomas, levando em consideração diferenças e atender a outros requisitos para vários locais.

Como arquiteto de softwares, você precisará saber se há internacionalização/localização requisitos, para que você possa considerá-los no projeto. Mesmo se não houver requisitos atualmente, se sua organização tem objetivos de negócios para negócios internacionais expansão, o software pode eventualmente ser usado em diferentes áreas ao redor do mundo. Se isso é o caso, você deve considerar esse fato em seu projeto inicial, pois é mais difícil fornecer esse recurso mais tarde.

Internacionalização é projetar software de forma a permitir a localização e suporte para diferentes idiomas, culturas e regiões. A internacionalização neste contexto é frequentemente abreviada para i18n, onde 18 é o número de letras entre i e n na palavra internacionalização. Às vezes, a palavra globalização é usada em vez de internacionalização. Quando for esse o caso, às vezes é abreviado como g11n.

Como parte da internacionalização, o software deve ser projetado de forma a adaptá-lo a diferentes localidades posteriormente não exigirão modificações no código. Por exemplo, todas strings traduzíveis, como legendas, mensagens, dicas e outros conteúdos, devem ser colocados em arquivos de recursos. O código pode então fazer referência a esses arquivos de recursos quando precisa que conteúdo. Mais tarde, ao localizar o aplicativo, essas strings podem ser traduzidas para diferentes locais sem ter que alterar nenhum código.

Unicode deve ser usado como o conjunto de caracteres para aplicativos que requerem internacionalização, uma vez que suporta todos os personagens de qualquer linguagem escrita no mundo todo. Se uma equipe de desenvolvimento fosse usar conjuntos de caracteres diferentes para diferentes idiomas, localizar um aplicativo seria muito mais difícil. Unicode irá facilitar exibindo caracteres de qualquer idioma que precise ser suportado, e porque há uma codificação única para cada caractere, os dados podem ser classificados, pesquisados e manipulados da mesma maneira.

Uma vez que um aplicativo está pronto para ser localizado, a seguir estão algumas coisas que precisam ser considerados:

- Tradução de legendas, mensagens, dicas e outros conteúdos para outro idioma
- Garantir que a interface do usuário possa acomodar as palavras adequadamente após serem
- traduzido (por exemplo, o espaçamento e a quebra de linha podem ser diferentes)
- Variação na grafia das palavras, mesmo para países que usam as mesmas idiomas, como avião/avião, maiúsculas/maiúsculas e organizações/organizações
- Diferenças de idioma e culturais podem exigir que o conteúdo seja reescrito para garantir que nada será mal interpretado ou interpretado de maneira errada
- Se o texto é lido da esquerda para a direita ou da direita para a esquerda
- Diferenças nos números de telefone
- Diferenças em pesos e medidas
- Diferentes formatos de data/hora, como se 01/06/2019 deve ser interpretado como 1 de junho ou 6 de janeiro
- Diferenças de fuso horário (UTC)
- Números de formatação, como símbolos de marcas decimais, separadores de grupos de dígitos e agrupamentos de dígitos
- Diferentes moedas, como símbolos monetários e se o símbolo monetário deve aparecer antes ou depois do valor monetário
- Quaisquer requisitos legais

4.7.7.12. Manter a portabilidade

Uma vez que um sistema de software é tornado portátil para diferentes ambientes, é importante que a portabilidade é mantida. Conforme as alterações são feitas no software como parte da manutenção, é possível que essas modificações possam afetar a portabilidade.

Ao fazer modificações, deve-se considerar se isso afetará ou não portabilidade. Durante o teste, deve-se verificar se as alterações feitas não afetaram adversamente afetar quaisquer requisitos de portabilidade.

4.7.7.13. Interoperabilidade

Interoperabilidade é o grau em que um sistema de software pode trocar e usar informações de outro sistema de software. Para que dois sistemas interoperem em um de forma útil, eles devem ser capazes de se comunicarem entre si (sintática interoperabilidade), bem como ser capaz de interpretar as informações trocadas de uma forma significativa e forma correta (interoperabilidade semântica).

O termo interoperabilidade está relacionado a, e às vezes é usado de forma intercambiável com, o termo integração.

4.7.7.13.1. Desafios com interoperabilidade

Há uma série de dificuldades em fornecer interoperabilidade entre as existentes ou novas sistemas de software. Ao lidar com os requisitos relacionados à interoperabilidade, mantenha-se em atenção às seguintes armadilhas em potencial:

- Mesmo quando um sistema de software segue um determinado padrão ao desenvolver interoperabilidade com outro sistema de software, o nível desejado de

interoperabilidade pode não ser atendida devido aos dois sistemas que interpretam as especificações de padrões de maneiras diferentes.

- A interoperabilidade entre dois sistemas de software às vezes é reduzida ou colocada em espera devido ao envolvimento de um sistema legado existente, seja diretamente ou indiretamente, quando há um desejo de manter a compatibilidade com o legado sistema. Uma organização pode não querer fazer modificações no legado sistema.
- Os testes de interoperabilidade às vezes são insuficientes e podem perder algumas questões de interoperabilidade.
- Os sistemas de software às vezes são lançados mesmo quando são conhecidas questões de interoperabilidade.
- Mesmo que a interoperabilidade seja alcançada entre dois sistemas de software, pode ser difícil de manter à medida que novas versões dos dois sistemas de software são lançadas. Isto não é incomum que a interoperabilidade seja interrompida, exigindo manutenção para mantê-lo.
- Pode haver problemas legais para interoperabilidade, portanto, esteja ciente de qualquer um que possa afetar seu sistema ao interoperar com outro. Isso pode ser particularmente verdadeiro se os dois sistemas de software são propriedade de organizações diferentes, e ainda mais portanto, se operam em jurisdições diferentes, como dois países diferentes. exemplo disso pode ser as leis de privacidade e segurança relacionadas à saúde informações sendo trocadas.

4.7.7.13.2. Localizando e trocando informações com outro sistema

Para alcançar a interoperabilidade entre dois sistemas de software, o sistema do consumidor deve ser capaz de localizar o outro e os dois sistemas devem ser capazes de trocar informações em uma forma semanticamente significativa.

O sistema do consumidor pode ou não precisar descobrir o outro sistema em tempo de execução. Se isso precisa acontecer em tempo de execução, um serviço de diretório conhecido é pesquisado por um ou mais atributos (por exemplo, um URL) para localizar o sistema. Deve-se notar que lá pode haver n níveis de direção durante o processo de localização. Por exemplo, uma vez que um local é encontrado, pode levar a outro local e assim por diante, até que o serviço seja descoberto.

Assim que o sistema for localizado, a resposta deve ser tratada. O serviço irá enviar de volta uma resposta ao solicitante, encaminhe uma resposta para outro sistema ou transmita uma resposta a quaisquer serviços interessados que possam estar ouvindo.

Para que os dois sistemas troquem informações, táticas de interoperabilidade, como orquestração e gerenciamento de interfaces. A orquestração envolve direção e gerenciar os serviços que são chamados e garantir que as etapas necessárias ocorram na sequência correta.

A fim de gerenciar interfaces para facilitar a troca de informações, recursos podem ser adicionados ou removidos para fins de interoperabilidade. Um recurso pode ser adicionado a uma interface especificamente para permitir a troca de dados. Um exemplo disso pode ser a funcionalidade relacionada ao armazenamento em buffer de dados. Além disso, os recursos podem ser removidos de uma interface se não deseja que esses recursos sejam expostos a outros sistemas. Por exemplo, funcionalidade para excluir certos dados podem ser removidos da interface se não quisermos que nenhum cliente externo tem essa capacidade.

4.7.7.13.3. Padrões de interoperabilidade

Quando a interoperabilidade envolve sistemas de software de diferentes organizações, uma das maneiras de alcançar a interoperabilidade é seguir um padrão comum. O padrão pode ser criado pelas duas organizações trabalhando juntas como parte de uma parceria, ou pode ser uma indústria existente, padrão nacional, internacional ou aberto.

Em alguns casos, toda uma indústria trabalhará em conjunto para chegar a um padrão. O padrão pode ser aquele recém-criado pela indústria, ou algum padrão existente pode ser usado como ponto de partida. O padrão de interoperabilidade acordado pode ser seguido durante o curso de desenvolvimento e usado durante o teste.

Outro aspecto dos padrões que pode facilitar a interoperabilidade é o uso de uma comum tecnologia, como acordo sobre o formato de intercâmbio de dados e a comunicação protocolo. Por exemplo, pode-se concordar que JavaScript Object Notation (JSON) é usado como formato de intercâmbio de dados e que Hypertext Transfer Protocol Secure (HTTPS) é usado como protocolo de transferência da web.

4.7.7.13.4. Teste de interoperabilidade

Além de testar a unidade e o sistema dos dois sistemas que devem interoperar, é fundamental para realizar testes de integração entre os dois sistemas. Os testes devem ser realizados para garantir que o sistema do consumidor pode localizar o outro sistema e que eles podem trocar informações corretamente. O ambiente de teste deve ser tão próximo do que será experimentado na produção quanto possível. Lembre-se de que, mesmo que ambos os sistemas sigam um padrão específico para interoperabilidade, e mesmo que ambos os sistemas tenham passado em seus próprios testes de conformidade baseados nesses padrões, não é incomum haver problemas com interoperabilidade. Alguns os problemas podem não ser descobertos até que os dois sistemas sejam testados juntos.

4.7.7.14. Testabilidade

Testabilidade é o grau em que um sistema de software oferece suporte a testes em seu determinado contexto. Quanto maior o nível de testabilidade, mais fácil será o sistema de software e seus componentes teste. Se um componente não é fácil de testar, pode indicar que o design não é ideal, levando a uma implementação desnecessariamente complexa. Uma parte significativa do desenvolvimento custos para sistemas de software estão relacionados a testes, portanto, se a arquitetura de software pode desempenhar um papel na testabilidade, pode haver benefícios de custo perceptíveis.

Um sistema de software que tem um nível mais alto de testabilidade torna mais fácil testar, o que resulta em um aumento tanto na eficiência quanto na eficácia dos testes. Eficiência de teste aumenta porque leva menos tempo e esforço para criar e executar testes. Testando a eficácia é aumentada porque você tem mais probabilidade de encontrar defeitos existentes no sistema de software, e é mais provável que você os encontre mais cedo.

Encontrar defeitos mais cedo tem um efeito tremendo no nível geral do sistema de software de qualidade. Não só é mais provável que os defeitos não cheguem à produção, mas quanto mais cedo um defeito é encontrado, menos caro será para consertar.

Alguns fatores que influenciam a testabilidade são os seguintes:

- Controlabilidade

- Observabilidade
- Isolabilidade
- Automatabilidade
- Complexidade do software

4.7.7.14.1. Controlabilidade

A controlabilidade representa o nível até o qual é possível controlar o estado do componente sendo testado. O componente que está sendo testado às vezes é chamado de sistema em teste (SUT) ou o componente em teste (CUT). Controlando o estado de um componente envolve ser capaz de ditar as entradas do componente, bem como o nível para que essas entradas exercem suas capacidades.

Ao projetar componentes, você deve procurar aumentar sua controlabilidade, pois este aumenta a testabilidade.

4.7.7.14.2. Observabilidade

A observabilidade representa o nível até o qual é possível observar o estado do componente sendo testado. Isso inclui ser capaz de observar entradas e saídas para que possamos poder determinar se o componente está funcionando corretamente.

Se não for possível para a estrutura de teste que está sendo usada para ver as entradas e saídas de um componente, então o componente não é observável, e não será possível confirmar se os resultados estão corretos.

Ao projetar componentes, você deve procurar aumentar sua observabilidade, pois é diretamente correlacionada à testabilidade.

4.7.7.14.3. Isolabilidade

Isolabilidade é o grau em que um componente pode ser isolado. O objetivo é ter testes que pode se concentrar em peças específicas de funcionalidade (por exemplo, testes de unidade) que não têm dependências de outros componentes. Isso nos permite criar e executar testes em um componente concluído, mesmo se outros componentes não estiverem completos.

Queremos evitar uma situação em que tenhamos que escrever uma grande quantidade de código antes de qualquer coisa pode ser testado, pois é desejável obter feedback o mais rápido possível. Se um problema é encontrado, será mais fácil determinar sua fonte e causa se os componentes exibem isolabilidade. Um aumento na isolabilidade aumenta a testabilidade de um componente.

4.7.7.14.4. Automatabilidade

Automatabilidade é o nível em que um processo ou ação pode ser automatizado. Se um sistema exhibe automatização, então testes automatizados podem ser criados e executados para o sistema.

O teste automatizado utiliza testes pré-programados que podem ser executados automaticamente. Esses os testes podem ser executados a qualquer momento, como antes de um check-in do código ou antes de ocorrer uma compilação. Se houver compilações automatizadas, os testes automatizados podem ser executados como parte desse processo. Os testes automatizados fornecem um feedback rápido sobre se um novo defeito foi introduzido no sistema. Como acabamos de notar, descobrir um defeito tão perto de quando foi introduzido é altamente benéfico. Ao projetar um sistema de software, você deve procurar aumentar a automatização, já que a capacidade de ter testes automatizados aumenta a testabilidade.

4.7.7.14.5. Complexidade do software

Como é o caso de outros atributos de qualidade, a complexidade do software desempenha um papel na testabilidade. Reduzindo dependências e isolando módulos (isolabilidade), diminui a complexidade do software.

Quando discutimos a sustentabilidade, mencionamos que minimizar a complexidade de elementos arquitetônicos podem ser alcançados reduzindo o número de linhas de código em módulos, aumentando a coesão e reduzindo o acoplamento. Técnicas que aumentam a manutenibilidade também aumenta a controlabilidade dos elementos. Isso tem um efeito direto e positivo impacto na testabilidade.

Além da complexidade estrutural dos componentes, existe a complexidade comportamental. Não determinismo é um tipo de complexidade comportamental. Se algum algoritmo que estamos testando for não determinístico, então isso significa que dadas as mesmas entradas, ele pode exibir um comportamento diferente com cada execução. Isso está em contraste com um algoritmo determinístico, que, se dado as mesmas entradas, irão se comportar da mesma maneira cada vez que for executado.

O código não determinístico é mais difícil de testar, então a primeira etapa seria identificar quaisquer áreas no sistema de software que são não determinísticas. Para tais áreas, a abordagem ideal, se possível, seria refatorar a lógica para torná-la determinística, ou permitir que a lógica seja ridicularizada como determinístico.

Um exemplo simples frequentemente visto na prática envolve testes e a hora atual. Na maioria linguagens de programação, existe uma maneira de obter a data e hora atuais do dia. Fazendo assim pode acoplar firmemente seu código ao ambiente do host. É não determinístico porque cada vez que fizermos uma ligação para saber a data e hora atuais, o resultado será diferente. Se houver lógica que usa este valor de uma forma que afetaria um teste de unidade, os resultados do teste irão variar devido à natureza não determinística do resultado. Esta lógica pode ser refatorada para envolver o ligue para obter a data e hora atuais em alguma outra aula. Isso permitirá que você injete essa dependência na classe que precisa dessa lógica. Nesse ponto, uma estrutura de teste pode simular a dependência, especificando qual será o resultado quando a chamada para obter o atual data e hora são feitas.

No entanto, há casos em que não será possível refatorar a lógica para torná-la determinística, ou torná-la de modo que a lógica possa ser ridicularizada como determinística. Um exemplo de isto é, se você tivesse um sistema multi-thread que é necessário para interagir com um externo componente, e esse componente externo gera eventos de uma maneira não determinística.

4.7.7.14.6. Importância da documentação de teste

Um aspecto da testabilidade é a facilidade com que os testes podem ser executados, incluindo testadores que não escreveu originalmente os casos de teste. A capacidade de reutilização e manutenção dos testes, parte da testabilidade, são melhorados com a existência de documentação de teste de qualidade. Quando se tratar com um sistema grande, pode ser difícil lembrar de todas as regras de negócios e alternativas caminhos relacionados a um determinado cenário.

Com metodologias ágeis de desenvolvimento de software, a documentação excessiva é vista como um risco do projeto, em vez de algo que irá reduzir o risco. As equipes de projeto buscam ser eficientes com documentação e produza apenas o necessário. Além disso, testes automatizados deve ser bastante auto documentado por meio de coisas como os nomes dos métodos de teste, classe de teste nomes e comentários.

No entanto, existem muitos tipos diferentes de teste além dos testes automatizados que precisam ser conduzidos, e alguns podem ser bastante envolvidos. Pode haver complexidade na lógica de negócios, nas entradas e saídas de testes, nos dados que são necessários para configurar testes específicos, e para garantir uma cobertura adequada em uma variedade de cenários de negócios. Alguma forma de teste a documentação pode realmente ser benéfica.

Os recursos podem, e muito provavelmente, mudar com o tempo. Um funcionário pode deixar uma organização ou testadores simplesmente recebem atribuições diferentes ao longo do tempo e um pedaço de a funcionalidade não é necessariamente propriedade de um único testador. A documentação de teste permite testadores diferentes para usar o caso de teste ao longo do tempo e facilita essas transições para que a qualidade não é sacrificada.

Para adicionar a esse ponto, é cada vez mais comum para os departamentos de desenvolvimento terceirizar trabalho de desenvolvimento, incluindo testes. Não só você pode ter que lidar com terceirizados recursos mudando, mas as tarefas de teste podem precisar passar de recursos internos para recursos terceirizados e vice-versa.

A documentação de artefatos, como designs e testes, é muito útil para que novos recursos podem adquirir velocidade rapidamente. É basicamente o conceito de conhecimento tribal e ser capaz de passe adiante as coisas que foram aprendidas anteriormente para outras pessoas. Projetos com bom a documentação normalmente demonstra um nível mais alto de maturidade organizacional e contribuir para o sucesso geral do projeto.

4.7.7.15. O que é um bom testador?

Enquanto estamos no tópico de testabilidade, gostaria de abordar as características de um bom testador. Como arquiteto de softwares, você pode fornecer orientação aos testadores. Bons testadores executar suas tarefas com um maior nível de eficiência, eficácia e meticulosidade.

Testadores que operam de forma eficiente o fazem de forma organizada e sua eficiência permite-lhes para concluir casos de teste com mais rapidez e encontrar defeitos com mais rapidez. Eles não estão apenas usando o software e encontrando bugs, mas usando uma abordagem sistemática para encontrar defeitos.

Os testadores que alcançam um maior nível de eficácia o fazem porque se concentram em atenção aos problemas com os quais os usuários se preocupam na versão lançada. Parte do processo de encontrar defeitos envolve documentar o problema e fornecê-los aos desenvolvedores para que eles podem ser consertados.

Testadores excepcionais são meticulosos em seu trabalho. Eles planejam suas atividades de teste e eles documentar seus casos de teste. Os testadores tentam ser o mais meticulosos possível, exercitando todos os aspectos da funcionalidade de um software e considerando uma variedade de cenários. Eles usam uma variedade de entradas e coisas de teste, como casos extremos, que são cenários que envolvem valores extremos de entrada, como aqueles no mínimo ou máximo de uma gama de possibilidades.

Outra característica de um bom testador é a capacidade de entender o comportamento do software, ambientes e recursos. Porque um testador não pode testar todas as entradas e cenários, um testador de software eficaz deve compreender o comportamento do software. Esse inclui o que o software está fazendo e quais coisas podem causar sua falha.

O software opera em um ambiente que interage com uma variedade de entradas e saídas. Existe a interface do usuário (IU) com a qual as pessoas interagem, mas também existe o kernel interface (sistema operacional), interfaces de software (por exemplo, em coisas como banco de dados sistemas) e interfaces do sistema de arquivos (por exemplo, erros relacionados ao acesso, leitura e gravar em arquivos). Um bom testador deve considerar a totalidade do ambiente em que o software opera.

Finalmente, o testador deve compreender os recursos do software. Embora possa haver um número limitado de recursos básicos (por exemplo, aceitação de entrada, produção de saída, armazenamento de dados e execução de cálculos), esses recursos podem ser combinados em funcionalidade. Um testador adequado deve considerar todos esses recursos para maximizar a probabilidade de que quaisquer defeitos existentes sejam encontrados.

4.7.7.16. Resumo

Os arquitetos de software devem prestar atenção especial aos atributos de qualidade, pois influenciam a arquitetura do software. O software deve atender aos atributos de qualidade designados, para identificá-los e especificá-los de uma forma que possam ser medidos e testados é importante.

Embora algumas partes interessadas possam se concentrar na funcionalidade, requisitos não funcionais tais como atributos de qualidade são um fator principal para saber se um sistema de software será ou não um sucesso. Alguns dos atributos de qualidade mais importantes incluem facilidade de manutenção, usabilidade, disponibilidade, portabilidade, interoperabilidade e testabilidade.

Agora que entendemos mais sobre os atributos de qualidade e o fato de que eles influenciam arquitetura de software, estamos prontos para explorar o design de arquitetura de software. Arquitetura design envolve a tomada de decisões para criar uma solução para requisitos funcionais, qualidade atributos e restrições. Você aprenderá o que está envolvido com o design de arquitetura, sobre princípios de design que você pode aproveitar em seus designs e sobre design de arquitetura processos.

5. Design de Arquiteturas de Software

O design da arquitetura de software é uma etapa fundamental na construção de sistemas de software de sucesso, e está capítulo começa explorando o que é design de arquitetura de software e por que é importante em um projeto de software.

Existem duas abordagens principais para o design de arquitetura: a de cima para baixo e de baixo para cima abordagens. Cada abordagem tem vantagens e desvantagens e, neste capítulo, você aprenda como selecionar a melhor abordagem para um determinado projeto. Projetar um software e arquitetura pode ser desafiadora, mas vamos dar uma olhada nos princípios de design e soluções que podem ser aproveitadas em um design.

Os processos de design de arquitetura fornecem orientação aos arquitetos de software para garantir que um o design atende aos requisitos, cenários de atributos de qualidade e restrições. Este capítulo cobre as atividades que são normalmente realizadas como parte de um processo de design de arquitetura e, em seguida, fornece uma visão geral de quatro processos: design orientado por atributos (ADD), A técnica da Microsoft para arquitetura e design, o método de design centrado na arquitetura (ACDM) e o método de desenvolvimento de arquitetura (ADM).

O capítulo será concluído explicando como usar um backlog de arquitetura para priorizar trabalhar e acompanhar o andamento dos projetos de arquitetura.

Neste capítulo, cobriremos os seguintes tópicos:

- Projeto de arquitetura de software
- A importância do design de arquitetura de software
- Abordagens de design de cima para baixo versus de baixo para cima
- Sistemas de software greenfield versus brownfield
- Drivers arquitetônicos
- Aproveitando os princípios de design e as soluções existentes
- Documentar o projeto de arquitetura de software
- Usando uma abordagem sistemática para design de arquitetura de software
- Design orientado por atributos (ADD)
- Técnica da Microsoft para arquitetura e design
- Método de design centrado na arquitetura (ACDM)
- Método de desenvolvimento de arquitetura (ADM)
- Acompanhar o progresso do design da arquitetura de software

O design da arquitetura de software envolve a tomada de decisões a fim de satisfazer requisitos, atributos de qualidade e restrições. É um processo de resolução de problemas que leva para a criação de um projeto de arquitetura.

O projeto de arquitetura de software compreende a definição das estruturas que irão compor a solução e documentando-os. As estruturas de um sistema de software são compostas por elementos e as relações entre os elementos. As propriedades e comportamentos dos elementos que são expostos publicamente, por meio de uma interface, devem ser identificados como parte do Projeto. O design permite que você entenda como os elementos se comportam e interagem

com uns aos outros. Implementações privadas dos elementos não são arquitetonicamente significativas e não precisa ser considerado como parte do projeto.

O design da arquitetura de software serve como orientação técnica para o desenvolvimento e normalmente ocorre iterativamente até que a arquitetura inicial esteja em um ponto onde o desenvolvimento equipe pode começar seu trabalho. Depois que uma arquitetura inicial é projetada, ela pode continuar a evoluir à medida que o desenvolvimento está ocorrendo. Por exemplo, podem ocorrer iterações de design adicionais para refatorar uma arquitetura para atender a novos requisitos ou atributos de qualidade.

O design da arquitetura de software é um processo criativo. Os arquitetos de software têm o privilégio de chegando a soluções para problemas complexos e pode usar a criatividade para fazê-lo. Pode ser uma das partes mais divertidas e recompensadoras de um projeto de software.

5.1. Tomando decisões de design

O conjunto de requisitos de software consiste em uma série de problemas de design que devem ser resolvidos. Para cada um desses problemas de design, como fornecer certas funcionalidades de negócios, respeitando uma restrição específica, atingindo os objetivos de desempenho ou fornecendo um certo nível de disponibilidade, pode haver várias maneiras de resolver o problema. Você precisará considerar os pontos fortes e fracos dessas alternativas, a fim de selecionar a mais adequada escolha.

Uma grande parte do design de arquitetura de software é tomar decisões de design para resolver problemas, que uma solução pode ser implementada. Como arquiteto de software, você liderará o processo de tomada de decisão.

É um processo colaborativo e geralmente os melhores projetos incorporam conhecimento e feedback de várias pessoas, como outros arquitetos de software e experientes desenvolvedores. Projetos conjuntos e revisão da arquitetura com outros são benéficos em vir com um design de arquitetura de software sólido.

O resultado do design é um conjunto de decisões que moldam a arquitetura de seu software. O design é documentado em artefatos que podem ser usados para a implementação de uma solução. Os arquitetos de software devem ter em mente que uma decisão tomada para um problema de design pode afetar outro. É por isso que o design da arquitetura de software é um processo iterativo.

Cada decisão para um problema de design pode não ser ideal para outro problema, mas o geral solução deve ser aceitável atendendo a todos os requisitos.

O perfeito é inimigo do bom, aforismo que tem origem no pensamento dos franceses. filósofo, Voltaire e outros, é aplicável ao design de arquitetura de software. UMA o projeto concluído pode não ser perfeito, pois haverá requisitos conflitantes que precisam ser atendidos e compensações feitas a fim de atendê-los. Se o design satisfizer todos os requisitos, então é um bom, mesmo que não seja perfeito.

5.2. Termos de design de arquitetura de software

Antes de prosseguirmos, vamos definir alguns dos termos que usaremos enquanto detalhando o processo de design de arquitetura de software. Esses termos podem variar, dependendo a organização e a equipe. Independentemente dos termos usados, o importante é que eles são usados de forma consistente pelos membros da equipe e que são compreendidos por toda a equipe membros.

Para os fins deste livro, usaremos os termos estrutura, elemento, sistema, subsistema, módulo e componente.

5.2.1. Estrutura

Estruturas são agrupamentos de elementos e relações entre eles. Qualquer coisa que seja complexa e composto por elementos pode ser referido como uma estrutura. Definimos software anteriormente arquitetura, em parte, por dizer que é composta pelas estruturas, seus elementos e o relações desses elementos entre si.

5.2.2. Elemento

Um elemento é um termo genérico que pode ser usado para representar qualquer um dos seguintes termos: sistema, subsistema, módulo ou componente. Se quisermos nos referir a peças de um software aplicação de uma forma geral, podemos nos referir a eles como elementos.

5.2.3. Sistema

O sistema de software representa todo o projeto de software, incluindo todos os seus subsistemas. Um sistema consiste em um ou mais subsistemas. É o nível mais alto de abstração em um projeto de arquitetura de software.

5.2.4. Subsistema

Os subsistemas são agrupamentos lógicos de elementos que constituem um sistema maior. O os subsistemas podem ser criados de várias maneiras, incluindo o particionamento de um sistema por funcionalidade.

Embora não precisem ser, os subsistemas podem representar software autônomo formulários. Um sistema de software geral pode ser composto de vários subsistemas e qualquer número deles pode ser um aplicativo independente. Esses aplicativos autônomos podem ser aplicativos externos que não foram desenvolvidos pela organização.

Organizar um sistema de software maior em subsistemas diminui a complexidade e permite desenvolvimento de software para ser mais bem gerenciado. Em alguns casos, um ou mais desenvolvimento equipes podem ser formadas para cada subsistema. Cada subsistema é composto por um ou mais módulos.

5.2.5. Módulo

Módulos, como subsistemas, são agrupamentos lógicos de elementos. Cada módulo está contido dentro de um subsistema e consiste em outros módulos e/ou componentes. Eles são tipicamente focados em uma única área lógica de responsabilidade.

As equipes de desenvolvimento atribuídas a um determinado subsistema serão responsáveis pelos módulos que compõem esse subsistema.

5.2.6. Componente

Os componentes são unidades de execução que representam algumas funcionalidades bem definidas. Elas normalmente encapsulam sua implementação e expõe suas propriedades e comportamentos por meio de uma interface.

Os componentes são o menor nível de abstração e normalmente têm um relativamente pequeno alcance. Os componentes podem ser agrupados para formar elementos mais complexos, como módulos.

5.3. A importância da arquitetura de software Projeto

Uma arquitetura de software é a base de um sistema de software. O design da arquitetura é significativa para a qualidade e o sucesso de longo prazo do software. Um adequado o design determina se os requisitos e atributos de qualidade podem ser satisfeitos.

Há uma série de razões pelas quais um bom design de arquitetura de software é fundamental para construção de software útil. Nesta seção, exploraremos os seguintes motivos:

- O design da arquitetura de software é quando as principais decisões são feitas em relação à arquitetura.
- Evitar decisões de design pode incorrer em dívidas técnicas.
- Um projeto de arquitetura de software comunica a arquitetura a outras pessoas.
- O design fornece orientação para os desenvolvedores.

O impacto do design da arquitetura de software não se limita a questões técnicas preocupações. Também influencia as partes não técnicas do projeto.

5.4. Tomando decisões importantes

É durante o design da arquitetura de software que as principais decisões são feitas que irão determinar se os requisitos, incluindo atributos de qualidade, podem ser satisfeitos. Arquitetura de software habilita ou inibe os atributos de qualidade, então as decisões de design desempenham um grande papel na decisão ou não eles podem ser encontrados.

Algumas das primeiras decisões são feitas durante o design. Se essas decisões precisam mudar, é mais fácil e menos caro mudar as decisões de arquitetura antecipadamente, antes mesmo de a codificação começou, do que fazer alterações mais tarde.

5.5. Evitar decisões de design pode incorrer em dívida de aspectos técnicos

As decisões críticas são feitas durante o projeto e, por esse motivo, há um custo para qualquer atrasar uma decisão de projeto ou não tomar nenhuma. Atrasar ou evitar determinado projeto decisões podem incorrer em dívidas técnicas.

A dívida técnica é semelhante à dívida financeira. No contexto do design, é o custo e esforço para o trabalho adicional que será necessário posteriormente devido às decisões que são feitas agora, ou porque as decisões não foram feitas.

Além de atrasar ou evitar decisões, uma decisão pode ser tomada sabendo que custar alguma quantia de dívida técnica. Como arquiteto de software, você pode decidir fazer um caminho mais fácil para uma solução, incorrendo em dívida técnica, embora haja uma solução melhor. Como é o caso da dívida financeira, a dívida técnica nem sempre é ruim. Às vezes, você vai querer pagar uma dívida depois para conseguir algo agora. Por exemplo, projetar uma melhor solução de longo prazo pode levar mais tempo e esforço, e você pode decidir por uma solução que leva menos tempo para colocar o software em produção para tirar vantagem de um mercado oportunidade.

Pode ser difícil medir com precisão o impacto da dívida técnica. Tenha em mente que em além do tempo e esforço que pode ser necessário mais tarde para tomar uma decisão que é feito ou evitado agora, o débito técnico pode ter outras repercussões negativas. Por exemplo, um design que não é ideal, levando a níveis mais baixos de modificabilidade e extensibilidade, pode prejudicar a capacidade da equipe de fornecer outras funcionalidades. Este é um custo adicional que deveria ser adicionado à dívida técnica.

O arquiteto de software precisa levar todos esses fatores em consideração ao decidir se deve ou não incorrer em dívida técnica.

5.6. Comunicar a arquitetura a outras pessoas

Os resultados do design da arquitetura permitem que você comunique a arquitetura do software para outros. Haverá uma variedade de pessoas que estarão potencialmente interessadas no design da arquitetura.

O projeto também irá melhorar as estimativas de custo e esforço, uma vez que influencia quais tarefas serão necessários para a implementação. Compreender a natureza do trabalho que temos pela frente e quais tipos de tarefas serão necessárias para concluir o projeto ajudarão os gerentes de projeto com seu planejamento. Ser capaz de estimar custo, esforço e os atributos de qualidade que serão atendidos também é útil para propostas de projetos.

5.7. Fornecendo orientação para desenvolvedores

Um projeto de arquitetura de software fornece orientação para a equipe de desenvolvimento, orientando opções de implementação, bem como fornecer treinamento sobre os detalhes técnicos do projeto. O design impõe restrições de implementação, tornando-o importante para tarefas de codificação.

Conhecer o design da arquitetura de software ajuda os desenvolvedores a estarem cientes da implementação escolhas disponíveis para eles, e minimiza a possibilidade de fazer uma decisão de implementação.

Também pode ser usado como treinamento para desenvolvedores. No início do projeto, o desenvolvimento a equipe precisará entender as decisões de design que foram feitas e as estruturas que foram projetados. Criação de projetos detalhados para componentes e implementação eles requerem uma compreensão do design da arquitetura. Se novos desenvolvedores se juntarem à equipe posteriormente, eles também podem usar o design de arquitetura como parte de sua integração.

5.7.1. Influenciando partes não técnicas do projeto

Outra razão pela qual o design da arquitetura de software é importante é o fato de que o design as decisões afetam outros aspectos do projeto de software além da arquitetura. Por exemplo, certas decisões de design de arquitetura podem afetar a compra de ferramentas e licenças, a contratação de membros da equipe, a organização do ambiente de desenvolvimento e como o software será eventualmente implantado.

5.8. Design de cima para baixo versus de baixo para cima aproximações

Existem duas abordagens fundamentais para o design da arquitetura de software. Um é de cima para baixo abordagem de design e a outra é uma abordagem de baixo para cima. Essas estratégias se aplicam a uma variedade de disciplinas, incluindo design de arquitetura de software. Vamos olhar para ambos em Mais detalhes.

5.8.1. Abordagem de cima para baixo

Uma abordagem de cima para baixo começa com todo o sistema no nível mais alto e, em seguida, um processo de decomposição começa a trabalhar para baixo em direção a mais detalhes. O ponto de partida é o nível mais alto de abstração. À medida que a decomposição avança, o design torna-se mais detalhado, até que o nível do componente seja alcançado.

Embora o design detalhado e os detalhes de implementação dos componentes não façam parte do projeto de arquitetura, as interfaces públicas dos componentes fazem parte do projeto. É as interfaces públicas que nos permitem raciocinar sobre como os componentes irão interagir com cada de outros.

Um projeto usando a abordagem de cima para baixo é normalmente executado iterativamente, com aumento níveis de decomposição. É particularmente eficaz se o domínio for bem compreendido.

Esta abordagem sistemática tem sido favorecida pelas empresas, uma vez que pode lidar com grandes e projetos complexos e porque o método de design é planejado. Uma abordagem sistemática para o projeto de arquitetura é atraente para as empresas porque pode ajudar com tempo e orçamento estimativas. No entanto, uma abordagem estrita de cima para baixo, que requer muito projeto de arquitetura tornou-se menos comum na arquitetura de software moderna.

5.8.2. Vantagens da abordagem de cima para baixo

Existem vários benefícios em usar uma abordagem de cima para baixo. É uma abordagem sistemática para projeta e divide o sistema em partes menores. Conforme um sistema é decomposto, ele empresta bem para a divisão do trabalho. Em projetos maiores com várias equipes, este trabalho pode ser dividido entre as equipes.

Conforme ocorre a decomposição adicional, as tarefas podem ser criadas para membros individuais da equipe. Isso oferece suporte ao gerenciamento de projetos na atribuição de tarefas, agendamento e orçamento.

Esse tipo de habilidade de planejamento é atraente para as empresas. As equipes de gestão de as organizações podem preferir, ou mesmo insistir, em uma abordagem de cima para baixo. No início do livro, nós discutimos como, como arquiteto de software, você pode ser solicitado a

ajudar com as estimativas do projeto, e uma abordagem de cima para baixo permitirá que você faça isso com mais precisão.

Embora essa abordagem funcione bem em projetos pequenos e grandes, pode ser particularmente útil para grandes projetos. Ao decompor um sistema em componentes menores, um grande projeto pode se tornar mais gerenciável à medida que o tamanho e a complexidade de cada componente são reduzidos.

5.8.3. Desvantagens da abordagem de cima para baixo

Uma abordagem estritamente de cima para baixo corre o risco de um grande design inicial (BDUF), às vezes referido como um design inicial grande (BUFD). O software é complexo e pode ser difícil criar toda a arquitetura desde o início. Falhas de design ou funcionalidade ausente na arquitetura não pode ser descoberta até mais tarde no processo, quando os componentes são projetados ou implementados. Se mudanças de arquitetura são necessárias em níveis mais altos da arquitetura depois que algum trabalho já estiver concluído, será mais difícil fazer as modificações.

Uma abordagem de cima para baixo funciona melhor quando o domínio é bem compreendido, o que nem sempre é o caso. Muitos projetos começam sem que o domínio seja totalmente compreendido. Mesmo quando é compreendido, as partes interessadas e os usuários às vezes podem não ter certeza sobre o que o software deve fazer e como deve funcionar.

Se várias equipes estão trabalhando em um projeto, cada uma responsável por um subsistema específico ou módulo, o compartilhamento e a reutilização do conhecimento podem ser difíceis com esta abordagem. Cada equipe pode trabalhar de forma independente das demais, o que tem suas vantagens, mas não facilita o compartilhamento de código ou conhecimento. O arquiteto de software pode ter que reconhecer áreas de reutilização, abstrair-las e comunicá-las às equipes. Outra forma de atenuar esse problema é fornecer oportunidades e ferramentas de colaboração para as equipes se comunicarem com cada uma das outras.

Se você usar a abordagem de cima para baixo, tome cuidado para não se tornar um arquiteto de torre de marfim. Se você projetar os níveis mais elevados de uma arquitetura e, em seguida, entregá-los aos desenvolvedores para lidar com o projeto detalhado de nível inferior, é fácil se desligar. Tanto quanto a sua organização e o projeto permitir, faça um esforço para se manter envolvido com a equipe. Se mudanças arquitetônicas são necessárias mais tarde, você já estará familiarizado com a implementação, o que o ajudará a fazer as alterações corretas.

5.8.4. Abordagem de baixo para cima

Em contraste com a abordagem de cima para baixo, a abordagem de baixo para cima começa com os componentes que são necessários para a solução e, em seguida, o design trabalha para cima em níveis mais elevados de abstração. Vários componentes podem ser usados juntos, como a construção de blocos, para criar outros componentes e, eventualmente, estruturas maiores. O processo continua até que todos os requisitos sejam atendidos.

Ao contrário da abordagem de cima para baixo, que começa com a estrutura de alto nível, não há uma abordagem inicial de projeto de arquitetura com abordagem ascendente. A arquitetura emerge como mais o trabalho está concluído. Portanto, isso às vezes é referido como design emergente ou arquitetura emergente.

A abordagem ascendente não exige que o domínio seja bem compreendido, pois a equipe concentra-se apenas em um pequeno pedaço de cada vez. O sistema cresce gradativamente conforme a equipe aprende mais sobre o domínio do problema, bem como a solução.

5.8.5. Vantagens da abordagem ascendente

Uma vantagem de uma abordagem de baixo para cima é o maior nível de simplicidade. A equipe só tem que se concentrar em peças individuais e construir apenas o que é necessário para uma iteração específica. Essa abordagem funciona bem com metodologias de desenvolvimento ágil. Com uma iterativa abordagem que lida com a mudança, a refatoração pode ocorrer para adicionar uma nova funcionalidade ou para alterar a funcionalidade existente. Cada iteração termina com uma versão funcional do software até que, eventualmente, todo o sistema seja construído. Práticas ágeis, como teste de unidade automatizado e integração contínua, são encorajados e podem levar a software de maior qualidade.

Uma abordagem de baixo para cima evita a possibilidade de um grande design inicial, o que pode levar a overdesigning uma solução. Alguns na comunidade ágil sentem que um grande esforço de design frente é tempo perdido e que um design emergente, ou nenhum design inicial (NDUF), seria mais efetivo.

A abordagem ascendente permite que a equipe de desenvolvimento comece a codificar muito cedo no processo, o que também significa que o teste pode ocorrer mais cedo. Isso inclui testes de unidade automatizados como bem como testes manuais por membros da equipe, como analistas de controle de qualidade e outros usuários. Obtendo o feedback no início do processo permite que a equipe identifique quaisquer alterações necessárias com antecedência.

Essa abordagem facilita a reutilização de código. Como a equipe está focada em um número limitado de componentes a qualquer momento, o reconhecimento de oportunidades de reutilização torna-se mais fácil.

5.8.6. Desvantagens da abordagem ascendente

Uma abordagem ascendente ou emergente pressupõe que a mudança é barata. Metodologias ágeis e as práticas fornecem uma abordagem que antecipa a mudança e pode se adaptar a ela. Contudo, dependendo da natureza da mudança, refatorar o design da arquitetura de software pode ser muito dispendioso.

Uma abordagem de baixo para cima, sem arquitetura inicial, pode levar a níveis mais baixos de manutenibilidade. Com a refatoração que pode ser necessária com esta abordagem, os problemas podem surgir. Se a equipe não for diligente, esse problema pode piorar com o tempo.

Todo o escopo do trabalho pode não ser conhecido ao usar essa abordagem. Isso o torna mais difícil de planejar e estimar todo o projeto, o que pode ser inaceitável para a empresa Programas.

Uma das desvantagens da abordagem de cima para baixo é que as falhas de design podem não ser detectado até mais tarde, levando a uma refatoração cara. No entanto, só porque não há inicial design com a abordagem de baixo para cima não o torna imune à descoberta de falhas de design mais tarde no projeto. Pode ser que só depois que a arquitetura surja, certo design as falhas tornam-se aparentes.

5.8.7. Qual abordagem devo usar?

Existem certos fatores a serem considerados ao decidir se o de cima para baixo ou de baixo para cima abordagem é melhor para um projeto de software. Os arquitetos de software podem achar vantajoso use uma abordagem de cima para baixo se mais de uma das opções a seguir for verdadeira:

- O projeto é grande em tamanho
- O projeto é complexo
- O software empresarial está sendo projetado para uma organização
- A equipe é grande ou há várias equipes trabalhando no projeto
- O domínio é bem compreendido

Pode ser mais apropriado usar uma abordagem ascendente se mais de um dos seguintes é verdade:

- O projeto é pequeno em tamanho
- O projeto não é muito complexo
- O software empresarial para uma organização não está sendo projetado
- A equipe é pequena ou há apenas uma equipe
- O domínio não é bem compreendido

Adotar uma abordagem extrema, como fazer um grande projeto de arquitetura inicial ou não projeto de arquitetura em tudo, normalmente não é ideal. Embora algumas situações o levem a selecionar uma abordagem de cima para baixo ou de baixo para cima, os arquitetos de software também devem considerar o uso de uma combinação das duas abordagens. Desta forma, você pode perceber alguns dos benefícios de ambas as abordagens, minimizando as desvantagens.

No início do projeto, ao invés de começar a codificar imediatamente, um top-down abordagem proporcionará a oportunidade de passar pelo menos algum tempo pensando sobre a estrutura geral do projeto. O design de uma arquitetura de alto nível fornece alguns estrutura que pode ser aproveitada para design e desenvolvimento adicional.

Um projeto de arquitetura de alto nível pode ser usado para definir e organizar as equipes, e fornece detalhes para gerenciamento de projetos para que possam realizar a alocação de recursos, programação e planejamento orçamentário. Como arquiteto de software, você pode ser solicitado a fornecer informações sobre esses tipos de atividades de gerenciamento de projeto, e tendo pelo menos um alto nível a arquitetura o ajudará nessas tarefas.

Aqueles que defendem uma abordagem estritamente de baixo para cima, em que a arquitetura emerge da implementação, tendem a pensar que a arquitetura de software inibe a agilidade e a capacidade para fazer alterações no software. No entanto, como foi mencionado no Capítulo 1, O Significado de Arquitetura de software, uma boa arquitetura de software realmente facilita fazer mudanças conforme bem como gerenciá-los. Uma boa arquitetura permite que você entenda o que seria necessário para fazer uma mudança específica.

Usar uma abordagem de cima para baixo para parte do design não requer um grande design inicial. Você pode se concentrar em questões de design arquitetonicamente significativas, e uma vez que um alto nível arquitetura é estabelecida, você pode empregar uma abordagem de baixo para cima. Componentes e módulos baseados na arquitetura de alto nível podem então ser projetados e implementados.

A qualidade do projeto de arquitetura não depende apenas da seleção da correta abordagem. As decisões de design corretas devem ser feitas durante o processo de design, como um Uma abordagem de cima para baixo ou de baixo para cima pode levar a projetos de arquitetura ruins. Um design criado com uma abordagem de cima para baixo pode perder os principais requisitos, o que pode levar a custos refatoração arquitetônica. Um projeto criado com uma abordagem de baixo para cima pode exigir refatoração substancial enquanto a equipe descobre como o sistema de software deve ser estruturada.

Nenhuma abordagem única é aplicável a todas as situações. Cada projeto, organização e equipe é diferente, portanto, a decisão de qual abordagem tomar pode variar. Mesmo com uma abordagem híbrida, a quantidade de design de arquitetura inicial necessária irá variar, então é sobre determinar quanto design é necessário. Isso faz parte do desafio de ser um software arquiteto. Bons arquitetos acabam aprendendo quanto design é apropriado para um determinado situação.

5.9. Software greenfield versus brownfield

Quando você está iniciando o processo de design, uma das primeiras considerações é se você está projetar um sistema greenfield ou brownfield. Os termos greenfield e brownfield são usados em várias disciplinas. É uma analogia com um projeto de construção, e se começará em terrenos verdes, como em terrenos subdesenvolvidos, ou terreno brownfield, referindo-se a terreno que foi desenvolvido anteriormente, mas não está em uso.

5.9.1. Sistemas Greenfield

Um sistema de software greenfield é um aplicativo de software completamente novo, no qual você pode começar do zero. Não há restrições com base em qualquer trabalho anterior. Um greenfield sistema pode ser projetado para um domínio bem conhecido ou para um novo domínio.

Um domínio bem compreendido é aquele que está maduro, e as possibilidades de inovação são muito limitados. Os exemplos incluem aplicativos de desktop do Windows, celular padrão aplicativos e aplicativos da web corporativos. Haverá estruturas, ferramentas e exemplos de arquiteturas para o software que você precisa construir. As arquiteturas de software de os aplicativos existentes podem ser usadas como um guia.

É mais provável que você esteja desenvolvendo software para um domínio bem conhecido e a vantagem é que haverá uma quantidade enorme de conhecimento que você pode aproveitar da experiência de quem criou aplicativos semelhantes.

Um sistema greenfield para um novo domínio também é um novo aplicativo de software que não precisa levar em consideração qualquer trabalho anterior. A diferença entre um sistema greenfield para um domínio maduro e outro para um novo domínio reside no fato de que um novo domínio não é tão bem compreendido e requer muito mais inovação.

Ao contrário de um domínio bem conhecido, você não encontrará tantas informações de suporte para um novo domínio. Em vez de depender de uma infinidade de arquiteturas de referência ou referir-se a um grande base de conhecimento, você se verá gastando tempo construindo protótipos para testar suas soluções.

Para novos domínios, pode ser benéfico projetar um protótipo descartável inicialmente. Estes são protótipos de alguma parte de um sistema de software para que você possa testá-lo, como obter feedback de usuários ou atributos de qualidade de teste. Eles irão ajudá-lo a obter uma compreensão do que tornará uma solução viável para um novo domínio.

Protótipos descartáveis não são construídos para uso a longo prazo, daí o termo descartável, então qualidades como manutenibilidade e reutilização não são o foco de tais protótipos. Se você está usando novas tecnologias, ou tecnologias que ainda não são familiares para você, um protótipo pode ser uma boa maneira de tentar uma solução.

5.9.2. Sistemas brownfield

Um sistema de software brownfield é um sistema de software existente. Se mudar para um existente sistema requer mudanças arquitetônicas, design de arquitetura será necessário. Modificações podem ser necessários para fins como correção de defeitos, implementação de novas funcionalidades, ou alterando a funcionalidade existente.

Mudanças arquitetônicas também podem ser realizadas no software existente para melhorá-lo em alguns maneira sem alterar nenhuma funcionalidade. Por exemplo, uma arquitetura para um existente o sistema de software pode ser refatorado para melhorar um atributo de qualidade específico. A maioria dos tempos, o trabalho em sistemas brownfield não envolve mudanças no atacado no geral arquitetura, a menos que seja necessário um grande retrabalho.

Um dos primeiros passos cruciais para o projeto de arquitetura de software de sistemas brownfield é obter uma compreensão da arquitetura existente. Você precisa entender o geral estrutura, os elementos e as relações entre esses elementos. A partir daí, o design não é tão diferente de um sistema greenfield que passou por algumas iterações para estabelecer uma arquitetura inicial.

5.10. Drivers arquitetônicos

Os drivers arquitetônicos são considerações que precisam ser feitas para o sistema de software que são arquitetonicamente significativos. Eles dirigem e orientam o design da arquitetura de software.

Os drivers arquitetônicos descrevem o que você está fazendo e por que está fazendo. Programas o projeto de arquitetura satisfaz os motivadores arquitetônicos. Os drivers arquitetônicos são entradas no processo de design e incluem:

- Objetivos de design
- Requisitos funcionais primários
- Cenários de atributos de qualidade
- Restrições
- Preocupações arquitetônicas

5.11. Objetivos de design

Os objetivos do projeto se concentram no propósito do projeto de arquitetura específico. Para o design específico em questão, quais são as razões por trás de porque o software está sendo projetado?

Os objetivos do projeto influenciam o projeto e, portanto, são um dos impulsionadores arquitetônicos.

Um objetivo comum de design é projetar uma arquitetura para uma solução, antes de desenvolvimento. O objetivo geral é facilitar a implementação de uma solução que irá satisfazer os requisitos.

Esse tipo de objetivo de projeto pode ser para um sistema greenfield ou brownfield. Como já exploramos, as diferenças entre esses tipos de sistemas podem levar você a foco em diferentes objetivos de design.

Projetar uma arquitetura de software para desenvolvimento não é o único tipo de objetivo de design. Como arquiteto de software, você pode se envolver com propostas de projetos. Por tal atividade de pré-venda, o objetivo do design pode se concentrar em chegar com o software capacidades, o prazo possível para entrega, uma divisão das tarefas de trabalho, e o viabilidade do projeto proposto. Se este for o propósito do projeto, então este tipo de o projeto inicial não será tão detalhado quanto aquele que você está projetando para desenvolvimento.

Para fins de uma proposta de projeto, não será necessário ser tão detalhado quanto você estaria em preparação para o desenvolvimento. Você pode ser obrigado a produzir um projeto para uma proposta de projeto em um curto espaço de tempo para cumprir um determinado prazo de vendas. Além disso, até que a venda seja concluída, você provavelmente não terá os fundos ou tempo para uma venda completa Projeto.

Da mesma forma, os arquitetos de software podem precisar criar um protótipo. Isso pode ser para um projeto proposta, mas também pode ser para testar uma nova tecnologia ou estrutura, para criar uma prova de conceito (POC) para alguma solução para um problema específico, ou para explorar como um determinado atributo de qualidade pode ser atendido de forma eficaz. Tal como acontece com as propostas de projeto, se o objetivo do design é construir um protótipo, o foco e escopo do design da arquitetura de software será diferente daquele que está sendo feito para o desenvolvimento.

É importante manter os objetivos do projeto em mente como um motivador arquitetônico quando o design da arquitetura de software está prestes a começar.

5.12. Requisitos funcionais primários

Outro tipo importante de entrada no projeto de arquitetura é o funcional principal requisitos que precisam ser satisfeitos. Requisitos funcionais primários são aqueles que são crítica para os objetivos de negócios da organização. No Capítulo 3, compreendendo o domínio, nós discutimos os domínios centrais, que se referem à parte do domínio que torna o software vale a pena escrever. Algumas das funcionalidades primárias virão do domínio principal. Isto é o que diferencia a organização dos concorrentes.

Embora satisfazer os requisitos funcionais seja um objetivo do design de arquitetura de software, mantenha Lembre-se de que nem todas as funcionalidades são afetadas pela arquitetura. Embora algumas funcionalidades sejam altamente afetadas pela arquitetura, outras funcionalidades também podem ser fornecidas com diferentes arquiteturas.

Mesmo nos casos em que a funcionalidade não é influenciada pela arquitetura diretamente, funcional os requisitos podem ser um motivador arquitetônico por outros motivos. Um exemplo

disso seria ser a necessidade de fazer modificações na funcionalidade posteriormente. Manutenção e a capacidade de modificação do software é afetada pela arquitetura do software.

5.13. Cenários de atributos de qualidade

Os atributos de qualidade são propriedades mensuráveis de um sistema de software. Eles são as ilidades, tais quanto à manutenção, usabilidade, testabilidade e interoperabilidade. Temos enfatizado a importância dos atributos de qualidade, uma vez que desempenham um papel tão importante no sucesso de sistemas de software e porque as decisões de arquitetura de software os afetarão.

Isso torna os atributos de qualidade um dos principais drivers arquitetônicos para a arquitetura de software Projeto. As decisões de design que são feitas determinarão quais atributos de qualidade serão conhecidos. Como um motivador arquitetônico, os atributos de qualidade são normalmente descritos no contexto de um cenário particular.

Um cenário de atributo de qualidade é uma breve descrição de como o sistema de software deve responder a um determinado estímulo. Os cenários tornam os atributos de qualidade mensuráveis e testáveis.

Por exemplo, um atributo de qualidade de desempenho ou um requisito que estabelece uma determinada função deve ser rápida não é mensurável ou testável. Um exemplo real de uma qualidade válida atributo relacionado ao desempenho seria o seguinte: Quando o usuário seleciona a opção Login, a página de login é exibida em dois segundos.

5.14. Priorizando cenários de atributos de qualidade

Antes do início do processo de design de arquitetura, os cenários de atributos de qualidade devem ser priorizados. É útil estar ciente da prioridade de cada cenário de atributo de qualidade quando projetar a arquitetura. Além de ser capaz de planejar adequadamente, como focar em atributos de qualidade de prioridade mais alta primeiro, pode haver compensações envolvidas ao habilitar certos atributos de qualidade. Compreender as prioridades o ajudará a fazer um design melhor decisões relacionadas aos atributos de qualidade e quaisquer compensações que precisem ser feitas.

Os cenários de atributos de qualidade podem ser priorizados classificando-os com base em dois critérios: importância do negócio e o risco técnico associado ao cenário. Uma escala de classificação de Alto (H), Médio (M) e Baixo (L) podem ser usados.

As partes interessadas podem ajudar a fornecer a classificação com base na importância do negócio, enquanto o arquiteto de software normalmente fornece a classificação com base no risco técnico. Uma vez que as classificações estão completas, cada atributo de qualidade deve ter uma combinação das duas classificações.

Se cada cenário de atributo de qualidade fosse atribuído a um número exclusivo, eles poderiam ser colocados em uma tabela como a seguinte:

Business importance/technical risk	L	M	H
L	6, 21	7, 13	15
M	3, 10, 11	14, 16, 17	1, 5
H	4, 18, 19, 20	2, 12	8, 9

Os cenários de atributos de qualidade localizados na parte inferior direita da tabela serão de maior importância. Os mais importantes serão aqueles com classificação H, H, indicando eles foram bem classificados em ambos os critérios. As iterações iniciais de design podem se concentrar nesses cenários primeiros. As iterações subsequentes podem considerar os cenários de atributos de qualidade mais importantes que permanecem, como H, M e M, H, até que todos os cenários de atributos de qualidade tenham sido considerados.

5.15. Restrições

Restrições são decisões impostas a um projeto de software que deve ser satisfeito pela arquitetura. Eles normalmente não podem ser alterados. Eles podem afetar a arquitetura do software design e são, portanto, um condutor arquitetônico.

As restrições são geralmente corrigidas desde o início do projeto e podem ser técnicas ou não técnico. Exemplos de restrições técnicas incluem a necessidade de usar uma tecnologia, tendo a capacidade de implantar em um determinado tipo de ambiente de destino, ou usando uma linguagem de programação específica. Exemplos de restrições não técnicas estão sendo exigidos cumprir um determinado regulamento, ou que o projeto deve cumprir um determinado prazo.

As restrições também podem ser classificadas como internas ou externas. Interno restrições se originam de dentro da organização e você pode ter algum controle sobre eles. Em contraste, as restrições externas vêm de fora da empresa e você não pode tenha qualquer controle sobre eles.

Como os outros drivers arquitetônicos, as restrições precisam ser consideradas no projeto como uma entrada no processo de design.

5.16. Preocupações arquitetônicas

As preocupações arquitetônicas são interesses do arquiteto de software que impactam o software arquitetura. Como resultado, eles são um motivador arquitetônico. Exatamente como requisitos funcionais e os atributos de qualidade são questões de design importantes para as partes interessadas, questões arquitetônicas são questões de design importantes para o arquiteto de software.

As preocupações arquitetônicas precisam ser consideradas parte do projeto, mas não são capturadas como requisitos funcionais. Em alguns casos, eles podem ser capturados como atributos de qualidade, em vez do que as preocupações arquitetônicas, ou uma preocupação arquitetônica pode levar a um novo atributo de qualidade cenários que precisam ser atendidos.

Por exemplo, um arquiteto de software pode ter preocupações relacionadas à instrumentação de software ou exploração madeireira. Se ainda não for registrado como parte de um atributo de qualidade, como capacidade de manutenção, o a preocupação arquitetônica pode levar a um novo atributo de qualidade.

Bons arquitetos de software serão capazes de reconhecer possíveis preocupações arquitetônicas com base em o tipo de software que estão projetando. Preocupações arquitetônicas também podem surgir de iterações de design de arquitetura anteriores, portanto, esteja ciente de que as mudanças na arquitetura podem levar a novas preocupações sendo criadas a partir deles.

5.17. Aproveitando os princípios de design e soluções

Projetar uma arquitetura de software do zero para um projeto com algum nível de complexidade pode ser uma tarefa desafiadora. No entanto, os arquitetos de software têm uma série de ferramentas em sua disposição ao projetar uma arquitetura.

Os problemas de design enfrentados por um projeto podem já ter sido resolvidos por outros, e ao invés de reinventando a roda, essas soluções podem ser aproveitadas em seu projeto de arquitetura. Esses princípios e soluções de design de arquitetura, que às vezes são chamados de design conceitos, são blocos de construção usados para projetar uma arquitetura de software.

5.18. Selecionando um conceito de design

Existem muitos conceitos de design, então um arquiteto de software precisa saber quais são adequados para um problema específico e, em seguida, selecione aquele que é mais apropriado entre as alternativas. Você também pode encontrar casos em que precisa combinar vários projetos conceitos para criar uma solução.

Dependendo do estágio do projeto de arquitetura em que você se encontra e da natureza do problema, certos conceitos de design farão mais sentido do que outros. Por exemplo, uma referência arquitetura seria útil ao criar a estrutura inicial da arquitetura, mas mais tarde no projeto, ao considerar um cenário de atributo de qualidade específico, uma tática pode ser usada.

Os arquitetos de software geralmente determinam quais conceitos de design estão disponíveis usando seus conhecimento e experiência, aproveitando o conhecimento e a experiência de seus colegas de equipe, e seguindo as melhores práticas.

Ao escolher um conceito de design específico entre várias alternativas que foram identificados, você vai querer pesar os prós e os contras, bem como o custo de cada alternativa.

Mantenha todas as restrições do projeto em mente ao selecionar os conceitos de design, pois uma restrição pode impedi-lo de usar certas alternativas.

Alguns dos conceitos de design disponíveis para você incluem padrões de arquitetura de software, arquiteturas de referência, táticas e software desenvolvido externamente.

5.19. Padrões de arquitetura de software

Ao projetar uma arquitetura de software, alguns dos problemas de design que você enfrentará já foi resolvido por outros. Os padrões de arquitetura de software fornecem soluções para problemas recorrentes de design de arquitetura. Os padrões são descobertos ao observar o que

as pessoas estavam conseguindo resolver um problema específico e, em seguida, documentando esses padrões para que possam ser reutilizados. Eles podem ser aproveitados em um projeto de arquitetura se o aplicativo de software tem o mesmo problema de design.

Os arquitetos de software devem aproveitar o trabalho e a experiência de outros quando eles têm um problema que pode ser resolvido por um padrão. A parte desafiadora é estar ciente do que padrões estão disponíveis e quais são aplicáveis ao problema que você está tentando resolver.

Como acontece com qualquer padrão de design, você não deve tentar forçar o uso de um. Você deve só usar um padrão de arquitetura se realmente resolver o problema de design que você tem e se for a melhor solução de acordo com o seu contexto.

5.20. Arquiteturas de referência

Uma arquitetura de referência é um modelo para uma arquitetura mais adequada para um determinado domínio. É composto de artefatos de design para uma arquitetura de software que fornece estruturas recomendadas, elementos e as relações entre os elementos.

5.20.1. Benefícios das arquiteturas de referência

Uma arquitetura de referência pode responder muitas das perguntas mais comuns para sistemas que precisa de um design específico. Eles podem ser muito úteis para arquitetos de software porque eles fornecem uma solução testada para um domínio de problema e reduzir algumas das complexidades envolvidos no projeto de uma arquitetura de software. As arquiteturas de referência são comprovadas, em ambos os contextos técnicos e de negócios, como soluções viáveis para determinados problemas.

Usar uma arquitetura de referência permite que a equipe forneça uma solução mais rápida e com menos erros. Reutilizar uma arquitetura oferece vantagens como entrega mais rápida de uma solução, esforço de design reduzido, custos reduzidos e maior qualidade.

Aproveitar as experiências de aplicativos de software anteriores e aprender com eles pode ser de grande valor para arquitetos de software. Eles nos ajudam a evitar certos erros e podem evitar atrasos dispendiosos que podem resultar do não uso de uma abordagem comprovada anteriormente.

5.21. Refatorando uma arquitetura de referência para suas necessidades

Assim como um projeto sem usar uma arquitetura de referência pode exigir várias iterações para alcançar o resultado, também é o caso quando se usa uma arquitetura de referência. Projeto as decisões precisarão ser feitas em relação à arquitetura de referência.

Durante as iterações para o design da arquitetura, a refatoração pode ocorrer em uma referência arquitetura para atender às necessidades específicas do aplicativo de software que está sendo projetado. A quantidade de refatoração necessária depende de quão próximo a arquitetura de referência atende os requisitos funcionais e de atributos de qualidade.

As arquiteturas de referência podem ser criadas em diferentes níveis de abstração. Se você quiser usar um e não está no nível de abstração que você precisa, você ainda pode ser capaz de aprender com ele, e use-o como um guia ao projetar sua própria arquitetura.

Para domínios bem compreendidos, pode haver uma série de arquiteturas de referência disponíveis para você. Em contraste, se você estiver projetando uma solução para um sistema greenfield que está em um novo domínio, pode haver poucos, se houver, disponíveis para você usar. Mesmo para esses tipos de projetos, porém, você pode encontrar uma arquitetura de referência que pode aproveitar, mesmo que seja apenas para uma parte do design. Só pode exigir mais refinamento e refatoração do que quando uma mais arquitetura de referência adequada está disponível.

Ao usar uma arquitetura de referência, você adota questões dessa arquitetura de referência que você precisará abordar. Se uma arquitetura de referência lida com um projeto específico problema, você precisará tomar decisões de design sobre esse problema, mesmo se você não tiver um requisito específico relacionado a ele. A decisão pode muito bem ser excluir algo de sua arquitetura que está na arquitetura de referência.

Por exemplo, se uma arquitetura de referência inclui instrumentação como uma questão transversal, você precisará tomar decisões de design sobre instrumentação durante seu design.

5.22. Criando sua própria arquitetura de referência

Uma vez que uma organização tenha uma arquitetura de software completa, que pode ou não ter usado uma arquitetura de referência, ele pode então se tornar uma arquitetura de referência. Quando uma organização precisa criar aplicativos de software, talvez como parte de um software linha de produtos, ele pode usar a arquitetura de um produto existente como uma arquitetura de referência.

Usar uma arquitetura de referência de sua própria organização é como usar uma referência arquitetura de outro lugar. Você colherá benefícios ao fazer isso, mas alguma quantidade de refatoração pode ser necessária para usá-la em um aplicativo específico. A vantagem adicional é que a arquitetura de referência provavelmente já seja adequada para o seu domínio.

5.22.1. Táticas

As táticas são técnicas comprovadas para influenciar cenários de atributos de qualidade. Eles se concentram em um atributo de qualidade único, por isso são mais simples do que outros conceitos de design, como arquitetura padrões e arquiteturas de referência, que visam resolver um maior número de problemas de design.

As táticas fornecem opções para satisfazer os atributos de qualidade e o uso de outros conceitos de design, como padrões de arquitetura ou uma estrutura construída externamente, junto com o código, são necessários para completar totalmente a tática.

Repassamos algumas táticas ao explorar os atributos de qualidade, como:

- Satisfazer um cenário de atributo de qualidade de manutenção, reduzindo a complexidade em um componente aumentando a coesão e reduzindo o acoplamento
- Aumentando a usabilidade em um cenário, fornecendo informações amigáveis e informativas mensagens para o usuário
- Implementar uma estratégia de nova tentativa em um processo para lidar com uma possível falha transitória em a fim de melhorar um cenário de atributo de qualidade de disponibilidade

- Satisfazendo um cenário de atributo de qualidade de portabilidade, aumentando a instalabilidade em garantindo que um processo de atualização de software para uma versão mais recente limpe adequadamente a versão mais antiga

5.23. Software desenvolvido externamente

Ao projetar uma arquitetura de software, você tomará decisões de design para vários de questões de design. Alguns desses problemas de design já têm soluções na forma de concreto implementações que foram desenvolvidas externamente. Em vez de construir uma solução interna para resolver um problema específico de design, você pode aproveitar o software que já foi desenvolvido fora da organização.

O software desenvolvido externamente pode vir em diferentes formas, como um componente, uma estrutura de aplicativo, um produto de software ou uma plataforma. Existem muitos exemplos de software desenvolvido externamente, como uma biblioteca de registro para a funcionalidade de registro, uma IU estrutura para criar interfaces de usuário ou uma plataforma de desenvolvimento para lógica do lado do servidor.

5.24. Comprar ou construir?

Uma das decisões que os arquitetos de software precisam fazer é a compra clássica ou construir dilema. Quando você precisar de uma solução para um problema de design específico, você precisa decidir se vai comprar ou construir. Ao usar o termo comprar, estamos nos referindo a usando algo construído externamente, e não necessariamente o fato de que pode ter um custo. Dependendo do tipo de solução que você está procurando, pode haver uma série de soluções gratuitas disponíveis para você, incluindo aquelas que são de código aberto.

Ao decidir se vai usar uma solução desenvolvida externamente ou construí-la internamente, você deve primeiro certificar-se de que entendeu o problema que está tentando resolver, e o âmbito desse problema. Você precisará pesquisar se software desenvolvido externamente existe que irá resolver o problema de design. Se o problema for exclusivo de sua organização, pode não haver nenhum software adequado disponível.

Você também deve saber se a organização tem ou não, ou pode obter, recursos para construir, manter e oferecer suporte a uma solução. Se a solução for construída internamente pelo projeto equipe, deve haver recursos suficientes, incluindo tempo, para construí-la.

5.25. Vantagens/desvantagens da construção

Uma vantagem de construí-lo internamente é que a solução será exclusiva para a sua organização e adaptados a ela. A organização terá controle total sobre a solução, incluindo propriedade total do código-fonte. Isso permitirá que a organização o modifique da maneira que desejar. Se surgir a necessidade de fazer alterações ou adicionar funcionalidade, a organização poderá fazê-lo com plena autoridade.

Outro benefício de construir você mesmo é que pode haver uma vantagem competitiva. Se a solução fornece algum recurso que os concorrentes não têm atualmente, construindo-o e possuí-lo pode fornecer uma vantagem estratégica para a organização.

As desvantagens de construí-lo sozinho são que exigirá tempo e recursos para fazer tão. O resultado pode não ter um conjunto de recursos tão robusto quanto um desenvolvido externamente solução. Por exemplo, se você precisa de um texto completo distribuído,

escalonável, de nível empresarial mecanismo de pesquisa como parte de seu aplicativo, provavelmente é impraticável construí-lo você mesmo em vez de usar uma solução comprovada que já existe.

5.26. Vantagens/desvantagens de comprar

Usar uma solução desenvolvida externamente tem seu próprio conjunto de vantagens. Isso vai economizar tempo, pois nenhum esforço será necessário para desenvolvê-lo. Pode ser de qualidade superior, supondo que já foi testado e usado na produção. Feedback de outros usuários do software pode ter exposto problemas que já foram corrigidos.

A solução externa pode ser continuamente aprimorada para atingir níveis mais elevados de qualidade e para apresentar novos recursos. Suporte e treinamento podem estar disponíveis para que sua equipe esteja capaz de alavancar.

No entanto, existem desvantagens em usar uma solução desenvolvida externamente. Pode haver um custo para usar tal solução. Dependendo do tipo de licença, você pode não ter acesso ao código-fonte e pode ser limitado em como a solução pode ser usada. Se você não pode modificar a solução, então a funcionalidade da solução será controlada por outra pessoa, e não pode atender exatamente às suas necessidades. Além disso, se houver problemas com a solução ou se você precisar dela alterado de alguma forma, você precisará contar com uma organização externa.

5.27. Pesquisando software externo

A fim de descobrir se existe software externo que será uma solução adequada para o problema sendo resolvido, ou a fim de selecionar uma solução externa de várias alternativas que possam estar disponíveis, alguma pesquisa será necessária.

O arquiteto de software deve considerar o seguinte:

- Resolve o problema de design?
- O custo do software é aceitável?
- É o tipo de licença que acompanha o software compatível com o projeto precisa?
- O software é fácil de usar? A equipe possui recursos que podem usá-lo?
- O software pode ser integrado com as outras tecnologias que serão usados no projeto?
- O software está maduro, fornecendo versões estáveis?
- Fornece o nível de suporte que pode ser necessário, seja esse suporte é o suporte pago ou por meio de uma comunidade de desenvolvimento?
- O software é amplamente conhecido, de forma que a organização possa facilmente contratar recursos familiarizados com ele?

Criar um ou mais protótipos que usam as possíveis soluções candidatas é uma boa maneira de avaliá-los e compará-los. Um POC para garantir que é uma solução viável é uma ideia inteligente.

5.28. Devo usar software de código aberto (OSS)?

Ao procurar uma solução desenvolvida externamente que irá resolver um problema de design, uma possibilidade é encontrar software de código aberto (OSS) que atenda às suas necessidades. OSS está escrito pela comunidade e se destina ao uso pela comunidade.

Dada a ampla disponibilidade e variedade de software de código aberto, existem muitas soluções disponíveis para uma variedade de problemas. É muito mais comum agora usar código aberto como parte de um aplicativo de software. Algumas organizações não permitem o uso de software de código aberto, mas se sua organização o faz, você deve considerá-lo como uma alternativa viável para uma determinada tarefa.

Uma consideração ao selecionar o software de código aberto é a licença que está associada com isto. A licença dita os termos e condições sob os quais o software pode ser usado, modificado e compartilhado. Um conjunto de licenças de código aberto que são populares é um grupo de licenças que foram aprovados pela Open Source Initiative (OSI). Alguns dos aprovados pelo OSI as licenças incluem (em ordem alfabética):

- Licença Apache 2.0
- Licença BSD 2 simplificada ou licença FreeBSD
- Licença BSD 3 cláusulas nova ou revisada
- Licença de desenvolvimento e distribuição comum
- Licença Pública Eclipse
- GNU General Public License (GPL)
- GNU Lesser General Public License (LGPL)
- Licença MIT
- Mozilla Public License 2.0

Existem diferenças nos termos e condições das várias licenças. Por exemplo, seu aplicativo pode incorporar software de código aberto que usa a licença MIT e você será capaz de distribuir seu aplicativo sem torná-lo open source.

Em contraste, se o seu aplicativo incorpora software que usa o GNU General Public License e, em seguida, distribuir seu aplicativo, seu aplicativo precisaria ser feito Código aberto. Isso é verdade mesmo se seu aplicativo for gratuito e você não alterar o software de origem que você está usando de alguma forma. Se o seu software for apenas para uso interno e for não distribuído, então seu aplicativo pode permanecer proprietário e de código-fonte fechado.

5.29. Vantagens de usar software de código aberto

Existem benefícios em usar software de código aberto, o que explica sua popularidade. Usando uma solução de código aberto para um problema de design oferece muitas das mesmas vantagens de usar aquele que foi comprado. Você não precisa perder tempo construindo a solução, pode ter um conjunto robusto de recursos e pode já ser uma solução testada e comprovada com muitos outros usuários.

Ao contrário do software que deve ser comprado, o software de código aberto está disponível gratuitamente, portanto, são economias de custo. Basta ter em mente a licença que acompanha o software.

Se o software de código aberto for uma solução popular com uma comunidade ativa, pode ser continuamente aprimorado com correções de bugs e novos recursos. Você será capaz de tirar vantagem deste trabalho. Bugs podem ser detectados e corrigidos rapidamente porque muitas pessoas estão usando e trabalhando no código. Esta é a ideia por trás da Lei de Linus, que leva

o nome de Linus Torvalds, o criador do kernel Linux. A Lei de Linus basicamente afirma que, dado o suficiente olhos, ou pessoas olhando para o código, todos os bugs são superficiais. Em outras palavras, com muitas pessoas olhando para o código-fonte, os problemas serão detectados mais cedo ou mais tarde, e alguém poderá fornecer uma correção.

Embora alguns vejam o software de código aberto como menos seguro devido à disponibilidade do código, algumas pessoas vêem isso como mais seguro porque há muitos olhos que estão usando, olhando para, e corrigir o código.

Outra vantagem do software de código aberto é o fato de você ter acesso ao código-fonte código. Se necessário, sua equipe de desenvolvimento poderá modificá-lo da mesma forma que você faria com uma solução interna.

5.30. Desvantagens de usar software de código aberto

Apesar de suas vantagens, existem algumas desvantagens em usar software de código aberto que você deveria considerar. Mesmo que o software seja gratuito, ainda existem custos relacionados ao uso isto. Alguém tem que gastar tempo integrando a solução ao sistema de software, e há um custo associado a esse esforço. Se o software de código aberto tiver que ser modificado de alguma forma para atender às necessidades do projeto, há um custo relacionado a esse trabalho também.

Se não houver ninguém na equipe que saiba como usar o software e for complexo o suficiente para que algum treinamento seja necessário, aprendendo a usar um software de código aberto também pode levar tempo.

Mesmo para um projeto de código aberto popular com uma comunidade ativa, não há garantia que o software continuará a ter suporte. Sempre há um risco do software caindo em desgraça. Se o projeto for abandonado, você não poderá contar com suporte para bug correções ou novos recursos, a menos que a equipe de desenvolvimento execute esse trabalho por conta própria.

Uma razão pela qual um projeto de software de código aberto pode se tornar menos seguro é se ninguém está ativamente trabalhando nisso. Mesmo que o projeto não tenha sido abandonado, ninguém está necessariamente lendo o código. O programador médio escreve muito mais código do que lê. A existência de alguns bugs de segurança proeminentes mostrou que é possível para segurança crítica vulnerabilidades não são detectadas por algum tempo. Apesar da Lei de Linus, o fato de que o código-fonte estão prontamente disponível apresenta um grau de risco de segurança. Indivíduos mal-intencionados podem analisar o código-fonte para identificar a segurança vulnerabilidades e tentar tirar proveito delas.

5.31. Documentando a arquitetura do software Projeto

Uma parte importante do design de arquitetura é documentar o design, incluindo muitas decisões de design que são feitas durante o processo. Isso normalmente vem na forma de esboçar vistas de arquitetura e documentar a lógica do projeto.

5.31.1. Esboçando o projeto de arquitetura

As arquiteturas de software são comumente documentadas por meio da criação da arquitetura Visualizações. Vistas de arquitetura são representações de uma arquitetura de software que são usadas para documente-o e comunique-o às várias partes interessadas. Múltiplas visualizações de uma arquitetura são normalmente necessárias, pois uma arquitetura de software é muito complexa para ser representada em um modelo único e abrangente.

A documentação formal de uma arquitetura de software por meio de visualizações será abordada. Capítulo 12, Documentação e revisão de arquiteturas de software, e normalmente não é feito como parte do processo de design. Embora esse tipo de documentação venha depois, informal a documentação, na forma de esboços, deve ocorrer durante o projeto de arquitetura.

Os esboços podem registrar as estruturas, elementos, relações entre os elementos e os conceitos de designs usados. Os esboços não precisam necessariamente usar nenhuma notação formal, mas devem ser claros. Embora não seja necessário esboçar tudo, no mínimo, você desejará esboçar decisões importantes e elementos de design. Esses esboços podem ser feitos em um quadro branco, em papel ou usando uma ferramenta de modelagem.

Documentar o design criando esboços durante o processo de design irá ajudá-lo a criar vistas de arquitetura mais tarde. Se você já tem esboços informais, quando chegar a hora para criar documentação formal, você verá que é uma tarefa mais fácil.

Documentar o projeto conforme ele ocorre também irá garantir que você não se esqueça de nenhum projeto detalhe quando se trata de criar as visualizações de arquitetura. Sua arquitetura será analisada e validado posteriormente para garantir que satisfaça os requisitos funcionais e o atributo de qualidade cenários, por isso é útil registrar detalhes durante o design que podem ser usados para explicar como os requisitos e atributos de qualidade foram satisfeitos pelo design da arquitetura.

Se você não for capaz de esboçar uma parte do seu design, você terá que considerar o possível razões para isso. Talvez não seja bem compreendido, muito complexo, você não colocou o suficiente pensou em como comunicá-lo, ou pode haver partes dele que não estão claras para você. Se esse for o caso, você deve revisitar o design até que seja capaz de esboçá-lo. Se você puder esboçar o design criado em uma iteração de forma fácil e clara, seu público será capaz para entender isso.

5.31.2. Documentando a lógica do design

O design da arquitetura de software envolve a tomada de muitas decisões de design, e o software os arquitetos devem documentar essas decisões junto com a lógica do projeto. Enquanto design os esboços podem explicar o que foi projetado, eles não dão nenhuma indicação quanto ao design justificativa.

Uma lógica de design explica as razões e justificativas por trás das decisões que são feitas durante o projeto da arquitetura do software. A lógica do design também pode incluir documentação sobre quais decisões não foram tomadas, bem como alternativas que foram considerados para as decisões que foram feitas. Motivos de rejeição podem ser registrados para cada alternativa que não foi selecionada.

A justificativa do design de gravação pode ser útil durante o processo de design, bem como uma vez que o design está completo. Os arquitetos de software que documentam seus fundamentos de design têm uma oportunidade de esclarecer seus pensamentos e argumentos à medida que capturam a lógica do design, o que pode até expor falhas em seu pensamento.

Uma vez que a lógica do design é documentada, qualquer pessoa que queira saber por que uma determinada decisão de projeto foi feita, mesmo depois de algum tempo, pode se referir a ele. Até indivíduos que estavam envolvidos com uma decisão de design, incluindo o arquiteto de software, pode esquecer o raciocínio por trás de uma decisão particular e será capaz de se referir à documentação.

A lógica do projeto deve se referir às estruturas específicas que foram projetadas e aos requisitos específicos que pretendiam cumprir. Algumas ferramentas de design de software fornecem funcionalidade que pode ajudar o arquiteto de software a capturar a lógica do projeto. Uma lógica de design completa fornece um histórico do processo de design da arquitetura de software. Há uma série de usos para a lógica de design, como para avaliação de design, verificação, transferência de conhecimento, comunicação, manutenção, documentação e reutilização.

5.31.3. Justificativa de design para avaliação de design

A lógica do projeto pode ser usada para avaliar diferentes projetos de arquitetura de software e suas escolhas de design. Os vários projetos podem ser comparados uns com os outros, e um compreensão pode ser obtida quanto às situações em que um projeto seria escolhido outro.

5.31.4. Fundamentação do projeto para verificação do projeto

O objetivo da verificação do projeto de arquitetura de software é garantir que o software sistema, conforme projetado, é o sistema de software pretendido. Ele verifica se o software arquitetura atende aos requisitos, incluindo os atributos de qualidade, e funciona como esperado. A lógica do projeto pode ser usada como parte da verificação.

5.31.5. Justificativa de design para transferência de conhecimento de design

A lógica do design pode ser usada para transferência de conhecimento para os membros da equipe, incluindo aqueles que pode se juntar à equipe mais tarde, durante o desenvolvimento ou após o software entrar em sua fase de manutenção.

Os membros da equipe podem aprender sobre as decisões de design e as razões por trás delas, revisar o fundamento lógico do design. É particularmente útil para transferência de conhecimento quando o arquiteto de software original e outros que colaboraram no design do software arquitetura, não estão mais disponíveis para fornecer as informações de outras maneiras.

5.31.6. Justificativa de design para comunicação de design

Será necessário em diferentes momentos comunicar o design do software arquitetura para várias partes interessadas e outros indivíduos. As informações fornecidas por a lógica do design agregam valor à comunicação geral. Além disso, a lógica do design pode ser usada por aqueles que estão revisando o software arquitetura para que eles possam aprender as razões por trás de decisões de design específicas.

Fundamentação do projeto para manutenção do projeto Durante a fase de manutenção de um projeto de software, é útil conhecer o design justificativa para as decisões tomadas no design da arquitetura de software. Quando um certo parte do software precisa ser alterada para manutenção, a lógica do design pode auxiliar na determinar quais áreas do software exigirão modificações.

Também pode ser usado para identificar pontos fracos no software e áreas do software que poderia ser melhorado. Por exemplo, com base em certas decisões de design, os atributos de qualidade podem ser habilitados ou inibidos, e se mudanças estão sendo consideradas que alterariam aquelas decisões, os membros da equipe podem estar cientes das razões por trás dessas decisões.

A lógica do projeto também apontará alternativas de projeto que não foram escolhidas, permitindo aqueles que consideram modificações para evitar alternativas de design anteriormente rejeitadas ou para pelo menos estar informado sobre as razões pelas quais essas alternativas foram rejeitadas a fim de tomar uma decisão educada.

5.31.7. Justificativa de design para documentação de design

A arquitetura de software deve ser documentada, e a lógica do design é uma importante parte dessa documentação. Se a documentação mostra apenas o design, quem está olhando para ele saberá o que foi projetado, mas não saberá por que foi projetado dessa forma. Eles também não estarão cientes das alternativas que foram consideradas e porque essas alternativas foram rejeitadas.

5.31.8. Justificativa de design para reutilização de design

A reutilização da arquitetura de software envolve a criação de vários aplicativos de software usando o núcleo ativos, permitindo que os componentes arquitetônicos sejam reutilizados em vários produtos de software.

As organizações procuram tirar proveito das eficiências que podem ser obtidas ao reutilizar componentes arquitetônicos para construir vários produtos de software como parte de um produto de software linha.

Capturar a lógica do projeto pode facilitar a reutilização de arquitetura com sucesso. Pode ajudar designers entender quais partes do aplicativo podem ser reutilizadas. Também pode fornecer alguns insights em onde modificações podem ser feitas em um componente para reutilizá-lo no aplicativo. Devido à variação entre os produtos de software, os componentes reutilizáveis são normalmente projetados como pontos de variação, ou lugares onde modificações podem ser feitas a fim de se adaptar o componente para uso em um produto de software específico. Compreender a lógica do design ajudará os designers a usar o componente corretamente e evitar modificações prejudiciais de sendo feito.

5.32. Usando uma abordagem sistemática para software projeto de arquitetura

Se você vai dedicar algum tempo para projetar a arquitetura de um sistema de software, e não apenas deixá-lo surgir após a implementação de recursos, você deve fazê-lo de forma sistemática.

Os arquitetos de software precisam garantir que a arquitetura que estão projetando irá satisfazer os drivers arquitetônicos e uma abordagem sistemática podem ajudar a atingir esse objetivo. No *Projetando Arquiteturas de Software, Uma Abordagem Prática*, o seguinte é dito sobre o uso de um processo de design de arquitetura:

"A questão é: como você realmente executa o design? Executar o design para garantir que os condutores estão satisfeitos requer um método baseado em princípios. Por "princípios", nos referimos a um método que leva em consideração todos os aspectos relevantes que são necessários para produzir um design adequado. Esse método fornece orientação necessária para garantir que os motoristas estão satisfeitos. "

Usar um processo de design de arquitetura estabelecido fornecerá a você, como arquiteto de software, com orientação sobre como projetar uma arquitetura que satisfaça o funcional requisitos e cenários de atributos de qualidade. Existem vários processos de design que pode ser usado para arquitetura de software. Embora sejam diferentes entre si, incluindo diferenças na terminologia, eles também têm alguns pontos comuns fundamentais.

6. Padrões de arquitetura de software

Os padrões de arquitetura de software são uma das ferramentas mais úteis que podem ser aproveitadas para projetar uma arquitetura de software. Alguns dos problemas de design que enfrentamos como arquitetos de software já tem soluções comprovadas. Arquitetos de software experientes conhecem padrões de arquitetura disponíveis e podem reconhecer quando um pode ser aplicado a um determinado cenário de design.

Este capítulo começa explicando o que são os padrões de arquitetura de software e como eles pode ser usado. Em seguida, entra em detalhes sobre alguns padrões de arquitetura comumente usados, incluindo arquitetura em camadas, arquitetura orientada a eventos (EDA), Model-View-Controller (MVC), Model-View-Presenter (MVP), Model-View-ViewModel (MVVM), Comando Segregação de responsabilidade de consulta (CQRS) e arquitetura orientada a serviços (SOA).

Neste capítulo, cobriremos os seguintes tópicos:

- Padrões de arquitetura de software
- Arquitetura em camadas
- Arquitetura orientada a eventos, incluindo notificações de eventos, estado realizado por evento, transferência e fonte de eventos
- Padrão Model-View-Controller
- Padrão Model-View-Presenter
- Padrão Model-View-ViewModel
- Segregação de responsabilidade de consulta de comando
- Arquitetura Orientada a Serviços

6.1. Padrões de arquitetura de software

Um padrão de arquitetura de software é uma solução para um problema recorrente que está bem entendida, em um determinado contexto. Cada padrão consiste em um contexto, um problema e um solução. O problema pode ser superar algum desafio, tirar proveito de alguns oportunidade, ou para satisfazer um ou mais atributos de qualidade. Os padrões codificam o conhecimento e experiência em uma solução que podemos reutilizar.

O uso de padrões simplifica o design e nos permite obter os benefícios de usar uma solução que é comprovado que resolve um problema de design específico. Ao trabalhar com outras pessoas que estão familiarizadas com padrões, fazer referência a um deles fornece uma abreviatura para fazer referência a uma solução, sem ter que explicar todos os seus detalhes. Como resultado, eles são úteis durante discussões para comunicar ideias.

Os padrões de arquitetura de software são semelhantes aos padrões de design, exceto que são mais amplos em escopo e são aplicados no nível de arquitetura. Os padrões de arquitetura tendem a ser mais granulação grosseira e foco em problemas arquitetônicos, enquanto os padrões de design são mais refinados e resolver problemas que ocorrem durante a implementação.

Um padrão de arquitetura de software fornece uma estrutura de alto nível e comportamento para software sistemas. É um agrupamento de decisões de design que foram repetidas e usadas com sucesso para um determinado contexto. Eles atendem e satisfazem os motivadores arquitetônicos e, como resultado, aqueles que decidimos usar podem realmente moldar as características e o comportamento da arquitetura.

Cada padrão tem suas próprias características, pontos fortes e fracos. Os padrões de arquitetura de software fornecem a estrutura e os principais componentes do software sistema em construção. Eles introduzem restrições de design, que reduzem a complexidade e ajudam a evitar decisões incorretas. Quando um padrão de arquitetura de software é seguido de forma consistente durante o projeto, podemos antecipar as propriedades que o sistema de software exibirá. Esse nos permite considerar se um projeto irá satisfazer os requisitos e atributos de qualidade do sistema.

6.2. Usando padrões de arquitetura de software

Muito parecido com os padrões de design, os padrões de arquitetura de software surgem depois de serem repetido com sucesso na prática. Como arquiteto de software, você descobrirá e usará um padrão existente quando apropriado.

Na maioria das vezes, você não inventará ou criando um padrão. É possível que você pode enfrentar desafios e problemas em um novo domínio, exigindo você a criar soluções verdadeiramente novas que não existem atualmente. No entanto, mesmo nesse caso, a solução ainda não será um padrão. Só depois que é repetido na prática e se torna conhecido como solução para um determinado contexto e problema, torna-se um padrão.

Os padrões de arquitetura de software podem ser aplicados a todo o sistema de software ou a um dos subsistemas. Consequentemente, mais de um padrão de arquitetura de software pode ser usado em um sistema de software único. Esses padrões podem ser combinados para resolver problemas.

6.3. Uso excessivo de padrões de arquitetura

Embora o aproveitamento de padrões de arquitetura seja uma ferramenta valiosa, não force o uso de um determinado padronizar. Um erro comum com padrões de design e arquitetura é usar um mesmo que seja não é apropriado fazê-lo.

Um desenvolvedor ou arquiteto pode tomar conhecimento de um determinado padrão e então se tornar com excesso de zelo em seu desejo de usá-lo. Eles podem aplicar um padrão apenas devido à sua familiaridade com isso.

A chave é obter conhecimento dos padrões que estão disponíveis e uma compreensão quanto aos cenários em que devem ser aplicados. Este conhecimento permite que um arquiteto de software para selecionar e usar o padrão apropriado. Um padrão de arquitetura de software deve ser apenas usado se for a melhor solução para um determinado problema de design e contexto.

6.4. Compreendendo a diferença entre estilos de arquitetura e padrões de arquitetura

Você pode encontrar o termo estilo de arquitetura e comparar seu significado com o termo padrão de arquitetura. Na maior parte, esses dois termos são usados alternadamente. No entanto, algumas pessoas fazem uma distinção entre os dois, então vamos parar um momento para explicar a diferença. Não é totalmente claro, pois as definições variam dependendo a quem você pergunta. Uma das maneiras pelas quais um estilo de arquitetura é definido é que ele é um conjunto de elementos, e o vocabulário a ser usado para esses elementos, que está disponível para ser usado em uma arquitetura. Ele restringe um projeto de arquitetura ao restringir as opções de projeto disponíveis.

Quando um sistema de software adere a um estilo de arquitetura específico, espera-se que exibam certas propriedades. Por exemplo, se seguirmos o estilo de arquitetura de micro serviço para um aplicativo, que vem com a restrição de que cada serviço deve ser independente dos outros, nós podemos esperar que tal sistema tenha certas propriedades. Um sistema que segue um micro serviço estilo de arquitetura será capaz de implantar serviços de forma independente, isolar falhas para um serviço particular, e usar uma tecnologia da escolha da equipe para um determinado serviço.

Um padrão de arquitetura de software é um arranjo particular dos elementos disponíveis em uma solução para um problema recorrente em um determinado contexto. Dada uma arquitetura particular estilo, podemos usar o vocabulário desse estilo para expressar como queremos usar os elementos disponível naquele estilo de uma determinada maneira. Quando este arranjo é uma solução conhecida para um problema comum e recorrente em um determinado contexto, é um padrão de arquitetura de software.

6.5. Arquitetura em camadas

Ao particionar um sistema de software complicado, a disposição em camadas é uma das mais comuns técnicas. Em uma arquitetura em camadas, o aplicativo de software é dividido em várias camadas horizontais, com cada camada localizada no topo de uma camada inferior. Cada camada é dependente em uma ou mais camadas abaixo dela (dependendo se as camadas estão abertas ou fechadas), mas é independente das camadas acima dele.

6.5.1. Camadas abertas versus camadas fechadas

As arquiteturas em camadas podem ter camadas projetadas para serem abertas ou fechadas. Com um fechado camada, as solicitações que estão fluindo para baixo na pilha da camada acima devem passar por ela e não pode contornar isso. Por exemplo, em uma arquitetura de três camadas com apresentação, negócios e camadas de dados, se a camada de negócios for fechada, a camada de apresentação deve enviar todas as solicitações para a camada de negócios e não pode contorná-la para enviar uma solicitação diretamente à camada de dados.

Camadas fechadas fornecem camadas de isolamento, o que torna o código mais fácil de alterar, escrever e compreendo. Isso torna as camadas independentes umas das outras, de modo que as alterações feitas em uma camada do aplicativo não afetarão os componentes das outras camadas. Se as camadas forem abertas, isso aumenta a complexidade. A capacidade de manutenção é reduzida porque várias camadas agora podem chamar para outra camada, aumentando o número de dependências e fazendo mais alterações difícil.

No entanto, pode haver situações em que seja vantajoso ter uma camada aberta. Um de eles é resolver um problema comum com a arquitetura em camadas, em que desnecessário o tráfego pode resultar quando cada camada deve ser passada, mesmo se uma ou mais delas estiver apenas passando solicitações para a próxima camada.

Em nosso exemplo de arquitetura de três camadas com camadas de apresentação, negócios e dados, digamos que introduzamos uma camada de serviços compartilhados entre as camadas de negócios e de dados. Esta camada de serviços compartilhados pode conter componentes reutilizáveis necessários para vários componentes na camada de negócios. Podemos optar por colocá-lo abaixo da camada de negócios para que apenas a camada de negócios tem acesso a ele. No entanto, agora todos os pedidos da empresa camada para a camada de dados deve passar pela camada de serviços compartilhados, embora nada seja necessário dessa camada. Se

abrirmos a camada de serviços compartilhados, as solicitações aos dados camada pode ser feita diretamente a partir da camada de negócios.

O ponto importante para os arquitetos de software entenderem ao projetar uma arquitetura é que há vantagens em camadas fechadas e na obtenção de camadas de isolamento. No entanto, arquitetos de software experientes entendem quando pode ser apropriado abra uma camada. Não é necessário abrir ou fechar todas as camadas. Você pode escolha seletivamente quais camadas, se houver, estão abertas.

6.5.2. Camadas lógicas versus camadas físicas

Você pode ter ouvido os termos camada e camada em referência a arquiteturas em camadas. Antes continuarmos discutindo arquiteturas em camadas, esses termos devem ser esclarecidos. Camadas são separações lógicas de um aplicativo de software e camadas são físicas.

Ao particionar a lógica do aplicativo, as camadas são uma forma de organizar a funcionalidade e componentes. Por exemplo, em uma arquitetura de três camadas, a lógica pode ser separada em apresentação, negócios e camadas de dados. Quando uma arquitetura de software é organizada em mais de uma camada, é conhecida como arquitetura multicamadas. Camadas diferentes não necessariamente devem estar localizado em máquinas físicas diferentes. É possível ter várias camadas na mesma máquina.

As camadas se preocupam com a localização física da funcionalidade e dos componentes. UMA arquitetura de três camadas com apresentação, negócios e camadas de dados implica que essas três camadas foram fisicamente implantadas em três máquinas separadas e cada uma está sendo executada em essas máquinas separadas. Quando uma arquitetura de software é particionada em várias camadas, é conhecido como uma arquitetura multicamadas.

Lembre-se de que algumas pessoas usam os dois termos alternadamente. Ao comunicar com outros, se a distinção for importante, você pode querer ser preciso em seu idioma e você pode precisar confirmar com a outra pessoa o que eles querem dizer quando usam um dos termos.

6.5.3. Vantagens das arquiteturas em camadas

Existem alguns benefícios importantes em usar uma arquitetura em camadas. Este padrão reduz complexidade alcançando uma Separation of Concerns (SoC). Cada camada é independente e você pode entendê-lo sozinho, sem as outras camadas. A complexidade pode ser abstraída em um aplicativo em camadas, o que nos permite lidar com problemas mais complexos.

As dependências entre as camadas podem ser minimizadas em uma arquitetura em camadas, que ainda reduz a complexidade. Por exemplo, a camada de apresentação não precisa depender diretamente na camada de dados e a camada de negócios não depende da camada de apresentação. Minimizar as dependências também permite que você substitua as implementações de uma determinada camada sem afetar as outras camadas.

Outra vantagem das arquiteturas em camadas é o fato de que podem fazer o desenvolvimento mais fácil. O padrão é difundido e bem conhecido por muitos desenvolvedores, o que torna seu uso fácil para a equipe de desenvolvimento. Devido à forma como a arquitetura separa a lógica da aplicação, combina bem com quantas organizações contratam seus recursos e alocar tarefas durante um projeto. Cada camada requer um conjunto de habilidades específico e adequado recursos podem ser atribuídos para trabalhar em cada camada. Por exemplo, desenvolvedores de IU para a camada de apresentação e desenvolvedores de back-end para as camadas de negócios e de dados.

Este padrão de arquitetura aumenta o atributo de qualidade de testabilidade de aplicativos de software. Particionar o aplicativo em camadas e usar interfaces para a interação entre camadas nos permite isolar uma camada para teste e simular ou criar stub nas outras camadas. Para por exemplo, você pode realizar testes de unidade em classes em sua camada de negócios sem a apresentação e camadas de dados. A camada de negócios não depende da camada de apresentação e a camada de dados pode ser simulada ou fragmentada.

Os aplicativos que usam uma arquitetura em camadas podem ter níveis mais elevados de capacidade de reutilização, se mais do que um aplicativo pode reutilizar a mesma camada. Por exemplo, se vários aplicativos visam o mesmo negócio e/ou camadas de dados, essas camadas são reutilizáveis.

Quando um aplicativo que usa uma arquitetura em camadas é implantado em diferentes camadas, há Benefícios adicionais:

- Há maior escalabilidade à medida que mais hardware pode ser adicionado a cada camada, fornecendo a capacidade de lidar com cargas de trabalho maiores.
- Um aplicativo multicamadas pode experimentar maiores níveis de disponibilidade quando várias máquinas são usadas por camada. O tempo de atividade é aumentado porque, se um hardware a falha ocorre em uma camada, outras máquinas podem assumir.
- Ter camadas separadas aumenta a segurança, pois os firewalls podem ser colocados entre as várias camadas.
- Se uma camada pode ser reutilizada para vários aplicativos, isso significa que a camada física também pode ser reutilizada.

6.5.4. Desvantagens das arquiteturas em camadas

Embora arquiteturas em camadas sejam comumente usadas, e por boas razões, existem desvantagens de usá-los. Embora as camadas possam ser projetadas para serem independentes, um a mudança de requisitos pode exigir mudanças em várias camadas. Este tipo de acoplamento diminui a agilidade geral do aplicativo de software.

Por exemplo, adicionar um novo campo exigirá alterações em várias camadas: a apresentação camada para que possa ser exibida, a camada de negócios para que possa ser validado/salvo/processado, e a camada de dados porque precisará ser adicionado ao base de dados. Isso pode complicar a implantação porque, mesmo para uma mudança como esta, um o aplicativo pode exigir a implantação de várias partes (ou mesmo de todo o aplicativo).

Outra desvantagem menor é o fato de que mais código será necessário para formulários. Isso é para fornecer as interfaces e outras lógicas que são necessárias para a comunicação entre as várias camadas.

As equipes de desenvolvimento devem ser diligentes em colocar o código na camada correta para não vazar lógica para uma camada que pertence a outra camada. Exemplos disso incluem colocação de negócios lógica na camada de apresentação ou colocar a lógica de acesso a dados na camada de negócios.

Embora os aplicativos com bom desempenho possam ser projetados com arquiteturas em camadas, se você estiver projetando um aplicativo de alto desempenho, deve estar ciente de que pode haver ineficiências em fazer uma solicitação passar por várias camadas. Além disso, mudar de uma camada para outra às vezes requer que as representações de dados sejam

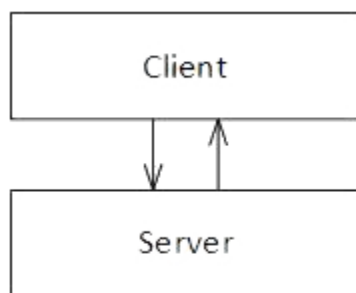
transformadas. Uma maneira de mitigar esta desvantagem é permitir que algumas camadas sejam abertas, mas isso só deve ser feito se é apropriado abrir uma camada.

Existem algumas desvantagens adicionais para arquiteturas em camadas quando são implantadas para vários níveis:

- A desvantagem de desempenho das arquiteturas em camadas já foi mencionada, mas quando essas camadas são implantadas para separar camadas físicas, como é comum, há um custo de desempenho adicional. Com hardware moderno, este o custo pode ser pequeno, mas ainda não será mais rápido do que um aplicativo executado em uma única máquina.
- Há um custo monetário maior associado a uma arquitetura multicamadas. Quanto mais máquinas forem usadas para a aplicação, maior será o custo geral.
- A menos que a hospedagem do aplicativo de software seja tratada por um provedor de nuvem ou caso contrário, foi terceirizado, uma equipe interna será necessária para gerenciar o hardware físico de um aplicativo multicamadas.

6.6. Arquitetura cliente-servidor (arquitetura de duas camadas)

As arquiteturas em camadas se tornaram muito predominantes com a popularidade do software cliente-servidor sistemas. Em um aplicativo distribuído que usa uma arquitetura cliente-servidor, também conhecida como arquitetura de duas camadas, clientes e servidores comunicam-se diretamente. Um cliente solicita algum recurso ou chama algum serviço fornecido por um servidor e o servidor responde às solicitações dos clientes. Pode haver vários clientes conectados a um único servidor:



A parte do cliente do aplicativo contém o código da interface do usuário e o servidor contém o banco de dados, que tradicionalmente tem sido um sistema de gerenciamento de banco de dados relacional (RDBMS). A maior parte da lógica do aplicativo em uma arquitetura cliente-servidor está localizada no servidor, mas parte dele também pode estar localizado no cliente. A lógica do aplicativo localizada no servidor pode existir em componentes de software, no banco de dados ou em ambos.

Quando o cliente contém uma parte significativa da lógica e está lidando com uma grande parte de a carga de trabalho, é conhecido como cliente grosso ou gordo. Quando o servidor está fazendo isso, o cliente é conhecido como cliente fino.

Em alguns aplicativos cliente-servidor, a lógica de negócios é espalhada entre o cliente e o servidor. Se a consistência não for aplicada, pode ser difícil saber sempre onde um determinado pedaço de lógica está localizado.

Se uma equipe não for diligente, a lógica de negócios pode ser duplicada no cliente e no servidor, violando o princípio DRY. Pode haver casos em que o mesmo pedaço de lógica seja necessário tanto no cliente quanto no servidor. Por exemplo, pode haver lógica de negócios necessária para a interface do usuário para validar uma parte dos dados antes de enviá-los ao servidor. O servidor pode precisar deste mesmo lógica de negócios porque também precisa realizar essa validação. Ao centralizar essa lógica pode exigir comunicação adicional entre o cliente e o servidor, a alternativa (duplicação) reduz a capacidade de manutenção. Se a lógica de negócios mudasse, teria que ser modificado em vários lugares.

6.6.1. Usando procedimentos armazenados para lógica de aplicativo

Se a lógica do aplicativo existir no banco de dados, ela é comumente encontrada em procedimentos armazenados. UM procedimento armazenado é um agrupamento de uma ou mais Structured Query Language (SQL) declarações que formam uma unidade lógica para realizar alguma tarefa. Pode ser usado para fazer uma combinação de recuperação, inserção, atualização e exclusão de dados.

Costumava ser popular usar procedimentos armazenados com aplicativos cliente-servidor porque seus o uso reduziu a quantidade de tráfego de rede entre o cliente e o servidor. Armazenados procedimentos podem conter qualquer número de instruções dentro deles e podem chamar outros procedimentos. Uma única chamada do cliente para o servidor é tudo o que é necessário para executar um armazenamento procedimento. Se essa lógica não foi encapsulada dentro de um procedimento armazenado, várias chamadas precisariam ser feitas pela rede entre o cliente e o servidor para executar a mesma lógica.

Os procedimentos armazenados são compilados na primeira vez em que são executados, momento em que uma execução plana é criado para que um mecanismo de consulta de banco de dados possa usar para otimizar seu uso em chamadas. Além de alguns benefícios de desempenho, há vantagens de segurança em usar procedimentos armazenados. Os usuários e aplicativos não precisam receber permissões para os objetos de banco de dados subjacentes que um procedimento armazenado usa, como tabelas de banco de dados. Um usuário ou aplicativo pode executar um procedimento armazenado, mas é o procedimento armazenado que tem controle sobre qual lógica é executada e quais objetos de banco de dados são usados.

Procedimentos armazenados aumentam a capacidade de reutilização, uma vez que um é escrito e compilado, pode ser reutilizado em vários locais.

Embora haja benefícios em usar procedimentos armazenados, há desvantagens. Existem construções de codificação limitadas, em comparação com linguagens de programação de alto nível, que são disponíveis para uso na lógica do aplicativo. Ter alguma de sua lógica de negócios localizada em armazenada procedimentos também significa que sua lógica de negócios não está centralizada.

No desenvolvimento de aplicativos modernos, a lógica do aplicativo não deve ser armazenada procedimentos. Ele pertence fora da camada de dados, independente e desacoplado do mecanismo usado para armazenar os dados. Com base nesse ponto, alguns relegaram procedimentos para operações simples de CRUD. No entanto, se for esse o caso, os procedimentos armazenados são não fornecendo muitos benefícios.

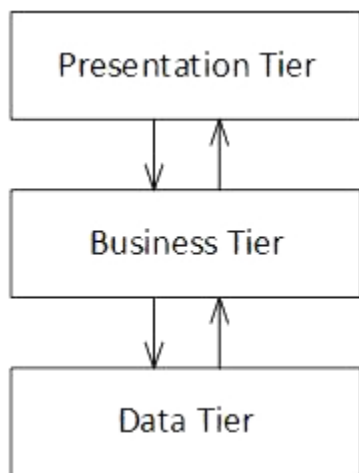
Embora a lógica do aplicativo não deva ser colocada em procedimentos armazenados, eles ainda podem ser de usar em algumas situações. Para consultas complexas (por exemplo, consultas SQL com tabela complexa junções e cláusulas WHERE) e consultas que requerem várias

instruções e grandes quantidades de dados, as vantagens de desempenho dos procedimentos armazenados podem ser úteis.

6.7. Arquitetura N-tier

Com uma arquitetura de n camadas, também conhecida como arquitetura multicamadas, existem várias camadas na arquitetura. Uma das variações mais utilizadas deste tipo de arquitetura é a arquitetura de três camadas. O surgimento da web coincidiu com uma mudança de arquiteturas de duas camadas (cliente-servidor) para arquiteturas de três camadas. Esta mudança não foi uma coincidência. Com aplicativos da web e o uso de navegadores da web, aplicativos rich cliente contendo lógica de negócios não eram ideais.

A arquitetura de três camadas separa a lógica em camadas de apresentação, negócios e dados:



6.7.1. Camada de apresentação

A camada de apresentação fornece funcionalidade para a IU do aplicativo. Deve fornecer um design visual atraente, pois é a parte do aplicativo com a qual os usuários interagem e veem. Os dados são apresentados ao usuário e a entrada é recebida dos usuários nesta camada. Aspectos do atributo de qualidade de usabilidade, que foi discutido no Capítulo 4, Atributos de qualidade de software, deve ser a preocupação da camada de apresentação.

Esta camada deve conter lógica para renderizar a interface do usuário, incluindo a colocação de dados nos componentes de IU apropriados, formatar os dados que são apresentados de forma adequada e ocultar/mostrar os componentes da IU conforme necessário.

Deve também fornecer alguma validação básica para ajudar os usuários a evitar ou minimizar erros, como para garantir que o tipo correto de dados está sendo inserido para um determinado controle e que os dados estão no formato correto. Os desenvolvedores devem ter cuidado para não introduzir lógica de negócios na validação, que deve ser tratada pela camada de negócios.

A camada de apresentação deve fornecer aos usuários um feedback útil, como amigável e mensagens informativas, dicas de ferramentas, feedback visual, como uma barra de progresso para longa duração processos e notificações para informar os usuários sobre a conclusão ou falha de operações assíncronas.

Os arquitetos de software devem se esforçar para projetar clientes finos que minimizem a quantidade de lógica que existe na camada de apresentação. A lógica na camada de

apresentação deve se concentrar no usuário preocupações de interface. Uma camada de apresentação desprovida de lógica de negócios será mais fácil de testar.

6.7.2. Camada de negócios

A camada de negócios, que às vezes é chamada de camada de aplicativo, fornece a implementação para a lógica de negócios do aplicativo, incluindo coisas como negócios regras, validações e lógica de cálculo. Entidades comerciais para o domínio do aplicativo são colocados nesta camada.

A camada de negócios coordena o aplicativo e executa a lógica. Pode executar detalhadamente processa e toma decisões lógicas. A camada de negócios é o centro do aplicativo e atua como um intermediário entre a apresentação e as camadas de dados. Ele fornece a camada de apresentação com serviços, comandos e dados que pode usar e interage com a camada de dados para recuperar e manipular dados.

6.7.3. Camada de dados

A camada de dados fornece funcionalidade para acessar e gerenciar dados. A camada de dados contém um armazenamento de dados para armazenamento persistente, como um RDBMS. Ele fornece serviços e dados para a camada de negócios.

Existem variações de arquiteturas de n camadas que vão além de apenas três camadas. Por exemplo, em alguns sistemas, há um acesso a dados ou camada de persistência, além de dados ou banco de dados camada. A camada de persistência contém componentes para acesso a dados, como um objeto relacional ferramenta de mapeamento (ORM), e a camada de banco de dados contém o armazenamento de dados real, como como um RDBMS. Uma razão para separá-los em duas camadas distintas é se você quiser capacidade de trocar seu acesso a dados ou tecnologia de banco de dados por um diferente.

6.8. Arquitetura orientada a eventos

Um evento é a ocorrência de algo considerado significativo em um aplicativo de software, tal como uma mudança de estado, que pode ser do interesse de outros aplicativos ou outros componentes dentro o mesmo aplicativo. Um exemplo de um evento é a colocação de um pedido de compra ou a postagem de uma nota de carta para um curso que um aluno está fazendo.

Uma arquitetura orientada a eventos (EDA) é uma arquitetura de software distribuída e assíncrona padrão que integra aplicativos e componentes por meio da produção e manuseio de eventos. Ao rastrear eventos, não perdemos nada de significativo relacionado ao negócio domínio.

Os EDAs são fracamente acoplados. O produtor de um evento não tem nenhum conhecimento em relação aos assinantes do evento ou quais ações podem ocorrer como resultado do evento. SOA pode complementar EDA porque as operações de serviço podem ser chamadas com base em eventos que estão sendo provocado. O inverso também pode ser projetado, de modo que as operações de serviço gerem eventos.

Os EDAs podem ser relativamente complexos, devido ao seu processamento distribuído assíncrono inerente. Como acontece com qualquer arquitetura distribuída, podem ocorrer problemas devido à falta de capacidade de resposta, problemas de desempenho ou falhas com mediadores de eventos e brokers de eventos (esses componentes serão descritos em breve).

6.8.1. Canais de eventos

Antes de cobrirmos as duas topologias de evento principais para EDAs, vamos examinar o conceito de evento canais porque ambas as topologias fazem uso deles.

As mensagens de eventos contêm dados sobre um evento e são criadas por produtores de eventos. Essas mensagens de eventos usam canais de eventos, que são fluxos de mensagens de eventos, para viajar para um processador de eventos.

Canais de eventos são normalmente implementados como filas de mensagens, que usam o ponto a ponto padrão de canal ou tópicos de mensagem, que usam o padrão publicar-assinar.

6.8.2. Filas de mensagens

As filas de mensagens garantem que haja um, e apenas um, receptor para uma mensagem. No contexto de canais de evento, isso significa que apenas um processador de evento receberá um evento de um canal de eventos. O padrão de canal ponto a ponto é utilizado para fila de mensagens implementações.

6.8.3. O padrão de canal ponto a ponto

O padrão de canal ponto a ponto é um padrão de mensagens usado quando queremos garantir que haverá exatamente um receptor para uma determinada mensagem. Se um canal tem vários receptores para que mais de uma mensagem possa ser consumida simultaneamente, e mais de um receptor tenta consumir uma mensagem, o canal do evento irá garantir que apenas um deles são bem-sucedidos. Uma vez que o canal de eventos está lidando com isso, ele remove qualquer necessidade de coordenação entre os processadores de eventos.

6.8.4. Tópicos de mensagens

Os tópicos de mensagens permitem que vários consumidores de eventos recebam uma mensagem de evento. O publicar e assinar padrão é usado para a implementação de tópicos de mensagem.

6.8.5. O padrão publicar-assinar

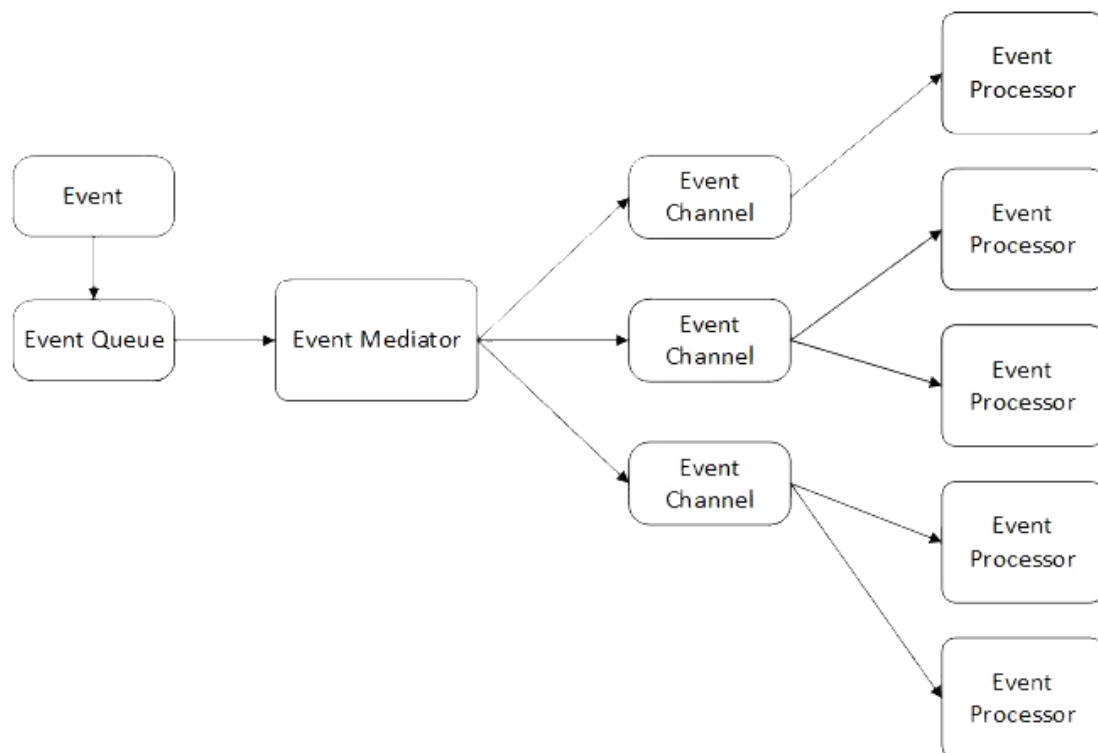
O padrão publicar-assinar, que às vezes é chamado de pub/sub para abreviar, é um padrão de mensagem que fornece uma maneira para um remetente (editor) transmitir uma mensagem para partes interessadas (assinantes).

Em vez de editores enviando mensagens diretamente para destinatários específicos, como no ponto-a-ponto padrão de canal, as mensagens podem ser enviadas sem qualquer conhecimento dos assinantes ou mesmo se não houver assinantes. Da mesma forma, permite que os assinantes mostrem interesse em uma mensagem particular sem qualquer conhecimento dos editores ou mesmo se não houver editores.

6.8.6. Topologias de arquitetura orientada a eventos

As duas topologias principais para EDAs são as topologias do mediador e do broker. A topologia do mediador usa uma topologia de mediador para EDAs usa uma única fila de eventos e um mediador de eventos para rotear eventos para os processadores de eventos relevantes. Esta topologia é comumente usada quando várias etapas são necessárias para processar um evento.

Com a topologia de mediador, os produtores de eventos enviam eventos para uma fila de eventos. Lá pode haver muitas filas de eventos em um EDA. As filas de eventos são responsáveis pelo envio de mensagens de eventos para o mediador do evento. Todos esses eventos, chamados de eventos iniciais, passam por um mediador de eventos. O mediador do evento então executa qualquer orquestração necessária:



Depois que o mediador de eventos orquestra cada evento da fila de eventos, ele cria um ou mais eventos de processamento assíncrono com base na orquestração. Esses eventos de processamento sejam enviados a um canal de eventos, que pode ser uma fila de mensagens ou um tópico de mensagens.

Os tópicos de mensagens são mais comumente usados com a topologia do mediador devido à orquestração envolvida, o que permite múltiplos processadores de eventos, que irão realizar tarefas diferentes, para receber uma mensagem de evento. Isso pode ser visto no diagrama anterior onde alguns dos canais de eventos estão enviando uma mensagem de evento para vários eventos processadores.

Os processadores de eventos ouvem nos canais de eventos para coletar os eventos e processá-los em linha com seu design. Na seção de estilos de processamento de eventos posteriormente neste capítulo, abordaremos os diferentes estilos de processamentos normalmente usados por processadores de eventos.

6.8.7. Implementações de mediador de eventos

Um mediador de eventos pode ser implementado de várias maneiras diferentes. Para simples orquestrações, um hub de integração pode ser usado. Eles normalmente permitem que você defina regras de mediação usando uma linguagem específica de domínio (DSL) para o roteamento de eventos. Linguagens específicas de domínio, ao contrário de uma linguagem de uso geral, como C #, Java ou UML, permite que expressões sejam escritas para um domínio particular.

Para uma orquestração mais complexa de eventos, Business Process Execution Language (BPEL) pode ser usado em conjunto com um motor BPEL. BPEL é uma linguagem baseada em XML que é usado para definir processos de negócios e seu comportamento. É frequentemente usado com SOA e serviços da web.

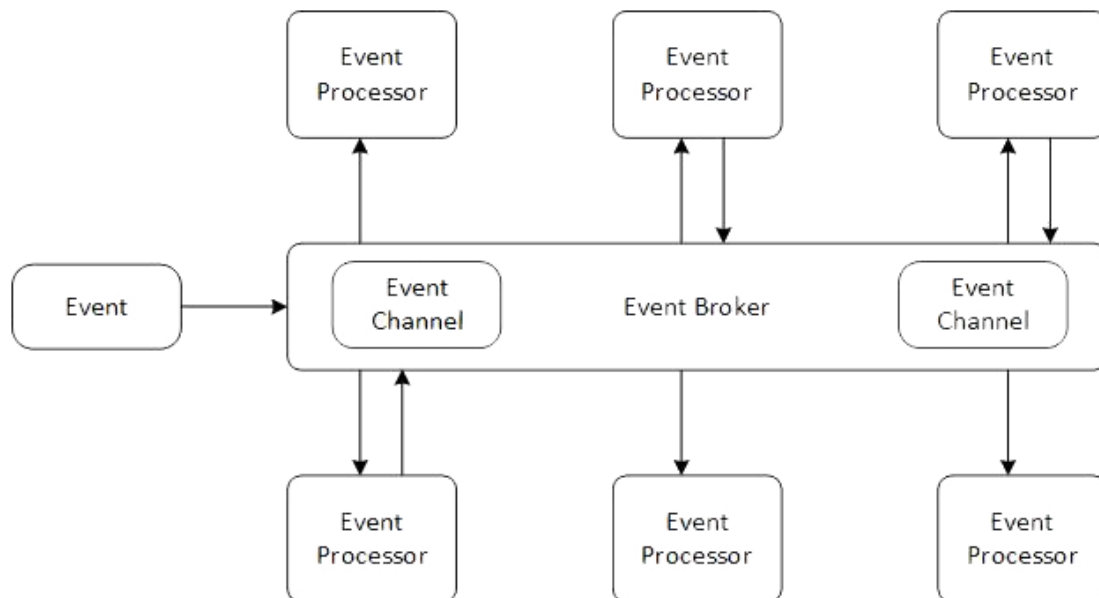
Grandes aplicativos de software com necessidades complexas de orquestração, que podem até incluir interações humanas, podem optar por implementar um mediador de eventos que usa um processo de negócios gerente (BPM). O gerenciamento de processos de negócios envolve modelagem, automação e execução de fluxos de trabalho de negócios. Alguns gerentes de processos de negócios usam processos de negócios Model and Notation (BPMN) para definir os processos de negócio. BPMN permite negócios modelagem de processos usando uma notação gráfica. Diagramas de processos de negócios criados com BPMN são semelhantes aos diagramas de atividades em UML.

Os arquitetos de software devem compreender as necessidades do aplicativo de software para selecionar uma implementação apropriada do mediador de eventos.

6.8.8. A topologia do broker

Em uma topologia de broker, as mensagens de evento criadas por produtores de evento entram em um broker de evento, às vezes chamado de barramento de eventos. O broker de eventos contém todos os canais de eventos usado para o fluxo de eventos. Os canais de eventos podem ser filas de mensagens, tópicos de mensagens ou alguma combinação dos dois.

Ao contrário da topologia do mediador, não há fila de eventos com a topologia do broker. O evento os processadores são responsáveis por pegar eventos de um broker de eventos:



Esta topologia é ideal quando o fluxo de processamento é bastante simples e não há necessidade de orquestração centralizada de eventos. Os eventos fluem para o processador de eventos a partir do broker de eventos e, como parte do processamento, novos eventos podem ser criados.

Isso pode ser visto no diagrama anterior, onde, em alguns casos, os eventos estão fluindo de os processadores de eventos de volta ao broker de eventos. Uma chave para a topologia do broker é o encadeamento eventos para executar uma determinada tarefa de negócios.

6.8.9. Estilos de processamento de eventos

Processadores de eventos são componentes que têm uma tarefa específica e contêm lógica para analisar e agir em eventos. Cada processador de eventos deve ser independente e fracamente acoplado com outros processadores de eventos.

Uma vez que as mensagens de evento chegam aos processadores de evento, existem três estilos predominantes de processamento eventos: processamento de evento simples (SEP), processamento de fluxo de evento (ESP) e evento complexo processamento (CEP). O tipo de processamento de evento necessário para um determinado aplicativo depende sobre a complexidade de processamento necessária. EDAs podem utilizar uma combinação destes três estilos.

6.8.9. Processamento de evento simples (SEP)

No SEP, eventos notáveis são imediatamente encaminhados para iniciar algum tipo de downstream ação. Este tipo de processamento pode ser usado para fluxos de trabalho em tempo real para reduzir a quantidade de tempo entre o evento ocorrendo e a ação resultante.

O processamento de eventos simples também inclui funcionalidades como a transformação de esquemas de eventos de um formulário para outro, gerando vários eventos com base na carga de um único evento e complementando uma carga útil do evento com dados adicionais.

6.8.10. Processamento de fluxo de eventos (ESP)

ESP envolve a análise de fluxos de dados de eventos e, em seguida, a tomada de qualquer ação necessária com base nessa análise. Os eventos são selecionados com base em várias condições e, em seguida, uma ação pode ser considerada para o evento ou o evento pode ser ignorado. Este tipo de processamento é ideal para fluxos em tempo real em que as decisões também estão envolvidas. Um exemplo é um sistema de negociação de ações em que um evento ocorre e entra em um evento stream quando um ticker da bolsa relata uma mudança no preço. Com base no preço, um algoritmo determina se uma ordem de compra ou venda deve ser criada e notifica o apropriado assinante, se necessário.

6.8.11. Processamento de eventos complexos (CEP)

No CEP, a análise é realizada para encontrar padrões em eventos para determinar se mais evento complexo ocorreu. Um evento complexo é um evento que resume ou representa um conjunto de outros eventos. Os eventos podem ser correlacionados em várias dimensões, como causais, temporal ou espacial.

O CEP pode ocorrer por um período mais longo, em comparação com os outros tipos de evento em processamento. Este tipo de processamento de eventos pode ser usado para detectar ameaças de negócios, oportunidades ou outras anomalias.

Um exemplo de funcionalidade que usa CEP é um mecanismo de fraude de cartão de crédito. Cada transação em um cartão de crédito é um evento e o sistema analisará um agrupamento desses eventos para um cartão de crédito específico para tentar encontrar um padrão que possa indicar a ocorrência de fraude. Se fraude de processamento é detectada, ação downstream é iniciada.

6.8.12. Tipos de funcionalidade orientada a eventos

EDA pode significar coisas diferentes. Existem três tipos principais de funcionalidade que podem normalmente são encontrados em sistemas que têm um EDA: notificação de evento, estado transportado por evento transferência e fonte de eventos. Os sistemas de software orientados a eventos podem fornecer uma combinação de os três.

6.8.13. Notificação de evento

Uma arquitetura que fornece notificação de eventos é aquela em que o sistema de software envia uma mensagem quando um evento ocorre. Esta funcionalidade é a mais comum em software sistemas que possuem um EDA. As topologias de mediador e broker nos permitem implementar notificações de eventos.

Há um acoplamento fraco entre o produtor do evento e os consumidores de qualquer evento, bem como entre a lógica que envia mensagens de evento e a lógica que responde aos eventos. Esse o acoplamento fraco nos permite mudar a lógica de um sem afetar o outro. Evento os componentes do processador são de propósito único e independentes de outros processadores de eventos, permitindo que sejam modificados sem afetar os outros.

A desvantagem do acoplamento fraco entre produtores de eventos e consumidores de eventos é que pode ser difícil ver o fluxo lógico das notificações de eventos. Esta complexidade adicional também torna mais difícil depurar e manter. Não há declarações específicas que você possa olhar em para ver qual lógica será executada. Uma variedade de consumidores de eventos, incluindo aqueles em sistemas de software diferentes daquele que produziu a notificação de

evento, podem reagir a um evento. Às vezes, a única maneira de entender o fluxo lógico é monitorar seus sistemas para veja o fluxo de mensagens de eventos.

6.8.14. Transferência de estado realizada por evento

A transferência de estado transportada por evento é uma variação da notificação de evento. Seu uso não é tão comum quanto notificação regular de eventos. Quando um consumidor de evento recebe uma notificação de evento, ele pode precisar de mais informações do produtor do evento para realizar a ação que desejam pegar.

Por exemplo, um sistema de vendas pode enviar uma nova notificação de evento de pedido e um sistema de envio pode se inscrever para este tipo de mensagens de evento. No entanto, para tomar as medidas adequadas, o sistema de remessa agora precisa de informações adicionais sobre o pedido, como a quantidade e tipo de itens de linha que estão no pedido. Isso requer que o sistema de envio consulte o sistema de vendas de alguma forma, como por meio de uma API, para obter essas informações.

Embora o editor do evento não precise saber nada sobre seus assinantes, o assinante é acoplado ao produtor no sentido de que precisa estar ciente do produtor e ter uma maneira de obter mais informações do produtor.

Retornos de chamada para o sistema que produziu uma notificação de evento para mais dados, a fim de lidar com um evento que aumenta a carga e o tráfego da rede. Uma maneira de resolver isso é adicionar estado informações aos eventos para que contenham informações suficientes para serem úteis para potenciais consumidores. Por exemplo, uma notificação de evento para um novo pedido pode conter o item de linha detalhes necessários para o sistema de remessa para que nenhum retorno de chamada seja necessário. O sistema de envio pode manter sua própria cópia apenas dos detalhes do pedido de que precisa.

Embora mais dados sejam repassados, ganhamos um maior nível de disponibilidade e resiliência. O sistema de remessa pode funcionar, pelo menos com pedidos já recebidos, mesmo se o sistema de pedidos estiver temporariamente indisponível. O sistema de envio não precisa ligar de volta para o sistema de pedidos após o recebimento da notificação de evento inicial, que pode ser particularmente benéfico se o contato e o recebimento de dados do sistema de pedidos forem lentos.

No entanto, com maior disponibilidade vem menor consistência. A replicação de alguns dados entre os sistemas de pedido e remessa diminui a consistência dos dados.

6.8.15. Fonte de eventos

Um sistema pode usar um armazenamento de dados para ler e atualizar o estado atual do aplicativo, mas e se existem requisitos para saber os detalhes das mudanças de estado que nos levaram ao atual apontar? Com o sourcing de eventos, os eventos que ocorrem em um sistema, como mudanças de estado, são persistidos em um armazenamento de eventos. Ter um registro completo de todos os eventos que ocorreram permite que ele sirva como uma fonte de verdade. A repetição de eventos de um registro de eventos pode ser usada para recriar o estado de um aplicativo.

A fonte de eventos funciona de maneira semelhante a um log de transações em um sistema de banco de dados. O log de transações registra todas as modificações feitas em um banco de dados. Esse permite reversões de transações e nos permite recriar o estado do sistema até um ponto específico, como imediatamente antes da ocorrência de uma falha.

Os eventos devem ser imutáveis, pois representam algo que já aconteceu. As ações podem ocorrer a jusante como resultado de um evento, portanto, se um evento pudesse ser alterado após o fato, ele pode colocar o sistema em um estado inconsistente. Se uma atualização ou o cancelamento de um evento é necessário, um evento de compensação deve ser criado.

A lógica de compensação pode ser executada com base em tais eventos e pode aplicar o necessário regras de negócios para aplicar contra operações. Isso irá garantir que o armazenamento de eventos ainda seja uma fonte da verdade e que podemos repetir todos os eventos para recriar o estado de um aplicativo.

Os benefícios da fonte de eventos incluem o fato de que pode ajudar na depuração de um sistema. Isto fornece a capacidade de pegar eventos e executá-los através do sistema para ver como o sistema vai se comportar. Isso pode ser usado para determinar a causa de um problema. Fonte de eventos também fornece auditoria detalhada. O registro completo dos eventos nos permite ver o que aconteceu, como aconteceu, quando aconteceu e outros detalhes.

Embora o sourcing de eventos possa ser muito útil, ele introduz alguma complexidade adicional em um sistema de software. Várias instâncias de um aplicativo e aplicativos multithread podem ser eventos persistentes em um armazenamento de eventos. O sistema deve ser projetado para garantir que os eventos são processados na ordem correta.

O código que processa eventos e o esquema de eventos podem mudar com o tempo. Consideração deve ser fornecido para garantir que eventos mais antigos, possivelmente com esquemas de eventos diferentes, ainda possam ser reproduzidos com a lógica atual.

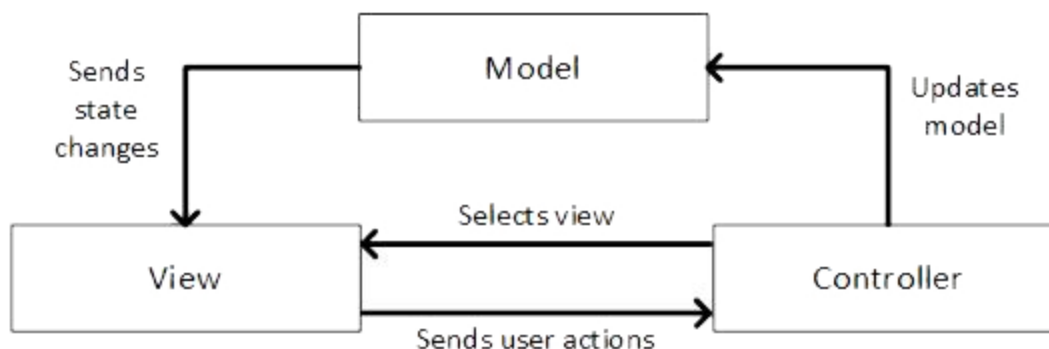
Se parte da sequência de eventos inclui o uso de um sistema externo, deve-se considerar ser fornecido para armazenar as respostas do sistema externo como eventos. Isso irá garantir que podemos reproduzir eventos com precisão para reconstruir o estado do aplicativo sem ter que chamar o sistema externo novamente.

6.9. O padrão Model-View-Controller

O padrão Model-View-Controller (MVC) é um padrão de arquitetura de software que é amplamente utilizado para a IU de um aplicativo. É particularmente adequado para aplicativos da web, embora também possa ser usado para outros tipos de aplicativos, como aplicativos de desktop.

O padrão fornece uma estrutura para a construção de interfaces de usuário e fornece uma separação de as diferentes responsabilidades envolvidas. Uma série de aplicativos e sites populares frameworks de desenvolvimento fazem uso desse padrão. Alguns exemplos incluem Ruby on Rails, ASP.NET MVC e Spring MVC.

O padrão MVC consiste no modelo, visualização e controlador:



O modelo, a visualização e o controlador têm responsabilidades distintas para a interface do usuário. Vamos dar uma olhada em cada um deles mais de perto.

6.9.1. Modelo

O modelo gerencia os dados do aplicativo e o estado. Entre suas responsabilidades está o processamento de dados de e para um armazenamento de dados, como um banco de dados. Um modelo é independente dos controladores e das visualizações, permitindo que sejam reutilizados com diferentes interfaces de usuário.

Isso também permite que eles sejam testados independentemente. Os modelos recebem diretivas dos controladores para recuperar e atualizar os dados. Os modelos também fornecem atualizações de estado do aplicativo. Em algumas variações do MVC, o modelo é passivo e deve receber uma solicitação para enviar uma atualização de estado do aplicativo. Em outras variações, uma visualização pode estar ativa e notificações push de mudanças de estado do modelo para uma visualização.

6.9.2. Visualização

A camada de visualização é responsável pela apresentação do aplicativo. É a parte do aplicativo que é visível para o usuário. A visualização exibe dados para o usuário em uma apropriada interface com base nas informações recebidas do controlador. Se o modelo está fornecendo o estado do aplicativo é atualizado diretamente para as visualizações, as visualizações também podem ser atualizadas com base nestas notificações.

Conforme os usuários manipulam uma visualização, como fornecer entrada ou fornecer alguma ação do usuário, o view irá enviar essas informações para um controlador.

6.9.3. Controlador

Conforme os usuários navegam em um aplicativo da web, as solicitações são encaminhadas para o controlador apropriado com base na configuração de roteamento. Um controlador atua como um intermediário entre o modelo e o visualizar.

Um controlador executa a lógica do aplicativo para selecionar a visualização apropriada e envia as informações de que ele precisa para processar a interface do usuário. As visualizações notificam os controladores do usuário ações para que o controlador possa responder a elas. Os controladores irão atualizar o modelo com base nas ações do usuário.

6.9.4. Vantagens do padrão MVC

Usar o padrão MVC permite uma separação de interesses. Separando a apresentação a partir dos dados, é mais fácil alterar um deles sem afetar o outro. Isso também torna cada parte mais

fácil de testar. No entanto, é difícil conseguir uma separação completa. Para exemplo, adicionar um novo campo ao aplicativo exigirá uma mudança nos dados e a apresentação.

O padrão MVC torna os objetos de apresentação mais reutilizáveis. Separando a interface do usuário dos dados permite que os componentes da IU sejam reutilizados. Isso também significa que um modelo pode ser reutilizado com mais de uma visualização.

A separação da apresentação da lógica de negócios e dados permite que os desenvolvedores se especializem em desenvolvimento front-end ou back-end. Isso também pode acelerar o processo de desenvolvimento, pois algumas tarefas podem ocorrer em paralelo. Por exemplo, um desenvolvedor pode trabalhar na interface do usuário enquanto outro trabalha na lógica de negócios.

6.9.5. Desvantagens do padrão MVC

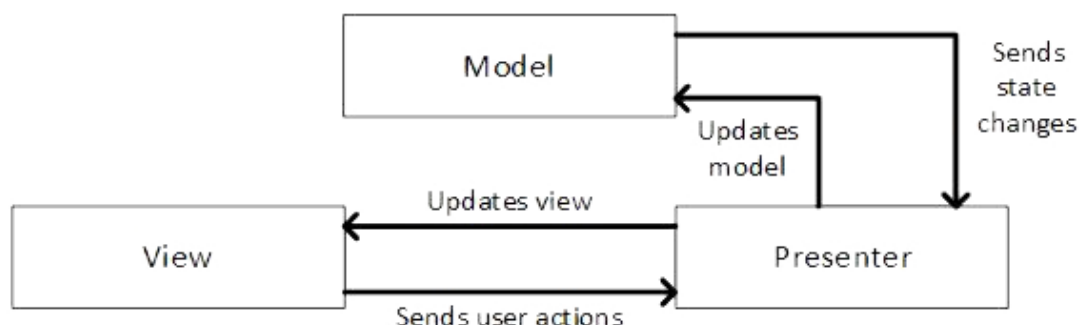
Se sua equipe de desenvolvimento não estiver configurada para que os desenvolvedores se concentrem no front-end ou desenvolvimento de backend, isso exige que os desenvolvedores sejam qualificados em ambas as áreas (full-stack desenvolvedores). Isso exigirá desenvolvedores qualificados em várias tecnologias.

Se um modelo está muito ativo e está fornecendo notificações diretamente para visualizações, mudanças frequentes a um modelo podem resultar em atualizações excessivas nas visualizações.

6.10. O padrão Model-View-Presenter

O padrão Model-View-Presenter (MVP) é uma variação do padrão MVC. Como o Padrão MVC, ele fornece uma separação entre a lógica da IU e a lógica de negócios. No entanto, o apresentador ocupa o lugar do controlador no padrão MVC.

Cada visualização no padrão MVP normalmente possui uma interface correspondente (interface de visualização). Os apresentadores são acoplados às interfaces de visualização. Em comparação com o padrão MVC, o view é mais fracamente acoplada ao modelo porque os dois não interagem um com o outro diretamente:



Os aplicativos da web e de desktop podem usar o padrão MVP. Os principais componentes deste padrão são o modelo, a visualização e o apresentador.

6.10.1. Modelo

Como era o caso com o padrão MVC, o modelo representa o modelo de negócios e os dados. Ele interage com o banco de dados para recuperar e atualizar dados. O modelo recebe mensagens do apresentador para atualizações e relatórios de alterações de estado para o apresentador.

Os modelos no padrão MVP não interagem diretamente com as vistas e apenas interagem com o apresentador.

6.10.2. Visualização

A visualização é responsável por exibir a interface do usuário e os dados. Cada visualização no MVP padrão implementa uma interface (interface de exibição). Conforme o usuário interage com a visualização, o view enviará mensagens ao apresentador para agir sobre os eventos e dados.

Os apresentadores são fracamente acoplados às visualizações por meio da interface de visualização. Visualizações são mais passiva no modelo MVP e conta com o apresentador para fornecer informações sobre o que exibição.

6.10.3. Apresentador

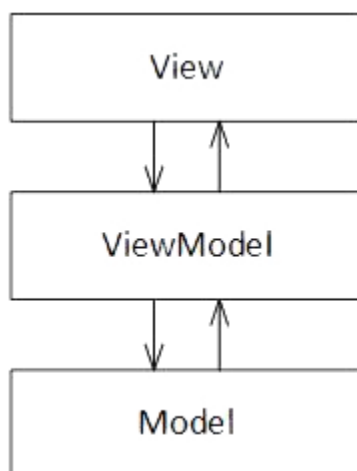
O apresentador é o intermediário entre o modelo e a visualização. Ele interage com ambos eles. Cada visualização possui um apresentador e a visualização notifica o apresentador sobre as ações do usuário. O apresentador atualiza o modelo e recebe mudanças de estado do modelo.

Um apresentador receberá dados do modelo e os formatará para exibição na exibição, tomando um papel ativo na lógica de apresentação. Os apresentadores encapsulam a lógica e as visualizações da apresentação desempenham um papel mais passivo.

Ao contrário do padrão MVC, onde um controlador pode interagir com múltiplas visualizações, no MVP padrão, cada apresentador normalmente lida com uma, e apenas uma, visualização.

6.11. O padrão Model-View-ViewModel

O padrão Model-View-ViewModel (MVVM) é outro padrão de arquitetura de software e compartilha semelhanças com MVC e MVP no sentido de que todos fornecem um SoC. Particionamento as várias responsabilidades tornam um aplicativo mais fácil de manter, estender e testar. O padrão MVVM separa a IU do resto do aplicativo:



Normalmente, há uma quantidade significativa de interação entre visualizações e ViewModels, facilitado pela vinculação de dados. O padrão MVVM funciona bem para aplicativos de desktop ricos, embora possa ser usado para outros tipos de aplicativos, como aplicativos da web e móveis.

Um exemplo de estrutura que pode ser usada para construir aplicativos MVVM é o Windows Apresentação Foundation (WPF). Os principais componentes do MVVM são Model, View e ViewModel. Vamos dar uma olhada em cada um deles com mais detalhes.

6.11.1. Modelo

O modelo no padrão MVVM desempenha um papel semelhante ao MVC e MVP. Representa o objeto de domínio de negócios e os dados. O modelo usa o banco de dados para recuperar e atualizar dados.

Em aplicativos MVVM, pode haver ligação direta com as propriedades do modelo. Como resultado, os modelos geralmente geram notificações de alteração de propriedade.

6.11.2. Visualização

A visualização é responsável pela interface do usuário. É a parte do aplicativo que é visível para os usuários. No padrão MVVM, a visualização está ativa. Ao contrário de uma função passiva onde a visão é completamente manipulado por um controlador ou apresentador, e não tem conhecimento do modelo, em visualizações MVVM estão cientes do modelo e ViewModel.

Embora as visualizações tratem de seus próprios eventos, elas não mantêm o estado. Eles devem retransmitir o usuário ações para o ViewModel, o que pode ser feito por meio de um mecanismo, como vinculação de dados ou comandos. Um objetivo com o padrão MVVM é minimizar a quantidade de código nas visualizações.

6.11.3. ViewModel

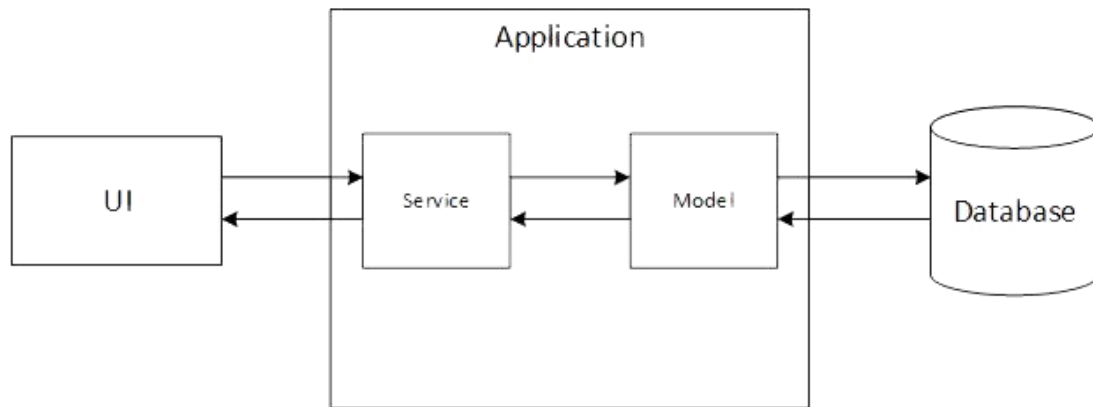
O ViewModel no padrão MVVM é semelhante aos objetos de controlador e apresentador que cobrimos com os padrões MVC e MVP em que eles coordenam entre a visão e o modelo. ViewModels fornecem dados para visualizações para exibição e manipulação, e contêm lógica de interação para se comunicar com visualizações e modelos. ViewModels deve ser capaz de manipulação de ações do usuário e entrada de dados enviados de visualizações. É o ViewModel que contém lógica de navegação para lidar com a movimentação para uma visualização diferente.

Views e ViewModels se comunicam por meio de vários métodos, como vinculação de dados, comandos, chamadas de método, propriedades e eventos.

6.12. A Responsabilidade da Consulta de Comando

6.12.1. Padrão de segregação

Segregação de responsabilidade de consulta de comando (CQRS) é um padrão em que o modelo que é usado para ler as informações é separado do modelo que é usado para atualizar as informações. Em uma arquitetura mais tradicional, um único modelo de objeto é usado para leitura e atualizando dados:



Compromissos tornam-se necessários para usar um modelo de objeto único, pois as classes de domínio são necessárias para servir a todos os propósitos. A mesma representação de uma entidade deve oferecer suporte a todas as operações de criação, leitura, atualização e exclusão (CRUD), tornando-as maiores do que precisa ser em todas as circunstâncias.

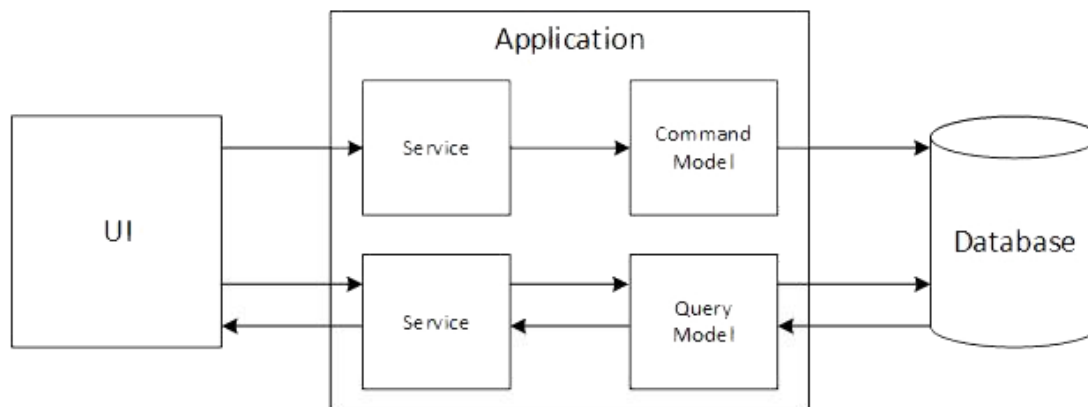
Eles contêm todas as propriedades de que o objeto precisará para vários cenários. Se a aula é mais do que apenas um objeto de transferência de dados (DTO), ele também pode conter métodos de comportamento.

Com esta abordagem, as aulas não são ideais para todas as situações em que precisam ser usados, pois muitas vezes há uma incompatibilidade entre o que é exigido pela leitura e gravação representações dos dados. Isso também pode tornar o gerenciamento de segurança e autorização mais complexo, pois cada classe é usada para operações de leitura e gravação.

Em um domínio colaborativo, várias operações podem estar ocorrendo em paralelo no mesmo conjunto de dados. Existe o risco de contenção de dados se os registros forem bloqueados ou se houver conflitos de atualização devido a atualizações simultâneas. As cargas de trabalho entre as tarefas de leitura e gravação são diferentes, o que significa que eles também têm diferentes requisitos de desempenho e escalabilidade.

6.13. O modelo de consulta e o modelo de comando

Uma maneira de superar os desafios de usar um único modelo de objeto para consultas e comandos é separar os dois. Este padrão, conhecido como CQRS, resulta em dois modelos. O modelo de consulta é responsável por leituras e o modelo de comando é responsável para atualizações:



Os objetos de consulta apenas retornam dados e não alteram o estado, enquanto os objetos de comando alteram o estado e não retornam dados. Outra maneira de olhar para este conceito é fazer uma pergunta (a consulta) não deve alterar a resposta. A fim de realizar uma ação que mudará o estado, um comando é usado.

Quando o sistema precisa ler os dados, ele passa pelo modelo de consulta e quando o sistema precisa atualizar os dados, ele passa pelo modelo de comando. Como parte do processamento de um comando, o sistema pode precisar ler dados, mas além do que é necessário para completar um comando, a leitura dos dados deve passar pelo modelo de consulta.

Embora não seja obrigatório, se o CQRS for levado para o próximo nível, os modelos de consulta e comando pode ser feito para utilizar bancos de dados separados. Isso dá a cada um dos dois modelos o seu próprio esquema, que pode ser otimizado para seu uso específico. Se forem usados bancos de dados separados, os dois bancos de dados devem ser mantidos em sincronia. Uma maneira de fazer isso é por meio de eventos.

6.13.1. Usando fonte de eventos com CQRS

Embora o CQRS possa ser usado sem eventos, eles se complementam, por isso é comum para sistemas que usam CQRS para alavancar o uso de eventos. Os eventos são uma forma de comunicar efetivamente as mudanças de estado para que o modelo de consulta possa se manter atualizado conforme o modelo de comando atualiza os dados.

Como vimos na seção Fonte de eventos anteriormente neste capítulo, a fonte de eventos envolve eventos persistentes que ocorrem em um sistema de tal forma que o armazenamento de eventos pode servir como o registro da verdade. Quando o modelo de comando altera o estado do sistema, os eventos podem ser gerados para que o modelo de consulta possa ser mantido em sincronia.

Manter os modelos de consulta e comando em sincronia é necessário quando a consulta e modelos de comando usam armazenamentos de dados separados. Além disso, um armazenamento de dados para o modelo de consulta pode conter dados desnormalizados que foram otimizados para consultas específicas. Tendo um armazenamento de eventos nos permite reproduzir eventos passados para recriar o estado atual do sistema, que pode ser útil para atualizar os dados desnormalizados no armazenamento de dados do modelo de consulta.

6.13.2. Vantagens do CQRS

O CQRS é adequado para domínios complexos e fornece uma separação de interesses que ajuda para minimizar e gerenciar a complexidade. Separando um sistema em consulta e comando os modelos o tornam mais sustentável, extensível e flexível. As equipes de desenvolvimento também podem ser organizadas de forma que uma equipe se concentre no modelo de consulta enquanto outra se concentra no modelo de comando.

Separar a responsabilidade entre comandos e consultas pode ajudar a melhorar desempenho, escalabilidade e segurança. O desempenho pode ser melhorado otimizando o esquema especificamente para cada modelo. O esquema para o modelo de consulta pode ser otimizado para consultas, enquanto o esquema para o modelo de comando pode ser otimizado para atualizações. Dados em o armazenamento de dados do modelo de consulta pode ser desnormalizado para aumentar o desempenho de consultas que o aplicativo precisa para executar.

As cargas de trabalho entre as operações de leitura e gravação serão diferentes e o uso do CQRS permite que você dimensionar cada um deles independentemente dos outros. A segurança é melhorada com CQRS porque torna mais fácil do que com um único modelo de objeto para garantir que apenas as classes certas podem atualizar dados.

A segurança pode ser mais fácil de implementar e testar ao usar CQRS porque cada classe é usada para leituras ou gravações, mas não ambas. Isso reduz a chance de expor inadvertidamente dados e operações que não devem estar disponíveis para um usuário específico em um determinado contexto.

6.13.3. Desvantagens do CQRS

Para sistemas que simplesmente precisam de operações básicas de CRUD, a implementação de um sistema CQRS pode introduzir complexidade desnecessária. Os sistemas CQRS têm maior complexidade, especialmente quando combinado com fonte de eventos. Por este motivo, é importante entenda que o CQRS não se aplica a todas as situações. Os arquitetos de software devem ser cientes de que o CQRS não precisa ser aplicado à totalidade de um sistema de software. Pode ser aplicado a apenas alguns subsistemas de uma arquitetura maior onde será o maior beneficiar.

Embora o uso de diferentes armazenamentos de dados para modelos de consulta e comando possa melhorar o desempenho e aumentar a segurança, você deve considerar que, ao realizar operações de leitura, pode estar lendo dados desatualizados.

Se bancos de dados separados são usados em um sistema CQRS, o banco de dados usado para leituras deve ser mantido atualizado com o banco de dados usado para gravações. O sistema de software seguirá um eventual modelo de consistência onde, se nenhuma nova atualização for feita em um determinado item, eventualmente todo o acesso a esse item irá adquirir os dados mais recentes.

Se o sistema está usando fonte de eventos ou algum outro mecanismo para manter os dois sincronização, haverá um certo atraso, mesmo que seja pequeno, antes de serem feitas consistente. Isso significa que quaisquer dados lidos podem estar obsoletos se o último as atualizações ainda não foram aplicadas. Isso está em contraste com um modelo de consistência forte, em que todas as alterações de dados são atômicas e uma transação não pode ser

concluída até que todas as mudanças foram concluídas com sucesso ou, no caso de uma falha, tudo foi desfeito.

6.14. Arquitetura Orientada a Serviços

Arquitetura orientada a serviços (SOA) é um padrão de arquitetura para o desenvolvimento de software sistemas, criando serviços interoperáveis fracamente acoplados que trabalham juntos para automatizar processos de negócios. Um serviço é uma parte de um aplicativo de software que executa uma tarefa específica tarefa, fornecendo funcionalidade a outras partes do mesmo aplicativo de software ou a outro Aplicações de Software. Alguns exemplos de consumidores de serviço incluem aplicativos da web, aplicativos móveis, aplicativos de desktop e outros serviços.

SOA alcança um SoC, que é um princípio de design que separa um sistema de software em partes, com cada parte abordando uma preocupação distinta. Discutimos SoC no Capítulo 6, Software Princípios e práticas de desenvolvimento. Um aspecto fundamental do SOA é que ele decompõe o aplicativo lógica em unidades menores que podem ser reutilizadas e distribuídas. Ao decompor um grande problema em preocupações menores e mais gerenciáveis satisfeitas pelos serviços, a complexidade é reduzida e a qualidade do software é melhorada.

Cada serviço em uma SOA encapsula uma certa parte da lógica. Essa lógica pode ser responsável para uma tarefa muito específica, um processo de negócios ou um subprocesso. Os serviços podem variar em tamanho e um serviço pode ser composto de vários outros serviços para realizar sua tarefa.

6.14.1. O que torna SOA diferente de outras distribuídas soluções?

Distribuir a lógica do aplicativo e separá-lo em unidades menores e mais gerenciáveis não é o que torna a SOA diferente das abordagens anteriores de computação distribuída. Naturalmente, você pode pensar que a maior diferença é o uso de serviços da web, mas tenha em mente que SOA não requer serviços da web, embora seja uma tecnologia perfeita para implementar com SOA. O que realmente diferencia SOA de um sistema tradicional arquitetura não é o uso de serviços da web, mas como seus componentes principais são projetados.

Embora SOA compartilhe semelhanças com soluções distribuídas anteriores, é muito mais do que apenas mais uma tentativa de criar software reutilizável. As diferenças, sem dúvida, fornecem novo valor significativo para as organizações, quando implementado de maneira adequada. Muitos benefícios podem ser realizados a partir de um SOA devidamente projetado.

6.14.2. Benefícios de usar um SOA

Existem vários benefícios em usar um SOA, incluindo:

- Aumenta o alinhamento entre negócios e tecnologia
- Promove a federação dentro de uma organização
- Permite diversidade de fornecedores
- Aumenta a interoperabilidade intrínseca
- Funciona bem com metodologias de desenvolvimento ágil

6.14.3. Aumenta o alinhamento entre negócios e tecnologia

Uma SOA leva a um maior alinhamento entre negócios e tecnologia. O cumprimento de requisitos de negócios precisa de lógica de negócios e processos de negócios para serem precisos representada em soluções de tecnologia. Lógica de negócios, na forma de entidades de negócios e processos de negócios, existe na forma de serviços físicos com uma SOA.

Esse alinhamento de negócios e tecnologia promove agilidade organizacional. Mudança é algo que praticamente todas as organizações devem enfrentar, e existe devido a uma variedade de fatores, como forças de mercado, mudanças tecnológicas, novas oportunidades de negócios e fusões corporativas. Independentemente da causa da mudança, SOA fornece agilidade organizacional por meio da abstração de serviço e do acoplamento fraco entre a lógica de negócios e de aplicativos.

Quando mudanças são necessárias, elas podem ser feitas mais facilmente para que o negócio permaneça em alinhamento com a tecnologia.

6.14.4. Promove a federação dentro de uma organização

A SOA promove federação. Federação em uma organização é um ambiente no qual os aplicativos e recursos de software trabalham juntos, ao mesmo tempo em que mantêm sua autonomia. Federação dá às organizações a liberdade de não serem obrigadas a substituir os seus sistemas existentes que devem funcionar juntos. Contanto que haja um comum, aberto e estrutura padronizada, aplicativos legados e não legados podem trabalhar juntos.

As organizações têm a flexibilidade de escolher se desejam substituir determinados sistemas, permitindo que eles usem uma abordagem em fases para a migração.

6.14.5. Permite diversidade de fornecedores

Outra vantagem de SOA é a diversidade de fornecedores. Além de permitir que organizações com fornecedores potencialmente díspares para trabalharem juntos, uma organização pode usar fornecedores diferentes internamente para obter as melhores soluções.

Embora não seja um objetivo de SOA aumentar a diversidade do fornecedor, ele oferece a opção de fornecedor diversidade quando há vantagem na introdução de novas tecnologias.

6.14.6. Aumenta a interoperabilidade intrínseca

SOA fornece maior interoperabilidade intrínseca para uma organização. Isso permite que compartilhamento de dados e reaproveitamento da lógica. Diferentes serviços podem ser reunidos para ajudar automatizar uma variedade de processos de negócios. Pode permitir que um sistema de software existente se integre com outros por meio de serviços da web. Maior interoperabilidade pode levar à realização de outros objetivos estratégicos.

6.14.7. Funciona bem com metodologias de desenvolvimento ágil

Outro benefício do SOA é que ele se presta bem ao desenvolvimento ágil de software metodologias. O fato de que sistemas de software complexos são divididos em serviços com unidades pequenas e gerenciáveis de lógica se adaptam bem a um processo iterativo e como as tarefas são atribuídos aos recursos.

Você também pode descobrir que ter desenvolvedores assumindo tarefas é mais fácil com SOA porque cada tarefa pode ser administrada em tamanho e mais facilmente compreendida. Embora este seja benéfico para qualquer desenvolvedor, é particularmente útil para desenvolvedores juniores ou aqueles que são novos em um projeto e pode não ter tanta experiência com a funcionalidade e negócios domínio.

6.14.8. Análise de custo-benefício de SOA

Como arquiteto de software, se estiver considerando SOA para um aplicativo, você precisará explicar as razões pelas quais você está considerando isso. Adotar um SOA tem algum custo, mas há pontos que você pode fazer para justificar os custos e maneiras de aliviá-los.

O custo de implementação de uma SOA pode superar os benefícios para algumas organizações, então cada caso deve ser considerado separadamente. Embora possa não ser apropriado para algumas organizações para implementar uma SOA, para outros, uma SOA devidamente projetada trará muitos benefícios, incluindo um retorno positivo do investimento.

A adoção de SOA pode ser um processo gradual e evolutivo. Porque um SOA contemporâneo promove federação, a criação de uma SOA não precisa ser um processo tudo ou nada. Um a organização não precisa substituir todos os sistemas existentes de uma vez. A lógica legada pode ser encapsulado e pode funcionar com uma nova lógica de aplicação. Como resultado, a adoção de SOA e seus custos relacionados podem ser distribuídos ao longo do tempo.

A adoção de SOA leva à redução das despesas de integração. Um SOA fracamente acoplado deve reduzir a complexidade e, portanto, reduzirá o custo de integração e gerenciamento de tais sistemas.

Os serviços fracamente acoplados são mais flexíveis e podem ser usados em mais situações. SOA pode aumentar a reutilização de ativos. É comum que cada aplicativo seja construído isoladamente, levando a maiores custos de desenvolvimento e maiores custos de manutenção ao longo do tempo. No entanto, pôr a criação de processos de negócios através da reutilização de serviços existentes, custos e tempo de entrada no mercado pode ser reduzido.

SOA aumenta a agilidade dos negócios. Mudança é algo que todas as organizações devem enfrentar. Independentemente da causa da mudança, usando serviços fracamente acoplados, a agilidade aumenta e o tempo e o custo para se adaptar às mudanças são reduzidos.

A adoção de SOA reduz o risco e a exposição do negócio. Um SOA devidamente projetado facilita o controle de processos de negócios, permite a implementação de segurança e privacidade políticas e fornece trilhas de auditoria para dados, o que pode reduzir o risco. Também pode ajudar com conformidade regulatória. As penalidades por não conformidade podem ser significativas, e SOA pode fornecer às organizações maior visibilidade de negócios que reduz o risco de mudança regulamentos.

6.14.9. Desafios com SOA

Embora a adoção de SOA ofereça muitos benefícios, também apresenta alguns novos complexidades e desafios. As soluções SOA podem permitir que as organizações façam mais, incluindo automatizando mais seus processos de negócios. Isso pode fazer com que as arquiteturas corporativas cresçam em escopo e funcionalidade em comparação com sistemas legados. Assumindo um grande o escopo da funcionalidade aumentará a complexidade de um sistema de software.

Em uma SOA, novas camadas podem ser adicionadas às arquiteturas de software, fornecendo mais áreas onde podem ocorrer falhas, tornando mais difícil identificá-las. Além disso, como um número crescente de serviços é criado, implantando novos serviços e novas versões de os serviços existentes devem ser gerenciados com cuidado para que a solução de problemas possa ser eficaz quando um erro ocorre com uma transação específica.

Outro desafio com a adoção bem-sucedida de SOA está relacionado às pessoas e não à tecnologia. SOA é um estilo arquitetônico maduro que existe há muito tempo. A tecnologia existe para permitir que as organizações automatizem uma variedade de processos de negócios complexos. Contudo, as pessoas podem ser um desafio para a adoção de SOA porque ainda existem técnicas e negócios profissionais que não estão familiarizados com SOA e realmente não sabem o que isso significa. Pessoas também pode ser naturalmente resistente a mudanças, e se sua organização ainda não estiver usando SOA, a mudança será necessária. Para que um SOA seja bem-sucedido, deve haver adesão de as pessoas na organização. Eles têm que estar comprometidos e isso se resume à cultura da equipe e das pessoas nela, incluindo gerentes, arquitetos de software, desenvolvedores e analistas de negócios.

6.14.10. Princípios-chave para orientação a serviços

As soluções orientadas a serviços são projetadas de forma que cumpram certos princípios-chave. Elas estão:

- Contrato de serviço padronizado
- Serviço de acoplamento solto
- Abstração de serviço
- Reutilização de serviço
- Autonomia de serviço
- Apátrida de serviço
- Descoberta de serviço
- Composibilidade de serviço

Esses princípios são detalhados no livro de Thomas Erl, *Service-Oriented Architecture, Second Edição*. Os princípios de orientação a serviços são aplicados à análise orientada a serviços e fases de design do ciclo de vida de entrega SOA.

6.14.11. Contrato de serviço padronizado

Cada serviço deve ter um contrato de serviço padronizado, consistindo em uma interface técnica e descrição do serviço. Mesmo que queiramos que os serviços sejam independentes, eles devem aderir a um acordo comum para que as unidades de lógica possam manter um certo nível de standardização.

A fim de ter contratos de serviço padronizados, todos os contratos de serviço dentro de um determinado o inventário de serviços deve seguir um conjunto de padrões de projeto. A padronização permite interoperabilidade e permite que a finalidade dos serviços seja mais facilmente compreendida.

6.14.12. Serviço de acoplamento solto

Os serviços devem ser fracamente acoplados e independentes uns dos outros. Os contratos de serviço devem ser projetados para ter independência dos consumidores de serviço e de suas implementações. Os serviços fracamente acoplados podem ser modificados de forma mais rápida e fácil. Contratos de serviço de dissociação de suas implementações permite que os contratos de serviço sejam modificados com impacto mínimo para consumidores de serviço e implementações de serviço. Ao minimizar as dependências entre serviços, cada serviço pode mudar e evoluir de forma independente, minimizando os efeitos de essas mudanças em outros serviços.

6.14.13. Abstração de serviço

Os contratos de serviço devem conter apenas informações que são necessárias para revelar, e serviço as implementações também devem ocultar seus detalhes. Qualquer informação que não seja essencial para o uso eficaz do serviço pode ser abstraída.

As decisões de design, como a tecnologia usada para um serviço, podem ser abstraídas. Esse segue o princípio de ocultação de informações que foi abordado no Capítulo 6, Software Princípios e práticas de desenvolvimento. Se uma decisão de projeto precisar ser alterada posteriormente, o objetivo é que pode ser feito com impacto mínimo.

6.14.14. Reutilização de serviço

Os serviços devem ser projetados com a reutilização em mente, com sua lógica de serviço sendo independente de uma tecnologia ou processo de negócios específico. Quando os serviços podem ser reutilizados para diferentes fins, as equipes de desenvolvimento de software obtêm maior produtividade, levando a economia de custos e tempo.

A capacidade de reutilização do serviço aumenta a agilidade organizacional porque as organizações podem usar serviços para responder às novas necessidades de automação comercial. Os serviços existentes podem ser compostos juntos para criar soluções para novos problemas ou para aproveitar uma nova oportunidade.

Os serviços reutilizáveis podem acelerar o desenvolvimento e permitir que um recurso ou produto alcance o mercado mais rápido. Em alguns casos, isso pode ser crítico para um projeto.

Decompor tarefas em mais serviços para reutilização requer mais análise e, potencialmente, apresenta mais complexidade. No entanto, quando os serviços reutilizáveis são projetados corretamente, há pode ser uma economia significativa de custos a longo prazo. Se surgir uma necessidade que seja satisfeita por um existente serviço, os recursos não precisam ser dedicados a trabalhar nele.

A reutilização de serviços leva a software de maior qualidade porque os serviços existentes já foram testados. Eles podem já estar em produção e, se houver algum defeito no serviço, eles podem já ter sido expostos e corrigidos.

6.14.15. Autonomia de serviço

Os serviços devem ser projetados para serem autônomos, com mais independência de seus ambientes de tempo de execução. O projeto deve buscar fornecer serviços com maior controle sobre seus ambientes de tempo de execução.

Quando os serviços podem operar com menos dependência de recursos em seu tempo de execução ambientes que eles não podem controlar, isso leva a um melhor desempenho e maior confiabilidade desses serviços em tempo de execução.

6.14.16. Apátrida de serviço

Os projetos de serviço devem se esforçar para minimizar a quantidade de gerenciamento de estado que ocorre, e separar os dados do estado dos serviços. Os serviços podem reduzir o consumo de recursos se não gerenciarem o estado quando é desnecessário, o que lhes permitirá lidar com mais solicitações de forma confiável. Tendo apátrida em serviços melhora a escalabilidade do serviço e melhora a capacidade de reutilização dos serviços.

6.14.17. Descoberta de serviço

Os serviços precisam ser descobertos. Incluindo metadados consistentes e significativos com um serviço, a finalidade do serviço e a funcionalidade que ele fornece podem ser comunicados. Os desenvolvedores de serviço são obrigados a fornecer esses metadados. Os serviços devem ser descobertos por humanos que pesquisam manualmente, bem como por softwares aplicativos pesquisando programaticamente. Os serviços devem estar cientes uns dos outros para que possam interagir.

6.14.18. Composibilidade de serviço

Os serviços devem ser projetados de forma que possam ser combinados. Esta é a capacidade de usar um serviço em qualquer número de outros serviços, e esses serviços podem ser compostos por outros Serviços.

Alguns outros princípios de orientação a serviços facilitam a composição de serviços. Serviço a composição está fortemente relacionada à reutilização do serviço. A capacidade de criar soluções por compor os serviços existentes fornece às organizações um dos mais importantes SOA benefícios: agilidade organizacional.

6.14.19. Estratégias de entrega SOA

Existem três estratégias principais de entrega de SOA: de cima para baixo, de baixo para cima e ágil. Uma entrega

A estratégia é necessária para coordenar a entrega de aplicativos, negócios e serviços de processo. As três estratégias de entrega de SOA refletem as principais abordagens da arquitetura de software projeto que foi abordado no Capítulo 5, Projetando arquiteturas de software.

6.14.20. A estratégia de cima para baixo

A estratégia de cima para baixo começa com a análise. Centra-se na lógica de negócios da organização e requer que os processos de negócios se tornem orientados a serviços. A abordagem de cima para baixo, quando feito corretamente, resulta em um SOA de alta qualidade. Cada serviço é analisado minuciosamente, e, como resultado, a reutilização é maximizada. A desvantagem é que essa abordagem requer muitos recursos, em termos de tempo e dinheiro. Há um pré-trabalho substancial que deve ocorrer com a estratégia de cima para baixo. Se uma organização tem tempo e dinheiro para investir em um projeto, então este pode ser um método eficaz abordagem.

Deve-se notar que, porque a análise ocorre no início, pode ser bastante tempo antes de quaisquer resultados serem obtidos. Isso pode ou não ser aceitável para um determinado projeto. A fim de realizar de forma significativa a análise orientada a serviços e o design orientado a serviços estágios do ciclo de vida SOA, a estratégia de cima para baixo deve ser usada pelo menos até certo ponto.

6.14.21. A estratégia de baixo para cima

A abordagem ascendente, em contraste, começa com os próprios serviços da web. Eles são criados conforme necessário. Os serviços da Web são projetados e implantados com base em precisa.

A integração com um sistema existente é uma motivação comum para usar o sistema ascendente estratégia. As organizações desejam adicionar serviços da web a um ambiente de

aplicativo existente para permitir a integração com um sistema legado. Um serviço de wrapper é criado para expor a lógica em um sistema existente.

Embora essa abordagem seja comum na indústria, não é uma abordagem válida para alcançar SOA. Para criar um SOA válido posteriormente, muito esforço e refatoração provavelmente será obrigatório. Os serviços da Web criados com essa abordagem podem não estar prontos para empresas. Eles são criados para atender a alguma necessidade, então, se você não tomar cuidado, eles não levarão em consideração a empresa como um todo.

6.14.22. A estratégia ágil

A terceira abordagem é uma estratégia ágil, que às vezes é conhecida como um encontro no meio abordagem. É um meio-termo entre as abordagens de cima para baixo e de baixo para cima. Nessa abordagem, a análise pode ocorrer simultaneamente com o design e o desenvolvimento. Assim que uma análise suficiente foi concluída, o design e o desenvolvimento podem começar. Enquanto tal esforços estão em andamento, a análise continua com outra funcionalidade. Esta abordagem combina bem com uma metodologia de desenvolvimento de software ágil e iterativa.

Este é o tipo de abordagem do melhor dos dois mundos em que um design adequado pode ser concluído produzirá todas as qualidades orientadas a serviços. Esta abordagem pode atender às necessidades imediatas ao mesmo tempo em que mantém as qualidades orientadas a serviços da arquitetura.

No entanto, à medida que mais análises são concluídas, esta abordagem pode exigir que os serviços concluídos sejam revisitados. Os serviços podem ficar desalinhados após análise contínua, exigindo que sejam refatorado.

6.14.23. Análise orientada a serviços

A análise orientada a serviços é um estágio no ciclo de vida do projeto SOA e é usada para decidir o que serviços devem ser construídos e qual lógica deve ser encapsulada por cada serviço. O a análise é um processo iterativo que ocorre uma vez para cada processo de negócios. Quando uma equipe está comprometida com a construção de uma SOA, ela deve realizar alguma forma de análise específico para orientação a serviços e além da análise padrão. Uma maneira que as organizações podem melhorar a modelagem de serviço é como eles incorporam a análise orientada a serviços e projetar em seu processo de desenvolvimento de software. Cada organização tem sua própria metodologia de desenvolvimento de software e deve determinar a melhor forma de incluir o serviço modelagem em seu próprio processo.

Arquitetura Orientada a Serviços, Segunda Edição, de Thomas Erl, detalha três etapas para orientação a serviços análise: definição de requisitos de automação de negócios, identificando sistemas de automação e serviços candidatos de modelagem. Definição de requisitos de automação de negócios A primeira etapa na análise orientada a serviços é definir os requisitos de automação de negócios para o processo de negócios que está sendo analisado na iteração atual. Os requisitos podem ser reunidos usando o método normal da organização de elicitar e capturar requisitos.

Com esses requisitos, o processo de negócios que queremos automatizar pode ser documentado em um alto nível. Os detalhes do processo de negócios são usados quando modelamos o candidato Serviços.

6.14.24. Identificação de sistemas de automação existentes

Uma vez que os requisitos foram estabelecidos para a iteração atual, a próxima etapa em a análise orientada a serviços envolve a identificação de quais partes, se houver, do processo de negócios lógica já está automatizada.

Levando em consideração os sistemas existentes que já podem automatizar a totalidade ou parte de qualquer um dos processos de negócios nos permite determinar quais partes dos processos de negócios ainda precisa ser automatizado. Esta informação serve como uma entrada quando modelamos o candidato Serviços.

6.14.25. Serviços candidatos de modelagem

A etapa final, modelagem de serviços candidatos, consiste em identificar a operação de serviço candidatos e agrupando-os em serviços candidatos. É importante notar que estes As operações e serviços candidatos são abstratos e um modelo lógico. Durante o design, outros fatores, como restrições e limitações, serão considerados. O projeto final de concreto pode ser diferente dos candidatos a serviço.

A modelagem de serviços candidatos deve ser um processo colaborativo entre técnicos e recursos de negócios. Analistas de negócios e especialistas em domínio podem usar seu conhecimento de negócios para ajudar a equipe técnica a definir candidatos a serviços.

6.14.26. Camadas de serviço e modelos de serviço

A lógica corporativa consiste em lógica de negócios e de aplicativo. A lógica de negócios é uma implementação dos requisitos de negócios e inclui o negócio de uma organização processos. Esses requisitos incluem coisas como restrições, dependências, pré-condições e pós-condições. A lógica do aplicativo é a implementação da lógica de negócios em uma solução de tecnologia. A lógica do aplicativo pode ser implementada em uma solução adquirida, um desenvolvimento personalizado solução, ou alguma combinação dos dois. A equipe de desenvolvimento trabalha para projetar e desenvolver a lógica do aplicativo. Tópicos como requisitos de desempenho, segurança restrições e dependências do fornecedor são consideradas na solução técnica.

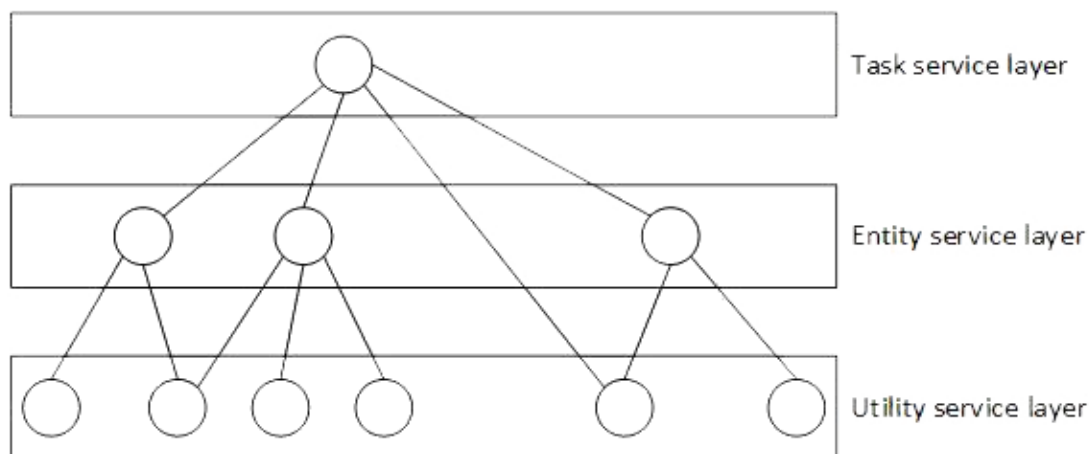
A orientação a serviços está relacionada ao negócio e à lógica do aplicativo porque SOA é uma maneira de representar, executar e compartilhar essa lógica. Os princípios de orientação a serviços podem ser aplicados a lógica de negócios e de aplicativo.

A função dos serviços é realizar os conceitos e princípios introduzidos pela orientação de serviço. Uma camada de serviço em uma arquitetura de software é normalmente colocada entre as camadas de negócios e aplicativos. Isso permite que os serviços representem a lógica de negócios e lógica de aplicação abstrata. Da mesma forma que diferentes aplicativos dentro do aplicativo de uma organização camada pode ser implementada em diferentes tecnologias, serviços dentro das camadas de serviço podem também ser implementado em diferentes tecnologias.

Abstração é uma das características importantes de um SOA e permite outras chaves características, como agilidade organizacional. A abstração é crítica porque abstrair a lógica de negócios e aplicativos permite uma solução orientada a serviços com Serviços. Atingir o nível apropriado de abstração não é uma tarefa trivial, mas pode ser realizado com uma equipe dedicada. Ao criar camadas de abstração, ou camadas de serviço, a equipe pode descobrir como os serviços devem representar o aplicativo e a lógica de negócios, e como para melhor promover a agilidade.

Durante a modelagem de serviço, torna-se aparente que existem alguns tipos comuns de Serviços. Esses tipos são modelos de serviço, que podem ser usados para classificar serviços candidatos. Esses serviços candidatos podem ser agrupados com base em seu modelo de serviço em uma camada de serviço.

Os três modelos de serviço comuns (e camadas) são serviço de tarefa, serviço de entidade e utilitário serviço:



6.14.27. Serviço de tarefas

Este tipo de serviço possui um contexto funcional não agnóstico, o que significa que contém lógica do processo de negócios e foi criado para uma tarefa ou processo de negócios específico.

Serviços de tarefas não têm um grande potencial de reutilização. Os serviços de tarefas normalmente compõem vários serviços em sua lógica e às vezes são referidos como serviços de negócios centrados em tarefas ou serviços de processos de negócios.

Se um serviço de tarefa tem lógica de orquestração extensa ou está hospedado em uma orquestração plataforma, às vezes é referido como um serviço de tarefa orquestrado. Lógica de orquestração automatiza o arranjo, coordenação e gerenciamento de vários serviços para realizar uma tarefa.

6.14.28. Serviço de entidade

Este modelo de serviço tem um contexto funcional agnóstico, o que significa que sua lógica não está vinculada a um único processo de negócios e reutilizável. Os serviços da entidade são serviços centrados nos negócios que são associados a uma ou mais entidades comerciais. Às vezes, eles são chamados de entidades centradas serviços comerciais ou serviços de entidades comerciais.

As entidades de negócios vêm do domínio de negócios e envolvem pessoas da equipe que compreender completamente o domínio de negócios e os processos de negócios serão benéficos em realizar a análise necessária para determinar os serviços apropriados da entidade.

Serviços da entidade podem compor serviços de utilidade em sua lógica.

6.14.29. Serviço de utilidade

Os serviços utilitários, como os serviços de entidade, têm um contexto funcional agnóstico. Eles contêm multiuso lógica e são altamente reutilizáveis. A diferença entre serviços da entidade e utilidade serviços é que os serviços de utilidade não estão associados a uma entidade de negócios ou lógica de negócios.

Os serviços de utilidade se preocupam com as funções relacionadas à tecnologia, como um software preocupações transversais do sistema. Os exemplos incluem registro, armazenamento em cache, notificações, autenticação e autorização.

6.14.30. Design orientado a serviços

A fase de design orientado a serviços começa quando a análise é concluída. Tendo um completo compreensão dos requisitos e uso dos modelos de serviço desde a fase de análise permitirá a criação de um design de serviço adequado.

A fase de design orientado a serviços usa os serviços candidatos lógicos que foram derivados durante a análise orientada a serviços e cria os designs de serviço físico. Antes de projetar a implementação dos serviços, a primeira etapa é projetar as interfaces físicas de serviço. Precisamos tomar decisões sobre as interfaces de serviço com base nos candidatos a serviço, os requisitos que precisam ser atendidos, bem como os padrões da organização e da indústria que será necessário ter um contrato de serviço padronizado. Uma vez que os contratos de serviço foram estabelecidos, a lógica e a implementação do serviço podem ser projetadas.

Projetar as interfaces de serviço e implementá-las são duas etapas distintas. Deveríamos concentrar-se totalmente nos contratos de serviço primeiro, independentemente de suas implementações. Algumas equipes projetar os dois simultaneamente ou pular para o estágio de desenvolvimento e apenas deixar o serviço interface emergir do serviço implementado.

No entanto, os arquitetos de software devem dedicar tempo para considerar os contratos de serviço antes as implementações. Não só precisamos garantir que os contratos de serviço satisfaçam os requisitos, mas eles precisam seguir os princípios-chave da orientação para serviços, que incluem o fato de que eles devem ser fracamente acoplados em suas implementações. Só depois os contratos de serviço foram estabelecidos, precisamos considerar o design do implementações.

6.14.31. Design de interface de serviço

Um dos principais objetivos do design orientado a serviços é derivar a interface física do serviço definições baseadas nos candidatos a serviço que foram determinados durante a orientação ao serviço análise. O design da interface de serviço é significativo porque a fase de design é a primeira vez que a tecnologia real é identificada.

Se você se lembra dos princípios-chave para orientação a serviços, eles precisam ser aplicados ao projeto das interfaces de serviço. Os contratos de serviço precisam ser padronizados entre si e dentro de um inventário de serviço. Eles devem ser fracamente acoplados em suas implementações, com decisões de design abstraídas de modo que as interfaces contenham apenas o que é essencial para consumidores de serviços.

O design da interface de serviço identifica a exposição interna e externa dos serviços. Para Por exemplo, um serviço de postagem de pedidos pode precisar ser usado tanto externa quanto internamente. Esse é um exemplo de serviço que pode precisar de mais de uma interface publicada para o mesmo serviço. Cada interface para o serviço pode expor diferentes operações

e exigirá diferentes níveis de segurança e autenticação. As diferenças devem ser determinadas e cada interface deve ser projetada antes do projeto das implementações.

Além dos desenvolvedores, uma interface de serviço desempenha um papel importante no teste e na qualidade garantia. Os testadores precisam da interface de serviço para projetar seus testes. Uma vez que a interface de serviço é conhecida, um equipamento de teste pode ser criado capaz de invocar o serviço que precisa ser testado. Os serviços precisam ser testados independentemente de outros serviços, bem como dentro dos serviços que irão consumi-lo.

6.14.32. Granularidade da interface de serviço

Tomar decisões sobre a granularidade da interface pode ser muito importante na orientação a serviços Projeto. A granularidade pode ter um impacto significativo no desempenho e em outras questões. Normalmente, uma interface de serviço contém mais de uma operação e as operações de um serviço deve ser semanticamente relacionado.

Operações de serviço refinadas oferecem maior flexibilidade para atender aos consumidores, mas levam a mais sobrecarga de rede, o que pode reduzir o desempenho. Quanto mais granulado operações de serviço que temos, menos flexíveis são, embora reduzam a rede sobrecarga e, portanto, pode melhorar o desempenho.

Os arquitetos de software devem procurar encontrar o equilíbrio certo entre o número de serviços e o número de operações em cada serviço. Você não quer agrupar muitas operações em um único serviço, mesmo que sejam semanticamente relacionadas, porque torna o serviço muito volumoso e difícil de entender. Também pode aumentar o número de serviços versões que precisarão ser lançadas daqui para frente, já que partes do serviço precisam ser modificado. No entanto, se sua interface de serviço for muito refinada, você pode acabar com um número desnecessariamente grande de interfaces de serviço.

6.14.33. Registros de serviço

Um registro de serviço contém informações sobre os serviços disponíveis e é um componente-chave de governança SOA. Ajuda a tornar os sistemas interoperáveis e facilita a descoberta de serviços web. Embora algumas organizações possam descobrir que um registro de serviço não é necessário, muitas implementações de SOA podem se beneficiar de um registro de serviço. Como organização começa a publicar e usar mais e mais serviços da web, alguns dos quais podem estar fora da organização, a necessidade de um registro centralizado se torna mais aparente.

Existem inúmeros benefícios em usar um registro de serviço. Promovendo a descoberta da web serviços, as organizações podem facilitar a reutilização e evitar a construção de vários serviços da web que realizar tarefas semelhantes. Os desenvolvedores podem consultar programaticamente um registro de serviço para descobrir serviços da web já existentes que podem satisfazer suas necessidades. Semelhante aos benefícios que são derivados de qualquer tipo de código reutilizado, a qualidade é melhorada e há um maior nível de confiabilidade com a reutilização de serviços da web. Os serviços da web reutilizados têm já foram testados e geralmente já estão sendo usados com sucesso em outra parte do sistema ou em outro sistema.

Os registros de serviço podem ser privados ou públicos. Como o nome indica, os registros públicos podem incluir qualquer organização. Isso inclui até organizações que não têm nenhuns serviços a oferecer. Os registros privados são restritos aos serviços que a organização se desenvolve ou serviços que alugou ou adquiriu.

Os benefícios dos registros de serviço público incluem a capacidade de encontrar as empresas certas e serviços para uma necessidade específica. Também pode levar a novos clientes ou permitir mais acesso a clientes atuais. Pode permitir que uma organização expanda suas ofertas e estenda seus alcances de mercado. Os registros de serviço podem ser uma ferramenta útil e poderosa para encontrar serviços, uma vez que podem ser pesquisados manualmente por pessoas ou programaticamente através de um API padronizada por um aplicativo. No entanto, devido a esses recursos, as organizações devem ter tempo para decidir se vai usar um registro público ou privado e quais serviços eles querem se registrar neles.

Um dos desafios de implementar um serviço de registro verdadeiramente útil e confiável é a administração do registro. Isso inclui mantê-lo atualizado adicionando novos serviços, remoção de serviços obsoletos e atualização de versões, descrições de serviço e serviço da web Localizações.

6.14.33. Descrições de serviço

Para que os serviços interajam uns com os outros, eles devem estar cientes uns dos outros. Serviço as descrições servem ao importante propósito de fornecer essa consciência. Eles providenciam informações sobre os serviços disponíveis para que os consumidores em potencial possam decidir se um serviço específico irá satisfazer suas necessidades.

As descrições de serviço ajudam a promover o acoplamento fraco, um princípio importante dos SOAs. As dependências entre os serviços são minimizadas, pois os serviços podem trabalhar juntos de forma simples por meio da consciência que têm um do outro por meio de suas descrições de serviço.

Qualquer serviço que deseja atuar como um receptor final deve ter uma descrição do serviço documentos. As descrições de serviço geralmente contêm informações abstratas e concretas. A parte abstrata detalha a interface do serviço sem entrar nos detalhes do específico tecnologias em uso. A beleza da abstração é que a integridade do serviço a descrição é mantida mesmo se os detalhes da implementação técnica forem alterados em o futuro. A descrição abstrata geralmente inclui uma visão geral de alto nível do serviço interface, incluindo as operações que pode realizar. Mensagens de entrada e saída do as operações também são detalhadas.

A parte concreta da descrição do serviço fornece detalhes sobre o transporte físico protocolo que está conectado à interface de serviço da web. Este transporte e localização específicos as informações incluem a ligação (requisitos para o serviço estabelecer uma conexão ou para uma conexão a ser estabelecida com um serviço), porta (endereço físico do serviço da web), e serviço (um grupo de terminais relacionados) para que o serviço da web possa ser usado.

Os possíveis desafios para o desenvolvimento de descrições de serviço incluem o seguinte:

- Decompor os serviços da web de forma adequada com base nas necessidades de negócios
- Determinar a finalidade e as responsabilidades exatas de um serviço específico
- Decidir sobre as operações que um serviço da web precisará fornecer
- Comunicar adequadamente a interface de um serviço na parte abstrata do serviço
- descrição para que os consumidores de serviço em potencial possam tomar uma decisão informada
- baseado em suas necessidades

6.14.34. Estruturação de namespaces

Um namespace é um Uniform Resource Locator (URI) exclusivo. Os namespaces são usados para agrupar serviços relacionados e elementos juntos e para diferenciar entre os diferentes que compartilham o mesmo nome. É importante que os arquitetos de software pensem nos namespaces. Por fornecer um namespace exclusivo, mesmo se sua organização usar serviços de outra, seus elementos terão a garantia de serem únicos. Mesmo que duas organizações tenham um serviço com o mesmo nome, eles serão diferenciados por seu namespace.

Além de fornecer nomes exclusivos, os namespaces são usados para organizar logicamente vários serviços e elementos. Um namespace apropriado deve fornecer significado ao serviço ou elemento para que alguém que esteja olhando para ele possa entender o serviço. Os namespaces facilitam nomear novos serviços, bem como localizar os existentes.

Para selecionar bons namespaces, temos que considerar como os namespaces são estruturados. UMA o nome de domínio da empresa é uma parte importante de um namespace e porque os nomes de domínio são exclusivos, geralmente fazem parte de um namespace. Normalmente, a função segue o domínio nome em um namespace. Isso permitirá a diferenciação entre o esquema (por exemplo, tipos de mensagens) e interfaces (por exemplo, serviços da web).

Uma área de negócios normalmente segue a função na estrutura de um namespace. Aqui é onde especialistas de domínio e analistas de negócios podem ajudar arquitetos de software a chegar a uma estrutura de negócios que faz sentido. Outra parte de um namespace comum é alguma forma de versão ou data. Isso permite a diferenciação entre várias versões do mesmo serviço ou elemento. O controle de versão é outro uso importante de namespaces.

6.14.35. Orquestração e coreografia

A orquestração de serviço e a coreografia de serviço desempenham papéis importantes em SOAs, como eles são abordagens para montar vários serviços para que possam trabalhar juntos. A orquestração representa a lógica do processo de negócios de uma forma padronizada usando serviços. Isto automatiza a execução de um fluxo de trabalho coordenando e gerenciando diferentes serviços. Na orquestração de serviço, há um processo centralizado contendo lógica fixa. Um orquestrador controla o processo decidindo quais serviços invocar e quando invocá-los. A orquestração é análoga ao maestro de uma orquestra, que unifica e dirige performers individuais para criar um desempenho geral.

A interoperabilidade para uma organização é promovida em soluções que usam orquestração devido aos terminais de integração que são introduzidos nos processos. Em uma SOA, orquestrações eles próprios são serviços. Isso promove a federação porque vários processos de negócios, potencialmente de diferentes aplicativos, podem ser mesclados.

A coreografia é outra forma de composição do serviço. Coreografias definem mensagem intercâmbios e pode envolver vários participantes, cada um dos quais pode assumir várias funções.

Em contraste com a orquestração, não há processo centralizado, ou orquestrador, que é controlá-lo. Com a coreografia, há um conjunto acordado de interações coordenadas que especificam as condições em que os dados serão trocados. Cada serviço em uma coreografia atua de forma autônoma para executar sua parte com base nas condições que foram estabelecidas e o ações dos outros participantes.

Tanto a orquestração quanto a coreografia podem ser usadas para lógica de processos de negócios pertencentes a uma organização única (intra organização) e colaboração entre várias organizações (interorganização). No entanto, a orquestração é menos provável de ser usada quando há múltiplas organizações envolvidas porque você precisaria possuir e operar a orquestração. A coreografia permite a colaboração sem ter uma única organização controlar todo o processo.

6.15. Resumo

Os arquitetos de software devem estar familiarizados com os padrões de arquitetura de software, pois eles são uma ferramenta poderosa ao projetar uma arquitetura de software. Os padrões de arquitetura fornecem uma solução comprovada para problemas recorrentes em um determinado contexto. Aproveitar os padrões de arquitetura dá ao arquiteto de softwares uma estrutura de alto nível do sistema de software e fornece um agrupamento de decisões de design que foram repetidas e usado com sucesso. Usá-los reduz a complexidade, colocando restrições no design e nos permite antecipar as qualidades que o sistema de software exibirá uma vez que seja implementado.

Neste capítulo, você aprendeu sobre alguns dos padrões comuns de arquitetura de software disponível, incluindo arquitetura em camadas, EDA, MVC, MVP, MVVM, CQRS e SOA.

O foco do próximo capítulo está em algumas das arquiteturas de software relativamente mais recentes padrões e paradigmas. Isso inclui arquitetura de micro serviço, arquitetura sem servidor, e aplicativos nativos da nuvem. À medida que a implantação de aplicativos de software na nuvem torna-se a tendência dominante, esses conceitos se tornam cruciais para qualquer arquiteto de software entender.

7. Arquetetando aplicativos modernos

Os aplicativos modernos implantados na nuvem têm diferentes expectativas e requisitos de aplicativos desenvolvidos no passado. Novos padrões e paradigmas de arquitetura de software foram introduzidos para atender a essas expectativas.

Neste capítulo, vamos explorar alguns desses padrões e abordagens de design de software e o desenvolvimento. Começaremos examinando a arquitetura monolítica, juntamente com a razões pelas quais deve ou não ser usado. Em seguida, exploraremos os micros serviço arquitetura (MSA), arquitetura sem servidor e aplicativos nativos da nuvem.

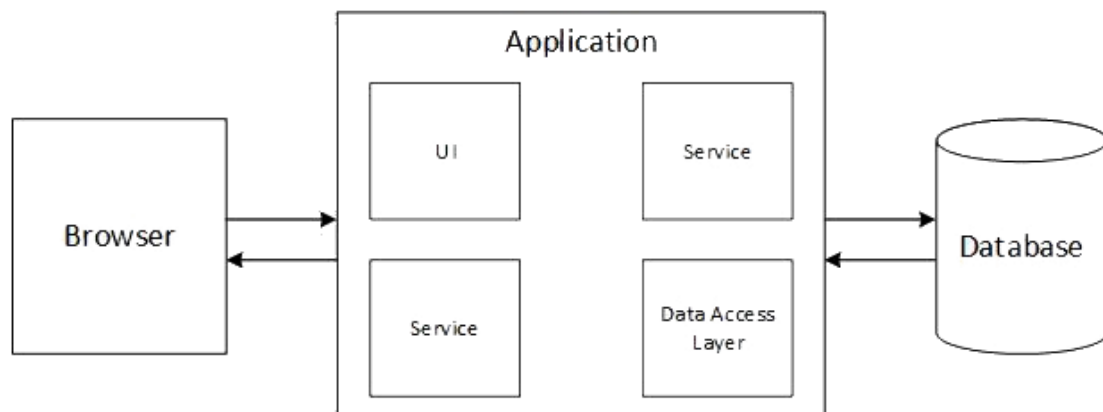
Neste capítulo, cobriremos os seguintes tópicos:

- Arquitetura monolítica
- Arquitetura de micro serviço
- Arquitetura sem servidor
- Aplicativos nativos da nuvem

7.1. Arquitetura monolítica

Uma arquitetura monolítica é aquela em que um aplicativo de software é projetado para funcionar como uma unidade única e independente. Os aplicativos que possuem esse tipo de arquitetura são comuns. Os componentes dentro de uma arquitetura monolítica são interconectados e interdependentes, resultando em código fortemente acoplado.

O diagrama a seguir mostra um aplicativo com arquitetura monolítica:



As diferentes preocupações de um aplicativo, como interface do usuário, lógica de negócios, autorização, registro e acesso ao banco de dados, não são mantidas separados em uma monolítica arquitetura. Essas diferentes peças de funcionalidade estão interligadas em um monolítico aplicativo.

7.1.1. Benefícios de uma arquitetura monolítica

Apesar de algumas desvantagens óbvias de usar uma arquitetura monolítica, se um aplicativo for relativamente pequeno, então há benefícios em usar um. Aplicações com uma monolítica arquitetura normalmente tem melhor desempenho. Com a interação entre a máquina executando o aplicativo e outras máquinas minimizadas, melhores níveis de desempenho são percebidos.

Pequenos aplicativos que possuem este tipo de arquitetura são mais fáceis de implantar devido à simplicidade da arquitetura de alto nível. Apesar da lógica fortemente acoplada, monolíticos aplicativos podem ser mais fáceis de testar e depurar porque são mais simples, com menos componentes a serem considerados.

Os aplicativos monolíticos são normalmente fáceis de escalar porque tudo o que é necessário é executar várias instâncias do mesmo aplicativo. No entanto, diferentes componentes do aplicativo têm diferentes necessidades de dimensionamento e não podemos dimensionar os componentes de forma independente com uma arquitetura monolítica. Estamos limitados a adicionar mais instâncias de todo o aplicativo em ordem para escalar.

7.1.2. Desvantagens de uma arquitetura monolítica

Embora uma arquitetura monolítica possa funcionar para alguns aplicativos, à medida que os aplicativos crescem em tamanho e complexidade, existem sérias desvantagens. Aplicações monolíticas inibem muito a agilidade da organização, pois se torna difícil fazer alterações no software. Um aspecto disso é o fato de que a implantação contínua é difícil de alcançar. Mesmo se uma mudança é feita para apenas um componente de um aplicativo monolítico, todo o sistema de software precisa ser implantado. As organizações devem dedicar mais recursos, como tempo e testadores, para implantar uma nova versão de um aplicativo monolítico.

Se o aplicativo for pequeno, pode ser fácil de manter devido à simplicidade da arquitetura. No entanto, aplicações monolíticas maiores e mais complexas começam a sofrer em termos de manutenção. Componentes fortemente acoplados tornam mais difícil fazer mudanças porque uma mudança em uma parte do aplicativo tem mais probabilidade de afetar outras partes do aplicativo.

A grande base de código de um aplicativo monolítico pode tornar difícil para os membros da equipe entenderem isso. Isso é especialmente verdadeiro para novos membros da equipe enquanto tentam se familiarizar com ele e começar a trabalhar com ele.

Mesmo carregando um aplicativo grande e monolítico em um ambiente de desenvolvimento integrado (IDE) e trabalhar com ele pode ser frustrante devido ao desempenho mais lento do IDE. Isso também leva mais tempo para esses aplicativos iniciarem, diminuindo a produtividade da equipe durante desenvolvimento.

Os aplicativos monolíticos exigem um compromisso com uma linguagem de programação específica e pilha de tecnologia. Uma vez que o aplicativo é escrito como uma única unidade, torna-se mais difícil para introduzir diferentes tipos de tecnologia. Em alguns casos, pode até tornar-se difícil mudar para uma versão mais recente da mesma tecnologia! Se uma migração para uma tecnologia diferente for necessária, exige que a organização se comprometa a reescrever todo o aplicativo.

Aplicativos maiores e mais complexos se beneficiam da divisão de responsabilidades entre várias equipes de desenvolvimento, como ter cada equipe enfocada em um determinado funcional área. No entanto, isso se torna difícil de fazer com uma aplicação monolítica, pois as mudanças são feitas por uma equipe de desenvolvimento pode afetar outra equipe de desenvolvimento.

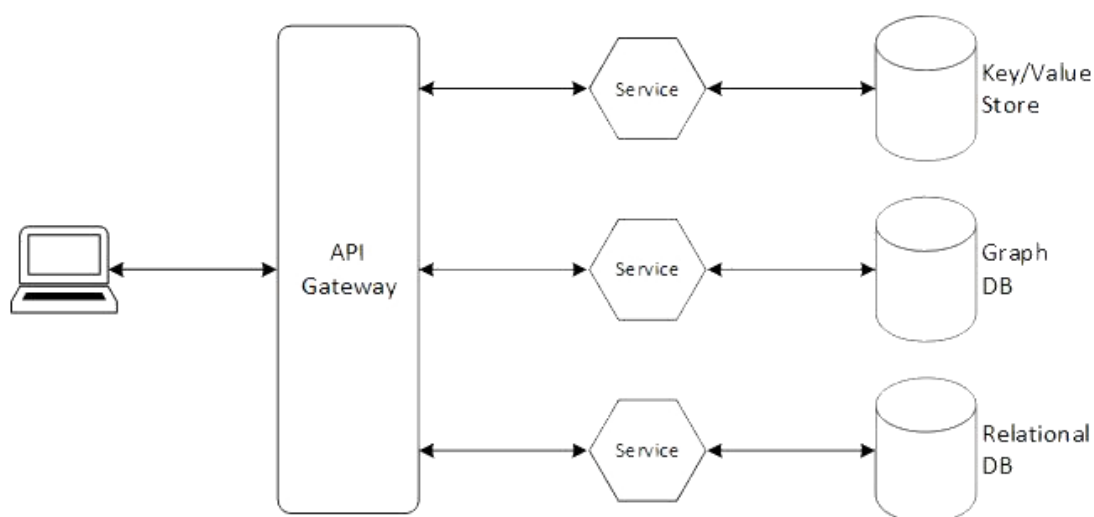
Devido a essas desvantagens, os aplicativos de software que são grandes e complexos devem ser movidos longe de ser uma aplicação monolítica. MSA e arquitetura sem servidor são alternativas para arquiteturas monolíticas. Eles abordam algumas das preocupações e limitações de aplicações monolíticas.

7.2. Arquitetura de micro serviço

O padrão de arquitetura de micro serviço (MSA) cria aplicativos de software usando serviços autônomos, com versão independente e independentes. Esses serviços usam bem definidos interfaces e comunicar-se entre si por meio de protocolos leves e padrão.

A interação com um micro serviço ocorre por meio de uma interface bem definida. Um micro serviço deve ser uma caixa preta para os consumidores do serviço, ocultando sua implementação e complexidade. Cada micro serviço se concentra em fazer uma coisa bem e podem trabalhar juntos com outros micros serviços para realizar tarefas mais complexas.

Uma arquitetura de micro serviço é particularmente adequada para softwares grandes e/ou complexos sistemas. Em contraste com a arquitetura monolítica, os aplicativos construídos usando um a arquitetura de micro serviço lida com a complexidade, dividindo o aplicativo em partes menores serviços que são mais fáceis de gerenciar. O diagrama a seguir ilustra um sistema que tem uma arquitetura de micro serviços:



As solicitações de entrada geralmente são tratadas por um gateway de API, que serve como a entrada apontar para o sistema. É um servidor HTTP que recebe solicitações de clientes e os encaminha para o micro serviço apropriado por meio de sua configuração de roteamento. No

serviço seção de descoberta posteriormente neste capítulo, daremos uma olhada mais de perto na descoberta de serviço para explicar como os gateways de API podem obter os locais das instâncias de serviço disponíveis.

7.2.1. SOA bem-feito

A arquitetura de micro serviço surgiu, não como uma solução em busca de um problema, mas como o resultado das deficiências e desvantagens da arquitetura tradicional orientada a serviços (SOA) e arquitetura monolítica.

SOA oferece uma série de benefícios, que foram abordados no Capítulo 7, Arquitetura de Software Padrões, como aumentar o alinhamento entre negócios e tecnologia, promovendo federação dentro de uma organização, permitindo a diversidade de fornecedores e aumentando interoperabilidade.

Embora a arquitetura de micro serviço seja uma variação da arquitetura orientada a serviços, há são as principais diferenças entre os dois. Algumas pessoas até se referem ao micro serviço arquitetura como SOA bem-feita.

Além de nos afastar de arquiteturas monolíticas, arquitetura de micro serviço melhora em SOA tradicional. Implementar SOA pode ser muito caro, difícil de implementar e pode ser um exagero para muitas aplicações. A arquitetura de micro serviço O padrão aborda algumas dessas desvantagens. Em vez de usar um barramento de serviço corporativo (ESB), como é comum com SOA, a arquitetura de micro serviço favorece a implementação do tipo ESB funcionalidade nos próprios serviços.

7.2.2. Características da arquitetura de micro serviço

Existem várias características relacionadas à arquitetura de micro serviço. Nisso seção, examinaremos as seguintes características:

- Serviços pequenos e focados
- Interfaces de serviço bem definidas
- Serviços autônomos e implantáveis de forma independente
- Armazenamento de dados independente
- Comunicando-se com protocolos leves
- Melhor isolamento de falhas
- Serviços pequenos e focados

A funcionalidade de um micro serviço deve ter escopo pequeno. Cada micro serviço deve siga o princípio da filosofia Unix que defende fazer uma coisa e fazê-la nós vamos. Manter o escopo pequeno para cada serviço permite que a complexidade do software seja melhor administrado.

Os aplicativos com uma arquitetura de micro serviço são mais fáceis de entender e modificar porque todo serviço tem uma responsabilidade focada. Isso permite que os serviços sejam desenvolvidos mais rapidamente.

Soluções menores são mais fáceis de carregar e usar em um IDE, o que aumenta a produtividade. Além disso, os novos membros da equipe podem se tornar produtivos em menos tempo.

Um micro serviço pode ser desenvolvido por uma pequena equipe. Isso permite que as organizações dividam facilmente trabalho entre várias equipes de desenvolvimento. Cada micro serviço pode ser trabalhado por uma equipe única, independente de outros micros serviços. Tarefas para micro serviços diferentes podem ser trabalhou em paralelo.

Se um sistema de software também estiver usando design orientado a domínio (DDD), o conceito de contextos funciona bem com micros serviços, pois ajuda no particionamento de serviços. Em alguns sistemas, cada micro serviço é implementado como seu próprio contexto limitado.

7.2.3. Interfaces de serviço bem definidas

Os micros serviços são tratados como caixas pretas, ocultando sua complexidade e implementação detalhes dos consumidores de serviço. Isso cria uma interface bem definida, com entrada e pontos de saída, o que é importante para ajudar a facilitar o trabalho conjunto de micro serviços. Serviços interagir uns com os outros através de suas interfaces.

7.2.4. Serviços autônomos e implantáveis de forma independente

Um aplicativo que usa uma arquitetura de micro serviço consiste em um sistema de Serviços. Os serviços devem ser fracamente acoplados, interagindo por meio de suas interfaces e não dependente da implementação do serviço.

Isso permite que a implementação de serviços mude e evolua independentemente um do outro Serviços. Contanto que a interface de serviço permaneça inalterada, a modificação de um micro serviço é menos provavelmente exigirá alterações em outras partes do aplicativo.

Os serviços autônomos são implantados de forma independente, tornando mais fácil implantá-los em Produção. Uma arquitetura de micro serviço permite implantação contínua porque é mais fácil para lançar atualizações para os serviços. Se uma alteração for feita em um micro serviço, ele pode ser implantado independentemente dos outros micros serviços.

A autonomia dos serviços aumenta a agilidade organizacional. Ele permite que as organizações se adaptem rapidamente às mudanças nos requisitos de negócios e aproveite as vantagens de novos negócios oportunidades.

7.2.5. Armazenamento de dados independente

Uma das características que suportam a autonomia do serviço é que cada micro serviço pode ter seu próprio armazenamento de dados. Isso ajuda os serviços a serem independentes e fracamente acoplados a outros Serviços. O armazenamento de dados de um serviço pode ser alterado sem que essas mudanças afetem outros serviços.

Projetar cada micro serviço para ter seu próprio armazenamento de dados pode vir na forma de um micro serviço com seu próprio banco de dados, mas essa não é a única maneira de manter um micro serviço armazenamento de dados privado. Se a tecnologia de armazenamento de dados é um gerenciamento de banco de dados relacional sistema (RDBMS), então, além da opção de ter um servidor de banco de dados separado, dados podem ser mantidos separados designando certas tabelas como propriedade de um serviço específico.

Outra opção é designar um esquema que deve ser usado apenas por um único micro serviço. Na seção de persistência poliglota posteriormente neste capítulo, você aprenderá sobre alguns benefícios de usar um banco de dados separado para cada micro serviço.

7.2.6. Melhor isolamento de falhas

Um sistema construído em uma arquitetura de micro serviço melhora o isolamento de falhas. Quando um o micro serviço cai, outros serviços ainda podem operar normalmente, permitindo

que outras partes do sistema permanecer operacional. Isso está em contraste com um aplicativo monolítico, onde uma falha pode potencialmente derrubar todo o sistema.

7.2.7. Comunicação com protocolos de mensagem leves

Os micros serviços devem se comunicar usando protocolos de mensagens leves e bem conhecidos. Não há nenhuma regra ditando um protocolo específico, e os micros serviços podem se comunicar de forma síncrona ou assíncrona. Uma implementação comum para micros serviços é ter eles expõem pontos de extremidade HTTP que são chamados por meio de chamadas de API REST. Para síncrona comunicação, o REST é um dos protocolos preferidos. É comum que REST seja usado com Java Script Object Notation (JSON). As operações de serviço podem aceitar e retornar dados em o formato JSON, pois é um formato de intercâmbio de dados leve e popular.

Alguns aplicativos podem exigir comunicação assíncrona. Uma mensagem comum protocolo usado para comunicação assíncrona com micros serviços é Advanced Message Protocolo de enfileiramento (AMQP). É um padrão aberto que pode conectar uma variedade de serviços, incluindo aqueles em diferentes plataformas e entre organizações. Foi projetado para segurança, confiabilidade e interoperabilidade.

AMQP pode oferecer suporte aos seguintes tipos de garantias de entrega de mensagem:

- Pelo menos uma vez: uma mensagem tem garantia de entrega, mas pode ser entregue várias vezes
- No máximo uma vez: uma mensagem tem a garantia de ser entregue uma vez ou nunca
- Exatamente uma vez: uma mensagem tem garantia de ser entregue uma vez e apenas uma vez

Outro protocolo popular com micro serviços é o gRPC. Ele foi projetado pelo Google como uma alternativa para REST e outros protocolos. É um protocolo de código aberto que foi projetado para ser mais rápido e compacto do que outros protocolos usados para sistemas distribuídos. gRPC é construído em buffers de protocolo, também conhecidos como protobufs, que é uma maneira de serializar dados que são neutros em termos de linguagem e plataforma. Isso permite que o gRPC conecte com eficiência os políglotas serviços porque suporta uma variedade de linguagens de programação. O aumento da prevalência de aplicativos e micros serviços em contêineres tornou o gRPC popular. Dado moderno cargas de trabalho, o gRPC é uma escolha atraente porque é um equipamento leve e de alto desempenho protocolo. O gRPC é inerentemente eficiente, mas também é baseado em HTTP/2. Isso produz benefícios como latência reduzida e maior compactação de dados.

O protocolo de comunicação que você seleciona para sua arquitetura de micro serviço realmente depende de seus requisitos. Não existe uma solução mágica que se aplique a todos situações. O contexto de suas preocupações de design conduzirá a seleção, como se suas necessidades de comunicação são síncronas ou assíncronas.

7.2.8. Criação de micros serviços políglotas

Uma das muitas vantagens de usar uma arquitetura de micros serviço é que ela oferece a você a opção de usar várias linguagens de programação, tempos de execução, estruturas e armazenamento de dados tecnologias.

Os aplicativos monolíticos se concentram no uso de uma linguagem de programação e tecnologia específicas pilha. Como esse tipo de aplicativo é escrito como uma única unidade, é

mais difícil de aceitar vantagem de diferentes tipos de tecnologia. No entanto, aplicativos complexos precisam resolver uma variedade de problemas. Ser capaz de selecionar tecnologias diferentes para problemas diferentes pode ser útil em vez de tentar resolver todos os problemas com uma única tecnologia.

Uma equipe de desenvolvimento pode selecionar as melhores soluções, dependendo da tarefa de que precisam completar. Uma arquitetura de micro serviço permite que as equipes experimentem e testem novos com o risco de se comprometer com eles para todo o sistema.

Ter micros serviços políglotas certamente não é necessário ao usar um micro serviço arquitetura. Em muitos casos, uma organização se concentrará em um número limitado de tecnologias, e os conjuntos de habilidades da equipe de desenvolvimento refletirão isso. No entanto, arquitetos de software devem estar cientes da opção e reconhecer oportunidades onde ela pode ser usada de forma eficaz.

Dois dos conceitos relacionados aos micros serviços políglotas são a programação políglota e persistência políglota.

7.2.9. Programação políglota

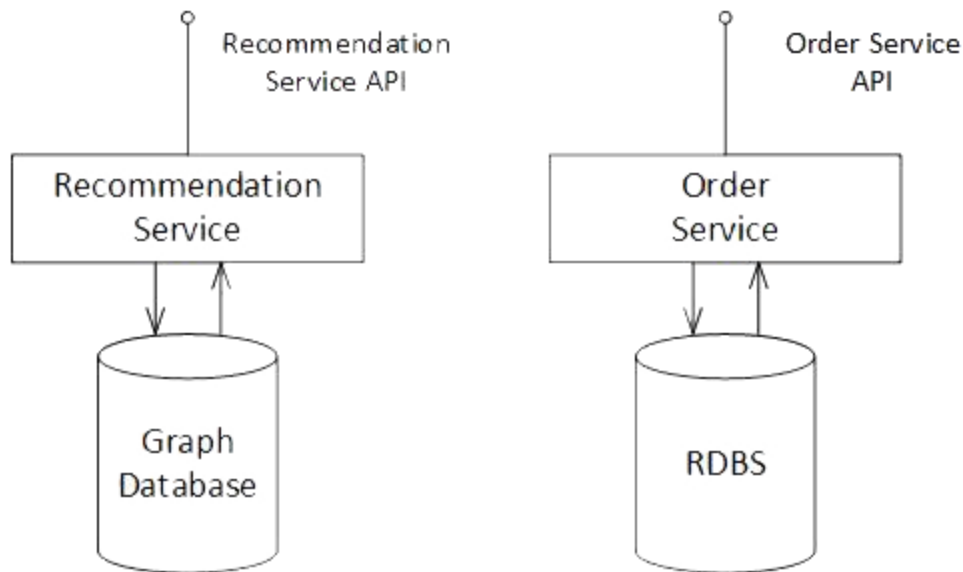
Com a programação políglota, um único aplicativo usa várias linguagens de programação em sua implementação. Pode ser útil aproveitar as vantagens de diferentes linguagens de programação para lidar com diferentes tarefas em um aplicativo.

Uma arquitetura de micro serviço permite que cada micro serviço seja desenvolvido usando a linguagem de programação que melhor se adapta ao problema em questão. Quando uma nova tecnologia se torna disponível, um aplicativo existente pode aproveitá-la para novos micros serviços que são desenvolvidos ou para novas versões de um micro serviço existente.

7.2.10. Persistência políglota

Semelhante à programação políglota, existe o conceito de persistência políglota, em que várias opções de persistência são usadas em um único aplicativo. Armazenamento de dados diferente as tecnologias são mais adequadas para diferentes tarefas e os micros serviços permitem que você faça vantagem disso.

Cada micro serviço é responsável por seu próprio armazenamento de dados, para que possa escolher o melhor armazenamento de dados tecnologia baseada no que está tentando alcançar. O diagrama a seguir descreve dois micros serviços, cada um com seu próprio banco de dados:



Um micro serviço dedicado a fornece recomendações de produtos com base em amigos, classificações, e compras anteriores seriam ideais para um banco de dados de gráficos. Um catálogo de produtos com detalhes sobre os produtos de uma empresa, exigindo leituras frequentes e gravações infrequentes, podem ser implementados com um banco de dados de documentos. Ao processar um pedido, o transacional recurso de um banco de dados relacional podem ser ideais.

Um único micro serviço pode até usar mais de uma tecnologia de armazenamento de dados, embora seja bastante incomum na prática. Ele apresenta uma complexidade extra, e se você quiser para fazer isso, você deve considerar se o seu micro serviço é muito grande em escopo. Naquilo caso, talvez o micro serviço precise ser dividido em mais de um.

7.2.11. Usando muitas tecnologias

Como acontece com muitas coisas, a programação poliglota e a persistência poliglota podem ser exageradas. A introdução de muitas tecnologias díspares em uma única organização tem um custo. Dominar uma tecnologia é difícil e você precisará de membros da equipe que sejam bem versados em as tecnologias que você está usando. O uso de diferentes tecnologias pode criar a necessidade de mais treinamento de desenvolvedores, o que custa dinheiro à organização, bem como aos seus funcionários

7.2.11.1. Tempo.

O uso de diferentes linguagens de programação e tecnologias de armazenamento de dados apresenta complexidade nos processos de construção, implantação e teste. O software precisará ser mantido continuamente após ser lançado para produção, exigindo um desenvolvimento diversificado equipe capaz de lidar com todas as diferentes tecnologias.

Ter a capacidade de tirar proveito de diferentes tecnologias pode ser um ativo valioso, mas consideração deve ser feita cada vez que usar um novo está sendo considerado para garantir que é a melhor decisão para o projeto.

7.2.11.2. Considerando a granularidade do serviço

A granularidade de um serviço se refere ao escopo de sua funcionalidade de negócios, que varia para cada serviço. Com micro serviços, o objetivo é ter serviços refinados para que cada um concentra-se em uma única capacidade de negócios.

É importante obter a granularidade dos micros serviços correta para garantir que o sistema consiste em serviços de tamanho apropriado. Um dos objetivos de uma arquitetura de micro serviço é decompor um domínio em serviços pequenos, focados e reutilizáveis. Serviços menores carregam com menos contexto, aumentando seu nível de reutilização.

7.2.11.3. Nano serviços

Os arquitetos e desenvolvedores de software que estão projetando micros serviços devem ter cuidado para não tornar a granularidade de seus serviços muito refinada, no entanto. Serviços cujos a granularidade é muito refinada são chamados de nano serviços e isso é considerado um anti padrão.

Um sistema com serviços muito pequenos tende a ter mais serviços e, conforme o número de os serviços em um sistema aumentam, assim como a quantidade de comunicação que deve ocorrer.

Os serviços usam recursos de rede que não são infinitos e os serviços são limitados por eles limitações. Ter muitos serviços pode levar à redução do desempenho de seus serviços e de sua aplicação geral.

Quando um sistema tem muitos nano serviços, também há um aumento na sobrecarga geral para os serviços. Cada serviço requer algum gerenciamento, incluindo coisas como configuração e entrada em um registro de serviço. Um número maior de serviços leva a quantidades crescentes de sobrecarga.

Os nano serviços podem levar a uma lógica fragmentada. Se uma única tarefa de negócios que se encaixa bem em um serviço único e coeso é decomposto em vários serviços menores, a lógica torna-se separado. Tornar os serviços desnecessariamente pequenos faz parte do nano serviço, anti padrão.

Quando a sobrecarga de um serviço supera sua utilidade, ele é um nano serviço e a refatoração deve ser considerada. Os na nos serviços podem ser refatorados combinando vários nano serviços em um serviço novo e maior ou movendo a funcionalidade de cada nano serviço em um serviço existente apropriado.

Pode haver exceções nas quais um nano serviço é dimensionado apropriadamente e não requer refatoração, por isso é sempre prudente usar o bom senso. Embora você não queira seu sistema consiste em muitos nano serviços, às vezes, a funcionalidade de um nano serviço específico simplesmente não pertence a nenhum outro lugar.

7.2.11.3. Compartilhamento de dependências entre micros serviços

As equipes de desenvolvimento devem evitar o compartilhamento de dependências, como estruturas e terceiras bibliotecas, entre micros serviços. Você pode ter vários micros serviços que compartilham a mesma dependência, por isso é natural pensar em compartilhá-los no host para torná-los disponível centralmente.

No entanto, cada micro serviço deve permanecer independente de outros micros serviços. Se nós quisermos para atualizar as dependências, não queremos afetar nenhum outro serviço.

Fazer isso seria aumentar o risco de introdução de defeitos e ampliar o escopo dos testes que precisam ser conduzidos em relação à mudança.

O compartilhamento de dependências também introduz afinidade de host, que queremos evitar. Micros serviços devem funcionar independentemente do host em que são implantados porque as suposições não podem ser feitas em relação ao host no qual um serviço será executado.

7.3. Micros serviços sem estado versus micro serviços com estado

Cada micro serviço pode ser sem estado ou com estado. Um sistema que usa micro serviços normalmente tem um aplicativo da web e/ou móvel sem estado que usa sem estado e/ou com estado Serviços.

Os micros serviços sem estado não mantêm nenhum estado nos serviços nas chamadas. Eles pegam em uma solicitação, processe-a e envie uma resposta de volta sem persistir nenhuma informação de estado.

Um micro serviço com estado persiste no estado de alguma forma para que funcione. Em vez de armazenar esse estado internamente, um micro serviço deve armazenar informações de estado externamente, em algum tipo de armazenamento de dados. Exemplos de um armazenamento de dados para persistir o estado incluem um sistema de gerenciamento de banco de dados relacional (RDBMS), um banco de dados NoSQL ou algum tipo de armazenamento na nuvem. Persistir o estado externamente fornece disponibilidade, confiabilidade, escalabilidade, e consistência para as informações do estado.

7.3.1. Descoberta de serviço

O cliente de um serviço, seja um gateway de API ou outro serviço, precisa ser capaz de descobrir a localização de uma instância de serviço. Em um ambiente distribuído tradicional, o serviço local (endereço IP e porta) são geralmente estáticos, e uma instância de serviço pode ser encontrada facilmente. Por exemplo, os locais de serviço podem ser lidos a partir de um arquivo de configuração.

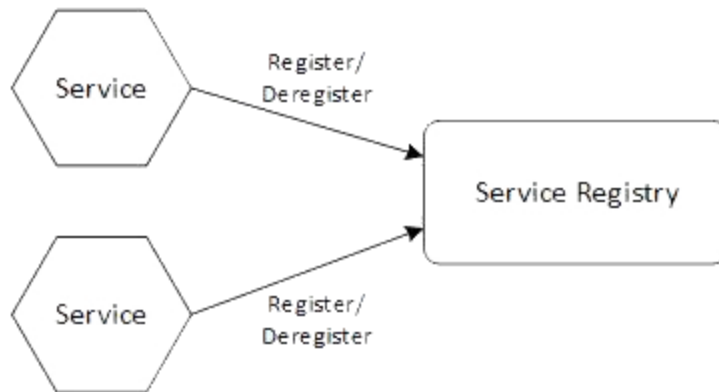
No entanto, a descoberta de serviço é mais complexa com um aplicativo baseado em nuvem usando micros serviços. O número e a localização das instâncias de serviço mudam dinamicamente na nuvem. Um registro de serviço pode ser usado para controlar as instâncias de serviço e seus locais.

7.3.2. Usando um registro de serviço

Um registro de serviço desempenha um papel fundamental na descoberta de serviço. É um banco de dados contendo serviço instâncias e suas localizações. O registro do serviço deve estar altamente disponível e atualizado para encontro. Para serem precisas, as instâncias de serviço devem ser registradas e canceladas com o registro de serviço. Isso pode ser feito por meio de auto registro ou de terceiros cadastros.

7.3.3. Padrão de auto registro

Usando o padrão de auto registro, as instâncias de serviço são responsáveis pelo registro com, e cancelar o registro do serviço de registro:



Quando uma instância de serviço é inicializada, ela deve se registrar no registro de serviço. De forma similar, quando uma instância de serviço é encerrada, ela deve cancelar seu próprio registro do registro de serviço.

Esta abordagem é relativamente direta, embora acople as instâncias de serviço ao registro de serviço. Se você tiver um sistema poliglota em que diferentes micros serviços são usando diferentes linguagens de programação e/ou estruturas, a lógica de registro de serviço deve existir para cada linguagem de programação e estrutura que é usada para o desenvolvimento de seus micros serviços.

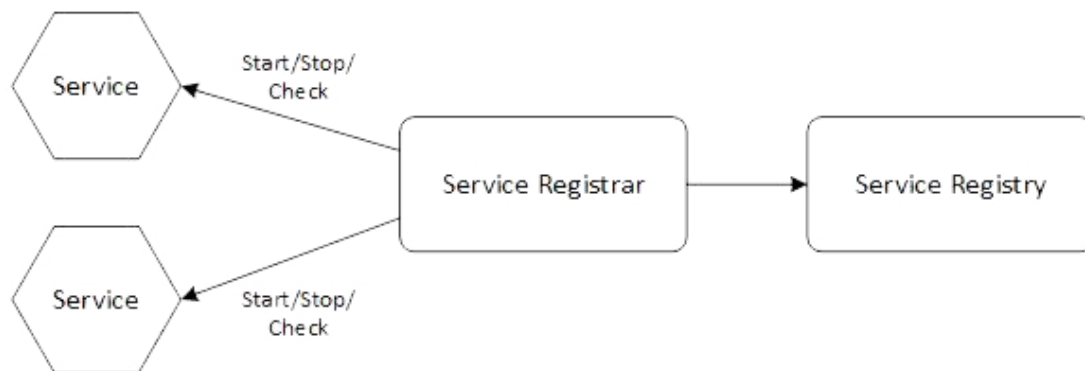
É um requisito comum ter instâncias de serviço registradas renovando periodicamente seus registros ou enviar uma solicitação de pulsação para indicar que eles ainda estão ativos e responsivos. Se uma instância de serviço não faz isso, então eles podem ser cancelados automaticamente. Fazendo isso vai lidar com situações em que uma instância de serviço está em execução, mas não está disponível para alguns razão. Essas instâncias de serviço podem não ser capazes de cancelar o registro do serviço registro.

Para aplicativos pequenos, o padrão de auto registro pode ser suficiente, mas aplicativos grandes desejarão usar o padrão de registro de terceiros.

7.3.4. Padrão de registro de terceiros

Ao usar o padrão de registro de terceiros, um componente dedicado, às vezes referido como o registrador de serviço, lida com o registro, cancelamento de registro e verificação do funcionamento de instâncias de serviço. Como o próprio registro de serviço, o registrador de serviço é um importante componente e, portanto, deve estar altamente disponível.

O diagrama a seguir ilustra o padrão de registro de terceiros:



Pesquisando as instâncias de serviço disponíveis ou assinando eventos relevantes, o registrador de serviço pode registrar novas instâncias de serviço e cancelar o registro de instâncias de serviço que não mais existem. Ele pode realizar verificações de saúde em instâncias de serviço e tomar as ações apropriadas com base no resultado.

Ao contrário do padrão de auto registro, as instâncias de serviço são desacopladas do serviço registro. Os serviços podem se concentrar em sua única responsabilidade e não precisam se preocupar com registro de serviço. Se diferentes micros serviços em seu sistema estiverem usando diferentes linguagens de programação e/ou frameworks, você não precisa implementar o serviço lógica de registro para cada um que é usado pela (s) equipe (s) de desenvolvimento.

Uma desvantagem desse padrão é que, a menos que o registrador de serviço seja um componente integrado de seu ambiente de implementação, é outro componente que deve ser configurado e gerenciado.

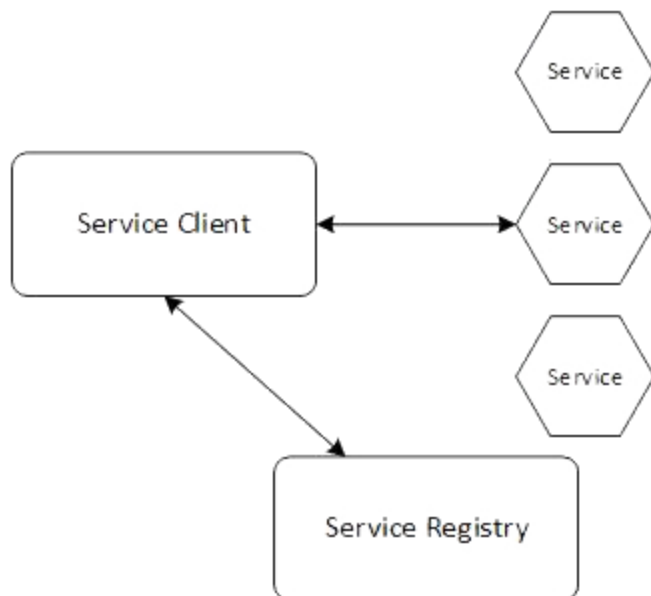
7.3.5. Tipos de descoberta de serviço

Existem dois padrões principais para descoberta de serviço:

- Padrão de descoberta do lado do cliente
- Padrão de descoberta do lado do servidor
- Vamos agora examinar os dois tipos em detalhes.

7.3.6. Padrão de descoberta do lado do cliente

Com o padrão de descoberta do lado do cliente, o cliente de serviço, seja um gateway de API ou outro serviço, consulta um registro de serviço para as localizações de instâncias de serviço disponíveis:



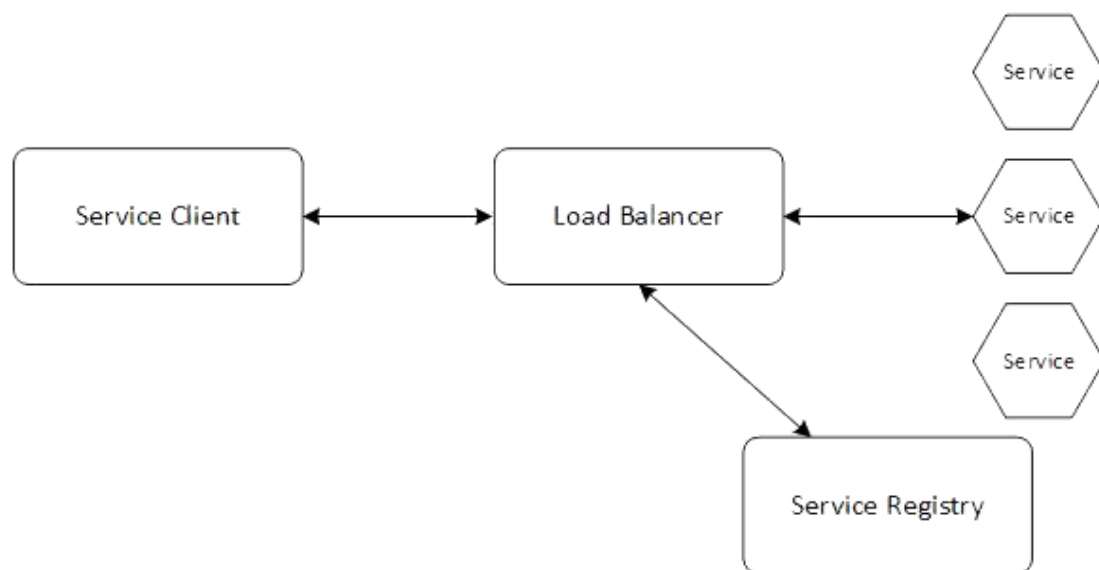
Uma vez que os locais das instâncias de serviço disponíveis são obtidos do registro de serviço, o cliente de serviço usa um algoritmo de balanceamento de carga para selecionar um deles. Nesse ponto, o cliente de serviço pode interagir com uma instância de serviço específica.

Esse padrão é direto, embora acople o cliente do serviço ao serviço registro. Para organizações que estão aproveitando o uso de programação múltipla linguagens e/ou estruturas para seu desenvolvimento de micro serviço, lógica de descoberta de serviço precisará ser escrito para cada linguagem de programação e estrutura que é usada para o desenvolvimento de micro serviços.

7.3.7. Padrão de descoberta do lado do servidor

O outro padrão principal para descoberta de serviço é o padrão de descoberta do lado do servidor. O cliente de serviço, como um gateway de API ou outro serviço, faz uma solicitação a um roteador. O roteador é normalmente um balanceador de carga.

O diagrama a seguir ilustra o padrão de descoberta do lado do servidor:



Neste padrão, é o balanceador de carga que consulta o registro de serviço para os locais de instâncias de serviço disponíveis. O registro do serviço pode ser integrado ao balanceador de carga ou pode ser um componente separado. O balanceador de carga é então responsável por encaminhar a solicitação para uma das instâncias de serviço disponíveis.

Ao contrário do padrão de descoberta do lado do cliente, os clientes de serviço são desacoplados do serviço registro. O código do cliente de serviço é mais simples, pois não precisa se preocupar com a interação com o registro de serviço ou implementação de um algoritmo de balanceamento de carga. Clientes de serviço podem bastar fazer uma solicitação ao balanceador de carga. Se a programação for diferente linguagens/frameworks estão sendo usados para desenvolvimento de micro serviços, não há necessidade de implementar lógica de descoberta de serviço para cada um.

Uma desvantagem desse padrão é que, a menos que o provedor de nuvem esteja fornecendo o balanceador de carga e funcionalidade associada (registro de serviço), é outro componente que deve ser instalado e gerenciado. Além disso, há mais saltos de rede envolvidos com este padrão devido para o balanceador de carga em comparação com o padrão de descoberta do lado do cliente.

7.4. Usar micro serviços não é para todos

Como é o caso de outros tipos de padrões de arquitetura, há desvantagens em usar uma arquitetura de micro serviço. Os benefícios de usar uma arquitetura de micro serviço devem superar a complexidade adicional, por isso não é a solução ideal para todos os tipos de aplicativos.

Como um sistema distribuído, uma arquitetura de micro serviço apresenta complexidade simplesmente não encontrado em um aplicativo monolítico. Quando vários serviços estão trabalhando juntos em um sistema distribuído e algo dá errado, há complexidade adicional em descobrir o que e onde algo falhou. Um serviço pode não responder a jusante e o sistema deve ser capaz de lidar com a interrupção.

Decompor um sistema complexo no conjunto certo de micros serviços podem ser difícil. Isto requer um conhecimento do domínio e pode ser um tanto quanto uma arte. Você não quer um sistema com serviços muito refinados, resultando em muitos serviços. Como o número de serviços aumenta, a gestão desses serviços torna-se cada vez mais complexo.

Ao mesmo tempo, você não quer que os serviços de um sistema sejam muito granulares, de modo que eles são responsáveis por muitas funcionalidades. A última coisa que você quer é um monte de serviços que são fortemente acoplados, fazendo com que tenham que ser implantados juntos. Se você não tomar cuidado, você vai acabar com uma arquitetura de micro serviço que é um monólito em disfarce.

O uso de vários bancos de dados é outro desafio ao usar uma arquitetura de micro serviço. É comum que uma transação comercial atualize várias entidades, o que exigirá o uso de vários micros serviços. Com cada um tendo seu próprio banco de dados, isso significa que as atualizações devem ocorrer em vários bancos de dados. Uma maneira de lidar com isso é por meio de terceirização de eventos e ter consistência eventual. Mesmo que seja aceitável, a implementação de fonte de eventos é outra complexidade adicional.

Os clientes de serviço, como um gateway de API e outros serviços, precisarão de uma maneira de conhecer a localização das instâncias de serviço disponíveis. A menos que um provedor de nuvem esteja prestando um serviço registro e funcionalidade associada, esses componentes adicionais precisarão ser configurados e gerenciados.

7.5. Arquitetura sem servidor

A arquitetura sem servidor permite o rápido desenvolvimento de aplicativos de software que podem lidar com vários níveis de tráfego em ambientes de produção. O termo sem servidor refere-se ao fato de que os serviços de computação são fornecidos sem exigir que você gerencie ou administre servidores. Seu código é executado sob demanda, conforme necessário.

Utilizar serviços de computação dessa maneira é semelhante há como o armazenamento em nuvem é usado. Com nuvem armazenamento, você não precisa gerenciar hardware físico e não precisa saber onde os dados são armazenados. Você usa tanto ou tão pouco armazenamento quanto você precisa.

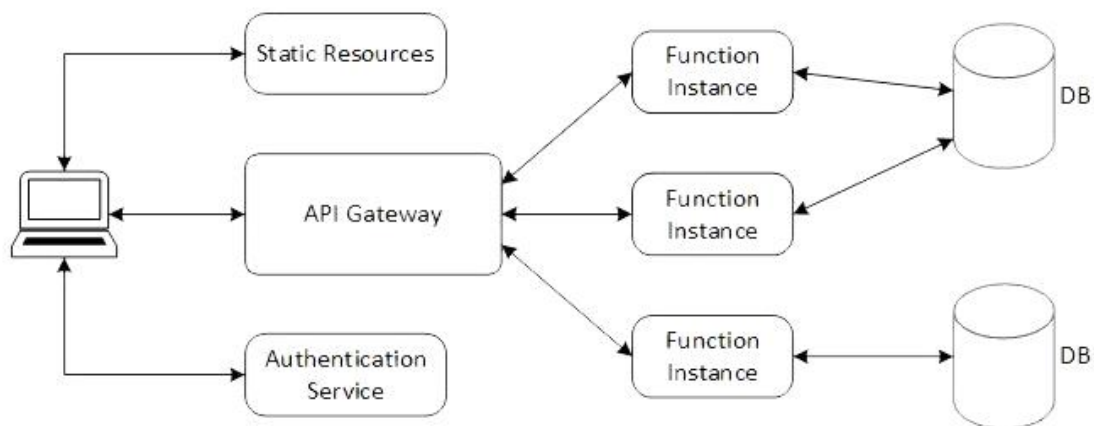
Da mesma forma, com a arquitetura sem servidor, você não precisa lidar com servidores físicos e a complexidade de como os recursos de computação são fornecidos está oculta para você.

7.4.5. Programas

os aplicativos usam tanta ou tão pouca capacidade de computação quanto precisa. A arquitetura sem servidor está amadurecendo e seu uso é cada vez mais comum. Pode ser o ideal escolha arquitetônica para alguns aplicativos de software. Uma série de fornecedores de nuvem, incluindo Amazon, Microsoft, Google e IBM fornecem serviços de computação.

A arquitetura sem servidor pode ser usada para muitos tipos de aplicativos de software e tarefas. Algum dos tipos comuns de aplicativos que são adequados para uma arquitetura sem servidor incluem web aplicativos, processamento de dados orientado a eventos, fluxos de trabalho de eventos, tarefas agendadas (trabalhos CRON), aplicativos móveis, chatbots e aplicativos da Internet das Coisas (IoT). Também é bom para tarefas de transformação de dados relacionadas a coisas como imagens/vídeo (por exemplo, para o para fins de compressão e/ou otimização), pacotes de voz e geração de PDF.

O diagrama a seguir mostra um exemplo de um sistema que tem uma arquitetura sem servidor:



Arquiteturas sem servidor usam função como serviço (FaaS) e back-end como serviço (BaaS) modelos para fornecer lógica de back-end para aplicativos de software. Os dois são usados juntos para fornecer a funcionalidade do aplicativo.

7.5.1. Função como serviço (FaaS)

Em uma arquitetura sem servidor, um pequeno trecho de código, como uma função, pode ser executado usando um serviço de computação efêmero para produzir um resultado. Isso é conhecido como Função como Serviço (FaaS). Por efêmero, queremos dizer que durará apenas um período limitado. O código executa em um contêiner que é girado na invocação e, em seguida, trazido de volta para baixo quando está completo.

As funções normalmente são chamadas devido a eventos ou solicitações HTTP. Quando uma função conclui seu processamento, ele pode retornar um valor para o chamador ou passar o resultado para outra função que será chamada como parte de um fluxo de trabalho. A saída pode ser estruturada, como um objeto de resposta HTTP, ou não estruturada, como uma string ou um número inteiro.

Cada função deve seguir o princípio de responsabilidade única (SRP) e servir a um único, propósito bem definido. As equipes de desenvolvimento podem codificar funções para realizar a lógica do lado do servidor e, em seguida, carregar o código para seu provedor de serviços para execução sob demanda. FaaS torna isso fácil de escrever código em seu laptop para executá-lo na nuvem.

As funções devem ser projetadas para serem idempotentes, de modo que várias execuções do mesmo pedido produza o mesmo resultado, e se o mesmo pedido é processado mais de uma vez, lá não deve haver nenhum efeito adverso.

Podem existir cópias de mensagens em vários servidores para fornecer redundância e alta disponibilidade. Para garantir pelo menos uma entrega, é possível que uma função seja invocada mais de uma vez. Por exemplo, um servidor com uma cópia de uma mensagem pode se tornar indisponível quando uma mensagem é recebida ou excluída, levando ao envio da mesma mensagem para uma função novamente.

As funções em FaaS podem ser síncronas ou assíncronas, dependendo das necessidades da tarefa. Uma das maneiras que as funções assíncronas funcionam no FaaS é que a plataforma retornará um identificador único que pode então ser usado para pesquisar o status do sistema assíncrono Operação.

Uma parte importante de uma arquitetura sem servidor é seu gateway de API. Um gateway de API é o ponto de entrada para um sistema. É um servidor HTTP que recebe solicitações de clientes e usa sua configuração de roteamento para encaminhá-los para o contêiner de função relevante. As funções são executadas em contêineres de computação e devem ser sem estado. O resultado da função FaaS é então enviado de volta ao gateway de API e, por fim, retornado ao cliente como uma resposta HTTP.

Implementações de FaaS que estão disponíveis de provedores incluem Amazon Web Services Lambda, Microsoft Azure Functions, Google Cloud Functions e IBM Cloud Functions.

7.5.2. Back-end como serviço (BaaS)

Backend as a Service (BaaS), que tem suas raízes no Mobile Backend as a Service (MBaaS), é um modelo que permite aos desenvolvedores tirar proveito dos aplicativos de serviço fornecidos pela terceiros. Isso reduz o tempo e os custos de desenvolvimento, pois as equipes não precisam escrever esses serviços internos. Em contraste com o FaaS, onde as equipes de desenvolvimento escrevem seu próprio código para as várias funções, o BaaS oferece o uso dos serviços existentes.

Exemplos da funcionalidade fornecida por aplicativos de serviço incluem um banco de dados, push notificações, armazenamento de arquivos e serviços de autenticação. Na arquitetura sem servidor diagrama, o serviço de autenticação e o serviço de armazenamento para os recursos estáticos são exemplos de BaaS.

7.5.3. Vantagens de arquiteturas sem servidor

Usar uma arquitetura sem servidor oferece muitos benefícios importantes, o que explica por que é se tornando cada vez mais popular.

7.5.3.1. *Poupança de custos*

Com uma arquitetura sem servidor, seu código só é executado quando necessário. Você obtém utilidade faturamento, em que você só é cobrado pelos recursos de computação reais usados. As organizações que mudam para uma arquitetura sem servidor irão reduzir os custos de hardware, pois há não mais servidores e infraestrutura de rede para apoiar. Além disso, as organizações não têm que contratar pessoal para dar suporte a toda essa infraestrutura. Essas economias de custo são semelhantes ao que as organizações podem economizar usando a infraestrutura como serviço (IaaS) ou plataforma como um Serviço (PaaS), exceto que também há custos de desenvolvimento reduzidos (aproveitando BaaS) e custos de dimensionamento (aproveitando FaaS).

7.5.3.2. *Escalável e flexível*

Você não pode sub provisionar ou super provisionar sua capacidade de computação com um servidor sem arquitetura. Você evita estar em uma situação em que não tem servidores suficientes durante períodos de pico de capacidade ou muitos servidores inativos durante os períodos fora de pico.

A escalabilidade e flexibilidade de arquiteturas sem servidor permitem a capacidade de computação para escalar para cima e para baixo conforme a demanda muda. Você só usa tanta capacidade de computação quanto você precisa e você é cobrado pelo valor que usar. O desperdício de capacidade de computação é mantido em um mínimo e as organizações terão custos reduzidos como resultado.

7.6. Concentre-se na construção de seus produtos principais

Outra das principais vantagens de usar uma arquitetura sem servidor é que não há servidores para gerenciar. Isso permite que as organizações se concentrem na criação de soluções e envio mais recursos.

Não ter que administrar uma infraestrutura aumenta a produtividade e reduz o tempo para mercado. Mesmo uma pequena equipe de desenvolvimento pode começar a construir um aplicativo e implantá-lo produção de forma relativamente rápida, porque eles não precisarão provisionar uma infraestrutura frente e eles não terão muito para gerenciar após a implantação. Uma equipe de desenvolvimento que quer construir um aplicativo rapidamente enquanto se preocupa menos com questões operacionais achar a arquitetura sem servidor atraente.

7.6.1. Desenvolvimento poliglota

Uma arquitetura sem servidor permite o desenvolvimento poliglota. As equipes de desenvolvimento são fornecidas com a capacidade de selecionar os melhores idiomas e tempos de execução com base na funcionalidade necessária. Também dá às equipes a chance de testar e experimentar facilmente tecnologias diferentes. Embora existam limitações quanto aos idiomas disponíveis para vocês, provedores de nuvem, estão expandindo sua seleção para que diferentes idiomas possam ser usados para as diferentes funções em desenvolvimento.

7.6.2. Desvantagens das arquiteturas sem servidor

Embora haja muitos benefícios em usar uma arquitetura sem servidor, você deve estar ciente que existem algumas desvantagens.

7.6.3. Dificuldades com depuração e monitoramento

Há complexidade em depurar sistemas distribuídos usando arquitetura sem servidor. Quando várias funções se integram para realizar uma tarefa e algo dá errado, pode ser difícil entender quando e por que o problema ocorreu. Os fornecedores fornecem ferramentas para depuração e monitoramento, mas ainda há um nível de imaturidade com o padrão de arquitetura sem servidor. Os aplicativos de arquitetura sem servidor se tornarão mais predominante, mas ainda é relativamente novo em comparação com outros tipos de arquitetura de software.

7.6.4. Problemas de multilocação

Os problemas de multilocação, embora não sejam exclusivos dos sistemas sem servidor, merecem consideração. Algum vez que softwares para clientes diferentes são executados na mesma máquina, há a possibilidade de um cliente afetar um cliente diferente. Exemplos disso incluem segurança problemas, como um cliente ser capaz de ver os dados de outro cliente ou desempenho problemas quando um cliente experimenta uma carga pesada que afeta o desempenho de outro cliente.

7.6.5. Bloqueio do fornecedor

O aprisionamento do fornecedor pode ser um problema com arquiteturas sem servidor. Você pode pensar que se movendo de um ambiente sem servidor para outro seria fácil, mas pode ser bastante envolvido. Além de seu código ser movido, cada provedor tem formatos específicos e métodos de implantação. Além disso, você pode aproveitar as vantagens de tecnologias e ferramentas específico para o seu fornecedor.

Algum grau de refatoração será necessário se você quiser trocar de fornecedor. Idealmente, seu O aplicativo de software não dependerá de um provedor de nuvem específico. Uma maneira de mitigar esta desvantagem é usar uma estrutura que empacota seu aplicativo de uma forma que permite para implantação em qualquer provedor de nuvem. O Serverless Framework (<https://serverless.com>) é um exemplo de tal estrutura.

7.6.6. Complexidade de projetar muitas funções

Os sistemas de software que possuem uma arquitetura sem servidor tendem a consistir em várias funções e há uma complexidade inerente ao projeto de muitas funções. Vai demorar para fazer decisões sobre a granularidade das funções fornecidas na arquitetura sem servidor. Um bom design fornece um equilíbrio entre ter muitas funções e ter funções que são muito grandes e difíceis de manter.

Há complexidade em encadear várias funções para executar transações. O projeto deve considerar como ele precisa lidar com uma situação na qual uma função em uma cadeia falha. Por exemplo, em caso de falha, o sistema pode precisar execute a lógica de compensação para cancelar uma transação.

7.6.7. Não há tantas otimizações de tempo de execução

As arquiteturas sem servidor não permitem muito em termos de otimizações de tempo de execução. Em um ambiente tradicional, otimizações podem ser feitas em relação à memória, processadores, discos e a rede. No entanto, os provedores de nuvem podem ajudar com otimizações para você.

7.6.8. Ainda imaturo

Padrões e práticas recomendadas para arquiteturas sem servidor não foram tão exaustivas estabelecido como outros tipos de arquitetura de software. No entanto, algumas organizações e as equipes de desenvolvimento não se importam de usar tecnologia de ponta, se puderem usá-la para seus vantagem. Com o passar do tempo, isso se tornará cada vez menos um problema.

7.7. Adotando uma abordagem híbrida para sem servidor

Usar uma arquitetura sem servidor para um sistema de software não precisa ser tudo ou nada abordagem. Além de novos aplicativos que aproveitam totalmente a arquitetura sem servidor, você pode escolha projetar uma parte do seu sistema com uma arquitetura sem servidor e usar um diferente padrão de arquitetura para as outras partes do sistema.

Por exemplo, uma organização pode optar por adotar uma abordagem híbrida e construir alguns novos recursos para um aplicativo existente em um ambiente sem servidor e usá-los com outros ambientes de arquitetura.

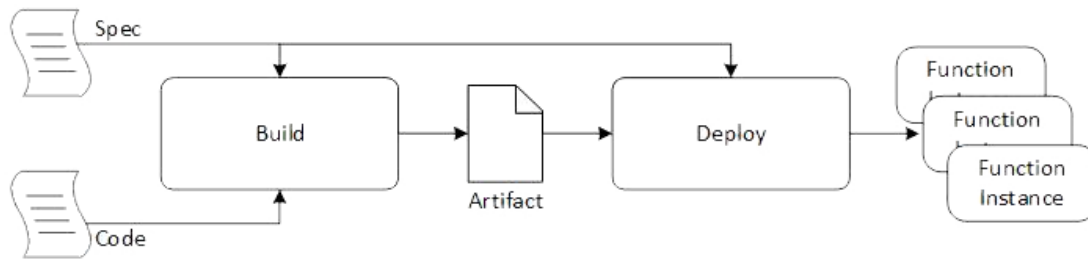
7.7.1. Implantação de função

Quando as funções são implantadas em um sistema sem servidor, elas passam por uma implantação pipeline. Embora as etapas no pipeline possam variar dependendo do provedor de nuvem, há são algumas etapas fundamentais que normalmente ocorrem.

Os desenvolvedores devem primeiro fazer upload da definição da função, que contém especificações sobre a função, bem como o código. As especificações e metadados incluem coisas como um identificador único, nome, descrição, identificador de versão, linguagem de

tempo de execução, recurso requisitos, tempo limite de execução (o tempo máximo que uma chamada de função pode executar até que seja encerrado), data/hora de criação e data/hora da última modificação. Quando uma função é invocada, é chamado para uma versão de função específica. O identificador de versão é usado para selecionar uma instância de função apropriada.

A ilustração a seguir mostra um pipeline de implantação de função típico:



Junto com a especificação, o próprio código e suas dependências devem ser fornecidos (para exemplo, como um arquivo ZIP ou Dockerfile). Se o código for armazenado em um repositório externo, o deve ser fornecido o caminho do local do código junto com as credenciais necessárias para o acesso.

Uma vez que a definição funcional foi carregada para o provedor de nuvem, o processo de construção usa-o para compilação para produzir um artefato. O artefato resultante pode ser um arquivo binário, um pacote ou uma imagem de contêiner.

O início de uma função de instância pode ser o resultado de uma inicialização a frio ou uma inicialização a quente. Com uma inicialização a quente, uma ou mais instâncias de função já foram implantadas e estão prontas para serem executado quando necessário. Uma inicialização a frio leva mais tempo, pois a função começa a partir de um estado não implantado. A função deve ser implantada e executada quando necessário.

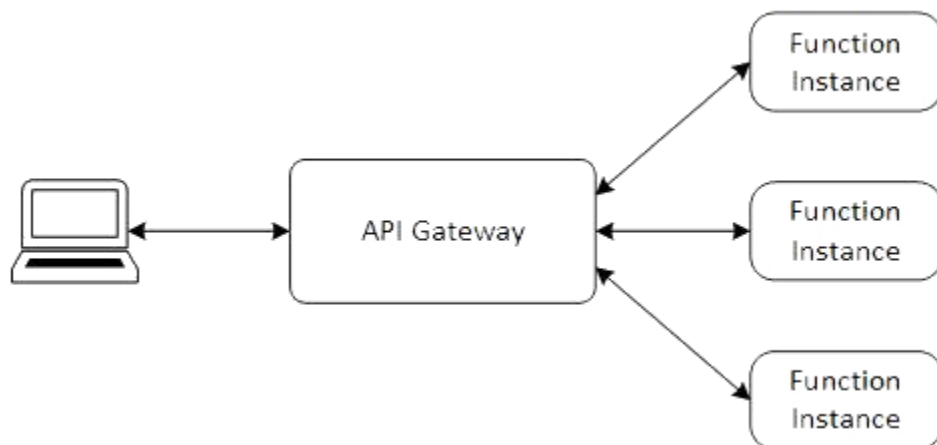
7.7.2. Invocação de função

Quando as funções são invocadas, existem quatro métodos principais de invocação que podem ser usados para invocar uma função:

- Pedido síncrono
- Solicitação assíncrona
- Fluxo de mensagens
- Trabalho em lote

7.7.2.1. Pedido síncrono

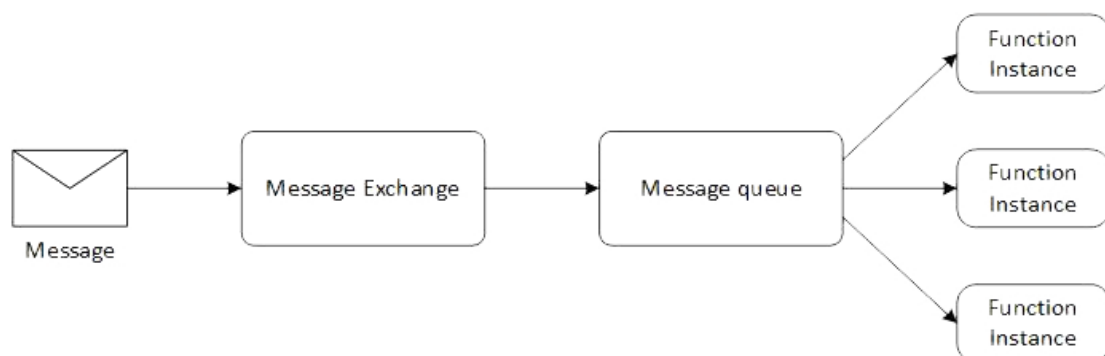
Quando um cliente faz uma solicitação síncrona, ele aguarda uma resposta. Um padrão de solicitação-resposta é usado para lidar com solicitações síncronas. Exemplos de solicitações incluem uma solicitação HTTP ou uma chamada gRPC. O diagrama a seguir mostra o fluxo de uma solicitação síncrona, que primeira passa por um gateway de API:



Para localizar uma instância de função, o gateway de API usará o serviço do lado do cliente descoberto ou passá-lo para um roteador (balanceador de carga) para descoberta de serviço do lado do servidor. Uma vez por a instância do serviço é localizada, a solicitação é passada a ela para processamento. Uma vez que a execução da função estiver concluída, a resposta é enviada de volta ao cliente.

7.7.2.2. *Solicitação assíncrona (fila de mensagens)*

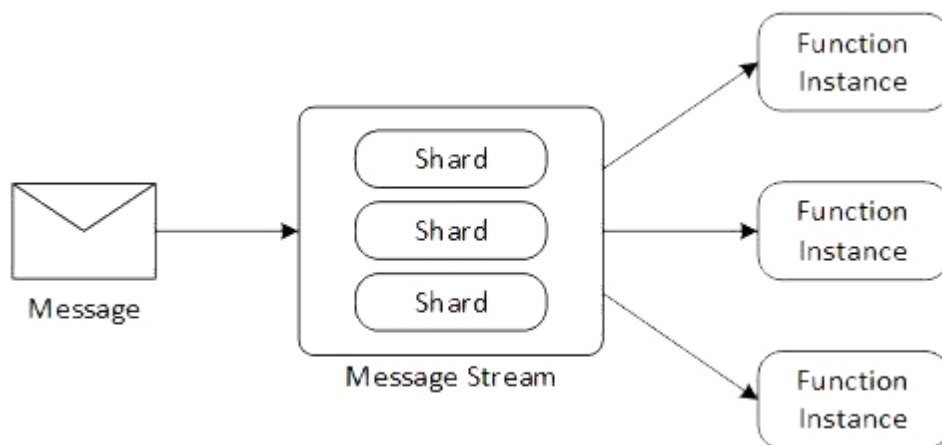
Quando você deseja processar solicitações de forma assíncrona, o padrão publicar-assinar pode ser usado. As solicitações de entrada são publicadas em uma bolsa. Troca e depois distribui mensagens para uma ou mais filas usando regras chamadas ligações. A partir daí, as instâncias de função são invocadas para processar mensagens publicadas:



Exemplos de filas de mensagens incluem RabbitMQ, AWS Simple Notification Service, Transporte de telemetria de enfileiramento de mensagens e trabalhos CRON agendados.

7.7.2.3. *Fluxo de mensagens*

Quando houver necessidade de processamento de mensagens em tempo real, um fluxo de mensagens pode ser usado. Os fluxos de mensagens podem ingerir, armazenar em buffer e processar grandes quantidades de dados de fluxo. Quando um novo fluxo é criado, normalmente é particionado em fragmentos, com cada fragmento indo para um trabalhador único (instância de função) para processamento:

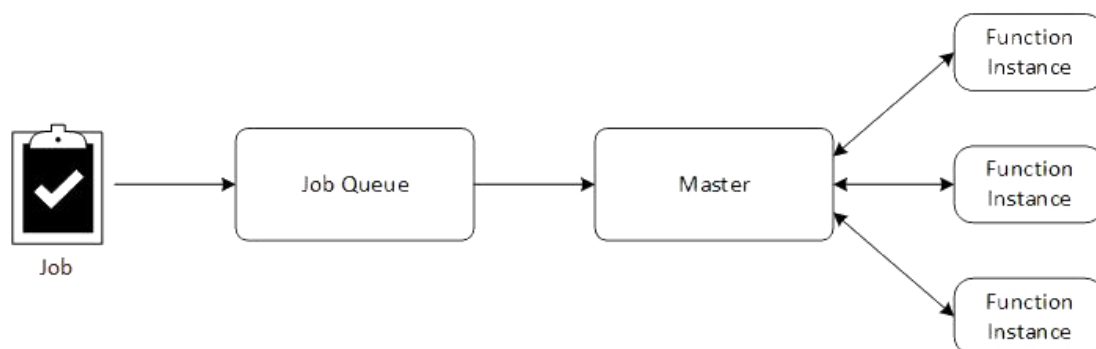


Exemplos de tecnologias para fluxos de mensagens sem servidor incluem AWS Kinesis, AWS DynamoDB Streams e Apache Kafka.

7.7.2.4. Trabalho em lote

Os trabalhos em lote são colocados em uma fila de trabalhos, sob demanda ou com base em uma programação. Para agilizar a execução dos trabalhos, o padrão mestre/trabalhador pode ser utilizado com empregos. O padrão mestre/trabalhador, que às vezes é referido como mestre/escravo padrão, acelera os trabalhos, dividindo-os em tarefas menores para que as tarefas possam ser processadas em paralelo.

O diagrama a seguir mostra o processamento de uma tarefa:



Um componente que desempenha o papel de Mestre divide o trabalho (conjunto de trabalho) em tarefas e lança trabalhadores. Neste caso, os trabalhadores são instâncias de função e podem processar tarefas em paralelo. Os trabalhadores continuam a puxar do conjunto de trabalho até que não haja mais tarefas a serem concluídas. O mestre reúne os resultados quando todos os trabalhadores terminam.

7.8. Aplicativos nativos da nuvem

A implantação de um aplicativo de software na nuvem não o torna nativo da nuvem. É sobre como o software é projetado e implementado, não apenas onde é executado. Nativo da nuvem

aplicativos foram projetados e desenvolvidos desde o início para serem implantados na nuvem. Ao fazer isso, os aplicativos podem aproveitar ao máximo seu ambiente de implementação.

No desenvolvimento de aplicativos modernos, a equipe de desenvolvimento precisa ter mais conhecimento sobre, e um grande interesse em, como seu aplicativo é executado na produção. Da mesma forma, a equipe de operações deve ser capaz de trabalhar com a equipe de desenvolvimento para melhorar, tempo, como o aplicativo é implantado e executado em um ambiente de produção.

7.8.1. Razões para mudar para a nuvem

Nos primeiros anos da computação em nuvem, as empresas hesitavam em adotar a nuvem tecnologias dentro de sua organização de TI. Eles tinham preocupações como perder o controle sobre sua infraestrutura, segurança, riscos de dados e confiabilidade.

Desde então, as empresas têm cada vez mais realocado suas cargas de trabalho de TI para a nuvem. Muitos as organizações já o fizeram ou têm planos de mover seus aplicativos para a nuvem.

Os principais provedores de nuvem, como Amazon, Microsoft e Google, estão experimentando uma explosão crescimento e a competição entre eles está se intensificando à medida que mais aplicativos são implantados na nuvem.

Há uma série de razões pelas quais as empresas estão movendo seus aplicativos e dados para a nuvem.

7.8.1.1. Reduzindo custos

A hospedagem baseada em nuvem reduz as despesas de capital, eliminando a necessidade de gastar dinheiro em ativos fixos, como hardware e software. Também reduz as despesas operacionais em reduzindo custos para coisas como equipe de suporte de TI e necessidades de eletricidade 24 horas por dia para energia e refrigeração.

7.8.1.2. Maior flexibilidade e escalabilidade

Tirar proveito da nuvem oferece às empresas maiores níveis de flexibilidade. Cargas de trabalho podem escalar para cima e para baixo com base na demanda muito rapidamente, mesmo quando grandes quantidades de computação recursos são repentinamente necessários. Grandes provedores de nuvem têm escala global. Além de sendo capazes de fornecer a quantidade certa de recursos de computação no momento certo, eles podem fazer, portanto, a partir da localização geográfica certa.

A computação em nuvem permite que as empresas cresçam com relativa facilidade, sem ter que fazer grandes ajustes em coisas como a quantidade de hardware.

7.8.1.3. Atualizações automáticas

A tarefa de manter o software de infraestrutura atualizado com o software e segurança mais recentes as atualizações são movidas para o provedor. As atualizações de hardware também são feitas pelo provedor de nuvem, incluindo upgrades para servidores, memória, poder de processamento e armazenamento em disco. Os data centers são atualizados regularmente com hardware de última geração, garantindo maior eficiência e desempenho mais rápido.

7.8.1.4. Recuperação de desastre

Backup, recuperação de desastres e continuidade de negócios são preocupações importantes para o software formulários. A computação em nuvem fornece esses serviços facilmente e com menos despesas do que fazendo isso por conta própria.

Para empresas menores, a despesa adicional para recuperação de desastres pode ser onerosa. A opção de implantar aplicativos na nuvem tornou mais fácil até mesmo para pequenos as empresas tenham recursos completos de backup e recuperação em caso de desastre.

7.9. O que são aplicativos nativos da nuvem?

Os aplicativos nativos da nuvem são projetados especificamente para o modelo de nuvem. Formulários projetados para a nuvem são capazes de tirar proveito disso como sua plataforma de destino, fornecendo organizações com vantagem competitiva, maior agilidade, implantação mais fácil, sob demanda escalabilidade, custos reduzidos, maior resiliência e outros benefícios.

As expectativas das aplicações modernas de hoje são diferentes daquelas do passado. Cloudnative aplicativos têm algumas características distintas que nos permitem atender a essas expectativas e requisitos. The Cloud Native Computing Foundation (CNCF) atualmente define o nativo da nuvem como usar uma pilha de software de código aberto para fazer aplicativos que são contêineres, orquestrados dinamicamente e orientados a micros serviços.

7.9.1. Containerized

Os contêineres são uma forma de empacotar um aplicativo de software. Eles são leves, autônomos pacotes de software. O aplicativo, junto com todas as suas bibliotecas e dependências, é agrupado em um pacote imutável. O conceito de contêineres para software é semelhante ao conceito de uso de contêineres físicos na indústria de navegação. Antes de padronizar os contêineres de transporte, o transporte poderia ser um processo ineficiente e complicado. Uma grande variedade de coisas em todas as formas e tamanhos podem ser transportado no mesmo navio de carga. Agora que existem contêineres de remessa padronizados tamanhos, sabemos o que e quanto caberá em um navio de contêiner, não importa o que seamos envio.

Da mesma forma, empacotar seu software com suas dependências em um contêiner baseado em os padrões nos permitem saber que ele pode ser executado em qualquer lugar que ofereça suporte a contêineres. Isto nos fornece previsibilidade, pois sabemos que o software funcionará conforme o esperado porque o contêiner é o mesmo, não importa onde seja executado. Elimina a experiência de ter um aplicativo não executado corretamente em uma máquina apenas para ter outra pessoa em uma máquina diferente afirmam que funciona na minha máquina. Erros inesperados devido a diferenças entre máquinas e ambientes pode ser drasticamente reduzido ou mesmo eliminado.

Em um aplicativo nativo da nuvem, cada parte do sistema é empacotada em seu próprio contêiner. Esse permite que cada parte seja reproduzível e permite que recursos sejam isolados para cada contêiner.

7.9.2. Orquestrado dinamicamente

Tornar seu aplicativo em contêineres não é suficiente. Um aplicativo nativo da nuvem precisará a capacidade de executar vários contêineres em várias máquinas. Isso é o que vai permitir a você para usar micros serviços e fornecer recursos de tolerância a falhas.

Depois de ter vários contêineres em execução em máquinas diferentes, você precisará orquestrá-los dinamicamente. O sistema deve iniciar o contêiner correto no momento certo, ser capaz de escalar contêineres adicionando e removendo-os com base na demanda e lançar recipientes em máquinas diferentes no caso de uma falha.

Existem várias ferramentas de agrupamento e orquestração de contêineres disponíveis. Atualmente, o mais popular é o Kubernetes, que às vezes é chamado de K8S porque há oito letras entre o K e o S. É um orquestrador de código aberto que foi originalmente desenvolvido pelo Google. Outras ferramentas de orquestração de contêineres além do Kubernetes incluem Docker Swarm e Apache Mesos.

Os provedores de nuvem também têm serviços de contêiner baseados em nuvem. Existe Amazon Elastic Container Service (Amazon ECS), que é o próprio serviço de orquestração de contêineres da Amazon para Amazon Web Services (AWS). Além do ECS, a Amazon oferece o Amazon Elastic Serviço de contêiner para Kubernetes (Amazon EKS), que é um serviço gerenciado para Kubernetes na AWS. A Amazon também oferece AWS Fargate, que é uma tecnologia que pode ser usado para Amazon ECS e EKS que permite executar contêineres sem ter que gerenciar servidores ou clusters.

O Azure Container Service (AKS) da Microsoft permite que você use um Kubernetes totalmente gerenciado serviço de orquestração de contêineres ou selecione um orquestrador alternativo, como não gerenciado Kubernetes, Docker ou Mesosphere DC/OS. O Google oferece o Google Kubernetes Engine, que é um ambiente gerenciado para Kubernetes.

7.10. Orientado a micro serviços

Um aplicativo nativo da nuvem deve ser particionado em micros serviços. Dividindo um aplicativo em serviços pequenos, autônomos, com versão independente e independentes aumenta a agilidade organizacional e a capacidade de manutenção do aplicativo.

7.10.1. Sem tempo de inatividade

Espera-se que os aplicativos de hoje estejam disponíveis o tempo todo, sem tempo de inatividade. Tem sempre foi uma meta para minimizar o tempo de inatividade, mas os dias em que era aceitável ter uma pequena janela de manutenção em que um aplicativo não estava disponível são encerradas.

Sistemas de software complexos terão falhas e devem ser projetados para esperar falhas para ocorrer. Os aplicativos nativos da nuvem são projetados para falhas e mantêm a tolerância a falhas em mente para que eles podem se recuperar rapidamente e minimizar o tempo de inatividade. Se um servidor físico falhar inesperadamente ou é retirado como parte da manutenção planejada, um sistema de failover redirecionar o tráfego para um servidor diferente. Os componentes de software devem ser projetados para que eles são fracamente acoplados, de forma que, se um falhar, um componente redundante pode assumir o controle.

Uma única falha não deve ter o efeito de derrubar todo o sistema de software. Para Por exemplo, se uma instância de um micro serviço falhar, outras instâncias podem receber solicitações de entrada. Se todas as instâncias de um micro serviço específico não estão operacionais, a falha é isolada apenas a uma parte do sistema, para que todo o sistema não falhe.

7.11. Entrega contínua

O aumento da concorrência e das expectativas do usuário significa que os aplicativos modernos têm ciclos de lançamento. Em vez de ter grandes lançamentos com meses (ou mesmo anos) de intervalo, a capacidade de ter atualizações de aplicativos em uma base mais regular (pense em semanas ou dias ao invés de meses) é uma necessidade.

Os aplicativos nativos da nuvem devem lançar atualizações de software rapidamente. Ciclos de lançamento mais curtos fornecem uma oportunidade de obter feedback dos usuários mais rapidamente. Ao invés de ter que esperar um longo período para receber feedback, a entrega contínua nos dá um muito mais estreito Loop de feedback. A equipe de desenvolvimento pode responder mais rapidamente aos usuários usando o feedback recebido para fazer ajustes e melhorar o software.

Os aplicativos nativos da nuvem aumentam a agilidade organizacional. Ao lançar o software e recebendo feedback regularmente, uma organização pode responder ao mercado, aos concorrentes e as necessidades de seus clientes rapidamente. Isso pode dar a uma organização um ambiente competitivo vantagem.

7.12. Suporte para uma variedade de dispositivos

Os aplicativos nativos da nuvem devem ser capazes de suportar uma variedade de dispositivos. Usuários de modernos os aplicativos usam dispositivos móveis, desktops, tablets e outros dispositivos. Eles esperam uma experiência unificada entre dispositivos, bem como a capacidade de alternar perfeitamente entre dispositivos. Para fornecer este tipo de suporte, os aplicativos nativos da nuvem garantem que serviços de back-end são capazes de fornecer a funcionalidade que uma variedade de dispositivos de front-end necessidade.

Com a IoT, muitos outros dispositivos agora estão conectados, o que significa que alguns aplicativos será necessário para apoiá-los. Para lidar com um número potencialmente grande de dispositivos e o volume de dados que eles produzem, sistemas altamente distribuídos projetados com um sistema nativo de nuvem abordagem são necessários.

7.13. Aplicativos de doze fatores

A metodologia de aplicativo de doze fatores é um conjunto de princípios que podem ser seguidos quando desenvolvimento de aplicativos a serem implantados na nuvem. Foi originalmente escrito por criadores do Heroku, que é uma plataforma de nuvem popular.

Os princípios da metodologia de aplicativo de doze fatores podem ser usados para projetar e desenvolver aplicativos nativos da nuvem. Um aplicativo que segue esta metodologia adere a certos restrições e está em conformidade com um contrato. Isso dá a ele um nível de previsibilidade que facilita implantação na nuvem. Esses fatores tornam o dimensionamento mais fácil, maximizam a portabilidade e garantir que o aplicativo possa ser implantado continuamente.

Em seu livro *The Twelve-Factor App*, Adam Wiggins descreve os doze fatores da seguinte forma:

- Base de código: uma base de código rastreada no controle de revisão, muitas implantações
- Dependências: declare e isole explicitamente as dependências
- Configuração: Armazenar configuração no ambiente

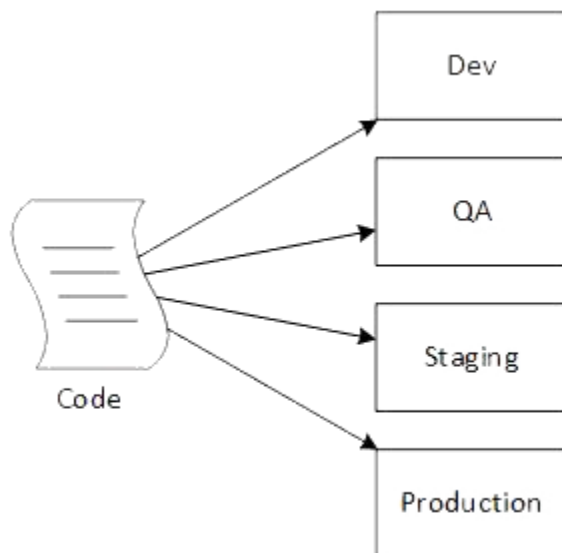
- Serviços de apoio: trate os serviços de apoio como recursos anexados
- Compilar, liberar, executar: separe estritamente os estágios de compilação e execução
- Processos: execute o aplicativo como um ou mais processos sem estado
- Vinculação de porta: Exportar serviços via vinculação de porta
- Simultaneidade: dimensionar através do modelo de processo
- Disposição: Maximize a robustez com inicialização rápida e desligamento normal
- Paridade de desenvolvimento/produção: mantenha o desenvolvimento, a preparação e a produção como o mais parecido possível
- Registros: trate os registros como fluxos de eventos
- Processos administrativos: execute tarefas de administração/gerenciamento de processos únicos

Nesta seção, vamos explorar os doze princípios de um aplicativo de doze fatores em mais detalhes.

7.13.1. Base de código

Um aplicativo nativo da nuvem deve ter uma, e apenas uma, base de código. Software maior os sistemas podem precisar ser decompostos em vários aplicativos, cada um dos quais deve ser tratado como um aplicativo separado com sua própria base de código.

Cada base de código deve ser rastreada em um sistema de controle de versão e pode ser usada para várias implantações do aplicativo. Por exemplo, uma única base de código pode ser implantada nos ambientes de desenvolvimento, controle de qualidade, teste e produção:



7.13.2. Dependências

As dependências do aplicativo devem ser declaradas explicitamente e isoladas. Um nativo da nuvem o aplicativo não deve depender da existência implícita de dependências. Sem suposições deve ser feito quanto ao que estará disponível no ambiente de execução, para que os aplicativos devem trazer suas dependências com eles e declarar suas dependências precisamente e

completamente. Os detalhes das dependências também devem incluir a versão específica que é necessário para cada dependência.

A maioria das linguagens de programação e estruturas modernas fornecem uma maneira de um aplicativo para gerenciar suas dependências. Às vezes, essas dependências vêm na forma de um pacote, que é uma distribuição de software que consiste em arquivos normalmente agrupados em alguns tipos de arquivo compactado. Um gerenciador de pacotes às vezes está disponível, que é uma ferramenta que pode ajudar para instalar, atualizar, configurar e remover pacotes.

7.13.3. Configuração

A configuração de um aplicativo consiste em valores que podem variar entre as implantações, como informações de conexão de banco de dados, URLs para serviços da web e informações do servidor SMTP para e-mails.

A configuração de um aplicativo nativo da nuvem deve ser armazenada no ambiente e não no código do aplicativo. A configuração irá variar dependendo do tipo de implantação (por exemplo, se a implantação é para desenvolvimento, teste ou produção), enquanto o código não. Deve haver uma separação estrita entre um código do aplicativo e sua configuração.

7.13.4. Serviços de apoio

Um serviço de apoio é qualquer serviço que o aplicativo usa na rede que é separado do próprio aplicativo. Exemplos de um serviço de apoio incluem um armazenamento de dados, um sistema de cache, um servidor SMTP, um servidor FTP e um sistema de mensagens/filas.

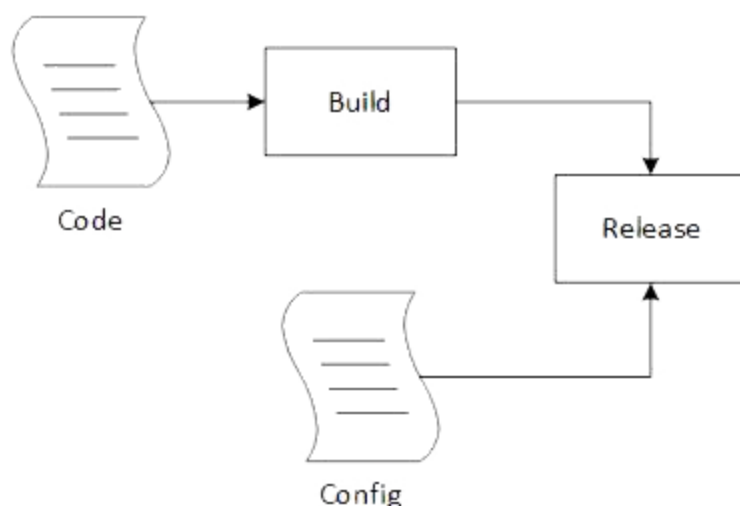
A ligação para cada serviço deve ser armazenada em uma configuração externa ao aplicativo. O aplicativo não deve se preocupar onde os serviços de apoio estão sendo executados e não deve fazer qualquer distinção entre serviços locais e de terceiros.

Todos os serviços de apoio devem ser tratados como recursos anexados, e o aplicativo deve ser capaz de anexar e desanexar serviços de apoio sem quaisquer alterações de código. Por exemplo, se um banco de dados cair, deve ser possível desanexá-lo e anexar um banco de dados diferente do um backup sem fazer alterações no código.

7.13.5. Construir/Liberar/Executar

Os aplicativos nativos da nuvem devem separar estritamente os estágios de construção, liberação e execução. O estágio de construção é onde o código é convertido em um pacote executável. De um aplicativo as dependências declaradas são consideradas durante este estágio.

O resultado do estágio de construção é uma construção:



No estágio de lançamento, a construção é combinada com informações de configuração externas ao aplicativo e é implementado em um ambiente de nuvem. Uma construção pode ser usada para múltiplas implantações. O resultado do estágio de liberação é um artefato imutável chamado um lançamento. Cada versão deve ter um identificador único, usando um esquema de controle de versão ou um carimbo de data/hora.

Assim que houver uma liberação, na fase de execução, também conhecida como runtime, o aplicativo é executado em seu ambiente. No momento em que as tarefas de construção e liberação são concluídas, o estágio de execução deve ser relativamente sem intercorrências e o aplicativo deve simplesmente funcionar.

7.13.6. Processos

Um aplicativo nativo da nuvem deve consistir em um ou mais processos sem estado, com quaisquer dados persistentes sendo armazenados usando um serviço de apoio. Os processos de um aplicativo que seguem a metodologia de aplicativo de doze fatores são sem estado e não compartilham nada.

A adesão a esta regra permitirá que seu aplicativo seja mais robusto e escalável com mais facilidade. Por exemplo, para que um aplicativo da web seja um aplicativo nativo da nuvem, ele não deve confiar em sessões pegajosas. Sessões pegajosas são onde um roteador/balanceador de carga pode atribuir um único servidor para um determinado usuário, de modo que as solicitações subsequentes do mesmo usuário sejam encaminhadas para o mesmo servidor. Entre outras coisas, isso pode permitir o armazenamento em cache dos dados da sessão do usuário na memória do processo do aplicativo, com a expectativa de que as solicitações subsequentes de o mesmo usuário serão roteadas para o mesmo processo e terá acesso aos dados em cache.

Em vez de usar sessões fixas, um aplicativo nativo da nuvem deve armazenar em cache por algum outro significa, como usar um sistema de cache distribuído como Redis ou Memcached.

7.13.7. Ligação de porta

Os aplicativos nativos da nuvem são totalmente independentes. Os serviços devem se tornar disponível para outros serviços por portas especificadas. Semelhante a qualquer serviço de apoio, seu aplicativo pode estar usando, a API que você expõe pode ser o serviço de apoio para outro aplicativo. Isso permite que um aplicativo seja o serviço de apoio de outro aplicativo.

Assim como o acesso a um serviço de apoio deve ser possível por meio de uma ligação simples, como um URL, outros aplicativos devem ser capazes de interagir com seus aplicativos por meio de um simples obrigatório.

7.13.8. Simultaneidade

Um aplicativo de software consiste em um ou mais processos e um aplicativo nativo da nuvem trata os processos como cidadãos de primeira classe. Os exemplos incluem um processo da web para lidar com HTTP solicitações e um processo de trabalho para lidar com uma tarefa assíncrona em segundo plano.

A ideia é que, ao executar vários processos para um aplicativo, ele possa ser executado de forma independente e simultaneamente. Isso permite que um aplicativo nativo da nuvem seja dimensionado horizontalmente conforme precisava.

7.13.9. Descartabilidade

Os processos de um aplicativo nativo da nuvem devem ser descartáveis para que possam ser iniciados ou interrompido a qualquer momento. Os processos devem ser projetados para que comecem tão rápido quanto possível e são capazes de desligar normalmente.

Os aplicativos nativos da nuvem devem ser robustos contra travamentos. No caso de um aplicativo trava, ele deve ter a capacidade de iniciar o backup sem a necessidade de esforço. Aplicações com essas qualidades relacionadas à descartabilidade permitem escalonamento elástico pôr o provedor de nuvem, bem como implantação rápida de código e/ou alterações de configuração. Se os tempos de inicialização para os processos de um aplicativo nativo da nuvem são muito lentos, isso pode levar a níveis mais baixos de disponibilidade durante períodos de alto volume de tráfego.

7.13.10. Paridade de desenvolvimento/produção

Os aplicativos nativos da nuvem devem minimizar as diferenças entre o desenvolvimento e produção, tanto quanto possível. As lacunas entre diferentes ambientes incluem diferenças nas ferramentas e serviços de apoio usados (incluindo quaisquer diferenças nas versões), o tempo entre quando algo é codificado e quando é lançado, e o fato de que diferentes as pessoas podem codificar o aplicativo em vez de implantá-lo.

As diferenças entre os ambientes de desenvolvimento e produção podem causar problemas para ir não detectado até que o software esteja em produção. Uma maneira de eliminar essas diferenças e atingir a paridade de desenvolvimento/produção é através do uso de contêineres. Anteriormente neste capítulo, discutimos contêineres e como eles são usados para empacotar um aplicativo, junto com todas as suas dependências, em um pacote imutável. Este contêiner pode então ser executado em qualquer lugar, incluindo ambientes de desenvolvimento, teste e produção.

Os contêineres nos dão previsibilidade porque sabemos que o aplicativo funcionará em uma maneira idêntica em diferentes ambientes.

7.13.11. Histórico

Os registros dão visibilidade à equipe de desenvolvimento e operações sobre o aplicativo, seu comportamento e suas exceções. Os registros são uma parte importante de um aplicativo e serão discutidos mais adiante em Capítulo 9, Questões transversais.

Os aplicativos nativos da nuvem não devem ser responsáveis pelo roteamento e armazenamento de seus fluxos de saída. Em vez de gravar ou gerenciar arquivos de log, cada processo deve tratar os logs como fluxos de eventos e simplesmente escrever seu fluxo de eventos na saída padrão (stdout) e erro padrão (stderr).

Em um ambiente de desenvolvimento, os desenvolvedores podem visualizar o fluxo para obter uma visão sobre o comportamento do aplicativo. Em um ambiente de produção, o fluxo deve ser capturado pelo ambiente de execução e, em seguida, encaminhado ao seu destino. Por exemplo, o riacho pode ser manipulado por uma ferramenta de gerenciamento de log e persistido em um armazenamento de dados.

Seguindo esta diretriz, os aplicativos nativos da nuvem podem ser escalonados dinamicamente para qualquer número das máquinas e não se preocupe em agregar todas as informações de log. Além disso, o método pelo qual as informações de registro são armazenadas e processadas pode ser alterado sem qualquer modificação do próprio aplicativo.

7.13.12. Processos administrativos

Periodicamente, haverá motivos pelos quais as tarefas administrativas precisarão ser executadas. Os exemplos incluem migração de banco de dados, limpeza de dados inválidos ou execução de análises para um relatório. Quando essas necessidades surgem, elas devem ser executadas como processos únicos em uma idêntica ambiente como produção. Isso significa que os scripts precisam ser executados no mesmo código e a mesma configuração de qualquer outro processo executado usando essa versão.

Os scripts administrativos/de gerenciamento para tarefas únicas devem ser comprometidos com o mesmo repositório de código como o resto do aplicativo e deve ser enviado com o resto do código em para manter a consistência e evitar problemas de sincronização entre ambientes.

7.14. Resumo

Os aplicativos de software hoje têm expectativas e requisitos que são diferentes dos passados. Existem demandas por maior disponibilidade, flexibilidade, tolerância a falhas, escalabilidade e confiabilidade. A entrega contínua e facilidade de implantação podem ser requisitos para organizações que desejam aumentar sua agilidade para manter seus aplicativos de software estreitamente alinhado com seus objetivos de negócios e oportunidades de mercado. Nós examinamos como MSA, arquitetura sem servidor e aplicativos nativos da nuvem podem atender a esses tipos de demandas e requisitos.

No próximo capítulo, examinaremos questões transversais. A maioria dos softwares aplicativos têm funcionalidades comuns que são necessárias em todo o aplicativo, inclusive em diferentes camadas do aplicativo. Esta funcionalidade é chamada de corte transversal preocupações, e vamos dar uma olhada em diferentes tipos de questões transversais e como leve-os em consideração durante o design e o desenvolvimento.